STUDY GUIDE

Operating Systems

Chapters 10–13: multiprocessor and real-time scheduling, I/O and disk scheduling, file management and embedded systems

This printable guide collects the notes, key terms and review questions (with answers and explanations) from the interactive guide. Use it to review after working through the simulations.

1. Multiprocessor, Multicore, and Real-Time Scheduling
2. I/O Management and Disk Scheduling
3. File Management
4. Embedded Operating Systems

Contents

1. **Chapter 10: Multiprocessor, Multicore, and Real-Time Scheduling**
   - 10.1 Multiprocessor and Multicore Scheduling
   - 10.2 Real-Time Scheduling
   - 10.3 Linux Scheduling
   - 10.4 UNIX SVR4 Scheduling
   - 10.5 UNIX FreeBSD Scheduling
   - 10.6 Windows Scheduling
2. **Chapter 11: I/O Management and Disk Scheduling**
   - 11.1 I/O Devices
   - 11.2 Organization of the I/O Function
   - 11.3 Operating System Design Issues
   - 11.4 I/O Buffering
   - 11.5 Disk Scheduling
   - 11.6 RAID
   - 11.7 Disk Cache
   - 11.8 UNIX SVR4 I/O
   - 11.9 Linux I/O
   - 11.10 Windows I/O
3. **Chapter 12: File Management**
   - 12.1 Overview
   - 12.2 File Organization and Access
   - 12.3 B-Trees
   - 12.4 File Directories
   - 12.5 File Sharing
   - 12.6 Record Blocking
   - 12.7 Secondary Storage Management
   - 12.8 UNIX File Management
   - 12.9 Linux Virtual File System
   - 12.10 Windows File System
   - 12.11 Android File Management
4. **Chapter 13: Embedded Operating Systems**
   - 13.1 Embedded Systems
   - 13.2 Characteristics of Embedded Operating Systems
   - 13.3 Embedded Linux
   - 13.4 TinyOS

Chapter 10 · Multiprocessor, Multicore, and Real-Time Scheduling

Many processors, shared caches and hard deadlines: scheduling stops being about one queue and one CPU.

Chapter 9 scheduled one processor. This chapter spreads the work over many processors and cores, and then adds deadlines that must be met. It looks at how tightly processors cooperate, how threads of one program are placed together, how real-time systems guarantee timing, and how Linux, UNIX SVR4, FreeBSD and Windows schedule in practice.

Learning objectives

- Classify multiprocessor systems and the granularity of parallelism in an application.
- Compare load sharing, gang scheduling, dedicated processor assignment and dynamic scheduling of threads.
- Explain what makes a system real-time and schedule tasks with deadline and rate monotonic scheduling.
- Describe priority inversion and how priority inheritance and priority ceilings prevent it.
- Compare the schedulers of Linux, UNIX SVR4, FreeBSD and Windows.

How Chapter 10 fits together

Chapter 9 chose the next process for a single processor. Chapter 10 asks what changes when there are many processors, and what changes when work has deadlines. In the overview timeline, five threads needing 3, 2, 4, 1 and 2 ticks (12 in all) share one queue and finish at t = 12 on one processor, t = 6 on two and t = 4 on four. Four processors give only a 3× speed-up (75% busy), because C alone needs 4 ticks and a thread runs on one processor at a time. C is due at t = 7: in arrival order on one processor it finishes at t = 9 and misses; moved to the front of the queue it finishes at t = 4 and makes it.

Many processors (10.1)

Multiprocessors range from loosely coupled clusters to tightly coupled machines that share one memory, and granularity measures how often the parallel parts of a program must synchronize. The design issues are assigning processes to processors (statically or dynamically), multiprogramming on each processor, and dispatching. Threads are placed by **load sharing** (one global queue), **gang scheduling** (related threads run at the same time), **dedicated processor assignment** or **dynamic scheduling**. On a multicore chip, threads that share data do well on cores that share a cache, while threads that compete for cache space are better kept apart.

Deadlines (10.2)

A **hard** real-time task must meet its deadline; a **soft** one should. A real-time OS needs determinism, responsiveness, user control, reliability and fail-soft operation. **Earliest deadline first** runs the task whose deadline is nearest; **rate monotonic scheduling** gives shorter periods higher fixed priorities and is

guaranteed when total utilization stays within $n(2^{1/n} - 1)$. **Priority inversion**, a high-priority task stuck behind a low-priority lock holder, is cured by **priority inheritance** or a **priority ceiling**.

Real systems (10.3 to 10.6)

Linux puts real-time classes (SCHED_FIFO, SCHED_RR) above normal tasks; the 2.6 kernels brought the O(1) scheduler, and from 2.6.23 the Completely Fair Scheduler ran normal tasks by smallest virtual runtime until EEVDF replaced it in 6.6 (2023). **UNIX SVR4** ranks every process on 160 global priorities in real-time, kernel and time-sharing classes, with preemption points and a tunable dispatch table. **FreeBSD**'s ULE scheduler keeps a run queue per core, uses an interactivity score to favor threads that mostly sleep, and balances load by pulling and pushing threads. **Windows** has 32 levels (16 real-time, 16 variable), boosts threads that stop waiting, lets the boost decay and rescues threads that have waited too long.

Why it matters next

Scheduling decides when work runs. Chapter 11 turns to the slowest work of all, input and output, where the order in which disk requests are served matters as much as the order of threads.

10.1 Multiprocessor and Multicore Scheduling

You should be able to

- Tell loosely coupled, functionally specialized and tightly coupled multiprocessors apart, and classify parallelism by its granularity.
- Explain the three design issues of multiprocessor scheduling: assigning processes to processors (static or dynamic, master/slave or peer), multiprogramming each processor, and the dispatching policy.
- Compare load sharing, gang scheduling, dedicated processor assignment and dynamic scheduling of threads, and compute the processor time a gang schedule wastes.
- Explain how shared caches on a multicore chip change where threads should run: cooperative sharing, contention and affinity.

Multiprocessor and multicore scheduling

With several processors the scheduler must also decide **which** processor runs each process, whether a program's threads should run **at the same moment**, and which **cache** holds a thread's data.

Three kinds of multiprocessor

- **Loosely coupled (distributed) multiprocessor**: a group of fairly independent computers, each with its own main memory and I/O channels (for example a cluster). They cooperate by messages.
- **Functionally specialized processors**: one general-purpose main processor in charge, helped by special-purpose processors such as an I/O processor or a graphics processor that work under its direction.
- **Tightly coupled multiprocessor**: several processors share one main memory under one operating system (symmetric multiprocessors, multicore chips). Any processor can run any process. This section is about this kind.

Granularity

Granularity is how often the parallel parts of an application must synchronize, measured as the number of instructions run between synchronization points.

Grain	Instructions between syncs	What it is	What it asks of the scheduler
Fine	fewer than 20	parallelism inside one instruction stream	handled by hardware and compilers, not OS scheduling
Medium	20 to 200	threads of one application	scheduling matters a lot: a thread that is not running stalls its partners
Coarse	200 to 2,000	concurrent processes in a multiprogrammed system	runs almost as on one processor, little or no software change
Very coarse	2,000 to about 1 million	distributed processing over network nodes	treated like coarse grain
Independent	no synchronization	unrelated processes, e.g. a time-sharing system	more run at once; average response time improves

Design issue 1: assigning processes to processors

- **Static assignment**: a process is tied to one processor for life; each processor has its own ready queue. Low scheduling overhead and easy group (gang) scheduling, but one processor may sit idle while another has a backlog.
- **Dynamic assignment**: one global queue; any free processor takes the next process. A variant keeps per-processor queues and moves waiting work to idle processors (**load balancing**). Costs: the shared queue must be locked, and a moved process finds a cold cache.

In the guide's lab (12 processes on 4 processors) the standard workload finishes at t = 16 with static assignment, with processors idle while work waited for 13 processor-time units; with one global queue or load balancing it finishes at t = 12 with no such idle time.

- **Master/slave architecture**: key kernel functions, scheduling included, always run on one master processor; slaves run user programs and ask the master for kernel services. Simple (little change from a uniprocessor OS; one processor controls memory and I/O, so conflicts are easy to resolve), but the master is a performance **bottleneck** and a **single point of failure**.
- **Peer architecture**: the kernel can run on any processor and each processor schedules itself from the pool of ready processes. The OS must make sure two processors never choose the same process and that no process is lost from a queue, usually with a lock on the shared queue. General-purpose kernels such as Linux, Windows and macOS take this approach.

Design issues 2 and 3: multiprogramming and dispatching

For coarse-grained or independent work, each processor should be multiprogrammed so it stays busy. For a medium-grained application on many processors, keeping every processor busy matters less than letting the application's threads run together.

Classic process scheduling uses one global queue (or one per priority level) feeding all processors, like one line served by many tellers; Linux and FreeBSD instead keep a queue per processor and balance the load.

The more processors there are, the less the choice of policy matters: a short process rarely waits long for *some* processor to free up. A classic simulation study found the difference between FCFS and round robin smaller with two processors than with one. The guide's experiment (4,000 processes, about 80% load per processor) shows the gap in mean turnaround keeps shrinking: in its first run with very mixed service times, FCFS's mean turnaround is about 54% longer than round robin's on 1 processor, 42% on 2, 31% on 4 and only 12% on 8. With similar service times FCFS wins, and that gap shrinks too (53% to 8%). So simple FCFS is often good enough on a multiprocessor.

Thread scheduling: four approaches

1. **Load sharing**: one global queue of ready threads; an idle processor takes the next one. Versions: **FCFS** (a new job's threads join the end of the queue), **smallest number of threads first** (the job with the fewest unscheduled threads goes first, ties by arrival; nonpreemptive), and **preemptive smallest number first** (an arriving job with fewer threads than a running job preempts that job's threads). Advantages: even load, no central scheduler (an idle processor runs the scheduling code itself), any uniprocessor policy can order the queue. Disadvantages: the queue needs mutual exclusion and becomes a bottleneck with many processors; preempted threads rarely resume on the same processor, so caches go cold; threads of one program rarely all run at once, which hurts closely coordinating threads. In the guide's example (3 processors; A with 6 threads, B with 1, C with 2) mean job turnaround is 6.00 with FCFS, 4.67 with smallest-number-first and 3.67 with the preemptive version, which also causes 2 migrations.
2. **Gang scheduling**: a set of related threads runs simultaneously on a set of processors, one thread per processor. It reduces blocking on synchronization and process switching, and one decision schedules many threads. Each time slice belongs to one gang, so a gang with fewer threads than processors leaves processors idle.
   Waste = $[(P - a)\cdot share_A + (P - b)\cdot share_B] / P$. Example: 4 processors, A has 4 threads, B has 1. Uniform (half each): 3/4 × 1/2 = **37.5%**. Weighted by threads (A 4/5, B 1/5): 3/4 × 1/5 = **15%**. With 6 processors, A = 6, B = 2: uniform 4/6 × 1/2 ≈ 33.3%, weighted 4/6 × 2/8 ≈ 16.7%.
3. **Dedicated processor assignment**: each application gets one processor per thread for its whole run; a processor whose thread blocks just idles. Justified because with very many processors utilization matters less than speed, and avoiding all process switches speeds the program. Experiments found speedup is best when an application's threads do not outnumber its processors and falls off beyond that.
4. **Dynamic scheduling**: the number of threads can change at run time; the OS partitions processors among jobs and each job maps its tasks to its processors. Allocation rules: (1) use idle processors first; (2) otherwise a **new arrival** gets one processor taken from a job holding more than one; (3) any unmet part of the request waits until a processor frees or the job withdraws it; (4) when processors are released, first give one to each waiting job that has none, then serve the rest first-come-first-served. In simulation studies it beat gang and dedicated assignment for applications that can adapt, but its overhead can cancel the gain.

Multicore thread scheduling

Cores on one chip often share a **last-level cache**. Classic schedulers place threads mainly to balance load; as core counts grow, avoiding off-chip memory trips matters as much, so kernels such as Linux also track which cores share a cache.

- **Cooperative resource sharing**: threads that share data (a producer and its consumer) should run on cores that share a cache.
- **Resource contention**: threads that each need lots of cache push each other's data out; a **contention-aware** scheduler spreads them over different caches.
- **Soft affinity**: prefer to run a thread again where it last ran, because its cache is warm; move it only to fix a bad imbalance. A strict limit on where a thread may run is hard affinity.

With two dual-core clusters you cannot always satisfy both rules: placing a sharing pair together may force two hungry threads together. In the guide's model (cache spills plus 2 × 2 MB when the shared buffer is split), together costs 4 MB of slow traffic per round and split costs 8 MB, so a scheduler must weigh the measured costs.

Key terms

Loosely coupled multiprocessor
A group of fairly independent computers, each with its own main memory and I/O channels, that work together over a network or fast link. Clusters and distributed systems are built this way.

Functionally specialized processors
A design in which one general-purpose main processor is helped by special-purpose processors, such as an I/O processor or a graphics processor, that work under its direction.

Tightly coupled multiprocessor
Several processors that share one main memory and are controlled by a single operating system. Symmetric multiprocessors and multicore chips are tightly coupled.

Granularity (synchronization granularity)
How often the parallel parts of an application must synchronize with each other, measured as the number of instructions executed between synchronization points. Fine grain means very often; coarse grain means rarely.

Static assignment
Tying each process to one processor for its whole life, with a separate ready queue for each processor.

Dynamic assignment
Letting a process run on whichever processor is free, usually by keeping one global ready queue that every processor takes work from.

Load balancing
Moving waiting processes or threads from a busy processor’s queue to an idle processor, so that no processor sits idle while another has a backlog.

Master/slave architecture
A multiprocessor OS design in which key kernel functions, scheduling included, always run on one master processor; the other (slave) processors run user programs and must ask the master for kernel services.

Peer architecture
A multiprocessor OS design in which the kernel can run on any processor and each processor schedules itself from the pool of ready processes.

Load sharing

Thread scheduling with one global queue of ready threads: whenever a processor is idle it takes the next thread from that queue. No thread is tied to a processor.

Gang scheduling

Scheduling a set of related threads, usually the threads of one process, to run at the same moment on a set of processors, one thread per processor. The older name coscheduling covers the same idea; gang scheduling usually means the strict form, where the whole group runs together or not at all.

Dedicated processor assignment

Giving an application as many processors as it has threads, for the whole of its run, so each thread has a processor to itself. A processor whose thread blocks simply sits idle.

Dynamic scheduling

Thread scheduling in which the number of threads in a process can change while it runs: the OS divides the processors among the jobs, and each job decides which of its threads use its processors.

Computer cluster

A group of complete, separate computers, each with its own memory and operating system, linked by a fast network and working together as if they were one bigger machine.

Last-level cache (LLC)

The cache level closest to main memory, often shared by all the cores of a chip or of a cluster of cores. A miss there means a slow trip off the chip to main memory.

Cooperative resource sharing

Threads that use the same data helping each other through a shared cache: on cores that share a cache, data one thread brings in is already there for the other.

Resource contention

Threads competing for the same limited resource, here the space in a shared cache: each pushes the other's data out, so both miss more often.

Contention-aware scheduling

Placing threads on cores so that threads that would fight over cache space use different caches, which cuts slow trips to main memory.

Soft affinity

A scheduler's preference, not a strict rule, to run a thread again on the processor where it last ran, because that processor's cache probably still holds its data. The scheduler may still move it to balance the load.

Review questions with answers

1. In which kind of multiprocessor do the processors share one main memory and run under a single operating system?

A. A loosely coupled system such as a cluster
B. A functionally specialized system with an I/O processor
C. A tightly coupled system such as a multicore chip
D. A distributed system linked by a fast network

Answer: C. A tightly coupled system such as a multicore chip
Tightly coupled processors share main memory under one OS, so any processor can run any process and the OS must decide who runs where.

2. Match each grain size with how many instructions typically run between synchronizations.

Fine	**Fewer than 20**
Medium	**20 to 200**
Coarse	**200 to 2,000**
Very coarse	**2,000 to about 1 million**
Independent	**No synchronization at all**

Fine grain is parallelism inside one instruction stream, medium is threads of one application, coarse is cooperating processes, very coarse is distributed processing over a network, and independent processes never synchronize.

3. What is the main drawback of static assignment, where each process is tied to one processor with its own queue?

A. Every processor must lock one shared queue before it can dispatch anything
B. A processor can sit idle while another processor still has a backlog of work
C. Processes keep migrating between processors and lose their cached data
D. It makes gang scheduling impossible because no process has fixed processors

Answer: B. A processor can sit idle while another processor still has a backlog of work
With per-processor queues, work cannot flow to an idle processor, so one processor may idle while another has several processes waiting.

4. In a master/slave multiprocessor, a failure of the master processor brings down the whole system.
Answer: True
Only the master runs the key kernel functions, scheduling included, so the slaves cannot get any kernel service once it fails: it is a single point of failure.

5. In a peer multiprocessor architecture, what extra problem must the operating system solve?

A. Sending every system call to one master processor and waiting for its reply
B. Moving each process to the processor with the most free cache before it runs
C. Making sure two processors never choose the same process and no process is lost
D. Running a separate copy of the OS, with its own memory, on every processor

Answer: C. Making sure two processors never choose the same process and no process is lost

Because every processor runs the kernel and schedules itself from a shared pool, the shared queues must be protected (for example with a lock) so that a process is neither taken twice nor lost.

6. Which are disadvantages of load sharing (one global queue of ready threads)? Select all that apply.

A. The shared queue needs mutual exclusion and can become a bottleneck with many processors
B. A preempted thread is unlikely to resume on the processor whose cache holds its data
C. The threads of one program are unlikely to all be running at the same moment
D. A dedicated scheduler processor must run all the time to hand out the threads

Answer: A, B, C

Load sharing spreads work evenly and needs no central scheduler (an idle processor runs the scheduling code itself), but the one queue is a contention point, threads lose cache warmth when they move, and related threads rarely run together.

7. Gang scheduling on 6 processors: application A has 6 threads and application B has 2. Each application gets half of the time. What percentage of the total processor time is wasted (idle)?

Answer: 33.3 %

While A runs, all 6 processors are busy. While B runs, 4 of 6 idle, for half the time: $4/6 \times 1/2 = 1/3 \approx 33.3\%$.

8. Same machine and applications (6 processors, A has 6 threads, B has 2), but now time is weighted by thread count: A gets 6/8 of the time and B gets 2/8. What percentage of processor time is wasted?

Answer: 16.7 %

Only B's slice wastes processors: 4 of 6 idle for 2/8 of the time, $4/6 \times 2/8 = 1/6 \approx 16.7\%$, half the waste of the uniform split.

9. Dedicated processor assignment leaves a processor idle whenever its thread blocks. Why can it still be a good choice?

A. It keeps every processor busy, because blocked threads give their processor to another job
B. It lets an application run many more threads than it has processors without any slowdown
C. It removes the need for the threads of an application to synchronize with each other
D. With very many processors, utilization matters less than avoiding process switches

Answer: D. With very many processors, utilization matters less than avoiding process switches

When processors are plentiful, each is a small share of the cost, so finishing applications fast (no process switching, warm caches) beats keeping every processor busy.

10. Under the dynamic scheduling policy, put the rules in the order the OS applies them, from a new request to a later release of processors.

Correct order:

1. Satisfy the request with idle processors first
2. If the job is new and nothing is idle, take one processor from a job that holds more than one
3. Leave whatever is still missing on a waiting list
4. On a release, give one processor to each waiting job that has none, then serve the rest first-come-first-served

Idle processors are used first; a brand-new job is guaranteed one processor if any job holds more than one; unmet demand waits; and freed processors go first to empty-handed jobs, then in arrival order.

11. On a multicore chip, two threads constantly pass a large buffer between them (a producer and its consumer). Where should a cache-aware scheduler place them?

A. On two cores that share the same last-level cache
B. On cores in different clusters, so each gets a whole cache
C. On whichever cores have the shortest queues at that moment
D. On cores as far apart as possible, to spread out the heat

Answer: A. On two cores that share the same last-level cache

This is cooperative resource sharing: on cores that share a cache, the buffer is stored once and data one thread writes is already there for the other.

12. Sort each statement: does it describe static assignment or dynamic assignment with one global queue?

Static assignment	Dynamic (global queue)
A process stays on one processor for its whole life	Any free processor takes the next waiting process
Each processor has its own ready queue	No processor idles while a process waits
Group (gang) scheduling is easy to arrange	The shared queue must be locked before each dispatch

Static assignment means per-processor queues, low overhead and easy gang scheduling, at the risk of imbalance. A global queue keeps every processor busy but every processor must lock the shared queue, and processes move between caches.

13. With several processors sharing one ready queue, the advantage of a clever dispatching policy over plain FCFS is usually larger than it is on a single processor.

Answer: False

The opposite: with more processors a short process rarely waits long for some processor, so the gap between policies such as FCFS and round robin shrinks, and simple FCFS is often good enough.

10.2 Real-Time Scheduling

You should be able to

- Explain what makes computing real-time and classify tasks as hard or soft, periodic or aperiodic.
- Describe the five requirements of a real-time OS and why immediate preemption cuts the wait for the processor.
- Name the four classes of real-time scheduling algorithms and schedule periodic and aperiodic tasks by earliest deadline.
- Apply rate monotonic scheduling and its utilization bound to decide whether a set of periodic tasks is safe.
- Explain priority inversion and how priority inheritance and priority ceilings keep it bounded.

What real-time means

In **real-time computing** a result is correct only if it is logically right **and** produced in time. Each real-time task has a **deadline**: a time by which it must start (starting deadline) or finish (completion deadline).

- **Hard real-time task**: missing the deadline causes unacceptable damage or a fatal error (an airbag, a pacemaker pulse, an emergency valve).
- **Soft real-time task**: the deadline is desirable; a late result still has some value, so it is still worth finishing (a video frame, a ticker display).
- **Periodic task**: comes back once per period T, or exactly T time units apart. **Aperiodic task**: arrives at unpredictable times, with a deadline to start, to finish, or both.

Real-time does not mean fast: it means a guaranteed worst case.

Five requirements of a real-time OS

1. **Determinism**: operations happen at fixed, known times or within known intervals. For interrupts it is the worst-case delay before the OS acknowledges one; long stretches with interrupts disabled are the main enemy.
2. **Responsiveness**: once acknowledged, how long until the interrupt is serviced: starting the handler, running it, plus delays from nested interrupts. Determinism + responsiveness = response time to an outside event.
3. **User control**: the designer sets fine-grained priorities, hard or soft, paging and swapping, which processes stay in main memory, disk transfer algorithms, and the rights of each priority band.
4. **Reliability**: failures may cost money, equipment or lives, so the bar is far higher than for a desktop system.
5. **Fail-soft operation**: fail in a way that keeps as much capability and data as possible instead of crashing. Its timing side is **stability**: if not every deadline can be met, the most critical tasks still meet theirs. Example: braking stays on time while the music pauses.

Typical RTOS features and preemption latency

Fast process or thread switch; small size; quick response to external interrupts; multitasking with IPC tools (semaphores, signals, events); special sequential files that collect data at high speed; preemptive

scheduling by priority; interrupts disabled only briefly; primitives to delay a task for a fixed time and to pause or resume tasks; special alarms and timeouts.

How long a newly ready real-time task waits:

- **Round robin**: its turn at the back of the queue (up to seconds).
- **Priority, nonpreemptive**: until the running task blocks or finishes (up to seconds).
- **Priority + preemption points**: until the next clock-based point (several ms).
- **Immediate preemption**: about 100 µs or less.

Four classes of real-time scheduling algorithms

- **Static table-driven**: analysis before run time builds a timetable of start times. For periodic tasks; inflexible (any change needs a new table).
- **Static priority-driven preemptive**: analysis before run time assigns fixed priorities; an ordinary preemptive priority scheduler runs the tasks. Example: rate monotonic scheduling.
- **Dynamic planning-based**: feasibility checked at run time; an arriving task is accepted only if all accepted deadlines can still be met.
- **Dynamic best effort**: no feasibility check; tries to meet deadlines and aborts a task whose deadline has passed. Common for aperiodic tasks.

A task may tell the scheduler its **ready time**, **starting deadline**, **completion deadline**, **processing time**, **resource requirements**, **priority** (a hard task may have an absolute priority) and **subtask structure** (a mandatory subtask with a hard deadline plus an optional one).

Deadline scheduling: periodic tasks

Earliest deadline first (EDF) runs, at every moment, the ready task whose deadline is nearest, preempting if needed. For independent preemptible tasks on one processor it is optimal: if any schedule meets every deadline, EDF does. For periodic tasks with deadline = next arrival, that means exactly $U \leq 1$.

Example: A every 20 ms needing 10 ms, B every 50 ms needing 25 ms ($U = 0.5 + 0.5 = 1.0$). A job unfinished at its deadline is missed and dropped.

- **Fixed priority, A above B**: A runs 0–10, 20–30, 40–50, so B's first job gets only 20 of its 25 ms by 50 and misses.
- **Fixed priority, B above A**: B runs 0–25, so A's first job misses at 20; later B runs 50–75 and A's fourth job misses at 80.
- **EDF**: A 0–10, B 10–20, A 20–30, B 30–45 (at 40, B's deadline 50 beats A's 60), A 45–55, B 55–60, A 60–70, B 70–90, A 90–100: all seven deadlines met.

Deadline scheduling: aperiodic tasks with starting deadlines

Each task needs 20 ms and, once started, runs to completion. A (arrives 10, start by 110), B (20, by 20), C (40, by 50), D (50, by 90), E (60, by 70).

- **Earliest deadline** (among tasks that have arrived): A 10–30, B's deadline passes at 20, idle 30–40, C 40–60, E 60–80, D 80–100. **B missed.**

- **Earliest deadline with unforced idle times** (earliest deadline, even if not yet arrived): unforced idle 10–20 while A waits, B 20–40, C 40–60, E 60–80, D 80–100, A 100–120. **All met.** Order: B, C, E, D, A. It needs arrival times in advance.
- **FCFS**: A 10–30, B missed, C 40–60, D 60–80, E's deadline passes at 70. **B and E missed.**

Rate monotonic scheduling (RMS)

RMS gives each periodic task a fixed priority by its period: shorter period (higher rate 1/T) = higher priority. Utilization $U = C_1/T_1 + \ldots + C_n/T_n$. Any scheduler needs $U \leq 1$. RMS is guaranteed to meet every deadline if $U \leq n(2^{1/n} - 1)$:

Bound for n = 1 to 6: 1.000, 0.828, 0.780, 0.757, 0.743, 0.735; as n grows it falls to ln 2 ≈ 0.693.

Worked example: C = 20, T = 100; C = 40, T = 150; C = 100, T = 350. U = 0.2000 + 0.2667 + 0.2857 = 0.7524 ≈ 0.752 ≤ 0.780 (pre-rounded terms add to 0.753: same verdict), so RMS meets every deadline.

The bound is **sufficient, not necessary**: between it and 1 you must check. Harmonic (10, 20), (10, 40), (20, 80) has U = 1.0 yet meets every deadline; (10, 20), (25, 50), also U = 1.0, misses under RMS but not under EDF. RMS stays popular: fixed priorities are simple and predictable, and under overload the short-period tasks keep their deadlines (stability); under EDF the task that misses is hard to predict.

Priority inversion

Priority inversion: a higher-priority task waits for a lower-priority one, usually because the lower task holds a lock. Some waiting is unavoidable, but it becomes **unbounded** when medium-priority tasks preempt the lock holder: the high task then also waits for them. The Mars Pathfinder lander (1997) kept resetting this way until engineers enabled priority inheritance on the lock.

- **Priority inheritance**: a lock holder runs at the priority of the highest task waiting for the lock, then drops back when it releases it.
- **Priority ceiling**: each resource gets a priority one level above its highest-priority user; a task that takes it runs at that ceiling at once and drops back on release. POSIX PTHREAD_PRIO_PROTECT mutexes work this way (often with the ceiling equal to the top user's priority: same effect).

Example: T3 (low) locks R at 1 for 4 units; T1 (high) arrives at 2 and needs R; T2 (medium, 6 units, no R) comes at 3. No protocol: T1 is blocked 3–12 and finishes at 15 (later if T2 is longer). Inheritance: T3 inherits priority 3 at 3, releases R at 6, T1 finishes at 9. Ceiling: T3 runs at 4 from time 1, so T1 cannot even preempt it; R is free at 5 and T1 again finishes at 9. Either cure limits T1's wait to one critical section.

Key terms

Real-time computing
Computing in which a result counts as correct only if it is both logically right and produced in time. A right answer delivered too late can be a failure.

Hard real-time task
A task that must meet its deadline: finishing late can cause unacceptable damage or a fatal error in the system, so a late result is worthless or harmful.

Soft real-time task

A task with a deadline that is desirable but not essential: a late result is worth less, yet it still makes sense to finish it.

Periodic task

A real-time task that repeats at a regular interval, its period T: once in every period, or exactly T time units apart.

Aperiodic task

A real-time task that arrives at unpredictable times but still has a deadline to start, to finish, or both.

Deadline

The time by which a real-time task must start (a starting deadline) or must finish (a completion deadline).

Determinism

How predictably an OS performs its operations at fixed, known times or within known intervals. For interrupts: the worst-case delay before the OS acknowledges one.

Responsiveness

How long an OS takes, once it has acknowledged an interrupt, to service it: starting the handler, running it, and any delay from nested interrupts.

Fail-soft operation

Failing in a way that keeps as much capability and data as possible instead of crashing outright. Its timing side is stability: if not every deadline can be met, the most critical tasks still meet theirs.

Preemption point

A place in the kernel, reached at regular intervals, where it checks whether a higher-priority task is waiting and, if one is, switches to it.

Static table-driven scheduling

Real-time scheduling in which analysis done before the system runs produces a fixed timetable saying when each periodic task starts. Predictable but inflexible: any change means rebuilding the table.

Static priority-driven preemptive scheduling

Real-time scheduling in which analysis done before the system runs gives each task a fixed priority, and an ordinary preemptive priority scheduler then runs the tasks. Rate monotonic scheduling is the standard example.

Dynamic planning-based scheduling

Real-time scheduling that checks feasibility at run time: an arriving task is accepted only if a plan exists that meets its deadline and the deadlines of every task already accepted.

Dynamic best effort scheduling

Real-time scheduling with no feasibility check: the system tries to meet every deadline, often with deadline-based priorities, and aborts a task whose deadline has passed. Common for aperiodic tasks.

Earliest deadline first (EDF)

A deadline scheduling policy that always runs the ready task whose deadline is nearest. For independent tasks that can be preempted on one processor, it meets every deadline whenever any schedule could.

Rate monotonic scheduling (RMS)

A fixed-priority policy for periodic tasks: the shorter a task's period (the higher its rate), the higher its priority.

Utilization bound
: For rate monotonic scheduling of n periodic tasks, the value n(2^(1/n) – 1). If the total utilization ΣC/T is at or below it, every deadline is guaranteed. It falls from 1 toward ln 2 ≈ 0.693 as n grows.

Priority inversion
: A higher-priority task is forced to wait for a lower-priority one, usually because the lower task holds a lock it needs. It becomes unbounded when medium-priority tasks keep preempting the lock holder.

Priority inheritance
: A cure for priority inversion: while a task holds a lock that a higher-priority task is waiting for, it runs at that waiting task's priority, and it drops back when it releases the lock.

Priority ceiling
: A cure for priority inversion: each shared resource gets a priority one level above its highest-priority user, and any task that takes the resource runs at that ceiling until it releases it.

Review questions with answers

1. A car's anti-lock braking controller must adjust brake pressure within a few milliseconds of a wheel starting to lock, or the car may skid. What kind of task is this?

A. A soft real-time task, since a slightly late adjustment still helps the driver
B. A hard real-time task, since missing the deadline can cause damage or injury
C. An aperiodic soft task, since wheel lock-ups happen at unpredictable moments
D. Not a real-time task, since the brakes still work without the controller

Answer: B. A hard real-time task, since missing the deadline can cause damage or injury

A hard real-time task is one whose missed deadline causes unacceptable damage or a fatal error. A soft task's late result still has some value.

2. Sort each task: hard or soft real-time?

Hard real-time	Soft real-time
Firing a car's airbag after a crash is detected	Showing the next frame of a streamed video
A pacemaker delivering its next pulse	Updating a stock-price ticker on a screen
Closing a valve during a chemical plant's emergency shutdown	Refreshing the weather widget on a phone

If lateness can hurt people or equipment, the task is hard. If a late result is merely less pleasant (a stutter, a stale number), it is soft.

3. Match each requirement of a real-time OS to what it is about.

Determinism	**Worst-case delay before an interrupt is acknowledged**
Responsiveness	**Time to service an interrupt once it has been acknowledged**
User control	**Designer sets priorities, paging and which tasks stay in memory**
Reliability	**Failures may cost money, equipment or lives, so must be very rare**
Fail-soft operation	**Keep as much capability and data as possible when a fault occurs**

Determinism and responsiveness together make up the response time to an outside event; user control, reliability and fail-soft operation complete the five.

4. Five aperiodic tasks each need 20 ms and must **start** by a deadline. A arrives at 10 (start by 110), B at 20 (by 20), C at 40 (by 50), D at 50 (by 90), E at 60 (by 70). A started task runs to completion. Under **earliest deadline with unforced idle times**, in what order do they run?

Correct order:

1. B
2. C
3. E
4. D
5. A

The scheduler always picks the earliest starting deadline among all unfinished tasks, even ones not yet arrived: it idles from 10 to 20 for B (by 20), then runs C (by 50), E (by 70), D (by 90) and A (by 110), and every task starts in time.

5. A kernel checks for a waiting higher-priority task only at preemption points that come every 10 ms. A real-time task becomes ready just after one of those points. About how long until it gets the processor?

A. About 100 µs, because the kernel switches as soon as the task is ready
B. Until the running task blocks or finishes, which could take seconds
C. One full round of time slices, one slice for every ready process
D. Close to 10 ms, the time remaining until the next preemption point

Answer: D. Close to 10 ms, the time remaining until the next preemption point

With preemption points the urgent task waits for the next point, so the delay is bounded by the point interval: here at most 10 ms, and close to that because the task became ready just after a point.

6. Three periodic tasks: C = 20 ms, T = 100 ms; C = 40 ms, T = 150 ms; C = 100 ms, T = 350 ms. What is their total processor utilization U = ΣC/T? (three decimals)

Answer: 0.7524

20/100 + 40/150 + 100/350 = 0.2000 + 0.2667 + 0.2857 = 0.7524, so 0.752 to three decimals (adding terms already rounded to 0.200 + 0.267 + 0.286 gives 0.753, which is also accepted). That is below the rate monotonic bound for three tasks, 0.780, so RMS will meet every deadline.

7. Three periodic tasks scheduled by rate monotonic scheduling have total utilization U = 0.85. The utilization bound for three tasks is 0.780. What can you conclude?

A. Some deadline will be missed, because U is above the bound
B. Every deadline will be met, because U is still below 1.0
C. Nothing yet: between the bound and 1, the quick test cannot decide
D. Switching to EDF would not help, since EDF has the same 0.780 limit

Answer: C. Nothing yet: between the bound and 1, the quick test cannot decide

U ≤ the bound guarantees success; U > 1 guarantees failure; in between you must analyse or simulate the actual task set.

8. On one processor, preemptive earliest deadline first meets every deadline of a set of independent periodic tasks (each deadline equal to the period) whenever their total utilization is at most 1.

Answer: True

For such task sets EDF is optimal: U ≤ 1 is both necessary and sufficient. Rate monotonic scheduling only guarantees success up to its lower bound.

9. Periodic task A (C = 10 ms, T = 20 ms) and task B (C = 25 ms, T = 50 ms) are released together at 0 and scheduled preemptively with fixed priority, A above B. How many ms of processing has B's first job received when its deadline arrives at 50?

Answer: 20 ms

A takes 0–10, 20–30 and 40–50; B gets only 10–20 and 30–40, so 20 of its 25 ms. B misses its deadline, although earliest deadline first meets every deadline for the same two tasks.

10. Which are typical features of a real-time operating system?

A. A fast process or thread switch
B. Preemptive scheduling based on priority
C. Long stretches with interrupts disabled to protect kernel data
D. Primitives to delay a task for a fixed amount of time
E. Round robin time slicing as the main scheduling policy
F. Special alarms and timeouts

Answer: A, B, D, F

An RTOS keeps interrupts disabled for the shortest possible stretches and schedules by priority with preemption, not plain round robin, so that urgent tasks get the processor at once.

11. Low-priority T3 holds lock R. High-priority T1 blocks on R. Then medium-priority T2, which never uses R, becomes ready. With **priority inheritance**, what happens next?

A. T3 runs at T1's priority until it releases R, so T2 cannot preempt it
B. T2 runs first, because it is ready and outranks T3's own priority
C. T1 preempts T3 and takes R, because T1 has the highest priority
D. T3 is aborted and R is freed, so that T1 can continue right away

Answer: A. T3 runs at T1's priority until it releases R, so T2 cannot preempt it

Under inheritance the lock holder temporarily runs at the priority of the highest task waiting for the lock, finishes its critical section, releases R and drops back.

12. Match each class of real-time scheduling algorithm to its description.

Static table-driven	**Offline analysis yields a timetable of start times**
Static priority-driven preemptive	**Offline analysis yields fixed priorities, as in RMS**
Dynamic planning-based	**A new task is accepted only if all deadlines can still be met**
Dynamic best effort	**No feasibility check; a task past its deadline is aborted**

Static approaches analyse before the system runs (producing a table or priorities); dynamic ones decide at run time, either with a feasibility check (planning) or without one (best effort).

10.3 Linux Scheduling

You should be able to

- Name the Linux scheduling classes and policies, and explain why real-time threads always run before normal ones.
- Trace SCHED_FIFO and SCHED_RR threads, including what happens on preemption, sleep and the end of a slice.
- Explain how the O(1) scheduler of Linux 2.6 picked a task in constant time with active and expired arrays and a bitmap.
- Compute CFS weights, processor shares, slices and virtual runtimes from nice values.
- Place the Linux schedulers on a timeline, from O(1) through CFS to EEVDF.

Linux scheduling

Scheduling classes and policies

Linux schedules threads (the kernel calls each one a task). Every thread belongs to one **scheduling class**, and the classes are served in a strict order: a lower class gets a processor only when every higher class has nothing ready on it.

1. **Deadline class**: `SCHED_DEADLINE` (since 3.14, 2014). A task declares a runtime, a deadline and a period; the kernel admits it only if there is enough spare processor capacity and runs the earliest deadline first.
2. **Real-time class**: `SCHED_FIFO` and `SCHED_RR`, real-time priority 1 (lowest) to 99 (highest).
3. **Fair class** for normal threads: `SCHED_OTHER` (called `SCHED_NORMAL` in the kernel source, the default), `SCHED_BATCH` (since 2.6.16: processor-bound jobs, mildly disfavoured when they wake) and `SCHED_IDLE` (since 2.6.23: weight 3, below nice +19's 15; beside one busy nice-0 thread it still gets about 0.3%, so it mostly uses time that would otherwise go idle). Normal threads share the processor according to their nice values, −20 to +19.

Real-time threads always run before normal threads. As a safety valve, by default real-time threads may use at most 95% of each second, so about 50 ms per second remains for normal threads even if a real-time thread loops forever.

Priority numbers

- Real-time priority: 1–99, **bigger is more urgent**.
- Nice value: −20 to +19, **smaller is more favoured** (a "nice" thread asks for less).
- Inside the kernel: one line of 140 levels, 0–139, **smaller is more important**. 0–99 are real-time (kernel number = 99 − real-time priority); 100–139 are normal (kernel number = 120 + nice). Example: real-time priority 30 → 69; nice +5 → 125; nice 0 → 120.

For two real-time threads the higher priority always wins. For two normal threads the nice value sets a *share*, not a winner.

SCHED_FIFO and SCHED_RR

- The scheduler runs the thread at the head of the highest-priority non-empty list.

- **SCHED_FIFO** has no time slice: the thread runs until it blocks, yields (`sched_yield`) or a higher-priority real-time thread becomes ready. It is never cut off by a thread of equal priority.
- A thread preempted by a higher priority goes back to the **head** of its list; a thread that wakes up or yields goes to the **tail**.
- **SCHED_RR** is the same plus a time slice (100 ms by default). When the slice runs out and another thread waits at the same priority, the thread goes to the tail of its list with a fresh slice. Linux refills a slice only when it reaches zero, so a preempted RR thread later finishes the rest of its slice.

Worked example (time unit 50 ms; A: FIFO, priority 90, needs 100 ms; B and C: priority 50, need 150 ms each; D: FIFO, priority 10, needs 100 ms; N: a normal thread needing 100 ms):

B and C use	Schedule (ms)
SCHED_FIFO	A 0–100, B 100–250, C 250–400, D 400–500, N 500–600
SCHED_RR	A 0–100, B 100–200, C 200–300, B 300–350, C 350–400, D 400–500, N 500–600
FIFO, A wakes at 150	A 0–100, B 100–150, A 150–200, B 200–300, C 300–450, D 450–550, N 550–650 (B resumes from the head)
FIFO, B sleeps at 150	A 0–100, B 100–150, C 150–300, B 300–400, D 400–500, N 500–600 (B rejoins at the tail)

Under FIFO, B and C run one after the other; under RR they alternate every slice. Either way D waits for both, and the normal thread N runs only after every real-time thread is done.

The O(1) scheduler (Linux 2.6.0, December 2003, to 2.6.22)

Linux 2.4 examined every runnable task at each decision, so its cost grew with the number of tasks. The O(1) scheduler made every decision cost the same:

- Each processor has its own **run queue** with two **arrays** of 140 lists, one list per priority: the **active** array and the **expired** array.
- A 140-bit **priority bitmap** marks the non-empty lists; finding the first set bit names the best list, and the task at its head runs.
- Higher priority means a longer slice: in 2.6.0, 10 ms at the lowest priority to 200 ms at the highest, about 100 ms at nice 0 (static priority 115 → 126 ms, 120 → 102 ms, 130 → 53 ms). From 2.6.9 to 2.6.22 the range was 5 ms (nice +19) to 800 ms (nice –20), exactly 100 ms at nice 0.
- A task that uses up its slice gets a fresh slice and moves to the **expired** array. When the active array is empty, the two array pointers swap. Every task with slice left runs before any task gets a second turn, so no normal task starves.
- **Sleep bonus**: dynamic priority = static priority – bonus + 5, with a bonus from 0 to 10 based on recent sleep time, kept inside 100–139. A nice-0 task that never sleeps sits at 125; one that mostly sleeps rises to 115. The slice still depends only on the static priority. Very interactive tasks could even skip the expired array. These guessing rules grew complicated and could be fooled.
- **Load balancing** (2.6.0–2.6.6): about every 200 ms on a busy processor, every 1 ms on an idle one, and whenever a queue runs empty, a processor pulls tasks from the busiest queue. From 2.6.7 the intervals depend on how closely the processors share hardware.

The Completely Fair Scheduler (2.6.23, October 2007, to 6.5)

CFS imitates an ideal processor that runs every ready task at once, each at its fair share.

- Each task keeps a **virtual runtime** (vruntime): `vruntime += time actually run × 1024 ÷ weight`.
- The rule: **run the task with the smallest vruntime**, the one that has been served least.
- **Load weight** from the nice value: 1024 at nice 0, about 1.25× more per step down and 1.25× less per step up (nice –5: 3121, –1: 1277, +1: 820, +5: 335, +19: 15). A busy task's share = its weight ÷ the total weight of the runnable tasks. One nice step between two busy tasks moves the split from 50/50 to about 56%/44%.

Worked example. Nice 0 (weight 1024) against nice +5 (weight 335): shares 1024 ÷ 1359 = 75.3% and 24.7%. Over one busy second the first runs 753.5 ms and the second 246.5 ms, yet both vruntimes grow by the same 753.5 ms: equal virtual time, unequal real time. A task that mostly sleeps keeps a small vruntime, so interactive work runs promptly without any special heuristics.

Slice lengths

- **Targeted latency**: the window in which every runnable task should get one turn, 6 ms by default from 2.6.36 (2010) to 6.5 (CFS started in 2.6.23 with 20 ms).
- **Minimum granularity**: the shortest turn, 0.75 ms by default from 2.6.36, so switching costs never swamp useful work.
- window = max(6 ms, n × 0.75 ms); slice of task i = window × w_i ÷ Σw. Up to 8 nice-0 tasks share the 6 ms window (4 tasks: 1.5 ms each); beyond 8 each turn stays 0.75 ms and the window grows (12 tasks: 9 ms).
- Example: three tasks with nice 0, 0 and +5 (total weight 2383): slices 2.58, 2.58 and 0.84 ms. After every task has had one turn, all three vruntimes have grown by the same 2.58 ms.
- The kernel scales both base values by 1 + $\log_2$ of the number of processors, up to 8 processors (24 ms and 3 ms on 8 processors).

The red-black tree and waking tasks

- Runnable tasks sit in a **red-black tree** (a self-balancing binary search tree) keyed by vruntime. The leftmost node is the next task; the kernel caches a pointer to it. Inserting or removing a task costs about $\log_2 n$ steps.
- The running task leaves the tree and is reinserted with its larger vruntime when its turn ends.
- A task waking from a long sleep would otherwise return with a tiny vruntime and hog the processor. CFS sets its vruntime to max(old value, smallest vruntime – half the targeted latency): a small head start of at most 3 ms with the base 6 ms latency, never a hoard. A new task likewise starts near the current smallest vruntime.

EEVDF (Linux 6.6, October 2023)

Earliest Eligible Virtual Deadline First replaced the CFS pick rule. Each task also gets a virtual deadline, and among the tasks that are still owed processor time (eligible), the one with the earliest virtual deadline runs. Weights, nice values and vruntime carry over. The targeted latency and minimum granularity settings gave way to a single base slice (0.75 ms by default).

O(1) versus CFS at a glance

	O(1) (2.6.0–2.6.22)	CFS (2.6.23–6.5)
Data structure	active + expired arrays of 140 lists, bitmap	red-black tree keyed by vruntime
Pick	head of the first non-empty list	leftmost task (smallest vruntime)
Nice value sets	priority level and slice length	weight, hence processor share
Interactivity	sleep bonus of up to ±5 levels	falls out of a small vruntime
Cost of a pick	constant	constant (cached leftmost); insert/remove log n

Key terms

Scheduling class
A family of scheduling rules inside the Linux kernel. Every thread belongs to one class; the kernel always serves the highest class that has a runnable thread, and each class orders its own threads.

SCHED_FIFO
A Linux real-time policy with no time slice: the thread keeps the processor until it blocks, gives the processor up, or a higher-priority real-time thread becomes ready.

SCHED_RR
A Linux real-time policy that works like SCHED_FIFO plus a time slice (100 ms by default): when the slice runs out, the thread goes to the back of the list for its priority.

SCHED_OTHER (SCHED_NORMAL)
The default Linux policy for ordinary threads; the kernel source calls it SCHED_NORMAL. These threads share the processor through the fair class, guided by their nice values.

SCHED_DEADLINE
A Linux policy (since kernel 3.14) for tasks that declare how much run time they need in every period and by when. The kernel serves the earliest deadline first, ahead of every other class.

Real-time priority
A number from 1 (lowest) to 99 (highest) given to a SCHED_FIFO or SCHED_RR thread. A ready thread with a higher real-time priority always runs before one with a lower number.

Run queue
The structure, one per processor, in which the Linux scheduler keeps the threads that are ready to run on that processor.

O(1) scheduler
The Linux scheduler from 2.6.0 (2003) to 2.6.22. It chose the next task in constant time, however many tasks were waiting, using priority lists, an active and an expired array and a bitmap.

Active and expired arrays
The two sets of 140 priority lists in each O(1) run queue. Tasks with slice left wait in the active array; a task that uses up its slice moves to the expired array; when the active array is empty, the two swap.

Priority bitmap
A row of 140 bits, one per priority list, where a 1 means “this list is not empty”. Finding the first 1 bit names the best non-empty list in a few instructions.

Sleep bonus
In the O(1) scheduler, a change of up to 5 priority levels either way: tasks that sleep a lot (usually interactive ones) move up, tasks that never sleep move down.

Load balancing
Moving ready tasks from busier processors' run queues to less busy ones, so that no processor sits idle while another has a backlog.

Completely Fair Scheduler (CFS)
The Linux scheduler for normal threads from 2.6.23 (2007) to 6.5. It always runs the task that has received the least weighted processor time, so each task gets a share in proportion to its weight.

Virtual runtime (vruntime)
A task's processor time scaled by its weight. It grows at the real rate for a nice-0 task, more slowly for a heavier task and faster for a lighter one. CFS runs the task with the smallest value.

Load weight
The number CFS derives from a nice value: 1024 at nice 0, about 1.25 times more for each step down and 1.25 times less for each step up. A task's share is its weight divided by the total weight of the runnable tasks.

Red-black tree
A self-balancing binary search tree. Its colouring rules keep the longest path from the root at most twice the shortest, so inserting or removing an item takes time proportional to log n.

Targeted latency
The CFS window (6 ms by default from 2.6.36; 20 ms in 2.6.23) within which every runnable task should get one turn. Each task's slice is its weight's share of that window.

Minimum granularity
The shortest turn CFS hands out (0.75 ms by default from 2.6.36), so that switching costs never swamp useful work. With many tasks, the window grows instead of the turns shrinking.

Earliest Eligible Virtual Deadline First (EEVDF)
The fair-class algorithm that replaced CFS in Linux 6.6 (2023). Among the tasks that are owed processor time, it runs the one whose virtual deadline comes first.

Review questions with answers

1. On one Linux processor, a SCHED_FIFO thread with real-time priority 40 is running. A SCHED_RR thread, also at priority 40, becomes ready. What happens next?

A. The SCHED_RR thread preempts it at once, because RR threads are favoured
B. The two threads start alternating, 100 ms each, immediately
C. The FIFO thread keeps running until it blocks, yields or is outranked
D. The kernel compares the two threads' nice values to pick a winner

Answer: C. The FIFO thread keeps running until it blocks, yields or is outranked
Threads of the same real-time priority share one list. A running SCHED_FIFO thread is never preempted by an equal-priority thread, so the newcomer waits at the tail of the priority-40 list.

2. A SCHED_FIFO thread is preempted because a higher-priority real-time thread wakes up. Where does the preempted thread go?

A. To the tail of the list for its priority
B. To the expired array, with a fresh slice
C. Down one priority level, as a penalty
D. Back to the head of the list for its priority

Answer: D. Back to the head of the list for its priority

A preempted real-time thread did nothing wrong, so it keeps its place at the head of its list and resumes as soon as no higher-priority thread is ready.

3. How did the Linux 2.6 O(1) scheduler find the next task in constant time?

A. It found the first set bit in a bitmap of non-empty priority lists
B. It sorted all runnable tasks by priority before every single decision
C. It kept every runnable task in a red-black tree ordered by priority
D. It always ran the task that had been waiting in the system the longest

Answer: A. It found the first set bit in a bitmap of non-empty priority lists

With 140 bits, one find-first-set operation identifies the highest-priority non-empty list, and the task at its head runs. The cost does not depend on how many tasks are waiting.

4. A buggy SCHED_FIFO thread at priority 99 spins in an endless loop on a one-processor Linux system with default settings. What happens to normal threads?

A. They never run again until the machine is restarted
B. They still get about 50 ms of each second
C. They get a fair half of the processor, as under CFS
D. The kernel ends the looping thread after 100 ms

Answer: B. They still get about 50 ms of each second

By default real-time threads may use at most 95% of each second, so about 50 ms per second remains for normal threads, enough to run a shell and stop the runaway.

5. Two processor-bound normal threads share one processor under CFS. One has nice 0 (weight 1024), the other nice +5 (weight 335). What percentage of the processor does the nice-0 thread receive?

Answer: 75.3 %

Share = weight ÷ total weight = 1024 ÷ (1024 + 335) = 1024 ÷ 1359 ≈ 75.3%. The nice +5 thread gets the other 24.7%.

6. CFS has a targeted latency of 6 ms and a minimum granularity of 0.75 ms. Twelve nice-0 tasks are runnable. How long, in ms, is the window in which each of them gets one turn?

Answer: 9 ms

Twelve equal turns of 6 ÷ 12 = 0.5 ms would be shorter than the 0.75 ms minimum, so each turn is 0.75 ms and the window stretches to 12 × 0.75 = 9 ms.

7. On a one-processor system running the O(1) scheduler, a processor-bound task that used up its time slice moved to the expired array and could not run again until the active and expired arrays swapped.

Answer: True

Expired tasks wait until every task in the active array has used its slice (or left); then the array pointers swap and the expired tasks become active again, with fresh slices.

8. Under CFS, a processor-bound task with nice +5 never runs while a processor-bound nice-0 task is runnable.

Answer: False

For normal threads the nice value sets a weight, not a strict rank. The nice +5 task still gets about a quarter of the processor; only real-time and deadline threads beat normal ones outright.

9. Put the Linux schedulers for normal threads in the order they were introduced.

Correct order:

1. The scan of every runnable task (2.4 and earlier)
2. The O(1) scheduler (2.6.0)
3. The Completely Fair Scheduler (2.6.23)
4. EEVDF (6.6)

The O(1) scheduler arrived with 2.6.0 in 2003, CFS replaced it in 2.6.23 in 2007, and EEVDF replaced the CFS pick rule in 6.6 in 2023.

10. Match each Linux policy to its behaviour.

SCHED_FIFO	**Real-time, no time slice**
SCHED_RR	**Real-time, equal priorities take turns**
SCHED_OTHER	**Default; shares by nice value**
SCHED_DEADLINE	**Declares runtime, deadline and period**
SCHED_IDLE	**Tiny weight; mostly uses otherwise idle time**

FIFO and RR are the real-time policies (RR adds a slice), SCHED_OTHER is the fair default, SCHED_DEADLINE (3.14) outranks everything, and SCHED_IDLE gives the least important background work a weight of only 3, below even nice +19.

11. Which scheduler does each feature belong to?

O(1) scheduler	CFS
Active and expired arrays Bitmap marking non-empty priority lists Sleep bonus of up to ±5 levels	Red-black tree sorted by vruntime Weight 1024 at nice 0 Targeted latency and minimum granularity

O(1) used arrays, a bitmap and a sleep-based bonus; CFS replaced all of them with vruntime, weights, a red-black tree and a latency-based slice.

12. A CFS task wakes up after sleeping for a long time. Which statements are true?

A. Its vruntime is raised to at least the smallest vruntime minus a small allowance
B. It keeps its old vruntime, however small, to catch up on missed time
C. It is inserted into the red-black tree according to its vruntime
D. It receives a fixed 100 ms slice regardless of its weight

Answer: A, C

CFS sets a woken task's vruntime to max(old value, smallest vruntime – half the targeted latency) and inserts it into the tree by that key. Keeping a tiny old value would let it hog the processor; slices come from weights, not a fixed 100 ms.

10.4 UNIX SVR4 Scheduling

You should be able to

- Explain the three goals of the SVR4 scheduler and how they differ from the traditional UNIX scheduler of Section 9.3.
- Place any global priority from 0 to 159 in its class (real-time, kernel or time-sharing) and state how that class sets priorities and quanta.
- Explain what a preemption point is and compute the dispatch latency of a real-time process with and without preemption points.
- Trace how the dispatcher uses the dispatch queues and the dqactmap bit map to choose the next process.
- Use a time-sharing dispatch table to predict how a process's priority and quantum change when it uses its quantum, sleeps, or waits too long.

UNIX SVR4 scheduling

UNIX System V Release 4 (SVR4, late 1980s) replaced the traditional UNIX scheduler of Section 9.3, which ranked processes by fixed bands plus one usage formula recomputed once a second, had no real-time class, and ran a kernel that could not be preempted. The redesign has three goals, in strict order of preference:

1. **Real-time processes** get the highest preference: for them, being late is as bad as being wrong.
2. **Kernel-mode work** comes next, so processes holding kernel resources finish quickly and release them.
3. **Ordinary user-mode (time-sharing) processes** come last and share the remaining processor time.

Four mechanisms deliver this: one scale of global priorities split into classes, preemption points in the kernel, per-level dispatch queues found through a bit map, and a tunable table that moves time-sharing priorities.

160 global priorities in three classes

Every process has a **global priority** from 0 to 159. **A larger number is more urgent** (159 is the top), the opposite of traditional UNIX, where a smaller number was more urgent.

Real-time class: 159 … 100 (60 levels)

Kernel class: 99 … 60 (40 levels)

Time-sharing class: 59 … 0 (60 levels)

Class	Global priorities	Who runs there	Priority	Quantum
Real-time	100–159	Processes that must react within a known time (machine control, audio, video). Only a privileged user can place a process here, using the `priocntl` system call.	Fixed; never changed by the scheduler	Fixed per priority level or per process
Kernel	60–99	The kernel's own system processes (for example the page-out daemon) and processes that have just woken from a sleep inside the kernel, while they finish their kernel work.	Fixed; chosen by what the process waited for	None (not time-sliced): runs until it sleeps again or returns to user mode; only a real-time process can preempt it, at a preemption point
Time-sharing	0–59	Ordinary user processes: shells, editors, compilers. The default class for user work.	Variable: down after a whole quantum, up after a sleep or a long wait	Set by the priority: 100 ms at 0 down to 10 ms at 59 in the standard SVR4 table

A real-time process outranks even kernel work, so a buggy real-time process stuck in a loop can freeze everything below it. Kernel-mode work sits above user work because a process inside the kernel often holds buffers or locks that others need. Strictly, the kernel class proper holds the system processes; a user process that wakes inside the kernel stays in its own class and only borrows a kernel-band priority until it returns to user mode, when ts_slpret sets its new time-sharing priority.

Preemption points

Traditional UNIX had a **nonpreemptible kernel**: a process inside a system call kept the processor until it slept or returned to user mode, so a newly ready process could wait as long as the longest system call. The kernel cannot simply be stopped anywhere, because midway through a call a shared structure may be half updated.

A **preemption point** is a place in a long kernel path where every kernel data structure is either consistent or locked. At each one the kernel checks a flag that says a real-time process is ready, and if it is set, switches right there. The worst **dispatch latency** (delay from becoming ready to running) is then the longest stretch between two points, not the longest system call.

Worked example. A 12 ms write system call has preemption points at 2.5, 5.5 and 9.5 ms. A real-time process R becomes ready at 6 ms. With preemption points R runs at 9.5 ms and waits 3.5 ms; with a nonpreemptible kernel it runs at 12 ms and waits 6 ms. Worst cases: 4 ms (the longest stretch, 5.5–9.5) versus 12 ms. If R becomes ready during the last stretch (after 9.5 ms), both kernels switch at 12 ms. Later

kernels went further and can be preempted almost anywhere (Solaris 2, and Linux 2.6 as a build option) by locking each shared structure separately.

Dispatch queues and the dqactmap bit map

- There is one **dispatch queue** (ready queue) per global priority: 160 queues. Processes in the same queue take turns round robin.
- **dqactmap** is a bit map with one bit per priority level; a bit is 1 exactly when that level's queue holds at least one ready process. The running process is in no queue.
- The dispatcher finds the highest set bit and runs the process at the head of that queue. 160 bits fit in five 32-bit words, so this is cheap and does not grow with the number of ready processes.
- A process whose quantum expires goes to the tail of its queue; a process that blocks leaves every queue. A process that becomes ready joins the tail of its level and sets that bit; if its priority is higher than the running process's, the running process is preempted (and goes back to the front of its own queue). When that happens depends on where the running process is: in user mode, the newcomer takes over at once; inside the kernel, a real-time newcomer takes over at the next preemption point, and any other newcomer waits until the running process sleeps or returns to user mode.

The time-sharing dispatch table (ts_dptbl)

Time-sharing priorities are driven by a tunable table with one row per priority 0–59. On each event the kernel reads a column of the process's current row:

Column	Meaning
ts_globpri	The global priority this row describes.
ts_quantum	The quantum for a process at this priority.
ts_tqexp	New, lower priority after the process uses its whole quantum ("time quantum expired").
ts_slpret	New, higher priority when it returns to user mode after a sleep ("sleep return").
ts_maxwait	Seconds a ready process may go without using up its quantum before the guard acts.
ts_lwait	New, higher priority for a process that waited longer than ts_maxwait: a starvation guard.

Example table used in this section (illustrative; real systems ship their own values, which an administrator can retune): quantum 100 ms for rows 0–9, 80 ms for 10–19, 60 ms for 20–29, 40 ms for 30–39, 20 ms for 40–49 and 10 ms for 50–59; tqexp = row – 10, slpret = row + 10, lwait = row + 20, each kept within 0–59; maxwait = 1 s.

Worked example. A process at row 35 has a 40 ms quantum. If it uses all 40 ms it moves to row 25 (quantum 60 ms). If instead it blocks for I/O and returns from the sleep, it moves to row 45 (quantum 20 ms). If it sits ready for more than 1 s without using up its quantum, it moves to row 55 (quantum 10 ms).

What the table does over time

A processor-bound process keeps using whole quanta, so it sinks, and its quanta grow as it falls. An I/O-bound process blocks before its quantum ends, so each sleep return lifts it toward the top rows, where quanta are short but it rarely needs a long one. In the simulation (A processor-bound from row 50; B I/O-bound from row 20 with 3 ms bursts and 25 ms I/O waits), A sinks from 50 to 0 by t = 228 ms with quanta of 10, 20, 40, 60, 80 and 100 ms; B first runs at 70 ms, once A has sunk to its row, then climbs to 59 by t =

182 ms. A burst longer than a row's quantum (for example 16 ms against a 10 ms quantum) makes the process alternate: down after a whole quantum, up after each sleep. Two simplifications: the run is shorter than one second, so ts_maxwait never fires, and a process that slept inside the kernel would first finish its kernel work at a kernel-band priority before dropping to its ts_slpret row.

History

The SVR4 design lives on in Solaris and illumos, which kept the 0–159 layout and added further classes (such as interactive, fair-share and fixed-priority) and a band above 159 for interrupt threads.

Key terms

UNIX SVR4 (System V Release 4)
AT&T's Release 4 of UNIX System V, from the late 1980s, which merged features of the main UNIX branches of its day. Its scheduler, taught here, lives on in Solaris and illumos.

Global priority
The single number, from 0 to 159, that SVR4 uses to rank every ready process whatever its class. A higher number means a more urgent process.

Scheduling class (priority class)
A group of priority levels that shares one set of rules for choosing priorities and quanta. SVR4 has three: real-time, kernel and time-sharing.

Real-time class
SVR4's top class, global priorities 100 to 159. Each process keeps a fixed priority and a fixed quantum, set when a privileged user places it in the class; the scheduler never changes them.

Kernel class (system class)
SVR4's middle band, global priorities 60 to 99: fixed priorities and no time slicing. The kernel's own system processes live here, and a user process that wakes from a sleep inside the kernel borrows a priority in this band until it returns to user mode.

Time-sharing class
SVR4's bottom class, global priorities 0 to 59, for ordinary user processes. A process's priority falls when it uses a whole quantum and rises when it returns from a sleep.

Preemption point
A place in kernel code where every kernel data structure is either consistent or locked, so the kernel can safely stop the current process there and switch to a waiting real-time process.

Dispatch latency
The delay between the moment an urgent process becomes ready and the moment it actually starts running.

Dispatch queue
In SVR4, the ready queue for one global priority level. There are 160 of them, and the processes in one queue take turns round robin.

dqactmap
SVR4's bit-map vector with one bit per priority level. A bit is 1 exactly when that level's dispatch queue holds at least one ready process.

Bit map (bit vector)
A row of bits in which each bit answers yes or no for one item, such as “is this queue non-empty?”. A processor can find the highest 1 bit in a word with a few instructions.

Dispatch table (ts_dptbl)
SVR4’s tunable table for the time-sharing class, with one row per priority 0 to 59. Each row gives that priority’s quantum and the new priority to use after each kind of event.

ts_globpri
The global priority that one row of the time-sharing dispatch table describes (0 to 59).

ts_quantum
The time quantum given to a time-sharing process that runs at this row’s priority.

ts_tqexp
The new, lower priority given to a process that used up its whole quantum (“time quantum expired”).

ts_slpret
The new, higher priority given to a process when it returns to user mode after a sleep (“sleep return”).

ts_maxwait
The number of seconds a ready process may go without using up its quantum before the starvation guard raises its priority to ts_lwait.

ts_lwait
The new, higher priority given to a ready process that has gone more than ts_maxwait seconds without using up its quantum: a guard against starvation.

priocntl
The SVR4 system call (and command) that places a process in a scheduling class and sets its class-specific priority and quantum.

Review questions with answers

1. In UNIX SVR4, which of these global priorities belongs to the kernel class?

A. 45
B. 130
C. 75
D. 160

Answer: C. 75

SVR4’s kernel class covers global priorities 60 to 99, so 75 is a kernel-class priority. Real-time is 100–159 and time-sharing is 0–59.

2. In UNIX SVR4, a process at global priority 90 is more urgent than a process at global priority 20.

Answer: True

In SVR4 a larger global priority is more urgent. 90 is in the kernel class and beats every time-sharing priority, including 20. (Traditional UNIX used the opposite direction, where a smaller number was more urgent.)

3. What makes a place in SVR4 kernel code suitable to be a preemption point?

A. The kernel switches back to user mode there after a call.
B. Every kernel structure there is consistent or locked.
C. A clock interrupt is certain to arrive there every 10 ms.
D. Only code that belongs to real-time processes runs there.

Answer: B. Every kernel structure there is consistent or locked.

A preemption point is a spot where no kernel data structure is half updated (each is consistent or protected by a lock), so the kernel can safely switch to a waiting real-time process there.

4. A system call's kernel path has preemption points at 2.5 ms, 5.5 ms and 9.5 ms and returns to user mode at 12 ms. A real-time process becomes ready 6 ms into the call. How many milliseconds does it wait before it can run?

Answer: 3.5 ms

The next preemption point after 6 ms is at 9.5 ms, so it waits 9.5 – 6 = 3.5 ms. A nonpreemptible kernel would make it wait until 12 ms, which is 6 ms.

5. In SVR4's dqactmap bit map, what does a bit set to 1 mean?

A. The process at that priority is running on the processor.
B. That priority level belongs to the real-time class.
C. A process at that priority has used up its whole quantum.
D. That level's dispatch queue holds at least one ready process.

Answer: D. That level's dispatch queue holds at least one ready process.

dqactmap has one bit per priority level, set exactly when that level's dispatch queue is not empty. The dispatcher runs the head of the queue with the highest set bit.

6. A time-sharing process uses up its quantum in SVR4. Put what happens next in order.

Correct order:

1. The running process uses up its whole quantum
2. Its row's ts_tqexp column gives its new, lower priority
3. It joins the tail of that priority's dispatch queue, and that bit is set
4. The dispatcher finds the highest set bit in dqactmap
5. The process at the head of that queue starts running

The table decides the new priority first, the process is queued at that level (setting its bit), and only then does the dispatcher look for the highest set bit and run the head of that queue.

7. Match each column of the SVR4 time-sharing dispatch table (ts_dptbl) to its meaning.

ts_quantum	**Length of a turn at this priority**
ts_tqexp	**New priority after using a whole quantum**
ts_slpret	**New priority on return from a sleep**
ts_maxwait	**Seconds a ready process may wait before the guard acts**
ts_lwait	**New priority for a process that waited too long**

Each row gives a quantum and three “next row” answers: one for a used-up quantum (lower), one for a sleep return (higher) and one, with ts_maxwait, that rescues a process at risk of starving.

8. In an example SVR4 dispatch table, row 23 has ts_quantum = 60 ms, ts_tqexp = 13, ts_slpret = 33 and ts_lwait = 43. A process at priority 23 blocks for disk I/O before its quantum ends, and later returns from that sleep. What is its new priority?

Answer: 33

Returning from a sleep uses the ts_slpret column, which is 33 in row 23. ts_tqexp (13) applies only after a whole quantum, and ts_lwait (43) only after waiting longer than ts_maxwait.

9. In SVR4, does each process below have a fixed priority, or one that changes with its behavior?

Fixed priority	Changes with behavior
A real-time audio mixer at priority 120 The kernel’s page-out daemon A process placed in the real-time class with priocntl	A compiler in the time-sharing class A text editor at time-sharing priority 40

Real-time and kernel-class priorities are fixed. Only time-sharing priorities move: down after a whole quantum, up after a sleep or a long wait.

10. Which statements about SVR4 time-sharing quanta are true?

A. In the standard table, priority 0 gets a longer quantum than priority 59.
B. Every row of the table gives the same quantum.
C. A processor-bound process tends to sink, and its quanta grow as it does.
D. An I/O-bound process tends to rise toward the rows with the shortest quanta.
E. Real-time processes take their quanta from the time-sharing table.

Answer: A, C, D

The standard table runs from 100 ms at priority 0 to 10 ms at 59. Using whole quanta pushes a busy process down to longer quanta; sleeping lifts an I/O-bound process to short quanta it rarely uses up. Real-time quanta are set separately and stay fixed.

11. Why does SVR4 rank kernel-mode work above ordinary time-sharing processes?

A. Such work often holds kernel buffers or locks that others need.
B. Kernel code is always shorter than user code, so it goes first.
C. The hardware blocks all interrupts until kernel work is finished.
D. Kernel-band processes carry deadlines, just as real-time ones do.

Answer: A. Such work often holds kernel buffers or locks that others need.
A process working inside the kernel may hold buffers, locks or other shared kernel resources. Letting it finish quickly releases them for everyone else.

12. The SVR4 kernel can be preempted at any instruction, which is how it bounds the wait of a real-time process.
Answer: False
SVR4 preempts the kernel only at preemption points, so the wait is bounded by the longest stretch between points. Kernels that can be preempted almost anywhere came later (for example Solaris 2, and Linux 2.6 as a build option).

10.5 UNIX FreeBSD Scheduling

You should be able to

- Name FreeBSD's five priority classes in order and explain why a lower priority number means a more urgent thread.
- Compute a thread's interactivity score from its sleep and run times and decide whether ULE treats it as interactive.
- Trace how the current, next and idle run queues give interactive threads quick service without starving batch threads.
- Explain processor affinity, topology awareness, and pull versus push migration on a multicore machine.

FreeBSD scheduling: the big picture

FreeBSD is an open-source descendant of Berkeley UNIX. Its scheduler has two goals that pull against each other: **scale** (keep every core of a multicore machine busy without fighting over shared data or wasting warm caches) and **respond** (serve a person at the keyboard at once, even while heavy jobs run). The **ULE scheduler** was built for both. It shipped as an option in FreeBSD 5.1 (2003) and became the default in 7.1 (2009). It replaced the **4BSD scheduler**, which used one global run queue for all processors and decay-usage priorities like traditional UNIX; 4BSD can still be selected when the kernel is built.

Priority classes

Every thread has a priority number from 0 to 255, and a **lower number is more urgent** (the same direction as traditional UNIX; SVR4 and Windows count the other way). The range is split into five classes. The classic layout (FreeBSD 5 to 8, the one usually taught):

Class	Range	Who lives there	How the priority is set
Bottom-half kernel (interrupt threads)	0–63	Kernel threads finishing hardware interrupt work (network, disk, clock)	Fixed per device type
Top-half kernel	64–127	Kernel threads doing top-half work (such as the page daemon) and, in the classic design, threads asleep in the kernel	Fixed per kernel thread, or chosen from the reason for sleeping
Real-time user	128–159	Threads marked real-time (rtprio, POSIX SCHED_FIFO / SCHED_RR)	Fixed, 32 levels
Time-sharing user	160–223	Ordinary threads: shells, editors, compilers	Recomputed from interactivity score, recent CPU use and nice value
Idle user	224–255	Spare-time work started with idprio	Fixed, 32 levels

In the classic design a thread could change band: a time-sharing thread asleep in the kernel (say, waiting for a disk read) took a top-half kernel priority, so it ran soon after the event and released kernel resources quickly; back in user mode it returned to its time-sharing priority. Since FreeBSD 8, ULE by default lifts such a sleeper only within the time-sharing band.

Later releases reordered the bands. FreeBSD 9 moved real-time user threads *above* top-half kernel threads (9 to 13: interrupt 0–47, real-time 48–79, kernel 80–119, time-sharing 120–223, idle 224–255). FreeBSD 14 and 15 keep that order and shrink the interrupt and kernel bands further, so time-sharing starts at 88 (14) or 56 (15). The 0–255 range, the lower-is-better rule and the idle band 224–255 never changed.

Per-processor run queues, affinity and topology

A single shared run queue hurts on many cores in two ways: every core must lock it to pick a thread (lock contention, the load-sharing bottleneck), and any core may pick any thread, so threads often restart on a core whose caches hold none of their data. ULE gives **each processor its own run queues**. A waking thread is normally queued on the processor that ran it last (**processor affinity**), because that cache may still hold its data. ULE is also **topology-aware**: it knows which cores are hardware-thread siblings, which share an L2 cache and which share only the L3.

Placing a waking thread (simplified): if it ran on its last core recently and that core is free, go home; else try a free core sharing a cache with it; else take the least loaded core. If it ran long ago its cache is cold, so ULE just takes the least loaded core.

The interactivity score

ULE watches each thread over a recent window (about the last 5 seconds) and adds up its **voluntary sleep time s** (blocked waiting for a keystroke, a disk read, a timer…; time spent waiting in a run queue does not count) and its **run time r**. With scaling factor **m = 50** (half the maximum score of 100):

- s > r (mostly sleeps): score = m / (s / r), between 0 and 50
- r > s (mostly runs): score = m + (m − m / (r / s)) = 100 − 50·s/r, between 50 and 100

- s = r: score = m = 50

A thread whose score is below the **interactivity threshold** (30 by default, adjustable) is **interactive**; otherwise it is a **batch** thread.

Worked examples. s = 90, r = 10: 50 / 9 ≈ 5.6, interactive. s = r: 50, batch. s = 10, r = 90: 50 + (50 – 50/9) ≈ 94.4, batch. A hog that runs 9 ms and naps 1 ms also scores about 94.4. **The threshold:** 50 / (s/r) = 30 gives s/r = 5/3, so the thread sleeps 5/8 = **62.5%** of the time; it must sleep more than that to count as interactive.

If you have seen another form. A simplified form sometimes quoted for the r > s line is m / (r / s) + m. It also stays between 50 and 100, so the verdict (batch) never changes, but it runs backwards: it would rate a pure CPU hog (s = 0) at only 50, and s = 10, r = 90 at 55.6 instead of 94.4. ULE's code subtracts, as above, so more running always means a higher score.

History keeps hogs honest. When the two totals pass the window, both are scaled down (by 4/5), so old behaviour fades gradually. In the step's model, an editor that starts compiling at 5 s crosses into batch at about 6.9 s, while a compiler that turns into an editor at 5 s only becomes interactive at about 9.9 s. Short naps cannot disguise a CPU hog, and a real change of behaviour is noticed within seconds.

Current, next and idle queues

In ULE's original design each processor keeps three queues for time-sharing and idle work:

1. The processor always runs the best (lowest-numbered) thread in the **current queue**.
2. Interactive threads (and real-time and kernel ones) go into current when they wake.
3. A batch thread whose slice ends goes to the **next queue**.
4. When current is empty, current and next **swap**. If both are empty, the **idle queue** runs.

An interactive thread therefore never waits for the batch threads parked in next, while each batch thread runs exactly once per swap, so none starves. A thread that stops sleeping loses its low score, so interactive threads cannot hog current. An idle-class thread gives up the processor the moment any other thread is ready.

In the step's trace (one processor, three batch threads, an editor woken every 20 ms, 10 ms slices, and a running time-sharing thread always finishing its slice) the editor waited 4.2 ms on average (worst 9 ms) when scored interactive, but 15.8 ms on average (worst 31 ms) when treated as batch. Real ULE does even better for interactive threads: when a better thread joins current, the running thread is made to step aside almost at once (at its next return to user mode), while a thread that joins next gets no such help. In recent FreeBSD a slice is about 94 ms, shrinking toward 16 ms as more threads wait. Since FreeBSD 7 the current/next pair has been replaced by one circular batch queue whose starting point rotates (interactive threads use a separate queue), with the same no-starvation guarantee.

Load balancing: pull and push

Per-processor queues drift out of balance: threads finish at different times, and waking threads return to their old core. ULE corrects this by **thread migration**, in two ways:

- **Pull migration (work stealing):** a processor about to go idle takes a waiting thread from a busier one (a running thread plus at least one waiting), searching its cache neighbours first. It reacts at once.

- **Push migration:** a periodic balancer pairs the most loaded processor with the least loaded one and moves a waiting thread. It catches imbalances among cores that are all busy. FreeBSD runs it about once a second, at a randomized interval so the cores do not all rebalance in lockstep (you may see “twice a second” quoted; the code has used an average of about one second since its early versions).

Migration is not free: the moved thread must refill caches on its new core (less if the two cores share a cache), so ULE moves threads only for real gaps and prefers near neighbours. In the step’s model (one core with eight short threads, three cores each busy with one long thread), no balancing finished at tick 20 with average turnaround 13.6 ticks; pull alone finished at 16 (12.4); push alone at 18 (12.4); pull and push together at 16 with the best average, 11.9.

Key terms

ULE scheduler
FreeBSD’s default thread scheduler since version 7.1 (2009). It keeps separate run queues for each processor, balances load between them, and spots interactive threads with an interactivity score.

4BSD scheduler
The older FreeBSD scheduler inherited from Berkeley UNIX: one global run queue shared by every processor, with priorities computed from decaying CPU usage. ULE replaced it as the default, but it can still be chosen when the kernel is built.

Bottom-half kernel (interrupt threads)
Kernel work started by hardware, not by a thread’s request: the handling of device interrupts. FreeBSD runs it in interrupt threads that hold the best priority numbers of all.

Top-half kernel
Kernel code that runs on behalf of a thread that entered the kernel itself, through a system call or a trap such as a page fault. Its priority band holds kernel threads and, in classic FreeBSD, any thread asleep inside the kernel.

Real-time user thread
A user thread given a fixed priority (with the rtprio command or the POSIX SCHED_FIFO and SCHED_RR policies). The scheduler never changes that priority.

Time-sharing thread
An ordinary user thread. The scheduler keeps recalculating its priority from its interactivity score, its recent CPU use and its nice value.

Idle-class thread
A user thread that may run only when no other thread wants the processor. In FreeBSD it is started with the idprio command.

Run queue
The scheduler’s list of threads that are ready to run, kept in priority order. Under ULE every processor has its own run queues.

Current queue
In the classic ULE design, the time-sharing run queue a processor is serving right now. Interactive (and real-time and kernel) threads are always placed here.

Next queue

In the classic ULE design, the run queue where batch threads wait for the next round. When the current queue becomes empty, the two queues swap roles.

Interactivity score
A number from 0 to 100 that ULE computes from a thread's recent voluntary sleep time and run time. Low means mostly sleeping (interactive); high means mostly computing (batch).

Interactivity threshold
The score below which ULE treats a thread as interactive: 30 by default, and adjustable by the administrator.

Voluntary sleep time
Time a thread spends blocked because it chose to wait for something, such as a keystroke or a disk read. Time spent waiting in a run queue does not count.

Batch thread
A time-sharing thread whose interactivity score is at or above the threshold. It mostly computes, so a short extra wait goes unnoticed.

Processor affinity
The preference for running a thread on the processor that ran it last, or on one that shares a cache with it, because that cache may still hold the thread's data.

Thread migration
Moving a ready thread from one processor's run queue to another's. It evens out the load, but the thread must refill the new processor's caches.

Pull migration (work stealing)
Load balancing started by a processor that is about to go idle: it takes a waiting thread from a busier processor.

Push migration
Load balancing started by a periodic balancer, which moves waiting threads from the most loaded processor to the least loaded one.

Topology-aware scheduling
Scheduling that knows which cores share caches (and which are hardware-thread siblings on one core) and prefers to keep or move threads among close neighbours.

Review questions with answers

1. In the classic FreeBSD priority layout, which range holds time-sharing user threads?

A. 128–159
B. 64–127
C. 224–255
D. 160–223

Answer: D. 160–223

Classic layout: interrupt threads 0–63, top-half kernel 64–127, real-time user 128–159, time-sharing user 160–223, idle user 224–255. Lower numbers are more urgent.

2. Put FreeBSD's priority classes in order from most urgent (lowest numbers) to least urgent.

Correct order:

1. Bottom-half kernel (interrupt threads)
2. Top-half kernel
3. Real-time user threads
4. Time-sharing user threads
5. Idle user threads

Hardware work comes first, then kernel work done for threads, then real-time, then ordinary time-sharing threads, and idle work last. (Since FreeBSD 9, real-time user threads sit above the top-half kernel band; the classic order is the one shown here.)

3. Over ULE's recent window a thread slept voluntarily for 90 ms and ran for 10 ms. With m = 50, what is its interactivity score?

Answer: 5.56

Sleep exceeds run, so score = m / (s / r) = 50 / (90 / 10) = 50 / 9 ≈ 5.6. That is far below 30, so the thread is interactive.

4. A thread slept voluntarily for 10 ms and ran for 90 ms. Using ULE's interactivity formula with m = 50, what is its score?

Answer: 94.44

Run exceeds sleep, so score = m + (m − m / (r / s)) = 50 + (50 − 50 / 9) ≈ 94.4. That is a batch thread. (The simplified form m / (r / s) + m would give 55.6, also batch, but it runs backwards.)

5. ULE's default interactivity threshold is 30 (m = 50). What percentage of its time must a thread spend sleeping for its score to equal exactly 30?

Answer: 62.5 %

50 / (s/r) = 30 gives s/r = 5/3, so sleep is 5 parts out of 8: 5/8 = 62.5%. Sleeping more than that makes the thread interactive.

6. In the classic ULE design, which run queue does each thread join?

Current queue	Next queue	Idle queue
A text editor waking up after a keystroke A real-time audio thread that just became ready	A compiler thread whose time slice just ran out A video encoder with an interactivity score of 80	A file indexer started with idprio to use spare time

Interactive, real-time and kernel threads go to current for quick service; batch threads go to next after their slice; idle-class threads wait in the idle queue and run only when current and next are both empty.

7. What does a ULE processor do when its current queue becomes empty but its next queue is not?

A. It swaps the current and next queues and carries on
B. It runs a thread from the idle queue straight away
C. It takes over the current queue of another processor
D. It raises every batch thread to interactive status

Answer: A. It swaps the current and next queues and carries on

The swap turns the waiting batch threads into the new current queue, so each batch thread gets one turn per round and none can starve.

8. Match each multiprocessor idea in ULE with what it means.

Pull migration	**A core with nothing to run takes a waiting thread from a busier core**
Push migration	**A periodic balancer moves threads from the busiest core to the least busy one**
Processor affinity	**Preferring the core whose cache still holds the thread's data**
Topology awareness	**Knowing which cores share caches and trying close neighbours first**

Pull reacts at once when a core would go idle; push fixes slower drifts; affinity and topology keep moves rare and short so caches stay useful.

9. Which are reasons ULE keeps separate run queues for each processor instead of one shared queue?

A. Cores no longer wait for one lock on a shared queue
B. A thread can return to the core whose cache holds its data
C. The cores can never become unevenly loaded
D. Interactive threads no longer need a score

Answer: A, B

Per-processor queues remove lock contention and keep caches warm. They can drift out of balance, which is why ULE needs pull and push migration, and the interactivity score is still needed to order threads within each processor.

10. A CPU-bound thread sleeps for 1 ms after every 9 ms it runs, hoping to look interactive. What does ULE decide?

A. It is interactive, because it sleeps regularly
B. It is real-time, because its pattern is periodic
C. It is batch, because its score is about 94
D. It is interactive for one slice, then batch

Answer: C. It is batch, because its score is about 94

r/s = 9, so score = 50 + (50 − 50/9) ≈ 94.4, far above 30. Because the window remembers recent history, short naps cannot disguise a CPU hog.

11. Which statement about FreeBSD's ULE scheduler is accurate?

A. It uses one global run queue shared by all processors
B. It became the default in FreeBSD 7.1, replacing 4BSD
C. It was designed for single-processor machines only
D. It treats a higher priority number as more urgent

Answer: B. It became the default in FreeBSD 7.1, replacing 4BSD

ULE appeared as an option in FreeBSD 5.1 (2003) and became the default in 7.1 (2009), replacing the single-queue 4BSD scheduler.

12. When ULE computes a thread's interactivity score, the time the thread spends waiting in a run queue for its turn counts as sleep time.

Answer: False

Only voluntary sleep counts: time blocked waiting for something such as a keystroke or a disk read. Waiting in a run queue is not sleeping; if it counted, a CPU hog stuck behind other threads would look interactive.

10.6 Windows Scheduling

You should be able to

- Explain how Windows uses 32 priority levels split into a real-time band and a variable band.
- Compute a thread's base priority from its process priority class and its relative thread priority.
- Trace how a priority boost raises a thread's current priority and how it decays back to the base.
- Describe how the balance set manager rescues starving threads and how quanta differ between client and server Windows.
- Predict where a ready thread runs on a multiprocessor, given its priority, hard affinity and ideal processor.

Windows scheduling

The Windows scheduler is built for **responsiveness**: the person using the machine should never feel it hesitate. It is **priority-driven and preemptive**: the dispatcher always runs the highest-priority ready thread, and a thread that becomes ready with a higher priority than a running one takes the processor at once, without waiting for the running thread's quantum to end. The unit of scheduling is the **thread**, not the process; a process is only a container, and its threads can sit at different priorities.

32 priority levels in two bands

Levels	Band	Behaviour
16–31	Real-time	Fixed priority, never boosted or lowered by the system; equal threads take turns. No deadline guarantee.
1–15	Variable	The current priority moves (boosts, decay) but never above 15.
0	Reserved	The zero-page thread, which zeroes free pages in spare time. (An idle processor runs its idle thread, outside the 32 queues.)

There is one ready queue per level and a 32-bit **ready summary** with a bit set for every non-empty queue. The dispatcher rules:

1. Run the first thread of the highest non-empty level (the highest set bit).
2. Threads at the same level take turns: a thread whose quantum ends goes to the **back** of its queue.
3. A higher-priority thread that becomes ready preempts the running thread immediately; the preempted thread goes to the **front** of its queue, since it had not used up its turn.

Base priority: priority class + relative thread priority

Each process has a **priority class** that sets a starting value: Idle 4, Below normal 6, Normal 8, Above normal 10, High 13, Realtime 24. Each thread has a **relative thread priority**: Lowest −2, Below normal −1, Normal 0, Above normal +1, Highest +2, plus two special cases. **Idle** drops the thread to the bottom of its band (1, or 16 in the Realtime class) and **Time critical** lifts it to the top (15, or 31 in the Realtime class). The result is the thread's **base priority**. Example: High (13) + Above normal (+1) = 14. Only privileged programs may use the Realtime class.

Class	Idle	Lowest	Below normal	Normal	Above normal	Highest	Time critical
Idle (4)	1	2	3	4	5	6	15
Below normal (6)	1	4	5	6	7	8	15
Normal (8)	1	6	7	8	9	10	15
Above normal (10)	1	8	9	10	11	12	15
High (13)	1	11	12	13	14	15	15
Realtime (24)	16	22	23	24	25	26	31

The bands are sealed: no combination lifts a non-real-time thread above 15, and no Realtime thread falls below 16.

Current priority: boosts and decay

A variable-level thread has a **base priority** (where it starts) and a **current priority** (what the scheduler uses now). When it stops waiting it receives a **priority boost**: its current priority becomes **base + increment**, never above 15. The increment is added to the base, not stacked on an earlier boost. Classic increments, chosen by the device drivers and the system:

- I/O completion: disk +1, network +2, keyboard or mouse +6, sound card +8.
- Wait on an event or semaphore ends: +1.
- A thread of the foreground process ending a wait: a small boost.
- A GUI thread woken by a window message: +2.

Priority decay: each time a boosted thread uses a full quantum, its current priority drops by one, until it is back at its base; it never goes lower. Threads at real-time levels are never boosted and never decay.

Worked example. An editor thread with base 8 waits for a key. The key press gives 8 + 6 = 14, so it preempts a CPU-bound thread at 8. After three full quanta it is at 14 → 13 → 12 → **11**; once it reaches 8 it simply takes turns with the other thread. A sound-card wake-up would give 8 + 8 = 16, which is capped at **15**.

Starvation rescue: the balance set manager

Strict priority can starve a low-priority ready thread behind a busy higher one. The **balance set manager**, a system thread, wakes about once a second and looks for ready threads that have waited about **4 seconds** or more. Each one is raised to priority **15** for one quantum: classically a doubled one (Windows 2000/XP); later versions give a short fixed amount instead. When that quantum ends the thread drops **straight back to its base**, with no step-by-step decay.

Worked example. L (priority 4) becomes ready at 0.3 s behind a CPU-bound thread at 10. The scans at 1–4 s find it has waited only 0.7–3.7 s. The scan at 5 s finds 4.7 s, so L runs at 15 for the classic doubled client quantum of 62.5 ms. The next rescue comes at 10 s. L ends up with about 1.1% of the processor: enough to make progress (for example, to release a lock a high-priority thread needs, ending a priority inversion), not a fair share. The rescue cannot help against a thread at a real-time level: 15 still loses to 16 or more.

Quantum length

Windows measures a quantum in **clock intervals**, the time between two clock interrupts (about 15.6 ms, precisely 15.625 ms, on most modern PCs; 10 ms on some older single-processor machines). Typical defaults, both configurable:

Edition	Quantum	At 15.625 ms
Client	2 clock intervals	31.25 ms
Client, foreground stretched 3×	6 clock intervals	93.75 ms
Server	12 clock intervals, no stretching	187.5 ms

Quantum stretching gives the threads of the **foreground process** (the one that owns the window in use) longer turns on client Windows. It changes how much processor time they get, not their priority. Example: one foreground and two background CPU-bound threads at the same level, taking turns, give the foreground thread 93.75 / (93.75 + 2 × 31.25) = **60%** of the processor instead of 33.3%. Short quanta keep a desktop responsive; long quanta let a server finish a request in one turn with fewer context switches. The “Adjust for best performance of: Programs / Background services” setting switches between the two styles.

Multiprocessor scheduling

- Any processor may run any thread unless the thread’s **hard affinity** (affinity mask, the set of processors it may use) forbids it. Hard affinity is a rule.
- Each thread also has an **ideal processor** (soft affinity): the one Windows prefers, so the thread finds its data still in that processor’s cache. It is only a preference.
- Windows tries to keep the highest-priority ready threads running across all processors. A newly ready thread takes an idle processor it may use (its ideal one if possible); if none is idle it may preempt a lower-priority thread on an allowed processor. (The lab uses the lowest-priority allowed one; real Windows checks only the thread on the newcomer’s ideal processor, so this is a goal, not a guarantee.)
- A freed processor runs the highest-priority waiting thread allowed on it.
- Price of hard affinity: a thread allowed only on one busy processor waits, even while lower-priority threads run elsewhere or another processor is idle.

- Machines with more than 64 logical processors divide them into processor groups.

Key terms

Priority class
A label on a Windows process (Idle, Below normal, Normal, Above normal, High or Realtime) that sets the starting priority for all of its threads.

Relative thread priority
A per-thread setting (Idle, Lowest, Below normal, Normal, Above normal, Highest, Time critical) that moves a thread up or down from its process's class priority.

Base priority
In Windows, the priority a thread starts from, fixed by its process's priority class plus its relative thread priority. A variable-level thread never drops below it.

Current priority
The priority the Windows scheduler actually uses right now. For a variable-level thread it can sit above the base priority after a boost and then slides back down.

Real-time levels
Windows priority levels 16 to 31. Threads here keep a fixed priority: the system never boosts or lowers them, and equal-priority threads take turns.

Variable levels
Windows priority levels 1 to 15, used by ordinary threads. The system raises and lowers a thread's current priority here, but never above 15.

Zero-page thread
A Windows system thread that runs at priority 0, the lowest level, and fills free memory pages with zeros in spare moments so they are ready to hand out.

Ready summary
A 32-bit mask with one bit per priority level; a bit is 1 when that level's ready queue is not empty, so the dispatcher finds the highest ready level in one quick scan.

Priority boost
A temporary rise in a variable-level thread's current priority, given when it stops waiting (for example when its I/O completes), so it gets the processor quickly.

Priority decay
The step-by-step return of a boosted thread to its base priority: one level lower after each full quantum it uses.

Balance set manager
A Windows system thread that wakes about once a second. One of its jobs is to find ready threads that have waited about 4 seconds and give them a one-time boost to priority 15.

Clock interval
The time between two clock interrupts, about 15.6 ms on most modern PCs. Windows measures a thread's quantum in clock intervals.

Foreground process

The process that owns the window the user is working in right now.

Quantum stretching
Giving the threads of the foreground process longer quanta (up to three times as long) on client Windows, without raising their priority.

Hard affinity (affinity mask)
The set of processors a thread is allowed to run on. The scheduler never places the thread on a processor outside this set.

Ideal processor (soft affinity)
The processor Windows prefers for a thread, so it keeps finding its data in that processor's cache. It is a preference, not a rule.

Review questions with answers

1. The Windows scheduler chooses among threads, not processes, so two threads of the same process can have different priorities.

Answer: True

Threads are the unit of scheduling. A process's class sets a starting point, and each thread's relative priority moves it up or down, so threads of one process can sit at different levels.

2. Which statement about the Windows real-time priority levels (16–31) is true?

A. Their priority is fixed: the system never boosts or lowers it
B. Windows guarantees that their threads always meet deadlines
C. Any program reaches them by putting its process in the High class
D. A large enough I/O boost can lift a Normal thread into this band

Answer: A. Their priority is fixed: the system never boosts or lowers it

Real-time levels are fixed-priority levels: no boosts, no decay, and equal threads take turns. "Real-time" describes the band, not a timing guarantee.

3. A process in the High priority class creates a thread whose relative priority is Above normal. What is the thread's base priority?

A. 13
B. 15
C. 14
D. 24

Answer: C. 14

Base priority = class value + offset: High is 13 and Above normal adds +1, giving 14.

4. Sort each combination of process class and relative thread priority by the band its base priority falls in.

Real-time band (16–31)	Variable band (1–15)
Realtime class, Idle thread Realtime class, Lowest thread	High class, Time critical thread Normal class, Highest thread Idle class, Time critical thread High class, Highest thread

Only the Realtime class reaches 16–31 (Idle gives 16, Lowest gives 22). Every other class stays at or below 15: Time critical gives exactly 15, and High + Highest is 13 + 2 = 15.

5. Match each completed I/O with the classic priority increment its driver gives the waiting thread.

Disk	**+1**
Network	**+2**
Keyboard or mouse	**+6**
Sound card	**+8**

The bigger the increment, the more a human is waiting on the result: input devices and sound get the largest boosts, a disk read the smallest. Drivers choose these values.

6. A thread with base priority 8 has been waiting for keyboard input. The key press arrives (+6), and the thread then uses three full quanta in a row without waiting again. What is its current priority now?

Answer: 11

The boost gives 8 + 6 = 14. Each full quantum it uses lowers it by one: 14 → 13 → 12 → 11. It would keep falling until it reached its base of 8.

7. A thread with base priority 13 that receives a +6 keyboard boost runs at priority 19.

Answer: False

Boosts never lift a thread above 15, the top of the variable band, so it runs at 15. A variable-level thread can never reach the real-time levels.

8. Put the life of a boosted editor thread in order.

Correct order:

1. The thread blocks, waiting for a key press
2. The key press completes and its current priority becomes base + 6
3. It preempts a lower-priority thread that is running
4. After each full quantum it uses, its current priority drops by one
5. Its current priority is back at its base and stays there

Wait, boost on wake-up, preempt, then decay one level per full quantum until the base priority is reached; decay never goes below the base.

9. A ready thread at priority 4 has not run for 4 seconds, because a CPU-bound thread at priority 10 never blocks. What does the balance set manager do about it?

A. It raises the waiting thread one level per second until it passes 10
B. It lowers the busy thread to priority 4 so that the two take turns
C. It moves the waiting thread up to 16 until all of its work is done
D. It raises the waiting thread to 15 for one quantum, then back to 4

Answer: D. It raises the waiting thread to 15 for one quantum, then back to 4

About once a second the balance set manager looks for threads that have been ready for about 4 s, lifts them to 15 for one quantum (classically a doubled one; later versions use a short fixed amount), and lets them drop straight back to their base afterwards.

10. A client edition of Windows gives threads a quantum of 2 clock intervals, and the clock interval is 15.625 ms. The foreground process's threads get quanta stretched to 3 times that. How many milliseconds is one foreground quantum?

Answer: 93.75 ms

2 × 15.625 ms = 31.25 ms for a normal quantum; 3 × 31.25 ms = 93.75 ms for a stretched foreground quantum.

11. Which statements about Windows quanta are true?

A. Server editions use longer quanta than client editions by default
B. Quantum stretching raises the priority of the foreground process's threads
C. A quantum is measured in clock intervals
D. A stretched foreground thread can run ahead of a higher-priority background thread
E. Among CPU-bound threads, longer quanta mean fewer context switches per second

Answer: A, C, E

Servers default to 12 clock intervals, clients to 2. Stretching only lengthens the foreground threads' turns; it never changes their priority, so a higher-priority thread still runs first. Among CPU-bound threads, which use their whole turn, longer turns mean fewer switches.

12. Thread X (priority 12) has a hard affinity that allows only CPU 2. It becomes ready while CPU 2 runs a priority-14 thread and CPU 0 is idle. What happens?

A. X runs on CPU 0, because an idle processor always beats affinity
B. X waits in the ready list until CPU 2 can take it
C. X preempts the priority-14 thread, since CPU 2 is its only processor
D. X runs on CPU 0 for now and moves back to CPU 2 once it is free

Answer: B. X waits in the ready list until CPU 2 can take it

A thread runs only on processors inside its affinity mask. The only allowed processor runs a higher-priority thread, so X waits, even though another processor sits idle. That is the price of hard affinity.

Chapter 11 · I/O Management and Disk Scheduling

Devices are slow, varied and everywhere. The OS hides their differences and keeps the disk arm from wasting time.

Input and output is where the computer meets the world, and it is the slowest and most varied part of any system. This chapter shows how I/O is organized, why buffers smooth out speed differences, how the order of disk requests changes performance, how RAID spreads data over many disks, how a disk cache avoids trips to the disk, and how UNIX, Linux and Windows build their I/O systems.

Learning objectives

- Summarize the kinds of I/O devices and the three ways to perform I/O.
- Explain the layered organization of the I/O function and the purpose of I/O buffering.
- Compute disk access times and compare disk scheduling policies.
- Describe the RAID levels and the design of a disk cache.
- Compare the I/O facilities of UNIX SVR4, Linux and Windows.

How Chapter 11 fits together

I/O is the slowest and most varied part of a computer. The overview follows one read() of a 4 KB block down the layers to the disk and back up by DMA and an interrupt. On the example disk (average seek 4 ms, 7,500 rpm, 500 sectors of 512 bytes per track) the device time is 4 + 4 + 0.128 = 8.128 ms, while all the software together takes about 0.1% of the read. An SSD needs about 0.1 ms, and a disk-cache hit finishes in about 2 µs, roughly 3,500 times faster than the hard disk.

Devices and organization (11.1 to 11.3)

Devices serve people, machines or other computers, and differ in data rate, application, complexity of control, unit of transfer, data representation and error conditions. The OS performs I/O by programmed I/O, interrupt-driven I/O or DMA, which moves a whole block and interrupts once at the end. It aims at efficiency and generality, and organizes I/O in layers; for a file: directory management, file system, physical organization, device I/O, and scheduling and control.

Buffering (11.4)

System buffers let input be read ahead and output be written behind. Single, double and circular buffering let the device and the process overlap and smooth out bursts, but no buffer makes a device faster than its average rate.

Disk scheduling (11.5)

A disk operation costs seek time plus rotational delay (together the access time) plus transfer time. Seeks dominate, so the order of waiting requests matters: FIFO is fair but slow, SSTF cuts seeks but can starve far requests, SCAN, C-SCAN, LOOK and C-LOOK sweep the disk, and N-step-SCAN and FSCAN stop one busy process from holding the arm.

RAID (11.6)

RAID makes several disks act as one logical drive. RAID 0 stripes with no redundancy, RAID 1 mirrors, RAID 2 and 3 access all disks in parallel, and RAID 4, 5 and 6 use independent access with parity: on one disk (4), rotated across all disks (5) or doubled (6). RAID survives failed disks, not bad writes, so it is no backup.

Disk cache (11.7)

A disk cache keeps recently used blocks in main memory so a repeat read never reaches the device. LRU, LFU and frequency-based replacement choose which block to drop.

Real systems (11.8 to 11.10)

UNIX SVR4 treats each device as a special file and moves data through the buffer cache, character queues or unbuffered DMA. Linux moved from single-queue I/O schedulers (deadline, anticipatory, CFQ) to the multi-queue block layer (mq-deadline, BFQ, Kyber) and writes dirty page-cache pages back later. Windows sends every request through its I/O manager as an I/O request packet, writes cached data lazily, offers asynchronous I/O and completion ports, and adds software RAID, shadow copies and BitLocker.

Why it matters next

Chapter 12 builds file systems on exactly these pieces: blocks on a disk, buffers, the cache, and the layers between a file name and a sector.

11.1 I/O Devices

You should be able to

- Sort external devices into human-readable, machine-readable and communication devices.
- Read a logarithmic data-rate chart and explain why device speeds span about nine orders of magnitude.
- Explain the six ways devices differ (data rate, application, complexity of control, unit of transfer, data representation, error conditions) with an example of each.
- Tell a block-oriented device from a stream-oriented one.

I/O devices: where the computer meets the world

Input/output (I/O) is every exchange of data between the computer and something outside it: a key press, a saved file, a network message. The devices involved are wildly different, and the operating system must drive all of them, hide their quirks from programs, and keep the processor busy while they work.

Three categories of external device

The classic grouping asks one question: **what is at the other end of the device?** Not the plug, not the speed.

Category	Communicates with	Examples
Human readable	the user	keyboard, mouse, display, printer, touchscreen, game controller
Machine readable	electronic equipment	disk drives, USB flash drives, SSDs, sensors, actuators, controllers
Communication	remote devices or computers	network interface cards, Wi-Fi adapters, modems, cellular radios

A **sensor** measures something physical (temperature, light, motion) and reports it as data; an **actuator** turns a command into a physical action (a motor turns, a valve opens). A **modem** (modulator-demodulator) turns digital data into signals for a telephone or cable line and back. Two traps: a printer is *human readable* even though a machine prints, because its pages are for people; and a USB keyboard and a USB flash drive share a port but sit in different categories.

Difference 1: data rate

The **data rate** is how many bits per second a device can move (bit/s, kbit/s = 10^3, Mbit/s = 10^6, Gbit/s = 10^9). Typical values, as orders of magnitude only:

Device	Order of magnitude (bit/s)	Typical value
Keyboard	10^2	80 bit/s
Mouse	10^3–10^4	4 kbit/s
Dial-up modem	10^4–10^5	56 kbit/s
Laser printer	10^6–10^7	3 Mbit/s
Image scanner	10^7	28 Mbit/s
Ethernet (10/100 Mbit/s)	10^7–10^8	100 Mbit/s
USB flash drive	10^8–10^9	800 Mbit/s
Hard disk drive	10^9	1.6 Gbit/s
Gigabit Ethernet	10^9	1 Gbit/s
Graphics display	10^9–10^{10}	3 Gbit/s
Solid-state drive (SSD)	10^{10}–10^{11}	28 Gbit/s
10/100 Gigabit Ethernet	10^{10}–10^{11}	10 Gbit/s

From a keyboard (about 80 bit/s) to a 100 Gbit/s link is a factor of about 1.3×10^9: roughly **nine orders of magnitude** (an order of magnitude is a factor of ten). On an ordinary linear chart a few hundred pixels wide, one pixel stands for a few times 10^8 bit/s, so every device up to 100 Mbit/s Ethernet (half of the twelve) would be thinner than one pixel; that is why device speeds are drawn on a **logarithmic scale**, where each equal step multiplies the value by 10.

Worked example 1. Keyboard ≈ 10^2 bit/s, hard disk ≈ 10^9 bit/s. One second of disk transfer equals $10^9 \div 10^2 = 10^7$ seconds of typing, about 116 days. With the typical values above (80 bit/s and 1.6 Gbit/s) it is 2×10^7 keyboard-seconds, about 231 days.

Worked example 2. A 1920 × 1080 display with 24-bit colour redrawn 60 times a second needs 1920 × 1080 × 24 × 60 = 2,985,984,000 bit/s ≈ 3.0 Gbit/s. A 4K screen needs about four times as much.

Why it matters: a 3 GHz processor runs about 3×10^8 clock cycles between two keystrokes, so one interrupt per key is cheap. A 10^{10} bit/s device delivers over a billion bytes a second, far too many for one interrupt each, so fast devices move whole blocks by DMA (section 11.2).

The six ways devices differ

Each difference is a reason the OS cannot treat all devices the same way.

1. **Data rate**: bits per second, spanning about nine orders of magnitude (above). Keyboard ≈ 10^2 vs hard disk ≈ 10^9; laser printer ≈ 10^6–10^7 vs network card 10^9–10^{11}.
2. **Application**: what the device is used for changes the software and policies it needs. A disk holding *files* needs file-management software (directories, space allocation, protection); a disk holding *paging space* needs virtual-memory hardware and software. A terminal used by an administrator may get a higher priority than an ordinary user's. A shared printer needs a **spooler**, a program that queues whole jobs on disk and feeds them to the printer one at a time; a network card needs protocol software that delivers each packet to the right process.
3. **Complexity of control**: how much work it takes to command the device and check its status. A keyboard or printer needs only a data register and a few status bits; a disk needs block numbers, directions, memory addresses, request queues, scheduling and retries; a network card needs circular lists of buffers, DMA and interrupt settings. The device's I/O module (controller) hides the lowest-level detail, such as moving a disk arm.
4. **Unit of transfer**: the size of one piece of data moved. A keyboard or terminal sends single characters as a stream; a printer receives a stream of bytes; a disk moves whole blocks of 512 or 4,096 bytes; a network card moves packets (on standard Ethernet up to 1,500 data bytes plus a header).
5. **Data representation**: how information is encoded as bits: the character code (ASCII, Unicode), formats of numbers and images, and conventions such as parity. A keyboard sends key numbers that the driver maps to characters through the layout; a disk stores raw bytes plus error-correcting code (ECC) bits; a printer expects a page-description language; network headers send multi-byte numbers most significant byte first (network byte order).
6. **Error conditions**: what can go wrong, how it is reported, how serious it is, and what can be done. A missed keystroke is simply retyped; an unreadable disk sector is retried and repaired with ECC if possible, otherwise reported and data may be lost; a paper jam needs a person; a network packet whose **checksum** (a short number computed from the data and sent with it) no longer matches is dropped, and a **protocol** (an agreed set of rules for the exchange) such as TCP has the sender send it again.

Parity example. A **parity bit** makes the count of 1s even (even parity) or odd (odd parity), so one flipped bit is detected. The letter A in 7-bit ASCII is 1000001 (two 1s): even parity adds 0, odd parity adds 1. For 1011001 (four 1s) odd parity adds 1, giving five 1s. Both ends of a link must use the same convention.

Block-oriented vs stream-oriented devices (preview of 11.4)

- **Block-oriented**: data is stored in fixed-size blocks, moved one whole block per transfer, and any block can be reached directly by its block number. Examples: disks, SSDs, USB flash drives.

- **Stream-oriented**: data moves as a sequence of bytes with no block structure and no addresses; a byte the program did not keep cannot be asked for again. Examples: keyboards, terminals, printers, mice, communication ports, and most devices that are not secondary storage.

Worked example. To change one byte on a disk with 4,096-byte blocks, the OS reads the whole block into memory, changes the byte, and writes the whole block back: 2 × 4,096 = 8,192 bytes move to change one. With 512-byte blocks it is 1,024 bytes.

Key terms

Human-readable device
A device that exchanges information with the person using the computer, such as a keyboard, mouse, display or printer.

Machine-readable device
A device that exchanges information with other electronic equipment rather than with a person, such as a disk drive, USB flash drive, sensor or actuator.

Communication device
A device that exchanges information with remote devices or computers, such as a network interface card or a modem.

Sensor
A device that measures something in the physical world (temperature, light, motion) and reports it to the computer as data.

Actuator
A device that turns a command from the computer into a physical action, such as a motor turning, a valve opening or a relay switching.

Modem
Short for modulator-demodulator: a device that turns digital data into signals that can travel over a telephone or cable line, and turns them back again at the other end.

Data rate
How many bits per second a device can send or receive. Written in bit/s, kbit/s (thousands), Mbit/s (millions) or Gbit/s (billions).

Order of magnitude
A factor of ten. Two quantities differ by three orders of magnitude when one is about 10 × 10 × 10 = 1,000 times the other.

Logarithmic scale (log scale)
A chart axis on which each equal step multiplies the value by the same factor (usually 10), so tiny and huge values fit on one chart.

Unit of transfer
The size of the piece of data a device moves in one operation: a single byte or character for some devices, a whole block of hundreds or thousands of bytes for others.

Block-oriented device
A device that stores data in fixed-size blocks, moves one whole block at a time, and lets any block be reached by its block number. Disks and USB flash drives are examples.

Stream-oriented device
A device that moves data as a sequence of bytes with no block structure and no block numbers. Keyboards, terminals, printers, mice and communication ports are examples.

Character code
An agreed table that gives each character a number, so that text can be stored and sent as bits. ASCII and Unicode are the common ones.

Parity bit
An extra bit added to a group of bits so that the total count of 1s is even (even parity) or odd (odd parity). A single flipped bit then breaks the rule and is detected.

Error-correcting code (ECC)
Extra bits stored with data, computed from it, that let a device detect and repair small errors when the data is read back.

Packet
A chunk of data sent over a network as one unit, with a header that says where it comes from and where it is going.

Checksum
A short number computed from a block of data and sent along with it. The receiver recomputes it; if the two disagree, the data was damaged on the way.

Protocol
An agreed set of rules for exchanging messages over a network: their format, the order of steps, and what to do about errors. TCP is a protocol that has lost or damaged packets sent again.

Spooler
A program that collects whole jobs for a shared slow device, such as a printer, in a queue on disk and feeds them to the device one at a time, so programs never wait for it.

Review questions with answers

1. Sort each device by what sits at the other end of it.

Human readable	Machine readable	Communication
Tablet touchscreen Inkjet printer Joystick	External SSD Motor actuator in a robot arm Thermostat temperature sensor	Cable modem Ethernet network card

Human-readable devices exchange data with the user, machine-readable devices with electronic equipment (storage, sensors, actuators), and communication devices with remote machines. A printer is human readable because its pages are meant for people.

2. Two identical disks are installed. One holds users' files; the other holds paging space for virtual memory. Which device difference explains why the OS puts different software in front of each?

A. Data rate: a paging disk must spin much faster
B. Application: its use decides the software it needs
C. Unit of transfer: paging moves single bytes, not blocks
D. Error conditions: only a file disk can report errors

Answer: B. Application: its use decides the software it needs

Application is about what a device is used for. Files need file-management software (directories, allocation, protection); paging space needs virtual-memory hardware and software.

3. A serial link uses **odd parity**. The 7 data bits of a character are `1011001`. Which parity bit is added?

A. 0, because the data already holds an even number of 1s
B. 1, because the last data bit is a 1
C. 0, because parity bits are only added to letters
D. 1, so the total number of 1s becomes odd

Answer: D. 1, so the total number of 1s becomes odd

1011001 holds four 1s. Odd parity needs an odd total, so the parity bit is 1, making five 1s. If any single bit later flips, the count becomes even and the receiver detects the error.

4. A network card receives a packet whose checksum does not match its contents. What usually happens next?

A. It is dropped, and a protocol such as TCP has it resent
B. The OS terminates the receiving process with an error
C. The OS waits for a person to replace the network card
D. The card repairs it with the disk drive's ECC bits

Answer: A. It is dropped, and a protocol such as TCP has it resent

Error conditions differ by device. Network errors are frequent and mild: the damaged packet is discarded and a reliable protocol notices the gap and gets it sent again, so programs rarely see the error.

5. A keyboard delivers about 10^2 bit/s and a hard disk about 10^9 bit/s. How many seconds of typing deliver as much data as **one second** of the disk?

Answer: 10000000 s

$10^9 \div 10^2 = 10^7 = 10{,}000{,}000$ seconds, about 116 days of nonstop typing. The two rates are seven orders of magnitude apart.

6. A display shows 1920 × 1080 pixels, 24 bits per pixel, redrawn 60 times a second. How many gigabits per second (Gbit/s) must flow to it? Give one decimal place.

Answer: 3 Gbit/s

1920 × 1080 × 24 × 60 = 2,985,984,000 bit/s ≈ 3.0 Gbit/s, which is why a graphics display sits near the top of the data-rate chart.

7. A disk uses 4,096-byte blocks. To change one byte inside a block, the OS reads the block and writes it back. How many bytes move between the disk and memory?

Answer: 8192 bytes

The unit of transfer is a whole block: 4,096 bytes in and 4,096 bytes out, 8,192 bytes in all, to change a single byte.

8. A laser printer is a machine-readable device, because a machine rather than a person puts the marks on the paper.

Answer: False

The category depends on who the device exchanges data with. A printer's output is meant for people to read, so it is a human-readable device.

9. A USB keyboard and a USB flash drive belong to different device categories, even though both plug into the same kind of port.

Answer: True

True: the connector does not decide the category. The keyboard exchanges data with a person (human readable); the flash drive stores data for the computer (machine readable).

10. Match each device difference to an example of it.

Data rate	**A keyboard sends about 10^2 bit/s, an SSD about 10^{10} bit/s**
Application	**An administrator's terminal gets higher priority than other users'**
Complexity of control	**A printer needs a few status bits; a disk needs a detailed command set**
Unit of transfer	**A terminal sends one character at a time; a disk moves 4,096-byte blocks**
Data representation	**Two devices disagree on whether parity is even or odd**
Error conditions	**A paper jam needs a person; a lost packet is resent automatically**

These are the six ways devices differ, and each one is a reason the OS cannot handle every device with the same code and the same policies.

11. Which of these are stream-oriented devices?

A. Terminal
B. Hard disk drive
C. Mouse
D. USB flash drive
E. Printer
F. Serial communication port

Answer: A, C, E, F

Stream-oriented devices move bytes in sequence with no block numbers: terminals, mice, printers and communication ports. Disks and USB flash drives store fixed-size numbered blocks, so they are block-oriented.

12. Put these devices in order of typical data rate, slowest first.

Correct order:

1. Keyboard
2. Dial-up modem
3. Laser printer
4. Hard disk drive
5. 100 Gigabit Ethernet

Roughly 10^2, 10^4–10^5, 10^6–10^7, 10^9 and 10^{11} bit/s: each step is more than ten times faster, which is why device speeds are compared on a log scale.

11.2 Organization of the I/O Function

You should be able to

- Compare programmed I/O, interrupt-driven I/O and DMA by what the processor and the requesting process do while a block moves.
- Place each technique in the two-question classification: interrupts or not, through the processor or straight to memory.
- Trace the six-step evolution of the I/O function, from direct device control to I/O processors.
- Set up a DMA transfer with its four parameters and explain when cycle stealing makes the processor pause.
- Count system-bus transfers per word for the detached, integrated and I/O-bus DMA configurations.

Organization of the I/O function

Every I/O operation needs four chores done: **command** the device, **wait** until it is ready, **carry** each word between the device and memory, and **notice** that the job is finished. Organizing the I/O function means deciding which hardware does each chore. The long-term trend is to move these chores off the processor so it can run programs instead.

The three techniques

- **Programmed I/O.** The processor sends a command to the I/O module on behalf of a process, then repeatedly reads the module's status register (polling, busy waiting) until the operation is done, and copies each word itself. The module never interrupts.
- **Interrupt-driven I/O.** The processor sends the command and goes on with other work. When the module is ready it raises an interrupt; the handler copies the word and sends the next command. Every word still passes through the processor, and every word costs one interrupt.
- **Direct memory access (DMA).** A DMA module moves a whole block straight between the I/O module and memory. The processor is involved only at the start (setup) and at the end (one completion interrupt).

	No interrupts	Uses interrupts
Transfer goes through the processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer	(not a standard technique)	DMA

The empty corner would mean moving data straight to memory but polling for the end, which gives back much of the gain, so the classic design pairs direct transfer with a completion interrupt. (Some very fast modern devices are polled for completion, when checking costs less than taking an interrupt.)

What happens to the process that asked

- **Blocking call:** the process cannot continue without its data, so the next code the processor runs is the OS's. It moves the process to the Blocked state and dispatches another ready process. When the I/O finishes (after the last per-word interrupt, or the single DMA interrupt) the OS moves the process to Ready.
- **Asynchronous request:** the call starts the transfer and returns at once, and the requesting process keeps running; the OS finishes the transfer in the background and reports later (a flag, an event or a signal) that it has finished. The process must not use the buffer before then.
- With **programmed I/O** the request type makes no difference: the processor itself is stuck in the polling loop, so no other work gets done (a timer interrupt could preempt the process, but it goes back to spinning later).

Nonblocking is not the same as asynchronous. A **nonblocking call** only promises not to wait: it returns at once with whatever it could do right now (perhaps only part of the data) or, if nothing was ready, a "would block" error (EAGAIN in POSIX). It leaves no transfer running, so the program must call again later. Example: a nonblocking read() on an empty pipe whose writer is still open fails with EAGAIN; a byte written afterwards is obtained only by a new read(). An **asynchronous request** (POSIX aio_read, Windows overlapped I/O in section 11.10) hands the whole transfer to the OS, which completes it in the background and reports completion; the buffer is filled without another call.

Where the processor's time goes: a worked example

Model in ticks: sending a command 1, moving a word through the processor 1, interrupt overhead H (saving the context, finding the cause, entering and leaving the handler), DMA setup 4 (one per item), each DMA word costs at most one stolen tick. For N words and a device that needs D ticks per word:

- Programmed I/O: processor time on I/O = N(D + 2), all of it, because the waiting is spent polling.
- Interrupt-driven: processor time on I/O = N(H + 2); the D ticks per word are free for other work. The transfer ends at N(D + H + 2) because the device idles while each handler runs.
- DMA: processor time on I/O = 4 + N + H (setup, stolen ticks, one interrupt); the transfer ends at 4 + ND + H.

Example: N = 8, D = 10, H = 4 gives 96, 48 and 16 processor ticks. Programmed I/O leaves nothing for other processes; DMA leaves the most and also finishes first (t = 88, against 96 and 128). Two special cases: if H > D (a fast device), interrupt-driven I/O costs *more* than polling, which is why fast block devices use DMA; and for a single word DMA's fixed cost (4 + 1 + H) is more than interrupt-driven I/O's cost for that word (1 + H + 1), so DMA pays off for multi-word blocks.

The evolution of the I/O function, in six steps

1. The processor directly controls a peripheral device (it drives the device's signals itself).
2. A controller or I/O module is added. The processor uses programmed I/O without interrupts, but no longer needs to know the device's details.
3. The same configuration with interrupts: the processor no longer waits for the device.
4. The I/O module gets direct control of memory through DMA: a block moves without the processor, except at the start and the end.
5. The I/O module becomes an **I/O channel**: a processor with a specialized I/O instruction set. The CPU places an **I/O program** (channel program) in **main memory** and tells the channel to run it; the channel performs the whole sequence and interrupts only when it is done.
6. The I/O module becomes an **I/O processor**: it has its own **local memory** and is a computer in its own right. It can control a large set of devices (for example many terminals) with minimal CPU involvement.

Each step frees the processor from one more chore: device signals (2), waiting (3), carrying each word (4), issuing each command of a multi-step job (5), and the routine control of many devices (6). The steps are a logical progression, not a strict timeline: IBM mainframes had channels from the late 1950s, decades before the original IBM PC (1981) used a separate DMA controller chip. Today most devices do their own DMA as **bus masters** (devices that take control of the bus and start their own transfers), and SSD controllers or smart network cards contain their own processor cores and memory, the I/O-processor idea.

DMA in detail

To start a transfer the processor gives the DMA module four items: **read or write**, the **address of the I/O device**, the **starting location in memory** (loaded into the address register) and the **number of words** (loaded into the count register). The DMA module then moves the block one word at a time over the bus, adding 1 to the address register and subtracting 1 from the count register after each word. When the count reaches 0 it sends one interrupt to the processor.

Cycle stealing: for each bus transfer the DMA module takes the bus for one bus cycle (one per word in most layouts, two with detached DMA; see below). If the processor needs the bus in that cycle (to fetch an instruction or an operand, or to write a result) it simply pauses for one cycle; if it is busy inside itself (decoding, executing) the steal costs nothing. This is not an interrupt: no context is saved and no handler runs. The processor runs a little slower, but far less than if it moved every word itself.

The DMA module checks nothing; it writes to the physical addresses it was given. So the OS gives it only a buffer the requesting process owns and keeps those frames locked in memory (frame locking) until the completion interrupt; otherwise a page moved mid-transfer would have its frame overwritten. Modern machines also add an IOMMU, hardware that limits which memory a device may reach.

DMA configurations

Configuration	How it is wired	System-bus cycles per word
Single-bus, detached DMA	Processor, memory, DMA module and I/O modules all share one system bus.	2 (I/O module → DMA module, DMA module → memory)
Single-bus, integrated DMA-I/O	DMA logic is part of an I/O module, or linked to one or more I/O modules by a private path.	1 (DMA module → memory)
I/O bus	A separate I/O bus links all I/O modules to one DMA module.	1; the DMA module needs only one I/O-side interface, and new devices just plug into the I/O bus

Example: a 512-word block needs 2 × 512 = 1024 system-bus transfers with detached DMA and 512 with either of the other two. Every system-bus cycle the DMA module uses is one the processor may lose, so fewer system-bus cycles per word means less slowdown during a transfer.

Key terms

I/O controller (device controller)
A simple I/O module: circuitry that drives one kind of device, turning short commands from the processor into the exact signals and timing that device needs.

Asynchronous I/O request
An I/O request that starts the transfer and returns at once. The OS finishes the transfer in the background and later reports that it is done (a flag, an event or a signal); until then the process must not use its buffer.

Nonblocking I/O call
A call that never makes the caller wait: it returns at once with whatever it could do right now, perhaps only part of the data, or a “would block” error (EAGAIN) if nothing was ready. It leaves no transfer running, so the program simply calls again later.

Interrupt overhead
The processor time an interrupt costs on top of the useful work it leads to: saving the context, finding the cause, entering and leaving the handler, and restoring the context.

I/O channel
An I/O module that is a small processor with its own specialized instruction set. It fetches and runs an I/O program stored in main memory, so it can carry out a whole sequence of I/O operations without the CPU.

I/O program (channel program)
A list of I/O instructions that the OS places in main memory for an I/O channel to execute, for example: seek, read a block, check the result, read the next block.

I/O processor
An I/O module with its own processor and its own local memory: a small computer in its own right that runs the day-to-day control of many devices with very little help from the CPU.

Local memory

Memory that belongs to an I/O processor itself, separate from main memory, where it keeps its own programs and data buffers.

Single-bus detached DMA

A DMA configuration in which the processor, memory, the DMA module and the I/O modules all share one system bus, so each word crosses that bus twice: I/O module → DMA module, then DMA module → memory (the reverse for output).

Single-bus integrated DMA-I/O

A DMA configuration in which the DMA logic is built into an I/O module or wired to one or more I/O modules by a private path, so each word crosses the system bus only once.

I/O bus

A separate bus that links many I/O modules to one DMA module. Words reach the DMA module without touching the system bus, the DMA module needs only one I/O-side connection, and a new device just plugs into the I/O bus.

Bus master

A device that can take control of a bus and start its own transfers. Most modern devices (for example on PCI Express) contain their own DMA logic and act as bus masters.

IOMMU

Hardware between devices and main memory that translates and checks the addresses a device uses for DMA, so a device can reach only the memory the OS has granted to it.

Review questions with answers

1. A process makes a **blocking** read() that the OS carries out with interrupt-driven I/O. What happens right after the read command is sent to the I/O module?

A. The processor checks the module's status register in a loop until the first word is ready.
B. The OS moves the process to the Blocked state and dispatches another ready process.
C. The process keeps running, and the OS tells it later that the data has arrived.
D. The OS swaps the process out to disk until the whole block has been transferred.

Answer: B. The OS moves the process to the Blocked state and dispatches another ready process.
A blocking call means the process cannot go on, so the next code the processor runs is the OS's: it blocks the process and schedules another one. The completion interrupt later lets the OS make it Ready again.

2. A program puts a pipe in nonblocking mode and calls read() while the pipe is empty (the writer still has it open). The call returns at once, and the OS fills the program's buffer later, when data arrives.

Answer: False
Nonblocking only means "do not wait": with nothing to read, read() fails at once with a would-block error (EAGAIN) and starts nothing, so the program must call it again. Filling the buffer later and reporting completion is what an asynchronous request (for example POSIX aio_read) does.

3. When a DMA module steals a bus cycle, the processor is delayed only if it needs the bus in that same cycle, and it saves no context.

Answer: True
Cycle stealing is not an interrupt. The processor merely pauses for that one bus cycle, and only if it needed the bus at that moment (to fetch or store); nothing is saved and no handler runs.

4. Put the six steps in the evolution of the I/O function in order, earliest first.

Correct order:

1. The processor directly controls each peripheral device
2. A controller (I/O module) is added and the processor uses programmed I/O without interrupts
3. Interrupts are added, so the processor no longer waits for the device
4. The I/O module gets direct access to memory (DMA)
5. The I/O module becomes an I/O channel that runs an I/O program held in main memory
6. The I/O module becomes an I/O processor with its own local memory

Each step takes one more chore off the processor: device details, then waiting, then moving each word, then sequencing whole I/O jobs, then the routine control of many devices.

5. Match each design to its description.

Single-bus detached DMA	**Every word crosses the system bus twice**
Single-bus integrated DMA-I/O	**DMA logic is built into an I/O module or wired to a few by private links**
I/O bus configuration	**One DMA interface serves all I/O modules through a separate bus**
Bus master device	**A device with its own DMA logic that starts its own transfers**

Detached DMA uses the shared system bus for both moves; integrated and I/O-bus designs keep the device-to-DMA move off it; a bus master builds the DMA logic into the device itself.

6. A disk sends a 512-word block to memory through a **single-bus detached** DMA module. How many system-bus transfers does the block need?

Answer: 1024 transfers

Each word crosses the system bus twice (I/O module → DMA module, then DMA module → memory): 2 × 512 = 1024. An integrated or I/O-bus design would need 512.

7. Interrupt-driven I/O moves a 10-word block. For each word the processor spends 1 tick sending the command, 5 ticks of interrupt overhead and 1 tick moving the word. How many processor ticks does the transfer cost in total?

Answer: 70 ticks

(1 + 5 + 1) × 10 = 70 ticks. With DMA the same block costs one setup, at most one stolen tick per word, and a single interrupt.

8. Which technique does each description fit?

Programmed I/O	Interrupt-driven I/O	DMA
The processor reads a status register in a loop until the device is ready The I/O module never interrupts the processor	Each word passes through the processor, but no time is spent checking status The processor takes one interrupt for every word	The processor hands over direction, device, start address and word count, then leaves One interrupt arrives after the whole block is in memory

Programmed I/O polls and is never interrupted; interrupt-driven I/O moves every word itself, one interrupt each; DMA needs only a setup and one interrupt per block.

9. Which items does the processor give a DMA module to start a block transfer? Choose all that apply.

A. Whether to read or write
B. The ID of the process that asked
C. The address of the I/O device
D. The starting location in memory
E. The scheduling priority of the transfer
F. The number of words to move

Answer: A, C, D, F

Direction, device, where in memory and how many words: that is all the DMA module needs. Which process asked is the OS's business, kept in the OS's own tables.

10. What makes an I/O processor different from an I/O channel?

A. An I/O processor raises interrupts, while a channel has to be polled by the CPU.
B. An I/O processor can move data directly into memory, while a channel cannot.
C. A channel has a specialized instruction set, while an I/O processor runs no program.
D. An I/O processor has its own local memory and is a computer in its own right.

Answer: D. An I/O processor has its own local memory and is a computer in its own right.

A channel executes an I/O program it fetches from main memory. An I/O processor adds local memory of its own, so it can run the routine control of many devices with minimal CPU involvement.

11. A device can deliver a new word every microsecond, but each interrupt costs the processor about 4 microseconds. Why is interrupt-driven I/O a poor way to move a large block from it?

A. Each interrupt costs more than the short wait it replaces, so even polling would be cheaper.
B. The device must wait for the processor to finish polling before it may send the next word.
C. Interrupts can be used only by character devices, never by devices that move blocks.
D. The DMA module would steal so many bus cycles that the interrupts could not be served.

Answer: A. Each interrupt costs more than the short wait it replaces, so even polling would be cheaper.

With one interrupt per word, a fast device makes the overhead dominate. A block from a fast device is a job for DMA: one setup and one interrupt for the whole block.

12. Why must the OS keep the frames of a DMA buffer locked in memory until the transfer ends?

A. Locked frames sit in a faster region of memory, so the processor can read the arriving words sooner.
B. The DMA module can reach only frames whose lock bit is set; every other frame is invisible to it.
C. The DMA module uses physical addresses, so if the page moved it would write into someone else's frame.
D. Locking stops the processor from pausing whenever the DMA module steals a bus cycle for the buffer.

Answer: C. The DMA module uses physical addresses, so if the page moved it would write into someone else's frame.

The DMA module was given a physical start address. If the OS replaced or moved that page mid-transfer, the module would keep writing into a frame that now belongs to another page or process.

11.3 Operating System Design Issues

You should be able to

- State the two objectives of I/O design, efficiency and generality, and explain why each one matters.
- Explain how multiprogramming overlaps computing with I/O, and why the disk often stays the bottleneck.
- Describe how general functions such as open, read, write, close, lock and unlock hide device details.
- Name the layers used for a local device, a communications port and a file system, and say what each layer does.
- Trace a read request down through the layers to the hardware and back, and relate each layer to its time scale.

Two objectives of I/O design

- **Efficiency.** I/O is usually the bottleneck: devices are far slower than the processor and main memory. The OS must keep the processor busy while devices work and make each device, above all the disk, waste as little time as possible.
- **Generality.** Handle all devices in a uniform way where possible, both in how processes see them and in how the OS manages them. Device details are hidden in low-level routines so that processes and upper OS levels see every device through a few general functions.

Without a general interface, 3 programs and 4 devices need 3 × 4 = 12 pieces of device code, and a new device means changing every program. With one, each device needs one driver in the OS, and a new device needs only a new driver.

Efficiency: overlapping I/O with computing

A processor runs billions of instructions per second, while one hard-disk access takes several milliseconds, long enough for over ten million instructions. **Multiprogramming** lets some processes wait for I/O while another computes. Even so, I/O may not keep up. When too few ready processes are in memory, the OS brings more in by **swapping**, but swapping is itself disk I/O. That is why disk efficiency gets the most attention: buffering (11.4), disk scheduling (11.5), RAID (11.6) and the disk cache (11.7).

Model: each process computes for C ms, then does one I/O of T ms, repeatedly.

- **One process:** processor and device take turns. Processor busy = C / (C + T). Example: C = 4, T = 8 gives 4 / 12 = 33.3%.
- **Several processes, a separate device each:** overlap raises utilization. With C = 4, T = 8: 2 processes give 66.7%, 3 give 100%.
- **Several processes, one shared disk:** if T > C the disk saturates and becomes the bottleneck. It finishes one I/O per T ms, each releasing one C ms burst, so the processor is busy at most C / T. With C = 4, T = 8 that is 50%, however many processes there are; with C = 3, T = 12 it is 25%. More processes only lengthen the disk queue.

A **bottleneck** is the slowest stage in a chain of work; it sets the pace for the whole chain.

Generality: the same calls for every device

The general functions are **open, close, read, write, lock, unlock**. The program's call is identical for every device; the routines below the interface differ:

Call	Keyboard	Disk file	Network connection
open	check access, find the driver	name → file identifier, check access rights	set up a connection (UNIX: socket + connect)
read	block until a line is typed; an interrupt per key	find the block; use a copy in memory or queue a disk read and block	block until packets arrive; each protocol layer strips its header
write	input only (on UNIX the terminal sends it to the screen)	copy to a buffer; the block is written later	cut into packets, add headers, send
lock / unlock	seldom needed (the foreground process gets the keys)	hold the file or a byte range; advisory by default on UNIX-like systems, enforced on Windows	seldom needed
close	stop delivering keys, free the descriptor	free records; changed blocks reach the disk later (`fsync` forces them)	send queued data, tell the other side, free the descriptor

Device independence: the same program works with different devices unchanged. UNIX `sort` reads the keyboard when typed alone and a file when typed as `sort < names.txt`.

Common mistake: generality does not give every device every ability. Reading from a printer fails with an error code, and device-specific settings use an escape hatch (`ioctl` on UNIX-like systems).

Layered (hierarchical) structure

- Each layer does one kind of job and relies on the next lower layer for more primitive functions.
- It hides the lower layers' details and offers simpler services to the layer above.
- Ideally a layer can be changed without changing the others.
- Layers follow time scale: near the hardware, events take nanoseconds to microseconds; near the user, milliseconds to seconds.

Rough time scales (local-device stack): hardware 1 ns–1 µs, scheduling and control 0.1–100 µs, device I/O 1 µs–10 ms, logical I/O 10 µs–0.1 s, user process 10 ms–10 s.

Example: swapping a parallel-port printer for a USB printer with a different printer language changes device I/O, scheduling and control, and the hardware; logical I/O and the user process stay the same.

Three layer stacks

```
Local peripheral device   Communications port          File system
User process              User process                 User process
Logical I/O               Communications architecture  Directory management
                          (e.g. TCP over IP)           File system
                                                       Physical organization
Device I/O                Device I/O                   Device I/O
Scheduling and control    Scheduling and control       Scheduling and control
Hardware                  Hardware                     Hardware
```

- **Logical I/O** (local device): treats the device as a logical resource. It manages general I/O functions for processes, which use a **device identifier** and simple commands: open, close, read, write. It ignores how the device is controlled. (Chapter 12's file-management model uses the name for a different layer, which gives programs records instead of disk blocks.)
- **Device I/O**: turns requested operations and data (buffered characters, records) into sequences of I/O instructions, channel commands and controller orders. Buffering may improve device use.
- **Scheduling and control**: queues and schedules I/O operations, controls them, handles interrupts, and collects and reports status. It is the layer that actually talks to the I/O module.
- **Communications architecture** (communications port): replaces logical I/O and is itself several layers, for example TCP/IP. IP moves packets between computers; TCP gives programs a reliable, in-order byte stream. Each layer adds a header on the way out and removes it on the way in.
- **Directory management** (file system): converts symbolic file names into identifiers that point to the file directly or through a descriptor record or index table; also handles directory operations such as add, delete and reorganize.
- **File system layer**: deals with the logical structure of files and the user operations open, close, read, write; manages **access rights**.
- **Physical organization**: converts logical references to files and records into physical secondary-storage addresses, allowing for the **track** and **sector** structure of the device (as virtual addresses are converted to physical ones). It also allocates disk space and main-memory buffers.

Worked example: one file read through the stack

1. open("/home/ana/notes.txt"): directory management turns the name into file 7731; the file system layer checks that ana may read it and returns descriptor 3.
2. read(3, buf, 4096) at file position 8192: bytes 8192–12287 are logical block 8192 ÷ 4096 = 2.
3. Physical organization: the file's block table gives disk block 9,034. With 512-byte sectors a block is 8 sectors, so the data are sectors 9,034 × 8 = 72,272 to 72,279; on a toy disk with 500 sectors per track that is track 144, sectors 272–279. A buffer is reserved.
4. Device I/O builds the controller command “read 8 sectors from sector 72,272 into the buffer by DMA”.
5. Scheduling and control queues it behind 2 earlier requests and blocks the process, later starts the operation; the drive seeks and reads (milliseconds); the interrupt reports success and the process becomes ready.
6. The data travel back up; the file system layer copies 4096 bytes into buf, the position becomes 12288, and read returns 4096.

Real drives hide their geometry: the OS sends only a sector number and the drive finds the track itself.

Keyboard read: the process blocks; each key press (l, s, Enter) raises an interrupt in this simple model (real keyboards also report each release), handled in microseconds; device I/O builds the line, and read returns 3. The process waited seconds while the layers below worked in microseconds.

Network receive: a 566-byte frame arrives by DMA; the driver strips the 14-byte Ethernet header, IP its 20-byte header, TCP its 20-byte header (after checking order and acknowledging), leaving 512 data bytes. read returns 512 although 1000 were asked for.

Key terms

Efficiency (I/O design goal)
: The goal of keeping slow I/O from wasting the fast processor: overlap I/O with computing and make each device, above all the disk, do its work with as little delay as possible.

Generality (I/O design goal)
: The goal of handling all devices in one uniform way, so processes and the upper parts of the OS use the same few general functions for every device.

Bottleneck
: The slowest stage in a chain of work. It sets the pace for the whole chain, so speeding up any other stage does not help.

Device independence
: Being able to use the same program, unchanged, with different devices: the program names a device or file, and the OS deals with the differences.

Layered structure (hierarchical design)
: Organizing software as a stack of layers. Each layer does one kind of job, uses only the services of the layer directly below it, hides that layer's details, and offers simpler services to the layer above.

Logical I/O
: A layer name used in two models. In the I/O model for a local device it is the top layer, which treats the device as a logical resource reached through a device identifier and simple commands (open, close, read, write); in the file-management model (chapter 12) it is the layer that gives programs records instead of disk blocks and keeps basic data about each file.

Device identifier (device ID)
: A name or number the OS uses to pick out one particular device, for example the keyboard or the first printer, without the program knowing how that device is wired.

Device I/O
: The I/O layer that turns requested operations and their data into the exact sequences of I/O instructions, channel commands and controller orders the device needs. It may buffer data to use the device better.

Scheduling and control
: The lowest I/O software layer: it queues and orders I/O operations, starts them on the hardware, handles their interrupts, and collects and reports their status. It is the layer that actually talks to the I/O module.

Communications architecture

The layer that replaces logical I/O for a communications port. It is itself a stack of layers (for example TCP/IP) that turn a program's data into packets and back.

Protocol
An agreed set of rules for the format, order and meaning of the messages two communicating parties exchange.

TCP/IP
The family of protocols the Internet runs on. IP moves packets from computer to computer; TCP, on top of it, gives programs a reliable, in-order stream of bytes.

Directory management
The top layer of the file-system I/O stack: it converts symbolic file names into the internal identifiers the file system uses, and carries out directory operations such as adding, deleting and reorganizing entries.

Symbolic file name
A file name chosen for people, such as notes.txt or /home/ana/notes.txt. The OS translates it into an internal identifier before it uses the file.

File system layer
The layer below directory management: it deals with the logical structure of files and with the operations users ask for (open, close, read, write), and it enforces access rights.

Access rights
The permissions that say which users or processes may do which operations, such as read, write, execute or delete, on a file.

Physical organization
In the file-system I/O stack, the layer that turns logical references to files and records into physical addresses on secondary storage (tracks and sectors). It also allocates disk space and main-memory buffers. (Chapter 7 uses the same name for a memory-management requirement.)

Track
One of the concentric rings on a disk surface along which data are recorded.

Sector
A fixed-size piece of a track, and the smallest unit a disk reads or writes: traditionally 512 bytes, 4096 bytes on many newer drives.

Review questions with answers

1. Which pair names the two main objectives in designing the I/O facility of an operating system?

A. Security and portability
B. Fairness and low cost
C. Efficiency and generality
D. Reliability and compatibility

Answer: C. Efficiency and generality

Efficiency keeps slow I/O from wasting the fast processor; generality lets processes and upper OS levels handle every device through the same few functions.

2. Why does disk I/O get more design attention than any other kind of I/O?

A. Disks are the only devices able to raise interrupts
B. Disks are slow but needed for files and swapping
C. Disks move data a byte at a time, like keyboards
D. Disks cannot use DMA, so the CPU copies each byte

Answer: B. Disks are slow but needed for files and swapping

A disk access takes milliseconds, millions of instruction times, and files, swapping and virtual memory all depend on the disk, so its speed limits the whole system.

3. A process alternates between 6 ms of computing and 9 ms of disk I/O, and no other process is ready to run. What percentage of the time is the processor busy?

Answer: 40 %

With one process the processor and the disk take turns: the processor is busy 6 ms out of every 6 + 9 = 15 ms, and 6 / 15 = 40%.

4. Many processes each compute for 3 ms and then do one 12 ms I/O on the same single disk, which serves one request at a time. In the long run, at most what percentage of the time can the processor be busy?

Answer: 25 %

The disk is the bottleneck: it finishes at most one I/O every 12 ms, and each finished I/O releases one 3 ms burst of computing, so the processor is busy at most 3 / 12 = 25%. Adding processes only lengthens the disk queue.

5. Generality means every device must support every general operation, so a program can read from a printer just as it reads from a file.

Answer: False

Generality means one uniform interface, not identical abilities. An operation a device cannot perform, such as reading from a printer, simply fails with an error code, and device-specific settings go through an escape hatch such as ioctl.

6. Which of these are general functions through which processes and the upper levels of the OS see devices?

A. read
B. lock
C. move the disk arm to track 144
D. close
E. set the DMA start address
F. open

Answer: A, B, D, F

open, close, read, write, lock and unlock work the same for every device. Moving an arm or programming a DMA address are device details that the lower layers hide.

7. Put the layers of the file-system I/O stack in order, from the one nearest the user to the one nearest the hardware.

Correct order:

1. Directory management
2. File system
3. Physical organization
4. Device I/O
5. Scheduling and control

Names become identifiers, the file system handles logical file operations and access rights, physical organization finds tracks and sectors, device I/O builds the commands, and scheduling and control drives the hardware.

8. Match each I/O layer to its job.

Logical I/O	**Device as a logical resource: device ID + simple commands**
Device I/O	**Builds I/O instructions, channel commands, controller orders**
Scheduling and control	**Queues operations, handles interrupts, reports status**
Directory management	**Symbolic file names → internal identifiers**
Physical organization	**Logical file references → tracks and sectors**
Communications architecture	**Replaces logical I/O for a port; layered, e.g. TCP/IP**

Each layer has one kind of job and hides it from the layers above.

9. The layered model of I/O has three stacks: one for a local peripheral device, one for a communications port and one for a file system. Sort each layer into the only stack that uses it, or into “All three” if every stack uses it.

Local device	Communications port	File system	All three
Logical I/O	Communications architecture	Directory management File system layer Physical organization	Device I/O Scheduling and control

The three stacks differ only in their upper layers. Device I/O and scheduling and control, the layers closest to the hardware, appear in all three.

10. In the layered I/O model, which layer actually interacts with the I/O module and handles the device interrupts?

A. Communications architecture
B. Logical I/O
C. Device I/O
D. Scheduling and control

Answer: D. Scheduling and control

Scheduling and control queues and schedules operations, starts them, handles interrupts, and collects and reports status. It is the layer that talks to the hardware.

11. How do the time scales of the I/O layers compare?

A. Layers near the hardware deal with the shortest time scales
B. All layers work on one time scale, set by the device speed
C. Layers near the user deal with the shortest time scales
D. Only the user process has a time scale; the other layers are instant

Answer: A. Layers near the hardware deal with the shortest time scales

Hardware signals and bus cycles take nanoseconds, interrupts are handled in microseconds, and whole requests seen by the user take milliseconds or more. The layers follow that order.

12. A file system uses 4096-byte blocks on a disk with 512-byte sectors, and the blocks are laid end to end starting at sector 0. Physical organization finds the wanted data in disk block 9034. At which sector number does that block begin?

Answer: 72272

Each block holds 4096 / 512 = 8 sectors, so block 9034 starts at sector 9034 × 8 = 72,272 and ends at sector 72,279.

11.4 I/O Buffering

You should be able to

- Explain why the OS moves data through its own buffers instead of straight into a process's memory, including locked pages and single-process deadlock.
- Tell block-oriented devices from stream-oriented devices and match each to a style of buffering.
- Compute the time per block with no buffer (T + C), a single buffer (max[C, T] + M) and a double buffer (about max[C, T]), and name the bottleneck.
- Describe line-at-a-time and byte-at-a-time buffering and circular buffers, and explain why no number of buffers cures a sustained overload.

Why the OS buffers I/O

Devices are slow compared with the processor, so every transfer raises two questions: who waits, and where does the data land? **I/O buffering** keeps data for a while in memory on its way between a device and a process. Input transfers can then happen **before** the process asks for the data (read-ahead), and output transfers can happen **after** the process has handed the data over and moved on. Speed differences are smoothed out instead of forcing one side to wait.

The trouble with transferring straight into a process's memory

- The process has nothing to do until the slow transfer ends.
- The target area (say addresses 4096–4607 of the process) must stay in main memory for the whole transfer, so its page must be **locked** before the request is issued. A process with a locked page cannot be swapped out completely, and locked frames reduce the OS's freedom to manage memory.
- Without the lock there is a **single-process deadlock**: the process issues the request and blocks; the OS swaps it out; the device is now ready but the target page is not in memory, so the I/O waits for the process to come back, while the process waits for the I/O. One process is enough.

The fix is a **system buffer** in the OS's own memory. The device fills (or drains) the system buffer, so the process's memory is not involved and the process can be swapped out freely. The cost is an extra copy between the system buffer and the process (time **M**) and bookkeeping about which buffer belongs to which process.

Block-oriented and stream-oriented devices

- **Block-oriented device:** stores data in fixed-size blocks, transfers one block at a time, and lets the OS ask for any block by its number. Examples: disks, SSDs, USB flash drives, memory cards (secondary storage).
- **Stream-oriented device:** moves a flow of bytes with no block structure and no block numbers. Examples: keyboards and terminals, mice and other pointing devices, printers, communication ports, and most other devices that are not secondary storage.

"Stream" is about structure, not speed. Block devices are buffered a block at a time; stream devices a line or a byte at a time.

Timing: no buffer, single buffer, double buffer

Let **T** = time to read one block, **C** = computing time per block between requests, **M** = time to copy a block from a system buffer into the process.

Scheme	How it works	Time per block
No buffer	The process asks, waits while the device reads into its memory, then computes. The two take turns.	T + C
Single buffer	While the process computes on block i, the OS reads block i+1 into the system buffer (**read-ahead**, or anticipated input). The copy M overlaps nothing, because the only buffer is busy during it.	max[C, T] + M
Double buffer (buffer swapping)	Two system buffers: the OS fills one while the other is being emptied into the process, then they swap roles. The device keeps reading during the copy.	about max[C, T]; exactly max[T, C + M]

Estimate versus exact. The classic double-buffer formula max[C, T] treats the copy as free. With that simplification, if C ≤ T the device runs at full speed, and if C > T the process never waits for I/O. But the copy still takes the process's time, so the process needs C + M per block and the exact time is **max[T, C + M]**; the two agree whenever C + M ≤ T. Counted exactly, the dividing line is C + M against T: if T is larger the device never rests; if C + M is larger the process never waits. The slower side is the **bottleneck**: it sets the pace. (No buffer and single buffer have no such gap: T + C and max[C, T] + M are exact in this model.)

Worked example 1 (device slower): T = 10 ms, C = 4 ms, M = 1 ms

- No buffer: 10 + 4 = 14 ms per block. The process waits 10 ms and the device rests 4 ms per block.
- Single buffer: max(4, 10) + 1 = 11 ms. The process waits 6 ms per block; the device rests 1 ms (during the copy).
- Double buffer: max(4, 10) = 10 ms, and the exact max[T, C + M] = max(10, 5) = 10 ms agrees because C + M ≤ T. The device never rests; only a faster device could help. 1,000 ÷ 10 = 100 blocks per second, against about 71 with no buffer.

Worked example 2 (process slower): T = 5 ms, C = 8 ms, M = 1 ms

- No buffer: 5 + 8 = 13 ms per block.
- Single buffer: max(8, 5) + 1 = 9 ms. The process never waits for I/O; the device rests 4 ms per block.
- Double buffer: the estimate max(8, 5) = 8 ms leaves out the copy; the exact time is max[T, C + M] = max(5, 9) = **9 ms**, the value a timeline (and this guide's exercises, which count every millisecond) gives. The process never waits for I/O; the device rests 4 ms per block. With steady times one buffer already hides the I/O here; the second buffer pays off when the device is the slower side or its speed varies.

Buffering stream devices

- **Line-at-a-time:** the buffer holds one line. Input reaches the process only when the user presses Enter, so the process is woken once per line, and editing (Backspace) happens inside the OS buffer. For output, the process copies a whole line into the buffer and carries on. This fits a **scroll-mode terminal**.
- **Byte-at-a-time:** each character is handed over the moment it arrives, so the OS and the process form a small producer/consumer pair. This fits a **forms-mode terminal**, where every keystroke can matter at once.
- **Double buffering a stream:** with two line buffers, a process writing lines need not wait unless it gets two lines ahead of the device (with one buffer it waits as soon as it is one line ahead). For byte-at-a-time streams a double buffer gives no special advantage over a single buffer twice as long.

Circular buffering

When a process does rapid bursts of I/O, two buffers may not be enough. A **circular buffer** is a ring of more than two system buffers run as a bounded buffer: the device (producer) fills empty slots in order, wrapping around, and the process (consumer) empties them in the same order. During a pause the device reads ahead and fills the ring; a burst then finds its blocks already waiting.

Example with a device that needs 4 ms per block and a process that takes bursts of 6 blocks (1 ms each) separated by 30 ms of computing: with 2 buffers the process waits about a quarter of the time; with 6 buffers, enough for a whole burst, it never waits.

What buffering can and cannot do

- Buffering smooths out **peaks** in I/O demand and lets device work overlap with process work.
- No amount of buffering lets a process keep going forever if it demands data faster, on average, than the device can supply. The buffers drain (input) or fill (output), and from then on the process waits for the device. In the example above, with only 10 ms of computing between bursts, the process waits at least a third of the time, however many buffers the ring has.
- Buffering does not make any single transfer faster; it changes **when** transfers happen.
- In a multiprogramming system, buffering is one of the tools that raise efficiency: memory and the processor stay available for other work while transfers are in progress.

Key terms

I/O buffering

Letting data wait in a reserved memory area on its way between a device and a process, so input can be fetched before it is asked for and output can be sent after the process has moved on.

System buffer
A buffer in the operating system's own memory, outside every process's address space, where data waits on its way between a device and a process.

Block-oriented device
A device that keeps data in fixed-size blocks, moves one whole block per transfer and lets the OS ask for any block by its number. Disks, SSDs and USB flash drives are examples.

Stream-oriented device
A device that sends or accepts data as a flow of bytes with no block structure and no block numbers. Keyboards, mice, printers and communication ports are examples.

Single-process deadlock
A standstill that needs only one process: it waits for its I/O to finish, while the I/O waits for the process's swapped-out memory to come back.

Single buffering
Using one system buffer: the OS transfers the next block into it while the process works on the block it already has.

Read-ahead (anticipated input)
Starting to read the next block before the process asks for it, betting that it will be wanted soon.

Double buffering (buffer swapping)
Using two system buffers: the OS fills (or empties) one while the process empties (or fills) the other, and then the two swap roles.

Circular buffering
Using more than two system buffers arranged in a ring and managed as a bounded buffer, so a process can make rapid bursts of I/O without waiting.

Line-at-a-time operation
Stream I/O in which the buffer holds one whole line: the process is handed input only when a complete line, ended by Enter, has arrived.

Byte-at-a-time operation
Stream I/O in which each character is handed over the moment it arrives, because every keystroke may matter.

Scroll-mode terminal
A terminal where the user types and edits a line, then sends it with Enter; finished lines scroll up the screen.

Forms-mode terminal
A terminal that shows a form with fields; each keystroke can matter at once, for example Tab jumping to the next field.

Bottleneck
The slowest stage in a chain of work. It sets the pace, so speeding up any other stage does not help.

Review questions with answers

1. A process asks for a block to be read straight into its own address space. Why must the OS lock the target page in main memory before issuing the request?

A. So that no other process can read the block before its owner does
B. So the transfer cannot land in a frame that no longer holds that page
C. So the device can use DMA rather than programmed I/O for the job
D. So the page table needs no update once the transfer has finished

Answer: B. So the transfer cannot land in a frame that no longer holds that page

If the page were swapped out while the request waited, the transfer would have no valid place to go, or it would overwrite whatever now occupies the frame. Locking keeps the page in place until the I/O completes.

2. Put the steps of a single-process deadlock in order.

Correct order:

1. The process asks for a block to be read into its own page
2. The process blocks while its request waits for the device
3. Memory runs short and the OS swaps the process out
4. The device is ready, but the target page is no longer in memory
5. The I/O waits for the process, and the process waits for the I/O

One process is enough: it waits for its I/O, and the I/O waits for the process's page to come back. Locking the page before the request, or reading into a system buffer, prevents it.

3. Sort each device: block-oriented or stream-oriented?

Block-oriented	Stream-oriented
Hard disk drive USB flash drive Solid-state drive	Keyboard Mouse Printer Communications port

Block-oriented devices hold fixed-size numbered blocks and move one block per transfer; they are the secondary-storage devices. Nearly everything else moves a plain stream of bytes.

4. With no buffering, reading a block takes T = 8 ms and the process computes for C = 5 ms on each block. What is the time per block?

Answer: 13 ms

Without a buffer the device and the process take turns: T + C = 8 + 5 = 13 ms.

5. With single buffering, T = 8 ms, C = 5 ms and M = 2 ms (the time to copy a block from the system buffer to the process). What is the time per block?

Answer: 10 ms

max[C, T] + M = max(5, 8) + 2 = 10 ms. Reading the next block overlaps the computing, but the copy overlaps nothing, because the only buffer is busy during it.

6. With double buffering, T = 8 ms, C = 5 ms and M = 2 ms. How many blocks per second can the process handle?

Answer: 125 blocks/s

The time per block is max[C, T] = 8 ms. Here even C + M = 7 ms is below T, so the device is the bottleneck and never stops. 1,000 ms ÷ 8 ms = 125 blocks per second.

7. With double buffering, a process computes longer on each block than the device needs to read one (C > T). Which statement is true?

A. The device runs nonstop and the process sometimes waits for it
B. The process ends up waiting for I/O on every second block
C. The process never waits for I/O; the device is idle part of the time
D. Both buffers fill up, so the process eventually has to wait

Answer: C. The process never waits for I/O; the device is idle part of the time

When C > T the process is the bottleneck. The device keeps the buffers stocked and then waits for a free one, so the process always finds its next block ready.

8. Match each technique to what it does.

Line-at-a-time input	**The process is woken once per complete line**
Byte-at-a-time input	**Every keystroke is handed to the process at once**
Double buffering	**The OS fills one buffer while the process empties the other**
Circular buffering	**A ring of more than two buffers run as a bounded buffer**

Line and byte modes are the two ways to buffer a stream; double and circular buffering add buffers so more device work can overlap with process work.

9. A program reads commands from a scroll-mode terminal with line-at-a-time buffering. The user types `lss`, presses Backspace, then Enter. What does the program receive?

A. Four separate wake-ups: l, s, s and Backspace
B. The line lss followed by a Backspace code
C. Nothing until the OS buffer is completely full
D. One wake-up with the edited line ls

Answer: D. One wake-up with the edited line ls

In line-at-a-time mode the OS collects and edits the line in its buffer and wakes the process once, when Enter completes it.

10. With two line buffers for line-at-a-time output, a process must wait as soon as it gets one line ahead of the printer.

Answer: False

With two buffers it waits only when it gets two lines ahead: one line can be printing from one buffer while the next waits in the other.

11. Which statements about I/O buffering are true?

A. Input can be read ahead, before the process asks for it
B. Output can be written out after the process has moved on
C. Enough buffers let a process outpace a slower device forever
D. A system buffer lets the OS swap a process out during its I/O
E. Buffering makes each single transfer finish sooner

Answer: A, B, D

Buffering moves transfers earlier (input) or later (output) and frees the process's memory during I/O. It does not speed up the device, and no number of buffers keeps up with a steady overload.

12. On average, a process asks for blocks faster than the disk can deliver them. What does adding more circular buffers achieve?

A. Nothing lasting: the buffers drain and the process waits
B. The process stops waiting once there are enough buffers
C. The disk is pushed into transferring each block faster
D. The OS starts dropping blocks so it can keep up with demand

Answer: A. Nothing lasting: the buffers drain and the process waits

Buffers smooth out bursts. When the average demand is higher than the device's rate, every buffer empties and the process waits for each block the device delivers.

11.5 Disk Scheduling

You should be able to

- Break a disk access into seek time, rotational delay and transfer time, and compute each from the disk's parameters.
- Explain with numbers why reading a file sequentially can be hundreds of times faster than reading the same sectors at random.
- Trace FIFO, SSTF, SCAN, C-SCAN, LOOK and C-LOOK on a request queue and compute total and average seek lengths.
- Compare the policies that choose by requester (RSS, FIFO, PRI, LIFO) with those that choose by requested track.
- Explain starvation and arm stickiness, and how N-step-SCAN and FSCAN prevent them.

Disk scheduling

A hard disk spends most of each request getting its head into position, not reading. Many processes share one disk, so requests queue up and the operating system chooses the order; that choice, made mainly to cut arm travel, is **disk scheduling**.

Inside a hard disk

- **Platter**: a rigid disc with magnetic coating on both surfaces; several sit on one **spindle**, which spins them together at a constant speed (commonly 5,400 to 15,000 rpm).

- **Track**: one ring of data on one surface. **Sector**: a fixed-size slice of a track, the smallest unit read or written (classically 512 bytes, 4,096 on most newer drives).
- **Read/write head**: flies just above each surface. The **disk arm** (actuator) moves all heads together, so it can be at only one track position at a time.
- **Cylinder**: the tracks, one per surface, under the heads at one arm position; reading within one cylinder needs no arm movement.

Where the time goes

Before the mechanics start, a request may wait for the **device** (the disk is busy with other requests) and, on some systems, for a **channel** (a data path shared by several disks). Then:

- **Seek time** T_s: moving the arm to the track. Straight-line model $T_s = m \times n + s$ (n tracks crossed, m time per track, s startup and settling time). Real arms speed up, coast and settle, so this is an approximation; average seeks today take a few ms.
- **Rotational delay** (latency): waiting for the sector to come round, on average half a revolution: 1/(2r) for r revolutions per second. 7,500 rpm = 125 rev/s, so one turn takes 8 ms and the average delay is 4 ms; at 15,000 rpm it is 2 ms.
- **Transfer time** $T = b / (r \times N)$: b bytes to move, N bytes per track.

Access time is seek time plus rotational delay. The whole operation takes on average $\mathbf{T_a = T_s + 1/(2r) + b/(rN)}$, plus any queueing. Worked example: crossing 100 tracks with m = 0.03 ms and s = 1 ms gives T_s = 4 ms; at 7,500 rpm with 500 sectors of 512 bytes per track, one sector transfers in 8/500 = 0.016 ms, so T_a = 4 + 4 + 0.016 = 8.016 ms. Positioning is 99.8% of it.

Sequential versus random: the order matters

Disk with 4 ms average seek, 7,500 rpm, 500 sectors of 512 bytes per track; file of 2,500 sectors (1.28 MB).

- **Sequential** (5 neighbouring full tracks): first track 4 + 4 + 8 = 16 ms; each of the other 4 tracks 4 + 8 = 12 ms (no further seek assumed). Total 64 ms.
- **Random** (sectors scattered): each sector 4 + 4 + 0.016 = 8.016 ms; × 2,500 = 20,040 ms ≈ 20.04 s.

Same disk, same bytes, about 313 times slower: only the visiting order differs.

Policies that choose by requester

- **RSS (random scheduling)**: a random waiting request. Never used for real; it is the benchmark other policies are judged against.
- **FIFO**: arrival order. Fair: nothing starves. It ignores the arm: excellent for one process reading a file in order, but with many processes taking turns almost every request is a long jump, and it performs like random order or worse.
- **PRI (priority)**: the importance of the requesting process decides, set outside the disk manager (for example short batch jobs and interactive users first). It does nothing for arm travel, low-priority work can wait a long time, and it suits database systems poorly.
- **LIFO**: newest request first. In transaction processing the newest request usually comes from the process just served and lies next to its previous one, so the arm barely moves. Old requests can be pushed back indefinitely: starvation.

Policies that choose by requested track

- **SSTF** (shortest service time first): the waiting request nearest the arm. Small seeks and short queues, but a far request can starve while closer ones keep arriving. Always taking the nearest is greedy and not always optimal.
- **SCAN** (elevator): sweep one way serving every request reached, to the last track, then reverse. It evens out service compared with SSTF, but it is biased against the region it just swept: requests just behind the arm wait for the return trip, while new arrivals just ahead are served at once (so the latest arrivals are favoured).
- **Edge tracks under SCAN**: you may read that SCAN favours the innermost and outermost tracks. True part: an edge track is passed twice in quick succession (just before and after the turn), then not again for nearly a full round trip, so a request arriving just before the turn is served almost at once and waits there vary the most. Averaged over many arrivals, though, edge tracks wait **longest**: most arrivals land in the long gap. With a steady sweep over tracks 0 to L, a request for track x arriving at a random moment waits on average $(x^2 + (L - x)^2) / L$ track-times: L/2 in the middle, L (twice as long) at the edges. With L = 199, track 20 waits 163, track 100 waits 99.5.
- **C-SCAN**: serve in one direction only; at the end return to the far end without serving and sweep again. Every track is passed once per cycle at even intervals, so every track has the same average wait (half a cycle).
- **LOOK** and **C-LOOK**: the usual refinements that turn at the last waiting request instead of the disk edge (C-LOOK jumps straight to the farthest waiting request). Many descriptions simply call these SCAN and C-SCAN.

For a batch that is all present at once, the least possible travel is a sweep to the nearer end of the requests and then to the other end.

The classic example

200 tracks (0 to 199), arm at track 100 moving toward higher tracks, queue in arrival order 55, 58, 39, 18, 90, 160, 150, 38, 184. Seek length = tracks crossed since the previous service, including any trip to the edge and any return (counted here).

Policy	Service order	Total	Average
FIFO	55, 58, 39, 18, 90, 160, 150, 38, 184	498	55.3
SSTF	90, 58, 55, 39, 38, 18, 150, 160, 184	248	27.6
SCAN	150, 160, 184, 90, 58, 55, 39, 38, 18	280	31.1
LOOK	150, 160, 184, 90, 58, 55, 39, 38, 18	250	27.8
C-SCAN	150, 160, 184, 18, 38, 39, 55, 58, 90	388	43.1
C-LOOK	150, 160, 184, 18, 38, 39, 55, 58, 90	322	35.8
LIFO	184, 38, 150, 160, 90, 18, 39, 58, 55	537	59.7

Which convention gives which number. FIFO (55.3) and SSTF (248 / 9 = 27.56, often printed as 27.5) do not depend on any convention. The widely printed 27.8 for SCAN and 35.8 for C-SCAN come from turning at the last request: they are the LOOK and C-LOOK rows above, with C-LOOK's jump from 184 back to 18 counted as 166 tracks. Running to the edge gives SCAN 31.1 and, counting the run to 199 and the return to

0, C-SCAN 43.1. Some sources leave the return trip out: C-SCAN then gives 189 tracks (21.0) and C-LOOK 156 (17.3). Random order averages about 69 here.

Starvation, arm stickiness, N-step-SCAN and FSCAN

Arm stickiness: one or a few processes sending requests to the same track so fast that SSTF, SCAN or C-SCAN keep the arm there (a request at the arm's own track always counts as nearest and as ahead), while everyone else waits.

- **N-step-SCAN**: cut the queue into subqueues of N requests; serve one subqueue at a time with SCAN; requests arriving meanwhile join later subqueues. N = 1 gives FIFO; a large N performs like SCAN. Every request is served within a bounded number of sweeps.
- **FSCAN**: two queues. When a sweep begins, all waiting requests are in one queue and the other is empty; new arrivals go to the other queue and wait for the next sweep.

Summary

Policy	How it chooses	Remarks
Selection according to the requester		
RSS	Random	Benchmark only
FIFO	Oldest first	Fairest; ignores the arm
PRI	Process priority	Control outside disk management
LIFO	Newest first	Uses locality; can starve old requests
Selection according to the requested item		
SSTF	Nearest track first	High utilization, short queues; starvation possible
SCAN	Sweep back and forth	Better spread of service than SSTF
C-SCAN	Sweep one way, return	Lowest variation in waiting time
N-step-SCAN	SCAN on N requests at a time	Service guarantee
FSCAN	SCAN on the queue frozen at sweep start	Responds to load; no stickiness

Real systems

Solid-state drives have no arm and no rotation, so reordering by position gains little there; Linux, for example, usually applies no reordering (its "none" scheduler) to fast NVMe drives. Section 11.9 follows how Linux's disk schedulers evolved.

Key terms

Track
: One ring of data on a disk surface. A surface holds many concentric tracks, numbered from the outer edge inward (or the reverse).

Sector
: A fixed-size slice of a track, the smallest unit the disk reads or writes: classically 512 bytes, on most newer drives 4,096 bytes.

Cylinder
All the tracks, one per surface, that sit under the read/write heads at one arm position. Data in one cylinder can be read without moving the arm.

Read/write head
The tiny sensor at the tip of the disk arm that reads and writes the magnetic data as the surface spins beneath it.

Disk arm
The moving arm (actuator) that carries the read/write heads across the surfaces to reach a chosen track. All heads move together.

Seek time
The time the disk arm needs to move the heads to the track that holds the wanted data.

Rotational delay (rotational latency)
The time spent waiting, once the head is on the right track, for the wanted sector to spin around under it. On average half a revolution.

Transfer time
The time for the wanted sectors to pass under the head while their data is read or written.

Access time
Seek time plus rotational delay: the time to get the head into position to read or write. Adding the transfer time gives the time for the whole operation.

Disk scheduling
The operating system's choice of which waiting disk request to serve next, mainly to cut the time the arm spends moving.

Random scheduling (RSS)
Picking the next disk request at random. Never used for real; it is the yardstick that other policies are measured against.

Last-in-first-out (LIFO)
Serving the newest request first. Good when a process's next request is near its last one; old requests can starve.

Shortest service time first (SSTF)
Serving the waiting request that needs the least arm movement from where the arm is now.

SCAN (elevator algorithm)
The arm sweeps in one direction, serving every request it reaches, until it reaches the last track (or the last request) in that direction, then reverses.

C-SCAN (circular SCAN)
The arm serves requests in one sweep direction only. At the end it returns to the far end without serving anything and sweeps the same way again.

LOOK
The usual refinement of SCAN: the arm reverses at the last waiting request in its direction instead of running on to the disk's edge.

C-LOOK

The same refinement of C-SCAN: after the last request in the sweep direction, the arm jumps straight back to the farthest waiting request at the other end.

N-step-SCAN

The queue is cut into subqueues of N requests. One subqueue at a time is served with SCAN, and new requests go into later subqueues.

FSCAN

Two queues: a sweep serves only the requests that were waiting when it began, while every new arrival waits in the other queue for the next sweep.

Arm stickiness

One or a few processes sending requests to the same track so fast that a nearest-first or sweeping policy keeps the arm there, while everyone else waits.

Review questions with answers

1. A disk spins at 15,000 rpm. What is its average rotational delay, in milliseconds?

Answer: 2 ms

15,000 rpm is 250 revolutions per second, so one turn takes 1/250 s = 4 ms. On average the sector is half a turn away: 1/(2r) = 2 ms.

2. A disk spins at 7,500 rpm and holds 500 sectors of 512 bytes on each track. Once the wanted sector starts to pass under the head, how long does reading that one sector take, in milliseconds?

Answer: 0.016 ms

One revolution takes 60,000 / 7,500 = 8 ms. One sector is 1/500 of the track, so it passes in 8 / 500 = 0.016 ms; the formula b/(rN) = 512 / (125 × 256,000) s gives the same.

3. The disk arm is at track 50. Requests arrive in this order: 80, 20, 60. With FIFO scheduling, how many tracks does the arm cross in total?

Answer: 130 tracks

FIFO serves them in arrival order: 50 → 80 is 30, 80 → 20 is 60, 20 → 60 is 40. Total 30 + 60 + 40 = 130 tracks.

4. The arm is at track 50. The queue holds requests for tracks 80, 20, 60, 45 and 90. Put them in the order SSTF serves them.

Correct order:

1. Track 45
2. Track 60
3. Track 80
4. Track 90
5. Track 20

SSTF always takes the nearest: 45 (5 away), then 60 (15), 80 (20), 90 (10), and only then the far request at 20 (70). Total 120 tracks.

5. A request for a distant track is waiting. Meanwhile new requests keep arriving for tracks a few tracks to either side of the arm. Which policy can leave the distant request waiting for as long as that stream continues?

A. FIFO
B. SSTF
C. C-SCAN
D. FSCAN

Answer: B. SSTF

SSTF always picks the request nearest the arm. A steady supply of nearby requests always beats the distant one, so it can starve.

6. Under C-SCAN, what does the arm do once it has served the last request in its sweep direction?

A. It reverses direction and serves requests on its way back to the start.
B. It stops at the edge and waits there until a new request turns up.
C. It returns to the other end without serving, then sweeps the same way.
D. It moves next to whichever waiting request is closest to where it is.

Answer: C. It returns to the other end without serving, then sweeps the same way.

C-SCAN serves in one direction only. It goes back to the far end without serving (C-LOOK jumps straight to the farthest waiting request) and sweeps again, which keeps waiting times even across tracks.

7. N-step-SCAN with N = 1 behaves exactly like which policy?

A. SCAN
B. SSTF
C. FSCAN
D. FIFO

Answer: D. FIFO

Each subqueue holds a single request, and subqueues are served in the order they were filled, which is arrival order: FIFO.

8. With priority (PRI) disk scheduling, what decides which request is served next?

A. The priority of the requesting process, set outside the disk manager
B. How far the arm must travel from its position to each request
C. The exact order in which the requests arrived in the disk queue
D. A random pick among all the requests that are currently waiting

Answer: A. The priority of the requesting process, set outside the disk manager

PRI takes the order from the importance of the requesting process, for example favouring short batch jobs and interactive users. The disk manager does not control it, so it does nothing for arm travel.

9. Does each policy choose by who made the request, or by which track it asks for?

By requester	By requested track
RSS FIFO PRI LIFO	SSTF SCAN C-SCAN N-step-SCAN FSCAN

RSS, FIFO, PRI and LIFO look only at the requester or arrival time. SSTF and the sweeping family look at where the requested data lies.

10. Match each policy to how it picks the next request.

LIFO	**Newest request first**
SSTF	**Least arm movement from the current position**
LOOK	**Sweeps, but reverses at the last request instead of the edge**
FSCAN	**New arrivals wait in a second queue until the sweep ends**
RSS	**Random order, used only as a benchmark**

Each name says how the choice is made: by recency (LIFO), by distance (SSTF), by sweep (LOOK), by frozen batch (FSCAN) or by chance (RSS).

11. Doubling a disk's rotation speed halves both its average rotational delay and the time to transfer a given number of bytes.

Answer: True

Both terms have the rotation speed r in the denominator: 1/(2r) and b/(rN). Seek time is not affected.

12. Which policies stop one process that keeps requesting the same track from holding the arm there (arm stickiness)?

A. SSTF
B. SCAN
C. N-step-SCAN
D. FSCAN
E. C-SCAN

Answer: C, D

N-step-SCAN and FSCAN serve a batch that is frozen when the sweep starts; new requests for the busy track wait for a later batch. SSTF, SCAN and C-SCAN accept a request at the arm's own track at once, so they can stick.

11.6 RAID

You should be able to

- Explain why arrays of disks raise both the data transfer capacity and the I/O request rate, and why they need redundancy.
- Map a logical strip to its disk and stripe, and describe striping (RAID 0) and mirroring (RAID 1).
- Compute XOR parity, regenerate a failed disk from it, and count the reads and writes of a small write.
- Explain how RAID 2 to 6 differ: parallel versus independent access, a dedicated versus distributed parity disk, single versus dual redundancy.
- Compare the levels by disks needed, usable capacity, failures survived and performance.

Why arrays of disks?

Processors have sped up thousands of times over the decades; a disk's moving arm and spinning platter only a few times. Using several disks at once helps in two ways: one **large request** split across many disks finishes sooner (higher **data transfer capacity**), and many **small requests** on different disks run at the same time (higher **I/O request rate**). More disks also means more failures, so arrays store **redundant** information.

RAID (redundant array of independent disks; the 1988 paper that coined the name said *inexpensive*) rests on three ideas:

1. The operating system sees the set of physical drives as **one logical drive**. A controller or driver maps each logical block to a disk and position.
2. Data is **striped**: spread across the drives in round-robin order.
3. **Redundant capacity** holds a mirror copy or check data (parity) so data can be recovered after a disk failure. RAID 0 is the one level without this.

RAID 0: striping, no redundancy

The space is cut into equal **strips** (one or more blocks or sectors). Strip 0 goes on disk 0, strip 1 on disk 1, and so on. A **stripe** is one row: one strip on every disk. With n disks, logical strip k lives on **disk k mod n** in **stripe ⌊k / n⌋**. Example: n = 4, strip 13 → disk 1, stripe 3.

- **Large requests** are fast when every disk delivers part of the data at once, if the path to memory (controller, buses) can carry all the streams. 12 strips on 4 disks take 3 strip-times instead of 12.
- **Small requests** are fast when strips are large enough that one request touches one disk and requests spread evenly (load balancing).
- No redundancy: losing any one disk loses data, so RAID 0 is *less* reliable than a single disk (roughly n times as likely to suffer a failure).

RAID 1: mirroring

Every strip is stored on two disks. A **read** can use either copy; the controller picks the one that can reach the data sooner (shorter seek plus rotational delay), and two reads can run at once. A **write** must update

both copies; the two writes run in parallel, but the write finishes only when the slower disk does. **Recovery** is simple: copy the twin to a new disk. **Cost:** 2N disks for N disks of data (50% usable). Good for system software and highly critical files.

RAID 2 and RAID 3: parallel access

In a **parallel access array** every disk takes part in every request: the spindles (the motors that spin the platters) are synchronized so all heads are at the same position, and strips are tiny (a byte or a word).

- **RAID 2** stores a **Hamming code** computed across corresponding bits on extra check disks. A single-error-correcting code with m check disks must name any one of the N + m disks or "no error", so it needs the smallest m with $2^m \geq N + m + 1$: 3 check disks for 4 data disks, 4 for 10. The number grows with the logarithm of N. It helps only where disk errors are frequent; modern drives correct bit errors internally, so RAID 2 is essentially unused.
- **RAID 3** needs a single parity disk however large the array, because a failed disk is known: parity only has to supply the missing bits, not locate them.
- Both give a very high transfer rate for large requests, but the array serves only one request at a time, so small-request rate is poor.

Parity with XOR

Exclusive-OR (⊕) gives 1 when two bits differ and 0 when they are equal; across many bits it gives 1 when the number of 1s is odd. With four data disks, **P = X3 ⊕ X2 ⊕ X1 ⊕ X0**, bit by bit. Because a value XOR-ed with itself is 0, any one value equals the XOR of all the others, e.g. **X1 = P ⊕ X3 ⊕ X2 ⊕ X0**.

Worked example. X0 = 1011, X1 = 0110, X2 = 1100, X3 = 0011 gives P = 0010. If disk X2 fails: P ⊕ X0 ⊕ X1 ⊕ X3 = 0010 ⊕ 1011 ⊕ 0110 ⊕ 0011 = 1100, exactly the lost value. Parity can rebuild only one missing strip per stripe, and only because the failed disk is known.

RAID 4: block-level parity and the small-write penalty

RAID 4, 5 and 6 are **independent access arrays**: strips are relatively large, so each disk can serve a different request at the same time, which suits high request rates. RAID 4 keeps a parity strip for each stripe on one dedicated parity disk.

Writing a single strip (a **small write**) takes four operations: read the old data strip, read the old parity strip, compute $\mathbf{P' = P \oplus X_{old} \oplus X_{new}}$, then write the new data and the new parity. That is **2 reads + 2 writes**, the **small-write penalty**. Example: P = 0010, X1 changes from 0110 to 1111, so P′ = 0010 ⊕ 0110 ⊕ 1111 = 1011 (recomputing from all four strips agrees). A write that fills a whole stripe needs no reads; its parity comes from the new data alone. Because every write needs the parity disk, that disk becomes a **bottleneck**: under random small writes it performs half of all the array's I/O operations.

RAID 5: distributed parity

RAID 5 is organized like RAID 4 but rotates the parity strips round-robin over all disks: for n disks, the parity strip is on a different disk for each of the first n stripes, then the pattern repeats. Writes to different stripes

can then use different disks, removing the parity-disk bottleneck. It still needs N + 1 disks and 2 reads + 2 writes per small write, and survives the loss of any one disk.

RAID 6: dual redundancy

RAID 6 stores two different check values per stripe on two different disks: **P**, ordinary XOR parity, and **Q**, an independent check computed with different arithmetic (commonly a Reed–Solomon code). Two independent equations can solve for two missing strips, so **any two disks may fail**. N disks of data need **N + 2** disks, and a small write must update data, P and Q: **3 reads + 3 writes**, a heavier write penalty than RAID 5. It matters because a rebuild of a large disk can take hours; during that time the array runs in **degraded mode** (missing data recomputed on every read). A second disk failure in that window loses data on a RAID 5 array, and a single unreadable sector on a surviving disk loses the stripe it belongs to; RAID 6 can still recover in both cases.

Comparison

Level	Disks for N data	Usable with n disks	Failures survived	Large transfers	Small requests	One small write
0	N	100%	0	Very high	Very high	1 write
1	2N	50%	1 per pair	Reads above one disk; writes like one disk	Reads up to 2× one disk; writes like one disk	2 writes
2	N + m	N / (N + m)	1	Highest	Low (one at a time)	All disks
3	N + 1	(n – 1)/n	1	Highest	Low (one at a time)	All disks
4	N + 1	(n – 1)/n	1	Reads near RAID 0; writes limited	Reads near RAID 0; writes poor (parity disk)	2 reads + 2 writes
5	N + 1	(n – 1)/n	1	Reads near RAID 0; writes pay for parity	Reads near RAID 0; writes cost 4 I/Os, spread out	2 reads + 2 writes
6	N + 2	(n – 2)/n	Any 2	Reads near RAID 0; writes pay for P and Q	Reads near RAID 0; writes cost 6 I/Os	3 reads + 3 writes

Example: 6 disks of 4 TB give 24 TB (RAID 0), 20 TB (RAID 5), 16 TB (RAID 6) or 12 TB (RAID 1). In practice today: levels 0, 1, 5, 6 and combinations of levels, such as RAID 10 (data striped across several mirrored pairs); RAID 2 and 3 are historical.

RAID is not a backup: it protects against a disk dying, but a deleted file, malware or a fire reaches every disk at once.

Key terms

RAID (redundant array of independent disks)
: A set of physical disk drives that the operating system sees as one logical drive. Data is spread across the drives and, at every level except RAID 0, extra redundant information lets the array survive a disk failure.

Logical drive
The single drive the operating system sees and uses, even though a controller or driver really spreads its blocks over several physical drives.

Strip
A fixed-size chunk of an array's data stored on one disk: one or more blocks or sectors in most levels, as little as a byte or a word in RAID 2 and 3.

Stripe
One row across the array: the strips that sit at the same position on every disk, including any parity strips in that row.

Striping
Laying consecutive strips of data round-robin across the disks of an array, so that several disks can work on the data at the same time.

Data transfer capacity
How many bytes per second an array can move for one large request. Striping raises it because several disks deliver parts of the same request at once.

I/O request rate
How many separate I/O requests an array can complete per second. It rises when independent requests land on different disks that work at the same time.

Mirroring
Keeping two identical copies of every strip on two different disks (RAID 1), so either copy can serve a read and one copy survives if a disk fails.

Exclusive-OR (XOR)
A bit operation, written ⊕, that gives 1 when its two input bits differ and 0 when they are equal. XOR-ing the same value twice cancels it: A ⊕ B ⊕ B = A.

Parity
A check value made by XOR-ing corresponding bits of the data strips. Together with all the other strips of its stripe it regenerates any one missing strip.

Hamming code
An error-correcting code whose extra check bits can locate and fix a single wrong bit. RAID 2 keeps these check bits on extra disks.

Parallel access array
An array (RAID 2 and RAID 3) in which every disk takes part in every request, with the spindles (the motors that spin the platters) synchronized so that all heads are at the same position.

Independent access array
An array (RAID 4, 5 and 6) in which each disk can serve a different request at the same time, which suits workloads made of many small requests.

Small-write penalty
The extra I/O a parity array needs when a write changes only one strip: read the old data and old parity, then write the new data and new parity (2 reads + 2 writes in RAID 4 and 5).

Distributed parity

Putting the parity strips of different stripes on different disks in rotation (RAID 5), so that no single disk has to take part in every write.

Dual redundancy

Storing two independent check values per stripe, P (ordinary XOR parity) and Q (a different calculation such as a Reed–Solomon code), on two different disks, as RAID 6 does, so any two disks can fail.

Degraded mode

How an array runs after a disk fails: it still answers every request, but it must recompute the missing data from the surviving disks, so it is slower and has less protection left.

Rebuild

Recreating every strip of a failed disk on a replacement disk by reading the surviving disks and recomputing the missing data.

Review questions with answers

1. A RAID 0 array has 4 disks, numbered from 0, with strips laid round-robin starting at disk 0. Where is logical strip 13?

A. Disk 3, stripe 1
B. Disk 1, stripe 3
C. Disk 1, stripe 4
D. Disk 0, stripe 3

Answer: B. Disk 1, stripe 3

With n disks, strip k lives on disk k mod n in stripe $\lfloor k / n \rfloor$: 13 mod 4 = 1 and $\lfloor 13 / 4 \rfloor = 3$.

2. A RAID 5 array is built from 6 disks of 4 TB each. How many terabytes are available for user data?

Answer: 20 TB

Parity takes one disk's worth of space (spread over all six disks), so usable space is (6 – 1) × 4 = 20 TB.

3. Four data disks and one parity disk hold 4-bit values: X0 = 1011, X1 = 0110, X3 = 0011 and parity P = 0010. Disk X2 fails. What did X2 hold?

A. 1100
B. 1110
C. 1111
D. 0010

Answer: A. 1100

X2 = P ⊕ X0 ⊕ X1 ⊕ X3 = 0010 ⊕ 1011 ⊕ 0110 ⊕ 0011 = 1100. XOR-ing P with the survivors cancels them and leaves the missing value.

4. In RAID 1, every read must be sent to both disks so the controller can compare the two copies.

Answer: False

Either copy can serve a read. The controller picks the disk that can reach the data sooner, and two different reads can run on the two disks at once. Only writes go to both disks.

5. In RAID 5, how many disk I/O operations (reads plus writes) does a write that changes only one data strip need?

Answer: 4 I/Os

Read the old data strip and the old parity strip, compute P′ = P ⊕ X_old ⊕ X_new, then write the new data and the new parity: 2 reads + 2 writes.

6. An array has 6 disks. Order these levels by usable capacity, from most to least.

Correct order:

1. RAID 0
2. RAID 5
3. RAID 6
4. RAID 1

With 6 disks: RAID 0 uses all 6 (100%), RAID 5 uses 5 (83%), RAID 6 uses 4 (67%), and RAID 1 uses 3 (50%).

7. Sort each level by how its disks serve requests.

Parallel access: every disk works on every request	**Independent access: disks can serve separate requests**
RAID 2 RAID 3	RAID 4 RAID 5 RAID 6

RAID 2 and 3 synchronize their disks and use tiny strips, so each request occupies the whole array. RAID 4, 5 and 6 use large strips, so different disks can work on different requests.

8. Match each level to the redundancy it stores.

RAID 0	**None at all**
RAID 1	**A full mirror copy of every strip**
RAID 2	**A Hamming code on several check disks**
RAID 4	**Parity strips on one dedicated disk**
RAID 5	**Parity strips rotated across all disks**
RAID 6	**Two independent checks, P and Q**

RAID 0 is pure striping; RAID 1 mirrors; RAID 2 uses an error-correcting code; RAID 4 and 5 both use XOR parity, on one disk or rotated; RAID 6 adds a second, independent check.

9. Why does RAID 3 need only one check disk, while RAID 2 needs several?

A. RAID 3 uses large strips, so far fewer bits need protecting
B. RAID 3 mirrors the data, so a check disk is rarely needed
C. The failed disk is known, so parity need not locate the error
D. RAID 3 only detects errors; it cannot rebuild a failed disk

Answer: C. The failed disk is known, so parity need not locate the error

A Hamming code must find which bit is wrong, which takes several check bits. A failed disk announces itself, so a single parity value is enough to recompute the missing bits.

10. Which statements about RAID 4 and RAID 5 are true?

A. Both need N + 1 disks for N disks of data
B. In RAID 4, every write involves the same parity disk
C. RAID 5 survives any two disk failures
D. A one-strip write in either level needs two reads and two writes
E. In RAID 5, some disks never hold any parity

Answer: A, B, D

Both levels add one disk of XOR parity and pay 2 reads + 2 writes per small write. RAID 4 keeps all parity on one disk (a write bottleneck); RAID 5 rotates parity so every disk holds some. Either survives only one failure.

11. A team needs an array that keeps all its data after **any two** disks fail, and it can accept slower writes. Which level fits?

A. RAID 5
B. RAID 1
C. RAID 0
D. RAID 6

Answer: D. RAID 6

RAID 6 stores two independent checks (P and Q) per stripe on different disks, so any two disks can fail. The price is N + 2 disks and 3 reads + 3 writes per small write.

11.7 Disk Cache

You should be able to

- Explain what a disk cache is, what happens on a hit and on a miss, and why locality makes it pay off.
- Compare the two ways of delivering a cached block to a process: copying it or passing a pointer to shared memory.
- Trace LRU as a stack of pointers and LFU with reference counts, and explain LFU's burst problem.
- Apply frequency-based replacement with two and three sections and explain what each section is for.
- Read a miss-ratio-versus-cache-size curve and explain why its shape depends on the workload.

What a disk cache is

A **disk cache** is a buffer in main memory that holds copies of some disk blocks (a block is one or more sectors, the unit the OS reads and writes). Every read request checks the cache first:

- **Hit:** the block is already in the cache, so it is delivered from main memory. The disk is not touched.
- **Miss:** the block is read from the disk into the cache, then delivered. The copy stays, in case it is wanted again.

It pays off because of **locality of reference**: programs keep coming back to the same blocks (a directory, an index, the file being edited), so a block used recently is likely to be used again soon. It is the same idea as the hardware cache memory of chapter 1, one level further down (between main memory and the disk) and done in software: OS code plus ordinary RAM.

Average time. With hit ratio H, hit time T1 and disk time T2, the average time per request is H × T1 + (1 – H) × (T1 + T2). Example: H = 0.9, T1 = 0.01 ms, T2 = 8 ms gives 0.9 × 0.01 + 0.1 × 8.01 = 0.009 + 0.801 = 0.81 ms, about ten times better than 8 ms per request without a cache. The **miss ratio** is misses ÷ requests = 1 – H.

Design issue 1: delivering the block

	Copy	Pointer (shared memory)
How	The OS copies the block from its cache slot into a buffer in the process's memory.	The OS maps the cache slot into the process's address space and returns its address.
Cost	One memory-to-memory copy per read; the block sits in memory twice.	No copy, one copy of the data.
Catch	None for the cache: it may reuse the slot at any time.	The slot is pinned (cannot be replaced) while the process uses it, and must be protected from the process's writes (read-only mapping or copy on write).

Real systems offer both: on Linux and other UNIX-like systems, read() copies cached file data into your buffer, while mmap() maps the cached pages into your address space. Unlike the classic pointer scheme, a mapped page need not stay pinned: the MMU lets the kernel take it back by unmapping it first, and the next touch faults it in again.

Writes need a policy too: **write-through** sends every change to the disk at once; **delayed write** keeps the changed (dirty) block in the cache and writes it later, when it is replaced or the cache is flushed. The UNIX buffer cache of section 11.8 uses delayed write.

Design issue 2: replacement

Least recently used (LRU)

The most common policy. The cache is viewed as an **LRU stack**: the most recently referenced block on top, the least recently referenced at the bottom. On a hit, the block moves to the top and those above it slide down one place. On a miss with a full cache, the bottom block is replaced and the new block goes on top. The stack is a list of **pointers** to cache slots; the blocks themselves never move in memory.

Example, 3 slots, requests 3 1 3 5 1 4 3 1: miss, miss, hit, miss, hit, miss (evicts 3), miss (evicts 5), hit. Three hits.

Least frequently used (LFU)

Keep a **reference count** for each cached block (a new block starts at 1; each hit adds 1) and evict the block with the smallest count. Weakness, the **burst problem**: a block referenced many times in a short burst (for example ten small records that sit in one block) gets a high count and stays long after it stops being useful, while newer blocks, starting at count 1, are evicted first. Example with 4 slots: block 9 read 10 times, then blocks 1, 2, 3, 4 in a loop three times. LRU scores 17 hits; LFU scores only 9, all from the burst, because block 9 (count 10) keeps one slot and blocks 1 to 4 take turns in the other three.

Frequency-based replacement (FBR)

1. Keep the LRU stack, split into a **new section** on top and an **old section** below.
2. A hit in the new section moves the block to the top but does **not** change its count, so a burst of quick repeats counts only once.
3. A hit in the old section adds 1 to the count and moves the block to the top.
4. On a miss, the block with the **smallest count in the old section** is replaced; if several share it, the least recently used one (lowest in the stack) goes. The new block enters on top with count 1.

On the burst example FBR keeps block 9's count at 1 and evicts it as soon as the loop needs the room: 17 hits, like LRU.

Three sections. In the two-section version, a block that has just slid out of the new section still has count 1, so it can be evicted at once if older blocks have higher counts. The refinement adds a **middle section** between new and old: hits there add 1, but only old-section blocks can be evicted, so newcomers get time to earn a count. Example with 4 slots: blocks 1, 2, 3 read four times each, then blocks 5, 6, 7 four times each. Two-section FBR keeps evicting the newcomer and scores 9 hits; three-section FBR scores 18.

Count aging. Counts that never shrink let a hot set that has gone cold hold its slots. The published FBR design ages its counts (halving them from time to time) so old popularity fades; the simple versions above leave that out. The aging switch in the miss-ratio explorer halves every count (rounding up) whenever the average count of the cached blocks passes 2.

Performance: miss ratio versus cache size

- The miss ratio generally falls as the cache grows, then flattens: a block read for the first time always misses, however big the cache.
- The shape depends on the workload. With no locality (every block equally likely), a cache holding k of N blocks hits about k/N of the time: a straight line, and all policies are alike. A loop over more blocks than the cache holds misses every time (each block is evicted just before it returns); once the loop fits, almost everything hits: a cliff.
- On a mix of hot blocks and one-time scans, counting policies beat LRU, because scans push hot blocks out of LRU's stack. When the hot set moves, counting without aging can lose to LRU; with aging, FBR leads again. When the hot set stays put, aging can cost a few points, because it forgets popularity that was still useful.
- Studies that replayed real disk traces found FBR somewhat better than LRU. No policy wins on every workload.

Key terms

Disk cache
A buffer in main memory that holds copies of some disk blocks. The OS looks there before going to the disk, so a block used again is delivered at memory speed.

Disk block
The fixed-size unit in which the OS reads and writes a disk, made of one or more sectors (a sector is the smallest piece the drive itself reads or writes).

Cache hit (hit)
A request that finds the wanted disk block already in the disk cache, so it is served from main memory with no disk access.

Cache miss (miss)
A request that does not find the wanted disk block in the disk cache, so the block is first read from the disk into the cache.

Hit ratio
The fraction of requests that are hits: hits ÷ requests.

Miss ratio
The fraction of requests that are misses: misses ÷ requests, which is 1 – hit ratio. Lower is better.

Locality of reference (locality)
The tendency of programs to keep reusing a small, slowly changing set of data, so a block used recently is likely to be used again soon.

Reference string
The sequence of block numbers requested, in order. Replacement policies are traced and compared on reference strings.

Least recently used (LRU)
The replacement policy that evicts the cached block that has gone unreferenced for the longest time.

LRU stack
The cached blocks ordered by last use, most recent on top and least recent at the bottom. It is kept as a list of pointers, so the blocks themselves never move.

Least frequently used (LFU)
The replacement policy that keeps a reference count for each cached block and evicts the block with the smallest count.

Reference count
A counter kept with each cached block that goes up when the block is referenced. LFU and FBR pick their victims with it.

Frequency-based replacement (FBR)
A replacement policy that keeps the LRU stack but splits it into sections, counts only hits outside the new section, and evicts the old-section block with the smallest count.

New section
In FBR, the top part of the LRU stack, holding the most recently used blocks. A hit here does not raise the block’s count, so quick repeats in a burst count only once.

Old section

In FBR, the bottom part of the LRU stack. Only blocks here may be replaced: the one with the smallest count goes, and LRU order breaks ties.

Middle section

In three-section FBR, the part of the stack between the new and old sections. Its blocks cannot be replaced, and a hit raises their count, so they get time to earn one.

Count aging

Shrinking every reference count from time to time, for example halving them all, so that popularity earned long ago fades away.

Write-through

A write policy in which every change to a cached block is also written to the disk at once.

Delayed write

A write policy in which a changed block stays in the cache, marked dirty, and is written to the disk later: when it is replaced or when the cache is flushed.

Review questions with answers

1. A process asks for disk block 812, and the OS finds a copy of block 812 in the disk cache. What happens next?

A. The OS reads block 812 from the disk anyway, to make sure the copy is current
B. The block is delivered from main memory, and the disk is not touched
C. The OS moves block 812 to the bottom of the LRU stack, marking it as used
D. The request waits in the disk queue until the arm reaches block 812

Answer: B. The block is delivered from main memory, and the disk is not touched

That is a hit: the block is delivered from main memory at memory speed, with no disk access. Only a miss reads from the disk.

2. A disk cache has a hit ratio of 0.9. A hit costs 0.01 ms. A miss costs a disk read of 8 ms plus the 0.01 ms to deliver the block from the cache. What is the average time per request, in milliseconds?

Answer: 0.81 ms

Average = H × T1 + (1 – H) × (T1 + T2) = 0.9 × 0.01 + 0.1 × 8.01 = 0.009 + 0.801 = 0.81 ms, about ten times faster than 8 ms per request with no cache.

3. Instead of copying a cached block into the process's buffer, the OS hands the process a pointer to the cache slot in shared memory. What is the trade-off?

A. It needs more memory, since the block is then stored once in the cache and once in the process
B. It lets the cache replace the slot whenever it likes, since the process holds only an address
C. It needs no protection at all, since the process can only ever read a block from the disk
D. It saves the copy, but the slot cannot be replaced while the process is still using it

Answer: D. It saves the copy, but the slot cannot be replaced while the process is still using it

Sharing removes a memory-to-memory copy and the duplicate data, but the slot is tied up (pinned) while the process uses it and must be protected from the process's writes.

4. A read request misses in a full disk cache that uses LRU. Put the steps in order.

Correct order:

1. Search the cache for the block and fail to find it
2. Choose the block at the bottom of the LRU stack as the victim
3. Read the requested block from the disk into the victim's slot
4. Deliver the block, now cached, to the process

A miss is detected first, then LRU picks the bottom block, its slot receives the new block from the disk (whose pointer goes on top of the stack), and finally the data reaches the process. The block cannot be delivered before it has been read.

5. An LRU disk cache has 3 slots and starts empty. The blocks requested are 3, 1, 3, 5, 1, 4, 3, 1. How many requests are hits?

Answer: 3

Miss 3, miss 1, hit 3, miss 5, hit 1, then 4 misses and evicts 3 (least recently used), so 3 misses and evicts 5, and the last 1 hits. Hits: 3.

6. Under LRU, a hit copies the block's data into the slot at the top of the cache.

Answer: False

The blocks stay where they are. LRU keeps a stack of pointers to the slots, and only the referenced block's pointer moves to the top.

7. Block 9 is read ten times in quick succession and then never again. Why might LFU keep it in the cache long afterwards?

A. LFU keeps the most recently used block on top, and block 9 never leaves the top
B. LFU never evicts any block that has been read more than once since it arrived
C. Its count reached 10, while blocks loaded later start at 1 and so look less valuable
D. The burst marked block 9 as dirty, and a dirty block cannot be evicted by LFU

Answer: C. Its count reached 10, while blocks loaded later start at 1 and so look less valuable

LFU counts never shrink, so a short burst leaves a high count behind. Newer blocks start at 1 and are evicted first, while the stale block stays: the burst problem.

8. In frequency-based replacement, a hit on a block in the new section does not increase the block's reference count.

Answer: True

That is FBR's key rule. Quick repeats land in the new section, so a burst counts only once; hits in the old (or middle) section do add 1.

9. An FBR cache has 5 slots: the new section is the top 2, the old section the bottom 3. From top to bottom the blocks and counts are 14 (1), 22 (1), 37 (3), 41 (2), 58 (2). A miss arrives. Which block is replaced?

A. Block 58
B. Block 22
C. Block 41
D. Block 37

Answer: A. Block 58

Only old-section blocks (37, 41, 58) are candidates. The smallest count there is 2 (41 and 58); the tie goes to the less recently used block, the lower one in the stack: 58.

10. Three-section FBR adds a middle section between the new and old sections. What does the middle section achieve?

A. It doubles the counts of the blocks inside it, so popular blocks are never replaced
B. It shields blocks that just left the new section from eviction while they earn a count
C. It stores the changed (dirty) blocks until they have been written back to the disk
D. It makes hits in the new section count after all, but only half as much as others

Answer: B. It shields blocks that just left the new section from eviction while they earn a count
A block leaving the new section still has count 1 and would be the first victim. Only old-section blocks can be evicted, so the middle section gives it time to collect hits there.

11. Sort each statement under the replacement policy it describes.

LRU	LFU	FBR
Evicts the block at the bottom of the stack Keeps no reference counts at all	Evicts the block with the smallest count anywhere in the cache A short burst can keep a dead block cached for a long time	Ignores hits on blocks in the new section when counting Chooses victims only from the old section

LRU uses recency alone (bottom of the stack goes). LFU uses counts alone, which a burst can inflate. FBR combines them: sections decide which hits count and which blocks may be evicted.

12. Which statements about miss ratio and cache size are true?

A. As the cache grows, the miss ratio generally falls
B. With no locality, a cache holding k of N equally likely blocks hits about k/N of the time
C. FBR has a lower miss ratio than LRU on every possible workload
D. A block requested for the very first time misses however big the cache is
E. Once the cache can hold half of the blocks a trace uses, the miss ratio is always close to zero

Answer: A, B, D
Bigger caches generally miss less, and first-time reads always miss. Without locality the hit ratio is just the fraction of blocks that fit. No policy wins everywhere, and a loop just larger than the cache can miss on every request, however close it is to fitting.

11.8 UNIX SVR4 I/O

You should be able to

- Explain how UNIX gives each device a special file and why one uniform interface helps every program.
- Trace a block request through the buffer cache: hash lookup, hit or miss, the LRU free list and delayed writes.
- Describe character queues as producer/consumer buffers in which each character can be read only once.
- Compare buffered and unbuffered I/O: copies made, speed, and the cost of locking a process in memory.
- Say which kinds of I/O disks, tapes, terminals, communication lines and printers use, and why.

One interface for every device

In UNIX every I/O device is represented by a **special file** (device special file), a name such as `/dev/term/a` that lives in the file system beside ordinary files. A program opens, reads, writes and closes it with exactly the same system calls it uses for a text file, and the kernel passes the request to the right device driver. Because devices look like files, every file tool and every shell redirection already works with every device, and the normal access bits decide who may use a device.

- **Type.** `ls -l` shows **b** for a **block special file** (used a block at a time through the buffer cache, e.g. a disk partition) or **c** for a **character special file** (reached without the buffer cache: terminals, printers, and the raw interfaces of disks and tapes).
- **Major device number:** picks the driver (an index into the kernel's table of block or character drivers).
- **Minor device number:** handed to the driver to say which unit is meant (which partition, which terminal line).
- **Raw device:** the character special file through which a disk or tape is used unbuffered, traditionally named with an r (`/dev/rdsk/...`). The same partition can therefore appear twice: once as a block file, once as a raw file.
- A special file only needs a driver, not hardware: `/dev/null` discards writes and reports end of file on reads.

Under the file interface the kernel moves data in one of two ways. **Buffered I/O** passes through kernel buffers: the **buffer cache** (for block devices) or **character queues** (for character devices). **Unbuffered I/O** uses DMA to move data straight between the device and the process's own memory.

The buffer cache

The buffer cache is a pool of main-memory buffers, each able to hold a copy of one disk block; it is essentially a disk cache. All I/O to block devices goes through it. Each buffer has a **buffer header**: device number, block number, status flags (busy, valid, delayed write, I/O in progress), a pointer to the data area, and links for its hash chain and for the free list.

Three lists

- **Free list:** buffers not in use right now, in least-recently-used (LRU) order. A released buffer joins the **tail**; a buffer needed for a new block is taken from the **head**. Free buffers keep their data and can still be found (hit) until they are taken.
- **Device list:** for each disk, every buffer currently associated with it.
- **Driver I/O queue:** for each device, the buffers being transferred or waiting to be transferred.

One buffer can be on several lists at once (a free buffer holding a valid block is on the free list, its hash chain and its disk's device list).

Finding a block: hashing

The key is the pair (device number, block number). A hash function maps it to a slot of a hash table, and buffers whose pairs land in the same slot are linked in a **hash chain** (separate chaining). Only that chain is searched, and each buffer is compared on both numbers, because different pairs can share a slot. Example with slot = (device + block) mod 4: block 21 of device 0 gives 21 mod 4 = 1; block 18 of device 1 gives 19 mod 4 = 3; block 4 of device 0 and block 7 of device 1 both land in slot 0.

Reads and writes

1. **Read hit:** the buffer is found; it is taken off the free list, the bytes are copied to the process, and it is released to the tail.
2. **Read miss:** the clean buffer nearest the head of the free list is taken, moved from its old hash chain to the new one, put on the driver I/O queue and filled from disk by DMA; then the data is copied to the process and the buffer goes to the tail.
3. **Write:** the buffer is found or obtained the same way (a partial-block write reads the block first), the process's bytes are copied in, and the buffer is marked **delayed write** (dirty). The process continues at once.
4. **Delayed write:** the disk is updated later, when the buffer reaches the head of the free list and is about to be reused (the kernel starts its write, takes the next clean buffer, and the written buffer returns to the head when the write finishes), or when **sync** flushes every dirty buffer (classic UNIX ran sync about every 30 seconds). A crash before then loses the change; programs that cannot afford that call `fsync`.

Worked trace (free list b5 empty, b1 = disk 0 block 9 delayed write, then b3, b0, b4, b2): reading d0:12 hits b3; reading d1:7 misses and uses the empty b5; reading d0:21 misses, finds b1 dirty at the head (its write starts) and reuses b0, evicting d0:4; a later read of d0:4 is therefore a miss.

The device fills a buffer by DMA. The copy between buffer and process is memory to memory, and UNIX kernels let the processor make it (routines such as copyin and copyout); some descriptions say DMA does that copy too. History: this design is the classic UNIX buffer cache of the 1970s and 1980s (up to System V Release 3; BSD later split the free list into several lists). SVR4 caches ordinary file data in virtual-memory pages instead and keeps the buffer cache mainly for file-system metadata, such as the on-disk records that describe each file (inodes, chapter 12), and Linux (2.4 onward) merged its buffer cache into one page cache.

Character queues

Character devices (terminals, communication lines, printers) use **character queues**: first-in, first-out queues of characters organized as a producer/consumer problem. Either the device produces and the process consumes (keyboard input), or the process produces and the device consumes (printer or screen output). Each character can be read only once: reading removes it, unlike a buffer-cache block, which stays for the next reader. A reader of an empty queue blocks; a writer facing a full output queue blocks until the device makes room; input that arrives when the queue is full is lost. In normal line mode a terminal driver hands over a whole line after Enter. Classic UNIX built these queues from linked small blocks of characters (clists); SVR4 moved terminal drivers onto STREAMS with the same idea.

Unbuffered I/O

DMA moves data directly between the device and the process's I/O area: the fastest method, with one move and no copy. The costs: the process is **locked in main memory** and cannot be swapped out while the transfer is in progress (the device is writing into its frames), and the device is **tied to that process** for the whole transfer, unavailable to others. Example: reading 64 KB buffered moves 64 KB device → buffers plus 64 KB buffers → process = 128 KB of movement; unbuffered moves 64 KB once. Typical users: swapping, tape backups, databases that keep their own buffers (on Linux today, the `O_DIRECT` open flag).

	Buffered	Unbuffered
Data moves	two (device ↔ buffer ↔ process)	one (device ↔ process)
Process during transfer	may be swapped out	locked in memory
Device	shared, block by block	tied to the process
Later reads	may hit the cache	nothing cached

Which device uses which I/O

Device	Buffer cache	Character queue	Unbuffered
Disk drive	✓		✓
Tape drive	✓		✓
Terminal		✓	
Communication line		✓	
Printer		✓	✓

Pattern: devices that store numbered blocks gain from caching; devices that only stream characters cannot (a keystroke is never read twice), so they get character queues; unbuffered I/O is the fast lane for large transfers.

Key terms

Special file (device special file)
A name in the file system, usually under /dev, that stands for an I/O device. Programs open, read, write and close it with the same calls they use for ordinary files, and the kernel hands the work to that device's driver.

Block special file
A special file of type b. Its device is used a block at a time through the buffer cache; a disk partition is the usual example.

Character special file
A special file of type c. Its driver is reached without the buffer cache: terminals and printers go through character queues, and raw disks and tapes do unbuffered transfers.

Major device number
The number in a special file that picks the driver. The kernel uses it as an index into its table of block drivers or of character drivers.

Minor device number
The number in a special file that is handed to the driver so it knows which of its units is meant, such as which disk partition or which terminal line.

Buffered I/O
I/O whose data passes through buffers owned by the kernel (the buffer cache or character queues) on its way between a device and a process.

Unbuffered I/O
I/O in which DMA moves data straight between the device and the process's own memory, with no kernel buffer in between. It is the fastest path, but the process must stay locked in main memory until the transfer ends.

Raw device
The character special file through which a disk or a tape is used for unbuffered I/O, bypassing the buffer cache. Traditionally its name carries an r, as in /dev/rdsk/....

Buffer cache
UNIX's pool of main-memory buffers, each holding a copy of one disk block. Block-device I/O passes through it, so a request for a block that is already there needs no disk access.

Buffer header
The small record that describes one buffer: which device and block it holds, status flags (busy, valid, delayed write), where its data area is, and its links on a hash chain and on the free list.

Free list
The list of buffers not in use at the moment, kept in least-recently-used order. A released buffer joins the tail; a buffer needed for a new block is taken from the head.

Device list
For each disk, the list of all the buffers currently associated with that disk.

Driver I/O queue
For each device, the list of buffers that are being transferred, or are waiting their turn to be transferred, by that device's driver.

Hash chain
A linked list of the buffers whose (device, block) pairs hash to the same table slot. To find a block, the kernel searches only that one short chain.

Delayed write
Leaving a modified buffer in the cache, marked dirty, and writing it to the disk later: when the buffer is about to be reused for another block, or when the cache is flushed.

Character queue
A first-in, first-out queue of characters between a character device and a process. One side puts characters in, the other takes them out, and each character can be taken out only once.

sync
The UNIX operation that writes every delayed-write buffer out to its disk. Classic systems ran it about every 30 seconds from a small background process.

Review questions with answers

1. In a UNIX special file, what does the **major** device number tell the kernel?

A. Which unit of the device, such as which partition, is meant
B. How many bytes the device can move in a single transfer
C. Which driver should handle requests for this file
D. Whether the file is a block or a character special file

Answer: C. Which driver should handle requests for this file

The major number indexes the kernel's table of block drivers or of character drivers, so it picks the driver. The minor number is passed to that driver to say which unit is meant.

2. A UNIX program needs special system calls, different from `read` and `write`, to send output to a printer.

Answer: False

The printer has a special file, so the ordinary open, write and close calls work on it; the kernel routes them to the printer's driver. That uniform interface is the whole point of special files.

3. A buffer cache finds blocks with slot = (device number + block number) mod 4. In which slot is block 18 of device 1 found?

Answer: 3

1 + 18 = 19, and 19 mod 4 = 3 (19 = 4 × 4 + 3). The kernel then searches only slot 3's hash chain, comparing both numbers in each buffer.

4. A buffer cache's free list, from head to tail, is b4 (marked delayed write), b2, b7, b1; the last three are clean. A read misses. Which buffer receives the new block?

A. b4
B. b2
C. b7
D. b1

Answer: B. b2

Allocation takes the least recently used buffer from the head, but a delayed-write buffer must reach the disk before it can be reused. The kernel starts b4's write, moves on, and takes b2.

5. Put the handling of a buffer-cache **read miss** in order.

Correct order:

1. Hash (device, block) and search that slot's chain
2. Find no matching buffer: a miss
3. Take the clean buffer nearest the head of the free list
4. Move it from its old hash chain to the new one
5. The driver reads the block into the buffer by DMA
6. Copy the data to the process and release the buffer to the tail of the free list

Search first; only a miss needs a buffer. The LRU victim loses its old identity, takes the new one, is filled from disk, and after the copy goes to the tail as the most recently used.

6. Match each buffer-cache structure to what it holds.

Free list	**Buffers not in use, in least-recently-used order**
Device list	**Every buffer currently tied to one disk**
Driver I/O queue	**Buffers being transferred or waiting for a transfer**
Hash chain	**Buffers whose (device, block) pairs map to the same slot**

One buffer can be on several of these at once: a free buffer holding a valid block is on the free list, its hash chain and its disk's device list together.

7. Sort each device by the kinds of I/O classic UNIX uses for it.

Buffer cache and unbuffered	Character queues	Character queue or unbuffered
Disk drive Tape drive	Terminal Communication line	Printer

Block devices (disks, tapes) can use the buffer cache and also unbuffered transfers. Terminals and communication lines stream characters, so they use character queues. Printers use a character queue or unbuffered transfers.

8. Which statements about **unbuffered** I/O in UNIX are true?

A. DMA moves data directly between the device and the process's memory
B. The process cannot be swapped out until the transfer ends
C. The data is first copied into a kernel buffer, then into the process
D. The device is tied to that process for the whole transfer
E. A second read of the same data is served from the buffer cache

Answer: A, B, D

Unbuffered I/O skips kernel buffers entirely: one DMA move, so the process must stay in memory and the device works for it alone. With no buffer, nothing is left in the cache for a later read.

9. Each character in a UNIX character queue can be read only once: reading it removes it from the queue.

Answer: True

A character queue is a producer/consumer queue, and the consumer takes each character out as it reads it. A block in the buffer cache, by contrast, stays there and can be read again.

10. Why must a process doing unbuffered I/O stay in main memory until its transfer ends?

A. The kernel still has to copy the data from its buffer into the process
B. Swapping is never allowed while any I/O device is busy
C. The process holds the device driver and must release it before leaving
D. DMA is writing into its page frames, so they cannot be handed on

Answer: D. DMA is writing into its page frames, so they cannot be handed on

The device's DMA writes into the process's own frames. If the process were swapped out and its frames given to another process, the transfer would overwrite that other process's memory.

11. In the classic UNIX buffer cache, when is a buffer marked **delayed write** copied to the disk?

A. When the buffer is about to be reused, or when sync flushes it
B. Immediately, before the write call returns to the process
C. Only when the process that changed the block finally exits
D. Only at shutdown; until then the newest copy lives just in memory

Answer: A. When the buffer is about to be reused, or when sync flushes it

A delayed-write buffer reaches the disk when it gets to the head of the free list and must be reused, or when sync writes out every dirty buffer.

12. A process reads 64 KB through the buffer cache: the disk fills kernel buffers, then the kernel copies the data into the process. How many KB of data movement does this take in total?

Answer: 128 KB

Each byte moves twice: 64 KB from the device into the buffers plus 64 KB from the buffers into the process = 128 KB. An unbuffered read moves the 64 KB once.

11.9 Linux I/O

You should be able to

- Trace how the classic Linux 2.4 elevator merges, sorts or appends each new request, and name its two weaknesses.
- Explain how the deadline scheduler's FIFO queues and expiry times bound waiting, and why reads get the shorter deadline.
- Explain when the anticipatory scheduler saves seeks and when its pause wastes time, and how CFQ shares the disk fairly among processes.
- Place each Linux I/O scheduler in its kernel versions, and name today's blk-mq choices: mq-deadline, BFQ, Kyber and none.
- Describe the unified page cache and the two triggers that send dirty pages to the disk.

Linux I/O: two stops before the disk

A file read or write in Linux passes two kernel layers. The **page cache** answers reads from memory when it can and absorbs writes as **dirty pages**. Requests that really need the device wait in a queue, and the **I/O scheduler** (Linux calls it the **elevator**) decides their order.

Version	What changed
2.4 (2001)	The Linux elevator; during 2.4 the buffer cache is folded into one unified page cache
2.6.0 (2003)	Pluggable schedulers: deadline, anticipatory (default), noop; writeback by pdflush threads
2.6.18 (2006)	CFQ (added in 2.6.6) becomes the default
2.6.32 (2009)	One flusher thread per storage device replaces pdflush
2.6.33 (2010)	Anticipatory scheduler removed
3.13 (2014)	Multi-queue block layer (blk-mq)
4.11 / 4.12 (2017)	mq-deadline (4.11), BFQ and Kyber (4.12) for blk-mq
5.0 (2019)	Single-queue layer deleted, with noop, deadline and CFQ

The Linux 2.4 elevator

One request queue, kept in the order the head will sweep it: sectors at or above the head in ascending order, then (after the sweep wraps) the sectors below the head. Each new request goes through four checks, in order:

1. **Merge.** If a pending request of the same direction (read or write) covers the sectors right next to the new one, the two are joined into one request: a *back merge* if the new request continues the old one, a *front merge* if it comes just before it. Example: pending write 120–127 plus new write 128–135 gives one write 120–135.
2. **Age guard.** If some pending request has waited too long, the newcomer is added at the tail, so nobody passes the old request again. The 2.4 code measured age not by the clock but by how many sectors of newer work had already been allowed to jump ahead of a request, with a smaller limit for reads than for writes (the lab simply counts jumps, with a limit of 3). (The real code still sorted the newcomer among the requests behind the old one rather than always using the very tail.)
3. **Sorted insertion** at the request's place in the sweep. Every request behind it has been jumped once more.
4. **Append at the tail** if no place in the sweep fits (for example, a request behind the head belongs to the next sweep).

Weaknesses: requests far from the head can wait a long time (the guard is crude), and reads share one sorted queue with writes with no real priority (only a smaller jump limit). Yet most reads are **synchronous requests**: the process is blocked until the data arrives. Most writes are asynchronous: they sit in the page cache and the process keeps running.

The deadline scheduler (2.6)

Three queues. Every request is in the **sorted queue** and in either the **read FIFO** or the **write FIFO**. Default **expiry times**: 500 ms for reads, 5 s for writes. Before each dispatch: if the oldest request of a FIFO has expired, it is served next and the sweep carries on from its sector; otherwise the next request in sector order is served. Reads get the shorter clock because programs wait for them. (The real code keeps separate sorted lists for reads and writes, dispatches in batches of up to 16 requests, checks the FIFOs only when a batch starts, and prefers reads but never passes over waiting writes more than twice in a row.)

Worked example (the lab): every request takes 40 ms; a backup job keeps four writes pending ahead of the head; read R1 at sector 40 arrives at t = 0, behind the head. With the plain elevator new writes keep landing ahead of the head, so R1 waits 920 ms. With the deadline scheduler R1 expires at 500 ms and is served at the next dispatch, t = 520 ms. A read that arrives at 1.2 s expires at 1.7 s.

The anticipatory scheduler (2.6.0 to 2.6.32)

A sequential reader issues a read, waits for it, computes briefly (its **think time**), then asks for the next block. In that gap its queue is empty, so a scheduler may wrongly conclude it is finished (**deceptive idleness**) and seek away, only to seek back a moment later. The anticipatory scheduler, built on the deadline scheduler and the default from 2.6.0 to 2.6.17, keeps the disk idle for up to about 6 ms after a read, waiting for a nearby request from the same process. The bet wins when think time is shorter than the wait and loses when it is longer, so the real scheduler kept per-process statistics (average think time and seek distance) and waited only when it was likely to pay. It was removed in 2.6.33; CFQ, and later BFQ, keep a similar short idle wait.

Worked example (the lab): A reads 6 blocks (2 ms each, think time 2 ms); B has 6 writes far away; a long seek costs 8 ms; a waiting read always goes first. Without anticipation every read costs a seek away and a seek back: 11 long seeks, A done at 102 ms, B at 112 ms. Waiting up to 6 ms: 1 long seek, 16 ms of idle disk, A done at 22 ms and even B done at 48 ms. With an 8 ms think time every wait times out: still 11 seeks plus 36 ms of wasted idling.

Completely Fair Queuing (CFQ)

Added in 2.6.6, the default from 2.6.18 until its removal in 5.0. Each process's synchronous requests go into its own queue; asynchronous writes share a few queues, one per priority level. The queues are served in round robin; each turn is a time slice of the disk (100 ms by default for synchronous queues), and within a slice requests are served in sector order. The **I/O priority** (0 highest to 7 lowest, set with *ionice*) scales the slice: slice = base + base/5 × (4 – priority), so with a 100 ms base, priority 0 gets 180 ms and priority 7 gets 40 ms. So a process that floods the disk cannot shut out a light user. With n busy processes and slice q, a process waits (n – 1) × q between turns plus the switching seeks; for 4 processes and 100 ms that is 300 ms. Longer slices: fewer seeks, longer waits.

Linux today: blk-mq

Solid-state drives have no arm and complete hundreds of thousands of requests per second, so one queue behind one lock became a bottleneck. The **multi-queue block layer (blk-mq)**, added in 3.13, gives each CPU its own software queue feeding one or more hardware queues. Since 5.0 it is the only block layer. The choices are:

- **mq-deadline**: sector order plus read and write expiry; the default for devices with a single hardware queue (such as SATA disks).
- **BFQ (Budget Fair Queueing)**: per-process budgets of sectors and weights; favours interactive and soft real-time programs; the most processor time per request.
- **Kyber**: limits requests in flight so reads and synchronous writes meet target latencies; very light.
- **none**: no reordering; the default for devices with many hardware queues, such as NVMe drives (SSDs attached straight to the PCI Express bus).

The active scheduler is shown in brackets in /sys/block/<device>/queue/scheduler. mq-deadline is a rewrite; the original deadline scheduler was deleted in 5.0.

The page cache and writeback

Older kernels had a page cache for file pages and a separate buffer cache for disk blocks, so data could be cached twice; during the 2.4 series they became one unified **page cache**. Delaying writes has two main benefits: dirty pages can later be written together, sorted by sector, in an order that suits the disk; and recently used pages can be used again from memory. A page changed many times may reach the disk only once. The cost: dirty data is lost in a power failure.

Writeback is done by **flusher threads** (a pool of pdflush threads in 2.6; one flusher per storage device from 2.6.32; later kernels run that work on general kernel worker threads). Dirty pages are written when:

- **free memory falls below a threshold**: clean pages can be freed immediately, so writing dirty pages back makes their frames reclaimable (current kernels state this trigger as a share of memory: background writeback starts when dirty pages exceed 10% of available memory by default);
- **a page has been dirty longer than an age limit**: 30 s by default, checked by a flusher that wakes every 5 s.

Worked example: a page first dirtied at t = 12 s becomes due at 42 s and is written at the next wake-up, 45 s. A short age limit means more disk writes but less data at risk; a long one, the reverse.

Key terms

I/O scheduler (elevator)
The part of the kernel that decides in which order pending disk requests are sent to the device. Linux calls it the elevator, because the classic design sweeps across the disk like a lift moving between floors.

Linux elevator
The I/O scheduler of Linux 2.4: one request queue per disk, kept in sector order, with request merging and a crude age limit against starvation.

Request merging
Joining a new request onto a pending request of the same direction (read or write) for the sectors right next to it, so the disk handles both in one operation. A back merge adds to the end of the pending request, a front merge to its start.

Deadline scheduler
A Linux 2.6 I/O scheduler that keeps the sorted queue but also puts every request in a read FIFO or a write FIFO with an expiry time, so no request waits much past its deadline.

Expiry time
How long the deadline scheduler lets a request wait before it is served out of sector order. The Linux defaults are 500 ms for a read and 5 s for a write.

Synchronous request
A request the issuing process must wait for before it can go on, such as most reads. An asynchronous request, such as a typical write, completes in the background while the process keeps running.

Anticipatory scheduler

A Linux 2.6 I/O scheduler built on the deadline scheduler. After a read finishes it may keep the disk idle for a few milliseconds (about 6 ms by default), betting that the same process will soon ask for a nearby block. Removed in 2.6.33.

Deceptive idleness
The false impression that a process has no more disk work, created in the short gap between one of its synchronous reads finishing and its next read arriving.

Think time
The time a process spends computing between the end of one I/O request and the start of its next one.

Completely Fair Queuing (CFQ)
A Linux I/O scheduler that gives each process its own queue and serves the queues in round robin, each with a time slice on the disk. The default scheduler of the single-queue block layer from 2.6.18 until that layer was removed in 5.0.

I/O priority
A per-process setting (changed with the ionice command) that schedulers such as CFQ and BFQ use to give a process longer or shorter turns at the disk.

Multi-queue block layer (blk-mq)
The Linux block layer introduced in 3.13: each CPU puts requests in its own software queue, and these feed one or more hardware queues, so fast devices are not held back by one shared, locked queue. The only block layer since 5.0.

mq-deadline
The deadline scheduler rewritten for blk-mq (Linux 4.11): sector order plus read and write expiry times. The usual default for devices with a single hardware queue, such as SATA disks.

Budget Fair Queueing (BFQ)
A blk-mq scheduler (Linux 4.12) that gives each process a budget of sectors per turn and shares the disk in proportion to weights, favouring interactive and soft real-time work.

Kyber
A small blk-mq scheduler (Linux 4.12) for fast devices. It limits how many requests are in flight so that reads and synchronous writes meet target latencies.

Page cache
The main memory Linux uses to hold copies of file and disk data. Since the 2.4 series it is one unified cache for both file pages and disk blocks.

Dirty page
A page in the page cache that has been changed in memory but not yet written to the disk.

Writeback
Copying dirty pages from the page cache to the disk. Afterwards they are clean, so their frames can be freed or reused without losing data.

Flusher thread
A kernel thread that performs writeback in the background. Linux 2.6 used a pool of pdflush threads; from 2.6.32 there is one flusher per storage device.

Review questions with answers

1. The Linux 2.4 elevator has a pending write for sectors 120–127 when a new write for sectors 128–135 arrives. What happens to the new request?

A. It is inserted in sector order just behind the pending write
B. It is appended at the tail, behind every other request
C. It is joined with the pending write into one request
D. It goes straight to the disk, ahead of the whole queue

Answer: C. It is joined with the pending write into one request

The first check is merging: a new request for the sectors right next to a pending request of the same direction is joined to it (a back merge here), giving one request for sectors 120–135.

2. Put the checks the Linux 2.4 elevator applies to each new request in order.

Correct order:

1. Try to merge it with a pending request for the neighbouring sectors
2. If some pending request is too old, add the new one at the tail
3. Otherwise insert it at its place in sector order
4. If no place in sector order fits, append it at the tail

Merge first (cheapest), then the starvation guard, then sorted insertion, and finally the tail as the fallback.

3. Under the deadline scheduler with its default settings, a read request enters the queue at t = 1.2 s. At what time, in seconds, does it expire?

Answer: 1.7 s

The default read expiry time is 500 ms = 0.5 s, so the request expires at 1.2 + 0.5 = 1.7 s. From then on, the next time the scheduler checks the read FIFO, it serves this request ahead of sector order.

4. In the deadline scheduler, every request sits in the sorted queue and also in exactly one of the two FIFO queues.

Answer: True

Each request is in the sorted (elevator) queue and in the read FIFO or the write FIFO, depending on its direction. The FIFO tells the scheduler which request has waited longest.

5. Why does the deadline scheduler give reads a much shorter expiry time (500 ms) than writes (5 s)?

A. A process usually waits for its read, but most writes finish in the background
B. Reads move far less data than writes, so they always finish sooner
C. Writes must be sorted before they reach the disk, so they need extra time
D. Disks read faster than they write, so a read needs less time to complete

Answer: A. A process usually waits for its read, but most writes finish in the background

Most reads are synchronous: the process is blocked until the data arrives. Most writes land in the page cache first, so the process keeps running while the write waits.

6. Which problem does the anticipatory scheduler attack?

A. A stream of reads that keeps every write waiting forever
B. Seeking away just before the same process asks for a nearby block
C. Many CPUs fighting over the lock of one request queue
D. Dirty pages that stay in memory much longer than they should

Answer: B. Seeking away just before the same process asks for a nearby block

Between two reads of a sequential reader, its queue looks empty (deceptive idleness). Waiting a few milliseconds for its next request avoids a seek away and a seek back.

7. Which statement about the anticipatory scheduler is true?

A. It became the default scheduler for blk-mq devices in Linux 5.0
B. It improves throughput whatever the think time of the reading process
C. It waits after every request for any process at all, reads or writes
D. It was removed in 2.6.33, and CFQ kept a similar short idle wait

Answer: D. It was removed in 2.6.33, and CFQ kept a similar short idle wait

The anticipatory scheduler was the default from 2.6.0 to 2.6.17 and was removed in 2.6.33. CFQ already idled briefly for a process's next request, so the idea lived on.

8. CFQ serves 4 always-busy processes, each with a 100 ms time slice. Ignoring the cost of switching, how many milliseconds does one process wait between the end of its slice and the start of its next one?

Answer: 300 ms

The other three processes each use their 100 ms slice first: 3 × 100 = 300 ms. With seeks between areas the real wait is a little longer.

9. Which I/O schedulers can a Linux 5.0 (or later) kernel still use, and which were removed?

Available in 5.0 and later	Removed
mq-deadline BFQ Kyber none (no reordering)	CFQ Anticipatory noop (single-queue)

Linux 5.0 deleted the single-queue block layer and with it noop, deadline and CFQ; the anticipatory scheduler was already gone in 2.6.33. blk-mq devices choose among mq-deadline, BFQ, Kyber and none.

10. Match each Linux version with the change it brought.

2.6.18	**CFQ becomes the default scheduler**
2.6.32	**Per-device flushers replace pdflush**
2.6.33	**The anticipatory scheduler is removed**
3.13	**The multi-queue block layer arrives**
5.0	**The single-queue layer and CFQ are deleted**

CFQ default (2.6.18), per-device flusher threads (2.6.32), anticipatory removed (2.6.33), blk-mq added (3.13), single-queue path removed (5.0).

11. The Linux page cache keeps file pages in main memory and writes dirty pages to the disk later rather than at once. Which are real benefits of this design?

A. Dirty pages can later be written together in an order that suits the disk
B. A page changed many times may reach the disk only once
C. Data that was just written can be read again from memory
D. No data can be lost if the power fails

Answer: A, B, C

Delaying writes lets the kernel sort and batch them and absorbs repeated changes; keeping the pages cached serves the next read from memory. The price is the opposite of the last choice: dirty data not yet written is lost in a power failure.

12. A page first becomes dirty at t = 12 s and is not touched again. Memory is plentiful. The age limit is 30 s and the flusher wakes at t = 5, 10, 15, … s. At what time, in seconds, is the page written to disk?

Answer: 45 s

The page becomes due at 12 + 30 = 42 s, but nothing checks until the flusher's next wake-up, which is at 45 s.

11.10 Windows I/O

You should be able to

- Name the four kinds of modules the Windows I/O manager works with and trace a read or a write through them.
- Explain how the cache manager uses lazy write and lazy commit, and what that speed costs if the power fails.
- Compare synchronous and asynchronous I/O, and pick the right one of the five ways Windows reports that an asynchronous request finished.
- Contrast hardware RAID with software RAID, and describe what volume shadow copies and BitLocker do for a whole volume.

Windows I/O: one manager, many drivers

Every Windows I/O request goes through the **I/O manager**, part of the executive. It checks the handle and buffer, describes the request in an **I/O request packet (IRP)** (operation, file object, offset, byte count, buffer) and sends it to the top driver for that file or device. Drivers pass the IRP down their stack and mark it complete; the I/O manager then returns the result and tells the caller.

The four kinds of modules the I/O manager works with

- **Cache manager**: caches file data for the whole I/O subsystem, for every file system, local or network. The cache lives in main memory and its size changes with free memory: it grows when memory is plentiful and shrinks when processes need it. It caches pieces of files (file + offset), not raw disk blocks. It speeds up writes with **lazy write** and transactions with **lazy commit**.
- **File system drivers** (NTFS, FAT, exFAT…): the I/O manager treats a file system driver as just another driver and routes requests for a volume to it. The file system turns file offsets into block requests and sends them to the drivers below.

- **Network drivers**: networking is built in. The redirector makes remote files look local (SMB protocol), TCP/IP is a protocol driver, and each network adapter has its own driver. They are drivers, not part of the executive.
- **Hardware device drivers**: operate physical devices and reach their registers only through entry points of the hardware abstraction layer (HAL). The routines have the same names on every platform, so driver source code is portable across processor types.

Request paths

1. **Cached read:** ReadFile → I/O manager builds an IRP → NTFS → cache manager finds the bytes in memory (hit) → copied to the buffer, IRP completed. No disk, no interrupt. (A "fast I/O" shortcut can even skip building the IRP for such reads.)
2. **Read miss:** the bytes are not cached, so a paging read goes back through NTFS to the disk drivers, which program the controller through the HAL and start DMA. The thread blocks; the disk interrupts, the data lands in a cache page and is copied to the buffer.
3. **Lazy write:** WriteFile → IRP → NTFS → cache manager copies the data into a cache page and marks it dirty → WriteFile returns at once. About a second later the lazy writer thread sends the dirty page down through NTFS to the disk.
4. **Remote read:** the IRP goes to the redirector, which sends an SMB read message down through TCP/IP and the adapter driver; the reply comes back up the same way.

Lazy write and lazy commit

Lazy write: record the update in the cache, return immediately, and write it to disk later in the background when the system is less busy. Many updates to one page between two lazy-writer runs become one disk write. The price: until it is written, an update exists only in memory. A program that cannot take that risk opens the file for write-through or calls FlushFileBuffers.

Worked example (lazy writer once per second, one disk write = 10 ms of waiting under write-through): a program rewrites block A 10 times in 2.4 s. Write-through: 10 disk writes and 10 × 10 = 100 ms of waiting. Lazy write: one disk write per run in which A was dirty = 3 disk writes, almost no waiting. If the power fails at 1.47 s, the 2 updates made after the 1.0 s run are lost.

Lazy commit: the transaction version. Instead of writing "this transaction committed" at once, the commit record is cached and a background thread writes it to the file system log later. If the power fails first, recovery treats the transaction as not committed, so the file system stays consistent.

Synchronous and asynchronous I/O

Synchronous: the thread blocks until the operation completes. **Asynchronous (overlapped)**: the call starts the operation and returns at once (open with FILE_FLAG_OVERLAPPED, pass an OVERLAPPED record); the thread keeps working and the system must later signal completion. It does not speed up the device; it overlaps the thread's work with the device.

For N reads that each need D ms of device time and C ms of processing: synchronous = N × (D + C). Asynchronous with a device that serves one request at a time = max(N·D + C, D + N·C). With a device that serves all at once = D + N·C. Example N = 4, D = 10, C = 6: 64 ms, 46 ms, 34 ms. Example N = 5, D = 8, C = 4: 60 ms and 44 ms.

Five ways to signal that an asynchronous request finished

Technique	How the thread learns	Good for / limitation
Signal the file object	Waits on the file handle, which is signaled when an operation on it completes.	Simple, nothing to create; one request at a time, since every operation on the file signals the same object.
Signal an event object	Each request gets its own event; wait on one, any or all of them.	Several simultaneous requests on one file or device; one event per request, at most 64 per wait call.
Asynchronous procedure call (APC)	ReadFileEx names a completion routine, queued on the thread's APC queue; it runs when the thread enters an alertable wait.	Completion handled on the issuing thread; never runs if the thread never waits alertably.
I/O completion port	Completions go to one queue served by a pool of workers; about one runs per processor.	Servers with many connections; more setup.
Polling	The thread checks the status the system writes into its OVERLAPPED record.	Loops that must never block; checking too often wastes processor time.

Hardware RAID vs software RAID

Hardware RAID: a disk controller or storage cabinet combines physical disks and creates and regenerates the redundant data itself; Windows sees one disk and sends one request per write. **Software RAID**: Windows combines space, even noncontiguous parts of disks, into a logical volume and does the redundancy in a driver on the host CPU. Per small write, software RAID 1 sends 2 writes, and software RAID 5 sends 4 requests (read old data, read old parity, write new data, write new parity, with new parity = old parity ⊕ old data ⊕ new data). Windows supported software RAID 1 and RAID 5, plus striping, first through the fault-tolerant disk driver of Windows NT Server and then, from Windows 2000, through dynamic disks (RAID 5 only on Server editions); since Windows 8 and Server 2012, Storage Spaces provides mirror and parity layouts and dynamic disks are deprecated.

Volume shadow copies

A shadow copy is a **point-in-time view** of a whole volume: the volume exactly as it was at one instant, for consistent backups while users work and for recovering earlier file versions. Before it is taken, applications such as databases are asked to flush and briefly pause writes. How the view is kept depends on the **provider**: some make a complete copy (for example a split mirror, which becomes independent of the original volume), some redirect new writes elsewhere (redirect on write), and the built-in Windows **system provider**, modelled in this section, uses **copy on write**: taking it copies nothing; a block is saved to a diff area just before its first overwrite after the snapshot; reading the snapshot takes changed blocks from the diff area and unchanged blocks from the live volume. Such a copy-on-write snapshot is not a backup by itself: if the original disk fails, so does the snapshot. It is the consistent source a backup copies from.

BitLocker

Full-volume encryption (since Windows Vista): a driver below the file system encrypts every sector written and decrypts every sector read, so programs notice nothing and a stolen disk shows only scrambled data. At boot the key is released by the TPM (Trusted Platform Module, a security chip on the motherboard) only

if the early boot files are unchanged, optionally with a PIN or USB startup key; a 48-digit recovery key is the backup.

Key terms

I/O manager
The part of the Windows executive that receives every I/O request, normally describes it in an I/O request packet and hands it to the right driver, so programs see one uniform interface for files, devices and networks.

I/O request packet (IRP)
The record the I/O manager builds for one I/O request: the operation (read, write...), the file or device, the buffer and the byte count. Drivers pass it down their stack and mark it complete when the work is done.

Cache manager
The Windows component that keeps recently used file data in main memory for every file system, local or network. The cache grows and shrinks with the amount of free memory.

Lazy write
Accepting a write into the cache, returning to the program at once, and copying the changed data to disk later in the background, when the system is less busy.

Lazy commit
The transaction version of lazy write: the record that a transaction committed is kept in the cache and written to the file system log later by a background thread, instead of immediately.

File system driver
A driver that implements one file system, such as NTFS or FAT. The I/O manager treats it like any other driver; it turns file requests into requests for blocks of the volume and passes them to the drivers below.

Network driver
One of the drivers that give Windows its built-in networking: the redirector that makes remote files look local (using the SMB protocol), protocol drivers such as TCP/IP, and the drivers of network adapters.

Hardware device driver
A driver that operates one physical device. It reaches the device's registers only through routines of the hardware abstraction layer, so the same driver source works on every platform Windows supports.

Synchronous I/O
I/O in which the calling thread is blocked until the operation is finished: when the call returns, the data has been transferred.

Asynchronous I/O (overlapped I/O)
I/O in which the call only starts the operation and returns at once, so the thread keeps working; the system must later tell the thread that the operation finished.

File object
The kernel object that stands for one open instance of a file or device. It is waitable: it becomes signaled when an I/O operation on it completes.

Event object

A Windows object that is either signaled or not; threads wait on it and are released when it is set, by another thread or, for I/O, by the I/O manager. One event per request lets a thread tell several pending requests apart.

Asynchronous procedure call (APC)
A routine queued to one particular thread, to be run by that thread. For I/O, the I/O manager queues the program's completion routine, which runs when the thread next enters an alertable wait.

Alertable wait
A wait, such as SleepEx with its alertable flag set, during which a thread agrees to run any APCs queued to it. The wait ends as soon as they have run.

I/O completion port
A kernel queue of finished I/O operations served by a pool of worker threads. Each worker takes one completion at a time, and the kernel keeps only about as many workers running as there are processors.

Volume
A storage area that holds one file system and has its own drive letter or folder: part of one disk, a whole disk, or space combined from several disks.

Hardware RAID
RAID done by a disk controller or storage cabinet: it combines several physical disks and keeps the redundant data itself, so Windows sees just one disk.

Software RAID
RAID done by the operating system: a Windows driver combines space from several disks, mirrors or computes parity in software, and presents the result as one volume.

Volume shadow copy
A point-in-time view of a whole volume: the volume exactly as it was at one moment, used for consistent backups and for recovering earlier versions of files. How it is stored depends on the provider; the built-in Windows system provider uses copy on write.

BitLocker
Windows full-volume encryption: a driver below the file system encrypts every sector written to the volume and decrypts every sector read, so a stolen disk shows only scrambled data.

Review questions with answers

1. A program reads part of a file on an NTFS volume. Where does the Windows I/O manager send the request first?

A. To the disk's hardware driver, which reads files by name
B. To the HAL, which then chooses the right file system
C. To the NTFS file system driver, routed like any other driver
D. To the cache manager, which owns every volume's files

Answer: C. To the NTFS file system driver, routed like any other driver
The I/O manager treats a file system driver as just another driver: requests for a volume go to its file system driver, which turns file offsets into block requests for the drivers below.

2. With lazy write, when WriteFile returns the new data is already safely on the disk.

Answer: False

Lazy write only copies the data into a cache page and marks it dirty; a background lazy-writer thread writes it out later. Until then a power failure loses it.

3. A program rewrites the same 4 KB block 25 times within 0.9 s. The lazy writer then runs once and writes every dirty block. How many disk writes does lazy write need for this block in that second?

Answer: 1 writes

Each lazy-writer run writes a dirty block once, however many times it changed: 25 updates become 1 disk write. Write-through would have needed 25.

4. A thread reads 5 blocks. Each read needs 8 ms of device time (the device serves one request at a time), then 4 ms of processing. With asynchronous I/O the thread starts all 5 reads at once and processes each result as soon as it is ready. Total time in ms?

Answer: 44 ms

The device is the bottleneck (8 ms ≥ 4 ms): it finishes the last read at 5 × 8 = 40 ms, and processing that result adds 4 ms, so 44 ms. Synchronous I/O would take 5 × (8 + 4) = 60 ms.

5. Match each completion technique to how the thread finds out.

Signaling the file object	**Waits on the file handle, set by any operation on that file**
Signaling an event object	**One event per request shows exactly which one finished**
Asynchronous procedure call	**A completion routine runs in the thread during an alertable wait**
I/O completion port	**A pool of workers takes completions from one shared queue**
Polling	**The thread checks a status word in its own memory**

Five techniques, from simplest (the file object) to most scalable (the completion port); polling never blocks at all.

6. A chat server on a 16-core machine keeps 20,000 connections open and wants to serve them with a few threads. Which completion technique fits best?

A. A completion port served by a small pool of worker threads
B. One event object per connection, all waited on together
C. An APC completion routine per connection, all on one thread
D. Polling the status of every connection in a tight loop

Answer: A. A completion port served by a small pool of worker threads

Completion ports exist for servers: every connection feeds one queue, a few workers take completions, and the kernel keeps about as many running as there are processors.

7. A thread starts a read with ReadFileEx and names a completion routine. When does that routine run?

A. At once, interrupting the thread wherever it happens to be
B. On whichever idle thread the system thread pool picks
C. In a brand-new thread that Windows creates just for it
D. When the issuing thread next enters an alertable wait

Answer: D. When the issuing thread next enters an alertable wait

The I/O manager queues an APC on the issuing thread. It runs when that thread enters an alertable wait, such as SleepEx with the alertable flag set.

8. What does **lazy commit** delay?

A. Writing the changed data of ordinary files back to disk
B. Writing a transaction's commit record to the file system log
C. Building the I/O request packet for a transaction's writes
D. Releasing every lock a transaction holds until the disk is idle

Answer: B. Writing a transaction's commit record to the file system log

With lazy commit the record that a transaction committed is cached, and a background thread writes it to the file system log later, so the committing thread does not wait for a disk write.

9. Hardware RAID or software RAID?

Hardware RAID	Software RAID
Windows sees just one disk Needs a RAID controller Windows sends one request per small write	Parity is computed by the host CPU Can combine parts of several disks into one volume Built into Windows, for example Storage Spaces

Hardware RAID hides the disks behind a controller that does the copying and parity itself. Software RAID is a Windows driver that sees every disk and does that work on the host CPU.

10. Windows' built-in system provider keeps a volume shadow copy with copy on write. Which statements about such a shadow copy are true?

A. Taking one copies the whole volume at that moment
B. A block is saved just before its first overwrite after the snapshot
C. A backup can copy consistent files while people keep working
D. Unchanged blocks are read from the live volume
E. It survives the failure of the original volume's disk

Answer: B, C, D

With copy on write nothing is copied when the snapshot is taken, old blocks are saved only when they would be overwritten, and unchanged blocks are shared with the live volume, so it does not protect against that disk dying. (Other providers can make a complete, independent copy instead, such as a split mirror.)

11. Put the steps of a read that misses the cache in order.

Correct order:

1. The program calls ReadFile
2. The I/O manager builds an IRP and sends it to NTFS
3. NTFS asks the cache manager, but the data is not cached
4. A paging read goes through NTFS to the disk drivers
5. The disk interrupts and the data lands in a cache page
6. The bytes are copied to the program and ReadFile returns

Down through the I/O manager and the file system to the cache; on a miss, a paging read goes down to the disk; the completion travels back up and the data is copied to the caller.

12. Because BitLocker works in a driver below the file system, programs read and write files on an encrypted volume exactly as they would on an unencrypted one.

Answer: True

The driver encrypts every sector on its way to the disk and decrypts it on the way back, so programs and NTFS see plain data. Only someone reading the raw disk without the key sees scrambled bytes.

Chapter 12 · File Management

Files outlive the programs that make them. The file system decides how they are named, found, shared and stored.

A file is data that must survive after its program ends. This chapter covers what a file system must provide, how records are organized and found, how B-trees keep large indexes fast, how directories name files, how files are shared and protected, how records are packed into blocks, and how disk space is allocated and tracked, then shows the file systems of UNIX, Linux, Windows and Android.

Learning objectives

- Describe the basic concepts of files and the layered structure of a file system.
- Compare the main file organizations and explain how a B-tree index works.
- Explain file directories, file sharing and record blocking.
- Compare file allocation methods and free-space management techniques.
- Describe the file systems of UNIX, Linux, Windows and Android.

How Chapter 12 fits together

A file system keeps a file's name, its attributes and its data in different places. The overview fetches the 85th record of /home/maya/members.dat: with 100-byte records, 40 to a 4,096-byte block, it is in file block 3. Each directory on the path costs two reads (header, then entries), 6 for /, home and maya, and the file's header 1 more. Locating file block 3 then takes no read with contiguous allocation, 2 with chained and 1 with indexed, plus the block itself: 8, 10 or 9 reads for 100 bytes, which is why systems cache directory entries and headers.

Files, layers and organization (12.1 to 12.3)

Fields form records, records form files, and linked data forms a database. File software is layered: access method, logical I/O, basic I/O supervisor, basic file system, device drivers. The organization follows the use: pile (arrival order), sequential (key order, for batch runs), indexed sequential (key order plus an index and an overflow file), indexed (one index per searched field) or hashed (key turned into a block address). B-trees keep big indexes short and wide: a few block reads find one key among millions.

Directories and sharing (12.4 and 12.5)

A directory maps names to files; an entry holds the attributes or points to a file header. A tree of directories gives every file a pathname, absolute from the root or relative to the working directory. Sharing needs access rights (knowledge up to deletion, for the owner, groups or everyone) and locks on files or records, so simultaneous updates cannot wreck each other.

Blocking and storage (12.6 and 12.7)

Programs read records but disks move blocks. Fixed blocking fits ⌊B / R⌋ records per block; variable-length spanned and unspanned blocking trade wasted space against simplicity. Contiguous allocation gives one run (fast, but free space fragments), chained allocation links the blocks (any free block will do,

random access is slow) and indexed allocation lists them in an index block (good for both). Free space is tracked by a bit table, chained free portions, an index or a free block list.

Real file systems (12.8 to 12.11)

UNIX keeps everything about a file but its name in an inode, whose direct and indirect pointers reach its blocks. Linux serves every file system through the VFS objects (superblock, inode, dentry, file) and a dentry cache. NTFS describes each file as attributes in a master file table record and logs metadata changes to recover after a crash. Android puts ext4 or F2FS in fixed partitions, gives each app its own Linux user and private directory, and keeps app data in SQLite databases.

Why it matters next

Chapter 13 turns to embedded systems, whose storage is often flash that is erased in large blocks and wears out: the same questions (where blocks go, how they are found, how they survive a power cut) with far fewer resources.

12.1 Overview

You should be able to

- Explain what a file is, the three properties that make files useful, and the six basic file operations.
- Tell fields, records, files and databases apart, and explain why UNIX and Windows see a file as a stream of bytes.
- List the objectives of a file management system and the minimal requirements its users place on it.
- Name the five layers of file system software, from device drivers up to access methods, and trace a record request through them.
- Follow a request through the file management functions, from directory lookup and access control down to physical I/O.

What a file is

A program's variables disappear when it ends. A **file** is a named collection of data, created by an application and kept by the OS on secondary storage (disk or SSD), so it outlives the program that made it. Three properties make files useful:

- **Long-term existence (persistence).** The file stays on secondary storage through program exits, logoffs and power cycles, until someone deletes it.
- **Sharable between processes.** A file has a name, so other processes and users can find it; access controls decide who may do what with it.
- **Structure.** The data inside is organized to suit the application, and files can be organized into hierarchies (folders) and related to one another.

The OS software that provides these services is the **file management system**; programs reach file data only through it. Six basic operations: **create** (a new, empty file and its directory entry), **delete** (remove the entry and free the space), **open** (find the file by name in the directory, check access rights, set up an open-file record and give the process a file descriptor with a position of 0), **close** (discard the open-file record; the file itself stays), **read** and **write** (move data at the current position, which then advances).

Open files are main-memory state and vanish at a restart; the file on disk does not (though real systems may cache the newest writes in memory for a while, by default up to about 30 seconds on Linux, so a power cut can lose them). Deleting an open file differs: Windows normally refuses; UNIX removes the name at once and frees the data after the last close.

Fields, records, files and databases

Level	What it is	Example
Field	The basic element of data: one value with a type and a length. Fixed-length fields always use the same number of bytes; variable-length fields need a length count in front or an end marker.	an employee's hire date
Record	Related fields that a program treats as a unit. Fixed-length if every field is fixed; variable-length if a field varies or the number of fields varies.	everything about one employee
File	A collection of similar records treated as one entity, with a unique name. Created, deleted and protected as a whole; access control usually works at file level.	employees.dat
Database	A collection of related data whose relationships are explicit, built for one or more applications. Managed by a **DBMS**, a separate program on top of the OS, which usually keeps its data in ordinary files.	employees, departments and payroll, linked

Example: an Employees record with id (4 bytes), name (1 length byte + characters), dept (3), hire date (8) and salary (4-byte integer) is 20 bytes plus the name's characters, so "Rosa Diaz" gives 4 + (1+9) + 3 + 8 + 4 = 29 bytes: a variable-length record. A Payroll record of emp_id (4), period (6) and gross pay (4) is always 14 bytes: fixed-length.

Byte streams: who decides the structure?

UNIX, Linux and Windows treat an ordinary file as a **byte stream**: bytes numbered from 0 with no structure the OS knows about. A read request simply says "n bytes starting at byte p". Records exist only in the programs that use the file. With fixed-length records of size R, numbered from 0, record k starts at byte k × R. *Worked example:* 20-byte records, so record 3 starts at 3 × 20 = 60 and occupies bytes 60 to 79. A program that assumes the wrong layout (say 16-byte records) still gets the bytes, but splits them in the wrong places; the OS cannot notice. Benefit: one simple interface for every kind of file. Cost: every program must agree on the layout. Not every system works this way: IBM mainframe data sets (z/OS), for example, declare a record format and length that the system itself uses.

Objectives of a file management system

1. Meet the data management needs of its users (store data and perform the operations they need).
2. Guarantee, as far as possible, that the data in files are valid.
3. Optimize performance: throughput for the whole system and response time for each user.
4. Provide I/O support for a variety of storage device types.
5. Minimize or eliminate the potential for lost or destroyed data.
6. Provide a standardized set of I/O interface routines to user processes.
7. Provide I/O support for multiple users, on multiuser systems.

Minimal user requirements (interactive, general-purpose system)

1. Create, delete, read, write and modify files.
2. Have controlled access to other users' files.
3. Control what types of access are allowed to one's own files.
4. Restructure files in a form suited to the problem.
5. Move data between files.
6. Back up files and recover them if they are damaged.
7. Access files by symbolic name rather than by numeric identifier.

Rule of thumb: an objective is a quality of the whole system; a requirement is an ability each user must have.

File system software architecture (five layers)

Each layer uses only the one below and offers a simpler service to the one above. From the user down:

1. **Access method**: the level closest to the user; the standard interface between applications and the file system. Different access methods suit different file structures (pile, sequential, indexed sequential, indexed, hashed).
2. **Logical I/O**: lets programs work with records rather than blocks (general-purpose record I/O) and keeps basic data about each file, such as its record size.
3. **Basic I/O supervisor**: starts and ends all file I/O; keeps control structures for device I/O, scheduling and file status; selects the device; schedules disk accesses for performance; assigns I/O buffers and secondary storage. Part of the OS.
4. **Basic file system (physical I/O)**: exchanges whole blocks with secondary storage, decides where blocks are placed and buffers them in main memory; does not understand the data or the file's structure.
5. **Device drivers**: the lowest level; talk directly to device controllers, start I/O operations and handle their completion (interrupts). Part of the OS.

Worked example: read record 42 of payroll. Records are 256 bytes and blocks 4096 bytes, so 4096 ÷ 256 = 16 records per block. Logical I/O finds record 42 in file block $\lfloor 42 \div 16 \rfloor = 2$, starting at byte (42 mod 16) × 256 = 2560 of that block (bytes 2560 to 2815). The supervisor selects disk 0, assigns a buffer and queues the request for disk scheduling. The basic file system knows file block 2 is disk block 9120 and moves its 4096 bytes into the buffer. The driver reads sectors 72960 to 72967 (512-byte sectors) and handles the completion interrupt. Back up the stack, logical I/O extracts bytes 2560 to 2815 and the access method hands record 42 to the program.

Name clash with section 11.3. Section 11.3's layered I/O model also has a layer called **logical I/O**, but there it is the top layer of the stack for a local device: it treats the whole device as a logical resource, reached through a device identifier and simple commands. In the file system stack here, logical I/O is the record layer between the access method and the basic I/O supervisor.

File management functions: the path of a request

User and program commands → directory management → user access control → access method (record operations) → record blocking → disk scheduling, file allocation and free storage

management → physical I/O.

- **Directory management** first identifies and locates the file from its name.
- **User access control** lets only authorized users perform the operation, before any data moves. An unauthorized request stops here (for example "permission denied"); nothing below runs.
- The **access method** suited to the file's structure turns the command into record operations, since users work at the record level.
- **Record blocking**: programs work with records but I/O moves blocks, so records are packed into blocks for output and unpacked after input. It bridges the logical view (names, users, records) and the physical view (blocks, disk).
- **File allocation** decides and records which disk blocks hold each block of the file; **free storage management** tracks unused blocks; **disk scheduling** orders the block requests.
- **Physical I/O** transfers the blocks.

Examples: a read of record 42 uses file allocation only to look up where block 2 lives (disk block 9120) and needs no free storage. Appending record 64 to a file whose 64 records exactly fill blocks 0 to 3 starts a new block 4 (slot 0): free storage management hands out a free disk block (5507), file allocation records block 4 → 5507, and the block is written, leaving 65 records in 5 blocks.

Key terms

File
A named collection of data that applications create and the OS keeps on secondary storage (disk or SSD), so it lasts after the program that made it ends and can be used again later, by the same or another program.

Long-term existence (persistence)
The property that a file stays on secondary storage until someone deletes it, through program exits, logoffs and power cycles.

File management system
The set of system software that gives users and applications their services for using files. Programs reach file data only through it.

Directory
A table kept by the file system that maps each file name to information about the file, including where its data is stored.

Field
The basic element of data: one value, such as a name, a date or a salary. It has a type and a length, and the length may be fixed or variable.

Record
A collection of related fields that a program treats as one unit, such as everything about one employee. It is fixed-length when it always holds the same fixed-length fields, and variable-length when a field varies in length or the number of fields varies.

Database
A collection of related data in which the relationships among the elements are explicit, designed to be used by one or more applications.

Database management system (DBMS)
The software that stores, organizes and searches a database. It is a separate program, not part of the OS, and it usually keeps the database in ordinary files that the OS manages.

Byte stream
A view of a file as nothing more than a numbered sequence of bytes. UNIX, Linux and Windows treat ordinary files this way and leave any record structure to the application.

Access method
The top layer of file system software and the standard interface between applications and the file system. Each access method suits one file structure, such as pile, sequential or indexed.

Logical I/O
In the file system software stack, the layer that lets programs read and write records instead of blocks, and keeps basic data about each file, such as its record size. (Section 11.3's I/O model uses the same name for a different layer: the top layer for a local device, which treats the whole device as a logical resource.)

Basic I/O supervisor
The file system layer that starts and ends every file I/O operation. It keeps the control structures for device I/O, scheduling and file status, picks the device, schedules disk accesses and assigns I/O buffers.

Basic file system (physical I/O)
The file system layer that exchanges whole blocks with secondary storage, places blocks on the device and buffers them in main memory, without knowing what the blocks contain.

Block
The unit, usually of one fixed size, in which data move between main memory and secondary storage, for example 4096 bytes. One disk block usually covers several sectors.

Directory management
The file management function that identifies and locates a file from its symbolic name by looking it up in the directory, and carries out directory operations such as adding and removing entries.

User access control
The file management function that checks whether the requesting user may perform the requested operation on the file, before any data moves.

Record blocking
Packing records into blocks before output and unpacking them after input, because programs work with records while I/O moves whole blocks.

File allocation
Deciding which blocks of secondary storage hold a file's data, and recording that mapping so every block of the file can be found again.

Free storage management
Keeping track of which blocks of secondary storage are unused, so new or growing files can be given space.

Review questions with answers

1. Which property of files means that a file is still there after the program that created it exits and the computer is restarted?

A. Sharing between many processes
B. A structure suited to the application
C. Long-term existence on storage
D. Access control set by its owner

Answer: C. Long-term existence on storage

Files live on secondary storage such as a disk, which keeps its contents without power, so a file persists until someone deletes it.

2. Sort each item into the level of data it describes.

Field	Record	File	Database
One employee's hire date The price of one product	All the facts about one employee, handled as a unit One order: customer, date and items, handled as a unit	Every employee's record, kept under the name employees.dat	Employee, department and payroll data with explicit links between them

A field is one value, a record is related fields treated as a unit, a file is a named collection of similar records, and a database adds explicit relationships among its data.

3. In UNIX and Windows, the operating system keeps track of where each record in an ordinary file begins and ends.

Answer: False

Both treat an ordinary file as a stream of bytes. Any record structure is known only to the applications that read and write the file.

4. A program keeps fixed-length 20-byte records in an ordinary byte-stream file, numbering the records from 0. At what byte offset does record 3 begin?

Answer: 60 bytes

Record k starts at k × record size, so record 3 starts at 3 × 20 = 60 and occupies bytes 60 to 79.

5. A file holds 256-byte records packed into 4096-byte blocks, with no record split across two blocks. Records and blocks are both numbered from 0. In which block of the file is record 42?

Answer: 2

4096 ÷ 256 = 16 records fit in each block. Record 42 is in block $\lfloor 42 \div 16 \rfloor = 2$, as the record at position 42 mod 16 = 10 inside it.

6. Put the layers of file system software in order, from the one closest to the user down to the one closest to the hardware.

Correct order:

1. Access method
2. Logical I/O
3. Basic I/O supervisor
4. Basic file system (physical I/O)
5. Device drivers

Applications call the access method; logical I/O works in records; the basic I/O supervisor starts and schedules I/O; the basic file system moves blocks; device drivers talk to the hardware.

7. Match each file system layer to its main job.

Device drivers	**Command the device controllers and handle the completion of each I/O**
Basic file system	**Move whole blocks to and from storage without understanding them**
Basic I/O supervisor	**Start and end file I/O, pick the device, schedule disk access**
Logical I/O	**Give programs record-level access and keep basic data on each file**
Access method	**Offer applications a standard interface that suits the file structure**

Each layer hides the details of the one below: commands become blocks, blocks become records, and records reach applications through an access method.

8. Is each statement an objective of the file management system as a whole, or a minimal requirement of each user?

System objective	User requirement
Guarantee, as far as possible, that the data in files are valid	Back up files and recover them after damage
Optimize throughput and response time	Refer to files by symbolic name
Provide a standard set of I/O interface routines to processes	Control what access others have to one's own files

Objectives describe qualities the designers want for the whole system; user requirements are abilities each individual user must have.

9. A program running for a user with no rights to a protected file asks to read it. At which file management function is the request refused?

A. Directory management
B. User access control
C. Disk scheduling
D. Record blocking

Answer: B. User access control

User access control runs right after the file is located and before any record or block is touched, so an unauthorized request stops there.

10. A program asks for one record of an existing file, by the file's name, and is allowed to read it. The block that holds the record is not yet in main memory, and the file neither grows nor shrinks. Which of these file management functions take part in carrying out the request?

A. Directory management
B. Free storage management
C. Record blocking
D. Disk scheduling

Answer: A, C, D

The file must be found by name, its block read and unblocked, and the disk request scheduled. Free storage management acts only when space is given to or taken back from a file, which a read does not do.

11. Which statement about a database is correct?

A. It is built into the OS file system, which stores the links between its files
B. It is any large file in which every record has the same fixed length
C. It is a folder that groups related files so the OS can search them
D. It is run by a separate DBMS and records explicit links among its data

Answer: D. It is run by a separate DBMS and records explicit links among its data

A database is a collection of related data with explicit relationships among its elements, managed by a database management system that is separate from the operating system.

12. The basic file system layer moves whole blocks between storage and main-memory buffers without understanding what those blocks contain.

Answer: True

The basic file system deals only in blocks. Records and file structure are the business of logical I/O and the access methods above it.

12.2 File Organization and Access

You should be able to

- Explain what a file organization is and weigh the five criteria (access time, ease of update, storage economy, simple maintenance, reliability) for a given use.
- Describe how a pile, a sequential file, an indexed sequential file, an indexed file and a direct (hashed) file store, add and find records.
- Compute the average number of accesses to find a record in a sequential file and through one- and two-level indexes.
- Explain overflow files, exhaustive and partial indexes, and how a hashed file handles collisions.
- Choose a suitable organization for an application and justify the choice with the strengths and weaknesses of each.

File organization

A **file organization** is the logical arrangement of a file's records: their order, any indexes, and where new records go. It is chosen to match how the records will be accessed. Five criteria are weighed:

- **Short access time**: reach the one record you want quickly.

- **Ease of update**: add, change and delete records with little work.
- **Economy of storage**: no wasted padding, duplicate copies or bulky extras.
- **Simple maintenance**: few structures to keep in step or rebuild.
- **Reliability**: records survive crashes and mistakes intact.

Their priority depends on the use: a batch file read start to finish hardly cares about fast single lookups; an online file read one record at a time cares a great deal; a write-once file is never updated. The criteria also pull against each other: an index buys access time with extra storage and extra update work.

The pile

Records are kept in the order they arrive. They may differ in length and in which fields they have, so each field describes itself (name and value). There is no structure, so finding a record is an **exhaustive search** through the file, parsing each record to find its key. A pile suits data gathered before anyone organizes it, or data that is hard to organize, and it uses space well when records vary.

The sequential file

Every record has the same fixed format (same fields, same order, same length) and the records are stored in order of a **key field**. No field names are stored. A full pass in key order is fast, which suits batch jobs that process every record. A single lookup is poor: scanning from the start reads on average **N/2** records (exactly (N + 1)/2 for a key that is present; 500,000 for 1,000,000 records). Inserting in key order would mean moving every later record, so additions go to a separate **log file** (transaction file) and are merged into the main file in a periodic **batch update** that writes a fresh copy. Until the merge, a lookup must check the log too. A variant keeps the records as a linked list in key order, so an insert only relinks pointers, at the price of following pointers on every scan.

The indexed sequential file

A sequential file plus two additions:

- An **index** holding the first key of each stretch (block) of the file and a pointer to it. To find a key, scan the index for the highest entry ≤ the key, jump to that place in the main file, and scan forward.
- An **overflow file**. A new record is written there, and the record just before it in key order (in the main file or already in overflow) gets a pointer to it; the new record takes over that record's old pointer. The main file and index never move. A scan follows each chain and then returns to the main file. Long chains slow searches, so the file is periodically **reorganized**: overflow records are merged in key order and the index is rebuilt, so no search detours through overflow (a file that has grown also gets a longer index).

A **multilevel index** treats the lowest index as a sequential file and builds a smaller index over it.

Cost. Counting every index entry or record compared as one access, and reading each index entry by entry, a search costs about half of each level plus half of the stretch of file it ends in:

- One level with I entries over N records (S = N/I per entry): about I/2 + S/2. With N = 1,000,000 and I = 1,000: 500 + 500 = 1,000 accesses, instead of 500,000.
- The sum is smallest when I = $\sqrt{N}$, giving about $\sqrt{N}$ accesses.

- Two levels, U top entries over L lower entries: about U/2 + (L/U)/2 + (N/L)/2. With U = 100 and L = 10,000: 50 + 50 + 50 = 150. With U = 10 and L = 1,000: 5 + 50 + 500 = 555.

These are estimates. An index scan learns it has gone far enough only by reading the first entry past the target, and the cost lab counts that read too: about one extra access per index level. Averaged over every key, the exact counts are 1,002 (not 1,000), 153.5 (not 150) and 558.4 (not 555); the lab's 300 random searches add a little sampling noise on top. Real systems read whole blocks, keep top index levels in memory and search a sorted index by halving it (binary search, about $\log_2 I$ steps), so disk reads are far fewer, but the ranking is the same.

The indexed file

The sorted order and the single key are dropped. Records go wherever there is room, may vary in length, and are reached **only through indexes**, usually one for each field people search by. An **exhaustive index** has one entry for every record; a **partial index** has entries only for records that contain the field. Each index is kept sorted so it can be searched quickly, by halving it (binary search) or through a tree of index levels, never entry by entry. Adding a record means updating every index whose field the record has. Indexed files suit jobs where data must be current and is searched many ways but rarely processed in full: airline reservations, inventory control.

The direct (hashed) file

A **hash function** turns the key straight into the address of a block, called a **bucket**, and the disk can read any block whose address is known, so a lookup takes about one access whatever the file size. Every record needs a key, and there is no sequential order. Two keys that hash to the same bucket cause a **collision**; the answer used here gives each bucket a few slots and, when it is full, chains the newcomer into an overflow area behind its bucket (one extra access per chain record). Example with 7 buckets and h(key) = key mod 7: 1234 = 7 × 176 + 2 goes to bucket 2; 2051 and 6608 both give 0 and fill bucket 0, so 3010 (also 0) goes to overflow and costs 2 accesses to find. A key that is absent is ruled out after reading just its bucket and chain. In the cost lab every record compared along a bucket's chain counts as one access, so with an average of α records per bucket a successful search costs about 1 + α/2 (1.5 at one record per bucket). Listing records in key order needs every bucket read and then a sort. Hashed files suit fixed-length records fetched one at a time and fast: directories, price tables, schedules, name lists.

Comparing the five

	Space: variable	Space: fixed	Update: same size	Update: larger	One record	Subset	All records
Pile	Excellent	Adequate	Excellent	Good	Poor	Poor	Adequate
Sequential	Poor	Excellent	Excellent	Poor	Poor	Adequate	Excellent
Indexed sequential	Poor	Good	Excellent	Poor	Good	Adequate	Good
Indexed	Good	Adequate	Good	Good	Excellent	Excellent	Adequate
Direct (hashed)	Poor	Adequate	Excellent	Poor	Excellent	Poor	Poor

Each column compares the five on one job; the update columns assume the record has already been found. Why: the pile stores only the fields a record has but repeats field names; fixed formats pad short records and cannot let one grow; a same-size change is overwritten in place everywhere, though an indexed file must also move the entry of any changed indexed field; a larger record goes to the end of a pile (leaving a dead copy), or anywhere in an indexed file with each index pointer updated; indexes point straight at matching records, but a full pass through an indexed file either comes out in no order or hops around the disk; hashing finds one record in about one access but scatters records, so subsets and full passes read every bucket.

Choosing

- Data gathered before its structure is known: **pile**.
- A batch run over every record in key order and little else (monthly billing): **sequential**.
- Key-order batch runs plus single lookups by key (a student file): **indexed sequential**.
- Searches on several fields with up-to-the-minute data (airline bookings, a parts warehouse): **indexed**.
- One record by one key, as fast as possible (price lookup at a checkout): **direct (hashed)**.

Key terms

File organization
: The logical arrangement of the records in a file: their order, any indexes, and where new records go. It is chosen to match the way the records will be accessed.

Pile
: The simplest file organization: records are stored in the order they arrive, may have different lengths and different fields, and each field names itself. Finding a record means searching the whole file.

Exhaustive search
: Finding a record by examining records one after another until it turns up, or until every record has been examined.

Sequential file
: A file in which every record has the same fixed format (same fields, same order, same length) and the records are stored in order of a key field.

Key field
The field whose value identifies a record (such as an employee number) and that decides the order of the records in a sequential file.

Log file (transaction file)
A separate file that collects new or changed records as they arrive, so they can later be merged into the main file in one batch run.

Batch update
Applying many accumulated changes to a file in one run, usually by merging a log file into the sorted main file and writing a fresh copy.

Indexed sequential file
A sequential file with two additions: an index that lets a search jump close to the wanted record, and an overflow file that holds new records until the next reorganization.

Index
A smaller, sorted table of key values with pointers to where the records (or the next index level) can be found.

Overflow file
In an indexed sequential file, the area where newly added records are written. Each one is linked from the record that comes just before it in key order.

Multilevel index
An index built on top of another index: the top level picks a stretch of the level below, which in turn picks a stretch of the file.

Reorganization
Rewriting an indexed sequential file so the overflow records are merged into the main file in key order and the index is rebuilt.

Indexed file
A file whose records have no particular order and are reached only through indexes, usually one index for each field that people search by.

Exhaustive index
An index with one entry for every record in the main file.

Partial index
An index with entries only for the records that contain the field of interest.

Direct file (hashed file)
A file in which a hash function turns each record's key into the address of a block (bucket), so a record can be fetched with about one access and no searching.

Bucket
In a hashed file, the block that a hash value selects; it holds the records whose keys hash to that value.

Collision
Two different keys that hash to the same bucket. The file needs a rule, such as chaining extra records into an overflow area, for where the newcomer goes.

Review questions with answers

1. In an indexed sequential file, where does a newly added record go?

A. Into its place in the main file, shifting the later records along
B. Into the overflow file, linked from its predecessor in key order
C. At the end of the main file, after which the index is re-sorted
D. Into a separate log file that is merged in at the next batch run

Answer: B. Into the overflow file, linked from its predecessor in key order

The new record is written to the overflow file, and the record just before it in key order gets a pointer to it, so the main file and the index never move until the next reorganization.

2. A sequential file holds 200,000 records in key order. A lookup reads records one by one from the start. About how many records does it examine, on average, to find a record that is in the file?

Answer: 100000 records

Every position is equally likely, so on average the search goes halfway: about N/2 = 200,000 / 2 = 100,000 records.

3. A file of 160,000 records has a single-level index of 400 entries, each covering an equal stretch of the file. Using the estimate I/2 + S/2 (S = records per index entry), about how many accesses does an average successful search take?

Answer: 400 accesses

Each entry covers S = 160,000 / 400 = 400 records. Half the index plus half a stretch: 400/2 + 400/2 = 200 + 200 = 400, compared with 80,000 without the index. (Also counting the index entry read just past the target gives about 402.)

4. A hashed file has 7 buckets numbered 0 to 6 and uses h(key) = key mod 7. Which bucket does the record with key 1234 go to?

Answer: 2

1234 = 7 × 176 + 2, so the remainder is 2 and the record belongs in bucket 2.

5. A partial index has an entry for every record in the file.

Answer: False

That is an exhaustive index. A partial index has entries only for the records that contain the field of interest.

6. Match each organization with the job it fits best.

Pile	**Data collected before anyone knows how it will be organized**
Sequential file	**A monthly batch run over every record in key order**
Indexed sequential file	**A key-order batch run plus quick lookups by key**
Indexed file	**Searches on several fields with up-to-the-minute data**
Direct (hashed) file	**One product's price, by barcode, instantly**

Each organization is chosen for its access pattern: unknown structure, full passes, passes plus lookups, many search fields, or single-key speed.

7. Which organization does each statement describe?

Pile	Sequential file	Direct (hashed) file
Each field carries its own name Records may differ in length and in which fields they have	Every record has the same fields in the same order New records wait in a log file for a batch merge	A record's position is computed from its key A full bucket sends the newcomer to an overflow area

Self-describing, varied records make a pile; fixed records in key order with a log make a sequential file; computing the location from the key makes a hashed file.

8. Put the steps of finding a record in an indexed sequential file in order.

Correct order:

1. Compare the target with index entries until one is larger than it
2. Take the entry just before that one: the highest key ≤ the target
3. Jump to the main-file position that entry points to
4. Scan forward, following any overflow chain, until the target or a larger key appears

The index cuts the search down to one stretch of the file; the scan then finishes the job, detouring through overflow records that sit between main-file records in key order.

9. Which statements about an indexed file are true?

A. Records may be variable in length
B. Records are kept physically sorted by one key field
C. Adding a record means updating every index whose field it has
D. A partial index skips records that lack the indexed field
E. Reading every record is its strongest operation

Answer: A, C, D

An indexed file has no physical order and allows variable-length records; every applicable index must be updated on an insert; and a full pass through scattered records is one of its weaker operations.

10. Which criterion drops out of the decision entirely for a file that is written once to media that can never be rewritten?

A. Ease of update
B. Short access time
C. Economy of storage
D. Reliability

Answer: A. Ease of update

Records on write-once media are never changed, so ease of update drops out of the decision entirely.

11. A single-level index covers a file of N records. A search reads the index entry by entry, then reads the chosen stretch of the file record by record. Which index size gives the fewest accesses on average?

A. One entry per record (N entries)
B. About $\log_2 N$ entries
C. About N/2 entries
D. About $\sqrt{N}$ entries

Answer: D. About $\sqrt{N}$ entries

The cost is about $l/2 + N/(2l)$. Growing the index shrinks the stretches and the other way round; the sum is smallest when $l/2 = N/(2l)$, that is $l = \sqrt{N}$, giving about $\sqrt{N}$ accesses. (An index searched by halving it instead would favour a bigger index.)

12. Why does a sequential file collect new records in a separate log file instead of inserting them directly?

A. A log file stores each record in less space than the main file
B. The log is kept sorted, so every search checks it before the main file
C. Placing each one in key order would mean rewriting much of the file
D. Records in the log are complete, so they never need to be merged in

Answer: C. Placing each one in key order would mean rewriting much of the file

Keeping the main file in key order would force every insert to move the records after it. Collecting additions and merging them in one batch pass is far cheaper.

12.3 B-Trees

You should be able to

- Explain why an index kept on disk needs a short, wide tree, and why each B-tree node is the size of one disk block.
- State the rules of a B-tree of minimum degree d and spot a tree that breaks one.
- Search a B-tree while counting disk reads, and insert keys with node splits, including the root split that makes the tree taller.
- Use the bound $h \le \log_d((n + 1) / 2)$ to compute how many reads a lookup can need, and compare it with a binary search tree.
- Describe how a B+ tree differs from a B-tree and why file systems and databases favour it.

Why file systems and databases use B-trees

An **index** pairs each **key** (a file name, an account number) with the place where its record is stored. Large indexes do not fit in memory, so they live on disk, and every step of a search costs one disk-block read. A random read from a hard disk takes several milliseconds, roughly 100,000 times longer than a memory read, so the goal is to make searches need **few reads**, not few comparisons.

A **B-tree** is a balanced search tree whose **node is one disk block**, packed with many sorted keys. One read then chooses among hundreds of children instead of two, so the tree stays short. Example: with 4,096-byte blocks and 8-byte keys and pointers, a node holds 170 keys (each with a record pointer) and 171 child pointers: $170 \times 16 + 171 \times 8 = 4{,}088$ bytes.

Inside a node

- Keys are kept in increasing order; each key carries a pointer to its record (or the record itself).
- A node with k keys that is not a leaf has k + 1 child pointers, one for each gap below, between and above its keys. The keys act as **separator keys**: child i holds only keys that lie between key i – 1 and key i.
- Searching inside a node happens in memory and is cheap; only following a child pointer costs a disk read.

The rules (minimum degree d ≥ 2)

1. **Most keys:** every node holds at most 2d – 1 keys, so at most 2d children.
2. **Fewest keys:** every node except the root holds at least d – 1 keys (an internal one has at least d children). The root needs at least 1 key, and 2 children unless it is the only node.
3. **Pointers:** an internal node with k keys has exactly k + 1 children; leaves have none.
4. **Order:** keys increase within a node, and every key in a subtree lies between the two separators around it.
5. **Balance:** all leaves are on the same level.

d	keys per non-root node	children per internal non-root node
2	1 to 3	2 to 4 (the "2-3-4 tree")
3	2 to 5	3 to 6
100	99 to 199	100 to 200

Half-full nodes guarantee a large **fan-out**; level leaves make the tree perfectly balanced. To check a tree, test each rule: a root with a single key is legal, but any other node below d – 1 keys, any node above 2d – 1 keys, a node whose child count is not keys + 1, a key on the wrong side of a separator, or leaves on two levels all break the definition.

Search

1. Read the root block.
2. Scan its keys in memory for the first key ≥ k. If it equals k, stop: found.
3. If the node is a leaf, stop: k is not in the index.
4. Otherwise read the one child whose range can hold k, and repeat from step 2.

The **height** h is the number of steps from the root down to a leaf (a lone root has height 0, and a tree of height h has h + 1 levels). A search reads one node per level, so at most **h + 1** reads. A search for a missing key always ends at a leaf, so it reads exactly h + 1 nodes. Example: in the tree [55] / [27 39] [73 84] / [4 22] [29 34 36] [42 43 54] [56 66] [74 82] [85 88 93] (d = 3, height 2), 55 is found in 1 read, 39 in 2, 82 in 3, and 60 is reported missing after 3.

Insertion with node splits

This guide uses **proactive (top-down) splitting**:

1. If the root is full (2d – 1 keys), split it first: its median goes into a brand-new root. This **root split** is the only way the tree gets taller, and it lowers every leaf by one level at once, so balance is kept.
2. Walk down as in a search. Before stepping into a child, check it: if it is full, split it; its median moves up into the current node, which is guaranteed to have room because it was split earlier if it was full.
3. At the leaf (which now surely has room), insert the key in sorted order.

Splitting a full node of 2d – 1 keys: the first d – 1 keys stay, the d-th key (the **median**) moves up into the parent, and the last d – 1 keys move to a new sibling node. With d = 3, [a b c d e] becomes [a b] and [d e], and c goes up. Because the split happens before the new key is added, the key that moves up is the full node's own median, never the newcomer. The other common method inserts first and splits overflowing nodes on the way back up; both keep every rule, though their trees can differ slightly.

Worked example (d = 3)

```
start      [20 40 60 80]
           [3 7 12] [24 28 33 36] [44 51] [63 66 70 74 77] [85 91 96]
insert 48  leaf [44 51] has room          → [44 48 51]
insert 72  leaf [63 66 70 74 77] is full  → 70 moves up: root [20 40 60 70 80];
           leaves [63 66] and [72 74 77] (72 placed)
insert 30  root is full                   → root split: 60 alone in a new root,
           children [20 40] and [70 80]; height 1 → 2; 30 → [24 28 30 33 36]
insert 26  leaf [24 28 30 33 36] is full  → 30 moves up into [20 30 40];
           leaves [24 26 28] and [33 36]
final      [60] / [20 30 40] [70 80] /
           [3 7 12] [24 26 28] [33 36] [44 48 51] [63 66] [72 74 77] [85 91 96]
```

Predicting a split: follow the search path of the new key; every node on it that already holds 2d – 1 keys splits as the walk reaches it (top one first), and its middle key moves up. If no node on the path is full, nothing splits. A full root splits even when the target leaf has room.

How tall can a B-tree get?

The tallest tree for n keys is the thinnest legal one: a root with 1 key, and every other node holding only d – 1 keys. Such a tree of height h ≥ 1 has at least $2\,d^{h-1}$ nodes on its bottom level and at least $2d^h - 1$ keys in total. Since that cannot exceed n:

$$\mathbf{h \le \log_d((n+1)/2)}$$

Worked example: d = 100, n = 1,000,000. $\log_{100}(500{,}000.5) \approx 2.85$, and a height is a whole number, so h ≤ 2 and a lookup reads at most 3 nodes (a height-3 tree would need at least $2 \times 100^3 - 1 = 1{,}999{,}999$ keys). Even with every node full (199 keys) the tree needs height 2, so it has exactly that height. A perfectly balanced binary search tree with a million keys has $\lceil \log_2(1{,}000{,}001) \rceil = 20$ levels: 20 reads, about 200 ms at roughly 10 ms per random hard-disk read, against about 30 ms for the B-tree. In practice the root, and often the next level, stays cached in memory, so a lookup costs only one or two real disk reads.

B+ trees

The variant that many file systems and most database indexes actually use. It changes three things:

- every key, with its record, lives in a **leaf**; internal nodes hold only copies of keys that steer searches;
- the leaves are **linked** left to right in key order;
- every search goes all the way to a leaf, so every lookup costs the same.
- as in any B-tree, all leaves stay on one level.

Record-free internal nodes hold more keys, so the fan-out is larger and the tree shorter. A **range query** (all keys from 17 to 75, say) goes down once and then walks along the leaf chain, never hopping back up; a plain B-tree must walk in order through several levels, keeping a root-to-leaf path in memory. A plain B-tree can stop early when the key sits in an internal node (finding 52 took 2 reads in the guide's example, against 3 in the B+ tree). NTFS indexes directory entries with B+ trees, XFS uses them for free space and file extents, and most relational databases build their indexes on them.

Deletion (in brief)

Deletion runs the same ideas backwards: when a node would drop below d – 1 keys, it borrows a key from a sibling (through the parent) or merges with a sibling; if the root loses its last key, the tree becomes one level shorter.

Key terms

B-tree
A balanced search tree built for disks. Each node fills one disk block and holds many keys in sorted order, so a search reads only a few nodes even when the tree holds millions of keys.

Index
A lookup structure that pairs each key with the place where its record is stored, so one record can be found without reading the whole file.

Key
The value used to find a record, such as a file name or an account number. Inside a B-tree node the keys are kept in increasing order.

Node (tree node)
One box of a tree. In a B-tree a node is one disk block holding a sorted list of keys and, unless it is a leaf, one pointer to a child for each gap between and around those keys.

Root (root node)
The single node at the top of a tree, where every search starts.

Leaf (leaf node)
A node with no children, at the bottom of a tree. In a B-tree every leaf sits on the same level.

Internal node (nonleaf node)
A node that has children. In a B-tree an internal node with k keys has exactly k + 1 children.

Minimum degree (d)
The number that fixes a B-tree's node sizes: every node holds at most 2d – 1 keys, and every node except the root holds at least d – 1 keys. d is at least 2.

Fan-out (branching factor)
How many children a node has. A large fan-out makes a tree short and wide.

Height (tree height)
The number of steps down from the root to a leaf. A tree whose root is its only node has height 0; a tree of height h has h + 1 levels.

Balanced tree
A tree in which every path from the root to a leaf has (nearly) the same length, so no search is much slower than another. In a B-tree every leaf is on exactly the same level.

Separator key
A key in an internal node that divides the key ranges of the two subtrees on its sides: smaller keys live to its left, larger keys to its right.

Split (node split)
What a B-tree does to a full node: the middle (median) key moves up into the parent, and the keys on either side of it become two separate nodes.

Median key
The middle key of a full node. With 2d – 1 keys it is the d-th key, with d – 1 keys on each side; it is the key that moves up when the node splits.

Root split
The split of a full root. A new root is created to hold the median key, so the tree grows one level taller; it is the only way a B-tree gains height.

Proactive splitting (top-down insertion)
An insertion method that splits every full node it meets on the way down, so a key that moves up always finds room in its parent and the insertion never has to climb back up.

Binary search tree
A tree whose nodes each hold one key and have at most two children, smaller keys to the left and larger keys to the right.

B+ tree
A B-tree variant that keeps every key and record in the leaves, uses internal nodes only to steer searches, and links the leaves in key order for fast sequential scans.

Range query
A request for every record whose key lies between two values, such as all accounts numbered 300 to 400, returned in key order.

Review questions with answers

1. Why is each node of a B-tree that indexes a file usually made exactly one disk block in size?

A. Smaller nodes would make each key comparison slower in memory
B. One read then fetches a whole node, and many keys per node keep the tree short
C. A disk can only store tree nodes when they are exactly one block long
D. Block-sized nodes let the tree skip the balance rule on most levels

Answer: B. One read then fetches a whole node, and many keys per node keep the tree short
The disk moves whole blocks, so a block-sized node costs one read. Filling it with many keys gives a large fan-out, which keeps the tree only a few levels tall.

2. A B-tree has minimum degree d = 4. What is the largest number of keys any one node may hold?

Answer: 7 keys

The limit is 2d – 1 = 2 × 4 – 1 = 7 keys, which gives at most 2d = 8 children.

3. In a B-tree of minimum degree d = 3, the root is allowed to hold just one key.

Answer: True

Every node except the root needs at least d – 1 = 2 keys. The root only needs 1 key (and 2 children if it is not a leaf), because a root split leaves exactly one key in the new root.

4. Which event is the only way a B-tree becomes one level taller?

A. A leaf receives its (2d – 1)-th key and becomes full
B. A full leaf splits and its median moves up into its parent
C. The full root splits and its median becomes a new root
D. A key larger than every key already stored is inserted

Answer: C. The full root splits and its median becomes a new root

Only a root split creates a new node above the old root. Every leaf moves down one level at the same moment, which is why the leaves always stay on one level.

5. A B-tree of minimum degree d = 3 inserts with top-down (proactive) splitting. A full leaf on the insert's path holds 12, 18, 25, 31, 40. Which key moves up into the parent when that leaf splits?

Answer: 25

A full node holds 2d – 1 = 5 keys. Its median is the d-th key, the 3rd: 25. The keys 12, 18 stay in one node and 31, 40 form the other, d – 1 = 2 keys each.

6. Put the steps of inserting a key k with top-down (proactive) splitting in order.

Correct order:

1. If the root is full, split it and put its median in a new root
2. At the current node, pick the child whose key range holds k
3. If that child is full, split it and move its median up into the current node
4. Step down into the child (or half) that holds k, and repeat until a leaf
5. Place k into the leaf, keeping its keys in order

The root is checked first, then the insertion walks down one level at a time, splitting any full child before entering it. That guarantees the leaf has room at the end.

7. A B-tree has minimum degree d = 100 and holds 1,000,000 keys. Using $h \le \log_d((n + 1) / 2)$, where the height h counts the steps from the root down to a leaf, what is the largest number of nodes a single search can read?

Answer: 3 nodes

$\log_{100}(500{,}000.5) \approx 2.85$, and a height is a whole number, so $h \le 2$. A search reads one node per level: h + 1 = 3 nodes. A balanced binary search tree with the same keys needs 20 levels.

8. Each line describes a tree that should be a B-tree of minimum degree d = 2 (1 to 3 keys per node). Is it valid, or does it break a rule?

Valid B-tree	Breaks a rule
Root [40] with leaves [10 20] and [50 60 70] Root [30 60] with leaves [10], [40 50] and [70]	Root [40] with leaves [10 20 30 35] and [50] Root [30 60] with just two leaves, [10 20] and [40 50] Root [40] with leaves [10 45] and [50 60] Root [40] with leaf [10 20] on the left and, on the right, node [60] over leaves [50] and [70]

The broken ones: a leaf with 4 keys exceeds 2d – 1 = 3; a root with 2 keys needs 3 children; 45 sits left of the separator 40; and the last tree has leaves on two different levels.

9. Match each term to what it does.

Minimum degree d	**Sets each node's size limits: d – 1 to 2d – 1 keys**
Separator key	**Divides the key ranges of the subtrees on its two sides**
Median key	**Moves up into the parent when a full node splits**
Root split	**The only operation that adds a level to the tree**
B+ tree leaf chain	**Lets a range query walk sideways in key order**

d fixes the node sizes; separators steer each search; the median is what a split pushes up; only a root split adds a level; and linked leaves turn a range query into a sideways walk.

10. Which statements are true of a B+ tree?

A. Every key, together with its record, is stored in a leaf
B. The leaves are linked to each other in key order
C. Inner nodes hold copies of keys that only steer searches
D. A search may stop early at an inner node holding the key
E. Leaves may sit on different levels to save space

Answer: A, B, C

A B+ tree keeps all data in leaves, chains the leaves, and uses inner nodes only as signposts, so every search ends at a leaf. Stopping early is a plain B-tree trait, and all leaves stay on one level.

11. A B-tree has height 3: every leaf is three steps below the root. A search for a key that is not stored in it reads how many nodes?

A. 1: the root's keys already show the key is absent
B. 3: one read for each step down from the root
C. It depends on how full each node happens to be
D. 4: one node on each level, down to a leaf

Answer: D. 4: one node on each level, down to a leaf

An unsuccessful search can only give up at a leaf, so it reads one node on every level: height + 1 = 4 nodes.

12. While searching a B-tree, why does it hardly matter how many keys are compared inside one node?

A. A comparison in memory is far cheaper than one disk read
B. The disk controller compares all of a node's keys in one step
C. Each node holds so few keys that comparisons hardly ever happen
D. The comparisons for each node are remembered from earlier searches

Answer: A. A comparison in memory is far cheaper than one disk read

Scanning even a few hundred keys in memory takes well under a microsecond, while one random disk read takes milliseconds. The number of reads, one per level, sets the cost.

12.4 File Directories

You should be able to

- List the four groups of information a directory can keep about a file, and explain why many systems move most of it into a file header such as a UNIX inode.
- Describe the five directory operations: search, create file, delete file, list directory and update directory.
- Compare single-level, two-level and tree-structured directories, including how each handles name clashes.
- Write absolute and relative pathnames, and resolve a path one directory lookup at a time, including . and ..

What a directory is

A **directory** maps each file name to the file itself: its attributes and where its data are stored. Every open by name starts with a directory lookup. A directory is itself a file, stored in disk blocks, but it belongs to the OS: programs may ask to list it and change it only by asking the OS to create, delete or rename files. Because programs use only names, the OS can move a file's data and fix up one entry.

What a directory entry can hold

A **directory entry** describes one file or subdirectory. The full list of information a directory could keep falls into four groups:

Group	Fields	Purpose
Basic information	file name (unique within its directory), file type (text, binary, executable load module, directory, ...), file organization (for systems that offer several, such as pile, sequential, indexed or hashed)	what the file is
Address information	volume (the device or volume holding the file), starting address (first block), size used, size allocated	where the data live
Access control information	owner (usually the creator; grants rights to others), access information (who else may use it: once a name and password per authorized user, now user/group IDs or access control lists), permitted actions (read, write, execute, send over a network)	who may do what
Usage information	date created, identity of creator, date of last read access, identity of last reader, date last modified, identity of last modifier, date of last backup, current usage (which processes have the file open, whether it is locked, whether it was changed in main memory but not yet written to disk)	history and current activity

Worked example. hw1.txt holds 9,500 bytes (size used). With 4,096-byte blocks it needs ⌈9,500 ÷ 4,096⌉ = 3 blocks, so its size allocated is 12,288 bytes and 2,788 bytes of the last block are reserved but empty.

Real systems keep a subset. Classic UNIX stored no creation time ("ctime" means last status change); ext4, NTFS and APFS do. Everyday file systems do not record the last reader or modifier. Current usage changes constantly, so it lives in main-memory tables of open files, not on disk.

Two layouts: everything in the entry, or name plus pointer

Some systems put every attribute in the entry (an MS-DOS FAT entry is a fixed 32 bytes: name, attribute flags, dates, first cluster, size). Others keep only the **name and a pointer** and move the rest into a separate **file header**. In UNIX-like systems the header is the **inode** (index node): a numbered record holding type, permissions, owner, size, timestamps and block addresses, everything except the name. A directory is then a list of (name, inode number) pairs; in ext4 the entry for hw1.txt is 16 bytes.

- Small entries pack many to a block, so a search reads fewer blocks.
- Writing to a file changes its size and modification time in the inode only; the directory changes only when a name is added, removed or renamed.
- The cost: when attributes are needed (an open, a long listing) the OS must also read the inode.

The five directory operations

1. **Search**: find the entry whose name matches. A linear search compares names in order; a failed search must examine every entry.
2. **Create file**: search first to make sure the name is free (a duplicate is a **name clash** and is refused), then add an entry (and, in UNIX, allocate and fill an inode).
3. **Delete file**: search for the entry and remove it. In UNIX the inode and its blocks become free once no other entry names that inode and no process still has the file open.
4. **List directory**: return all the entries, or part of them.
5. **Update directory**: when an attribute stored in an entry changes (size, dates), rewrite that entry.

Comparing names one by one gets slow in a directory with thousands of entries, so modern file systems index the names instead: ext4 with a hashed tree, NTFS with a B+-tree. Older designs such as FAT still search linearly.

Three directory structures

- **Single-level**: one list holds every file of every user. Simple, but every name must be unique across all users, files cannot be grouped, and a large list is slow.
- **Two-level**: a **master directory** has one entry per user, leading to that user's own **user directory**, a flat list of their files. Names need only be unique within one user directory, and the master entry can carry access control. A user still cannot group files.
- **Tree-structured (hierarchical)**: the master directory becomes the **root directory**; user directories, and any directory below them, may hold files and further **subdirectories**. Names need only be unique within one directory. This is the structure of essentially every general-purpose OS today.

Search cost at scale. A successful linear search of n equally likely entries averages (n + 1) ÷ 2 comparisons, once per directory searched. With 500 users × 80 files: a single list of 40,000 averages 20,000.5; two levels average (500 + 1) ÷ 2 + (80 + 1) ÷ 2 = 250.5 + 40.5 = 291; a tree in which each user splits 80 files into 8 folders of 10 averages 250.5 + 4.5 + 5.5 = 260.5.

Pathnames

```
/
├── ana/  notes.txt · music/ song.mp3 · school/ (os/ hw1.txt lab2.c notes.txt · math/
notes.txt)
├── ben/  notes.txt · games/ chess.exe
└── chen/ todo.txt
```

- A **pathname** names a file by its route: the directories passed through from the top, then the file name, joined by a **path separator**: `/ana/school/os/hw1.txt`. UNIX, Linux and macOS use `/`; Windows uses `\` and begins a full path with a drive letter: `C:\ana\school\os\hw1.txt`.
- Two files may share a name if their pathnames differ: the example tree has four notes.txt files and no clash.
- An **absolute pathname** starts at the root (a leading `/`), so it names the same file wherever you are.
- Each process has a **working (current) directory**, changed by a shell's `cd` and printed by `pwd`. A **relative pathname** has no leading `/` and is read from the working directory: from `/ana/school`, `os/hw1.txt` is /ana/school/os/hw1.txt.
- `.` means this directory and `..` the **parent directory**. Classic UNIX file systems, and ext4 today, store both as real entries in every directory (the root's `..` leads back to the root); Windows and some newer file systems handle them while parsing the path instead, with the same meaning.

Resolving a path

The OS reads a path from left to right and performs one directory search per name, starting at the root for an absolute path and at the working directory for a relative one. From working directory /ana/school:

Path	Result	Searches
`/ana/school/os/hw1.txt`	/ana/school/os/hw1.txt	4
`os/hw1.txt`	the same file	2
`../notes.txt`	/ana/notes.txt	2
`../../ben/games/chess.exe`	/ben/games/chess.exe	5
`/ben/games/chess.exe`	the same file	3
`music/song.mp3`	error: no entry music in /ana/school	1
`os/hw1.txt/x`	error: hw1.txt is not a directory	2

A working directory gives nearby files short names and fewer searches; for distant files the absolute path can be shorter.

Writing a relative path

1. Write the working directory and the target as absolute paths.
2. Find the deepest directory they share.
3. Write one `..` for each level from the working directory up to it.
4. Add the names from there down to the target.

Example: from /ana/school/os to /ana/notes.txt, the deepest shared directory is /ana, two levels up, so the path is `../../notes.txt`.

Key terms

Directory entry
One record in a directory that describes one file or subdirectory. It always holds the name, plus either the file's attributes and location or a pointer to a separate record that holds them.

File attributes (metadata)
Facts about a file that the file system keeps outside the file's contents: its type, size, owner, permissions, dates and where its data are stored.

File header
A record kept apart from the directory that holds one file's attributes and the address of its data. When a system uses headers, a directory entry holds only a name and a pointer to the header.

Inode (index node)
The file header of UNIX-like file systems: a numbered record that holds everything about one file except its name, such as type, permissions, owner, size, timestamps and the addresses of its data blocks. A directory entry pairs a name with an inode number.

Single-level directory
A directory structure with one flat list of every file in the system, shared by all users.

Two-level directory
A directory structure with a master directory that has one entry per user; each entry leads to that user's own flat directory of files.

Master directory
The top directory of a two-level or tree-structured scheme. Its entries lead to the user directories; in a tree it is the root directory.

User directory
A directory that belongs to one user and lists that user's files (and, in a tree, that user's subdirectories).

Tree-structured directory (hierarchical directory)
A directory structure in which a master (root) directory leads to user directories, and any directory may hold both files and further subdirectories.

Subdirectory
A directory that is listed as an entry inside another directory.

Root directory
The directory at the top of the tree. It has no parent, and every absolute pathname starts there.

Pathname
The name of a file written as the route to it: the directories passed through, then the file name, joined by a separator, as in /ana/school/os/hw1.txt.

Path separator
The character placed between the names in a pathname: / in UNIX, Linux and macOS, \ in Windows.

Absolute pathname
A pathname that starts at the root directory (in UNIX it begins with /). It names the same file no matter which directory you are working in.

Relative pathname
A pathname that does not start at the root. The system reads it starting from the working directory.

Working directory (current directory)
The directory a process is currently "in". Each process has its own; relative pathnames are resolved from it, and a shell changes it with the cd command.

Parent directory
The directory that contains a given directory, one level closer to the root. Pathnames write it as two dots (..).

Name clash
An attempt to give a new file a name that another entry in the same directory already uses. The directory must refuse it, or lookups by name would become ambiguous.

Review questions with answers

1. Which of these is **not** one of the standard operations on a directory?

A. Update directory entries
B. Search for a file name
C. Read the data inside a file
D. List directory contents

Answer: C. Read the data inside a file

The five directory operations are search, create file, delete file, list directory and update directory. Reading a file's data is an operation on the file itself; the directory is only used to find it.

2. Sort each piece of information into the group of directory-entry information it belongs to.

Basic	Address	Access control	Usage
File type File organization	Starting address Size allocated	Owner Permitted actions	Date of last backup Which processes have it open

Basic information says what the file is; address information says where its data lives; access control says who may do what; usage information records its history and current activity.

3. In a UNIX file system, appending data to a file rewrites that file's entry in its directory, because the entry records the file's size.

Answer: False

A UNIX directory entry holds only a name and an inode number. The size and modification time live in the inode, so only the inode is rewritten; the directory changes only when a name is added, removed or renamed.

4. On a system with a two-level directory, users ana and ben each try to create a file named `notes.txt`. What happens?

A. Refused: names must be unique across the whole system
B. Both succeed: names need only be unique per user directory
C. Both succeed, but only if one of them first makes a subdirectory
D. The OS silently renames the second file to notes(1).txt

Answer: B. Both succeed: names need only be unique per user directory

The master directory leads to a separate user directory for each user, and a name only has to be unique within one directory. ana's notes.txt and ben's notes.txt sit in different directories.

5. A process's working directory is `/home/kim/web`. Which file does the relative pathname `../photos/cat.jpg` name?

A. /home/kim/web/photos/cat.jpg
B. /home/photos/cat.jpg
C. /photos/cat.jpg
D. /home/kim/photos/cat.jpg

Answer: D. /home/kim/photos/cat.jpg

Start at /home/kim/web, take `..` up to /home/kim, then go down into photos and find cat.jpg: /home/kim/photos/cat.jpg.

6. Starting at the root, how many directory searches does the OS make to resolve the absolute pathname `/usr/share/dict/words`?

Answer: 4

One search per name after the root: search / for usr, usr for share, share for dict, and dict for words. That is 4 searches.

7. A two-level directory has 300 user entries in its master directory, and every user directory holds 60 files. Both are searched one entry at a time. On average, how many name comparisons does it take to find a file that exists (every user and file equally likely)?

Answer: 181

A successful linear search of n entries averages (n + 1) ÷ 2 comparisons. (300 + 1) ÷ 2 + (60 + 1) ÷ 2 = 150.5 + 30.5 = 181. One flat list of all 18,000 files would average 9,000.5.

8. Match each term to its description.

Single-level directory	**One list shared by every user**
Two-level directory	**A master directory plus one flat directory per user**
Tree-structured directory	**Any directory may hold files and subdirectories**
Working directory	**Where a relative pathname starts**
Root directory	**Where every absolute pathname starts**

The three structures add levels step by step; the working directory and the root are the two possible starting points of a path lookup.

9. Which statements about pathnames are true?

A. Two files may share a name if their pathnames differ
B. A relative pathname names the same file whatever the working directory is
C. `..` refers to the parent directory
D. Windows separates names with `\`, UNIX with `/`
E. An absolute pathname in UNIX begins with `/`

Answer: A, C, D, E

Names only have to be unique inside one directory, so full pathnames tell same-named files apart. A relative pathname depends on the working directory; an absolute one does not.

10. Why do many file systems keep only a name and a pointer in each directory entry, with the other attributes in a separate header (an inode in UNIX)?

A. Small entries: more fit per block, so a search reads fewer blocks
B. It makes every file name unique across the whole disk, not just one directory
C. It lets user programs write directory entries directly, without the OS
D. It removes the need to store access control information for each file

Answer: A. Small entries: more fit per block, so a search reads fewer blocks

A name plus a number is a few bytes, so a directory block holds many entries and a search reads fewer blocks. The attributes are still kept, once, in the header.

11. The working directory is `/ana/school/os`. Put the steps of resolving `../math/notes.txt` in order.

Correct order:

1. Start at the working directory, /ana/school/os
2. Look up .. and move to the parent, /ana/school
3. Search /ana/school for the name math
4. Search /ana/school/math for the name notes.txt
5. Return the file /ana/school/math/notes.txt

A relative path starts at the working directory, and the OS handles one name at a time from left to right: .. first, then math, then notes.txt.

12. In a tree-structured directory, a file and a subdirectory that sit in the same directory cannot have the same name.

Answer: True

A lookup searches a directory by name only, so every name in one directory must be unique, whatever kind of entry it is. A second entry with that name would be a name clash.

12.5 File Sharing

You should be able to

- Name the two issues file sharing raises: access rights and the management of simultaneous access.
- List the classic access rights in order, from none to deletion, and explain what each one adds to the rights below it.
- Explain the role of the owner and grant rights to a specific user, a user group or all users, then decide whether a request is allowed.
- Show how two unprotected updates to the same record lose one of them, and how whole-file and record locking prevent it.
- Compare lock granularities by throughput and bookkeeping, and recognize the readers/writers and deadlock problems that locking brings back.

File sharing: two issues

When several users share a file, the file system must deal with two issues:

1. **Access rights**: who may do what to the file.
2. **Management of simultaneous access**: what happens when several users use, and especially update, the file at the same time.

Access rights: the classic hierarchy

The classic scheme lists eight levels. Each right **includes every right before it**, so granting one level grants all the levels below it.

#	Right	What it adds
0	None	The user may not even learn that the file exists. To enforce this, the user is not allowed to read the directory that lists the file.
1	Knowledge	The user can learn that the file exists and who owns it, and can then ask the owner for more rights. The file cannot be opened.
2	Execution	The user can load and run the file as a program but cannot copy it. Licensed (proprietary) programs are often shared this way.
3	Reading	The user can read the file for any purpose, including copying it and running it.
4	Appending	The user can add data, usually only at the end, but cannot modify or delete existing contents. Example: a sign-up sheet or a log.
5	Updating	The user can modify, delete and add data anywhere, including the first writing of the file and rewriting it completely.
6	Changing protection	The user can change the access rights other users hold. Normally only the owner has this right; an owner who extends it to others can usually limit which rights may be changed.
7	Deletion	The user can delete the file from the file system.

Why this order? Each level allows strictly more. Execution is below reading because a run-only program cannot be copied, while reading allows copying and running. Appending is below updating because adding at the end cannot damage existing data.

Real systems often use independent switches instead of one ladder. Classic UNIX keeps read, write and execute bits for owner, group and others (write without read is possible, and deleting a file depends on write permission on its *directory*). Windows can grant append data, delete and change permissions separately.

The owner and classes of users

One user is the **owner** of each file, at first the user who created it. The owner holds every right and may grant rights to others. Rights can be granted to three classes of users:

- **Specific user**: one individual, identified by user ID.
- **User group**: a named set of users who all receive the same rights, without being listed one by one.
- **All**: every user of the system. A file granted to all users is a **public file**.

When a user belongs to several classes the system needs a rule. **Combining** takes the highest right any matching class grants, so a grant can only add rights. The **most-specific** rule checks owner, then named user, then group, then all users, and uses only the first class that matches. Classic UNIX permission bits work this way with three classes (owner, group, others); POSIX access-control lists add named-user entries, checked right after the owner. That lets an owner shut out one person (named entry None) while everyone else may read. Windows access-control lists lean towards combining: the allow entries for the user and every group the user is in add up, but an explicit deny entry wins.

Example: Ben has Updating by name, group lab has Reading, all users have Knowledge. Combining gives Ben Updating: Ben may edit a row but not change rights. Eve, in no group, only has Knowledge.

Simultaneous access and the lost update

Reading together is harmless; updating is not. An update is three actions: **read** the record into private memory, **change** the private copy, **write** it back. If a second update reads the record in the gap before the first writes, one of the two changes is overwritten: a **lost update**.

Worked example: record 7 holds 1000. Ana subtracts 300 and Ben subtracts 200. Both read 1000; Ana writes 700; Ben writes 800. The file says 800, but the true value is 1000 – 300 – 200 = 500. Ana's update is lost.

The OS or file system must therefore impose a discipline, usually a **lock**: a process takes the lock before reading and releases it after writing, so read, change, write becomes one unit that no other process using the lock can break into. A process that finds the lock taken is **blocked** by the OS (it does not poll) and woken when the lock is released.

Whole-file locking versus record locking

- **Whole-file locking**: lock the entire file for the length of an update. Simple (one lock), but every update waits its turn, even for different records.
- **Record locking**: lock only the record (or byte range) being updated. Updates to different records proceed in parallel; only updates to the same record wait. The cost: more locks and bookkeeping, and an update that locks several records can deadlock.

The size of the unit one lock protects is the **lock granularity**. For n updates that all arrive together, each holding its lock for t ms:

- No locks: all done after t, but for every record wanted by k updates, k – 1 of them are lost.
- Whole-file lock: all done after n × t.
- Record locks: all done after t × (largest number of updates wanting one record).

Example: six updates want records 6, 1, 5, 8, 8, 3 with t = 30 ms. No locks: 30 ms, 1 update lost (record 8). Whole file: 6 × 30 = 180 ms (33.3 updates per second). Record locks: record 8 is wanted twice, so 2 × 30 = 60 ms (100 updates per second), with 5 record locks held at once.

Real systems. UNIX and Linux lock byte ranges with `fcntl` (POSIX calls them record locks); these are normally **advisory**: they hold back only processes that also ask for the lock. Linux removed its optional mandatory mode in version 5.15. Windows byte-range locks (`LockFileEx`) are mandatory for ordinary file reads and writes: the OS refuses a conflicting ReadFile or WriteFile. Reads and writes through a memory-mapped view of the file are not checked against these locks, so mapped access needs its own coordination.

Old problems return: readers/writers and deadlock

Locks come in two kinds. Many readers can hold a **shared (read) lock** together; a writer needs an **exclusive (write) lock**, which no one else may hold at the same time.

Already held	Asks shared	Asks exclusive
nothing	granted	granted
shared	granted (share)	waits
exclusive	waits	waits

This is the readers/writers problem again. If readers may jump ahead of a waiting writer ("readers first"), a steady stream of readers can starve the writer; serving requests first come, first served prevents that. UNIX `flock` and `fcntl` and Windows `LockFileEx` all offer both kinds.

Deadlock. A transfer from record 3 to record 8 locks r3 then r8; a transfer the other way locks r8 then r3. If each grabs its first record, each then waits for the record the other holds: hold and wait plus circular wait, so neither can ever continue. The chapter 6 cure is **resource ordering**: number the records and always lock them in increasing order. Then no process can hold a higher record while waiting for a lower one, and no cycle can form.

Key terms

File sharing
: Letting more than one user or process use the same file. The file system must then control who may do what to it, and what happens when several of them use it at the same moment.

Access rights
: The operations a particular user is allowed to perform on a particular file, such as reading it, adding to it or deleting it.

Access-rights hierarchy
: The classic ordering of file rights: none, knowledge, execution, reading, appending, updating, changing protection, deletion. Each right includes every right that comes before it.

Knowledge right
: The right to learn that a file exists and who owns it, so the user can ask the owner for more rights, without being able to open the file.

Execution right
: The right to load and run a program file without being able to read it as data or copy it. Often used for licensed programs.

Reading right
: The right to read a file for any purpose, which includes copying it and, for a program, running it.

Appending right
: The right to add new data to a file, usually only at its end, without changing or removing anything already in it.

Updating right
: The right to change, erase and add data anywhere in a file, including writing it for the first time and rewriting it completely.

Changing-protection right
: The right to change which access rights other users hold on a file. Normally only the owner has it.

Deletion right
The right to remove a file from the file system altogether. It sits at the top of the classic hierarchy.

Owner
The user in charge of a file, at first the user who created it. The owner holds every access right and decides which rights other users receive.

User group
A named set of users that receives access rights together: every member gets the rights granted to the group, without being listed one by one.

Public file
A file whose rights are granted to the class “all users”, so every user of the system receives them.

Simultaneous access
Two or more users or processes using the same file during overlapping periods of time. Harmless while everyone only reads; it needs control as soon as anyone changes the file.

Whole-file locking
Locking an entire file for the length of an update, so no other process can update any part of it until the lock is released.

Record locking
Locking only the record, or byte range, being updated, so other processes can work on other parts of the same file at the same time.

Lock granularity
The size of the unit one lock protects: a whole file is coarse, a single record is fine. Finer locks let more work run in parallel but need more locks and more bookkeeping.

Shared lock (read lock)
A lock that many readers can hold at the same time. It keeps writers out while the readers work.

Exclusive lock (write lock)
A lock that only one process can hold, and only while nobody holds any other lock on the same data. Writers take it.

Advisory lock
A lock the system records but does not enforce on ordinary reads and writes: it protects only processes that all agree to ask for the lock first.

Review questions with answers

1. Put the classic file access rights in order, from the weakest to the strongest.

Correct order:

1. None
2. Knowledge
3. Execution
4. Reading
5. Appending
6. Updating
7. Changing protection
8. Deletion

Each right includes every right before it: knowing a file exists, running it, reading it, adding to it, changing it, changing who may use it, and finally deleting it.

2. A user can see that a file named payroll.db exists and who owns it, but every attempt to open it is refused. Which right does the user hold?

A. None
B. Execution
C. Knowledge
D. Reading

Answer: C. Knowledge

The knowledge right lets a user learn that a file exists and who owns it, so the user can ask the owner for more rights. Nothing more.

3. A company wants staff to run a licensed design program without being able to copy it. Which is the highest right it should grant them on the program file?

A. Reading
B. Execution
C. Appending
D. Knowledge

Answer: B. Execution

Execution means “load and run, but do not copy”. That is why it sits below reading, which allows copying.

4. In the classic scheme, the right to change which rights other users hold on a file (changing protection) is normally held only by the file’s owner.

Answer: True

Changing protection sits above updating, so editing a file does not let you hand out rights. Normally only the owner holds it; an owner who passes it on can usually limit which rights may be changed.

5. Match each kind of entry in a file's classic grant list to the people it covers.

Specific user	**One person, identified by user ID**
User group	**A named set of users who get the same rights**
All users	**Everyone on the system: the file is public**
Owner	**At first the creator; holds every right**

An owner can grant rights to one named user, to a group (a set of users defined once), or to all users, which makes the file public. The owner always holds every right.

6. A system uses only the most specific class that matches a user (owner, then named user, then group, then all users). The owner grants Ben the right None by name and grants all users Reading. What may Ben do with the file?

A. Nothing; the named entry hides the file
B. Read it, since the public grant is higher
C. See that it exists, but never open it
D. Read it, but only through a group grant

Answer: A. Nothing; the named entry hides the file

Under the most-specific rule the named entry decides, so Ben gets None even though everyone else may read. UNIX checks owner, group, others in this way, and POSIX access-control lists add named-user entries right after the owner, which lets an owner shut out one person.

7. Two processes update record 7, which holds 1000. P subtracts 300 and Q subtracts 200. There is no locking: both read 1000 before either writes, then P writes, then Q writes. What does record 7 hold at the end?

A. 500
B. 700
C. 1000
D. 800

Answer: D. 800

Q computed 1000 – 200 = 800 from a stale copy and overwrote P's 700. P's update is lost: the classic lost update that a lock prevents.

8. Five processes each update a **different** record of the same file. They all arrive at the same moment, and each update holds its lock for 40 ms. With whole-file locking, how many milliseconds pass until all five are finished?

Answer: 200 ms

A whole-file lock lets only one update in at a time, even for different records: 5 × 40 ms = 200 ms.

9. Six updates arrive at the same moment and want records 2, 5, 2, 9, 5 and 2 of one file. Each holds its lock for 30 ms. With record locking, after how many milliseconds are all six finished?

Answer: 90 ms

Only updates to the same record wait for each other. Record 2 is wanted three times, so its updates queue: 3 × 30 ms = 90 ms. Records 5 and 9 finish sooner, in parallel.

10. Six updates want records 2, 5, 2, 9, 5 and 2 of one file, and there is no locking at all: every update reads its record before any of them writes. How many of the six updates are lost?

Answer: 3

For each record, only the last write survives. Record 2 had three updates (2 lost), record 5 had two (1 lost), record 9 had one (0 lost): 2 + 1 + 0 = 3.

11. Which locking method does each statement describe?

Whole-file locking	Record locking
Only one lock to keep track of for the file Two processes that update different records still wait for each other	Updates to different records can proceed at the same time A transfer that locks two records one after the other can deadlock

A whole-file lock is simple but serializes every update. Record locks allow parallel work on different records, but need many locks, and an update that takes several of them can end up in a circular wait.

12. Which statements about processes that lock shared files are true?

A. Many readers may share a record, but a writer needs it alone (readers/writers)
B. Two processes can each hold a lock the other needs (deadlock)
C. A writer can wait forever if new readers keep jumping ahead (starvation)
D. A process waiting for a file lock must keep polling it in a loop

Answer: A, B, C

Locking brings back the readers/writers problem, deadlock and starvation from earlier chapters. A waiting process does not poll: the OS blocks it and wakes it when the lock is released.

12.6 Record Blocking

You should be able to

- Explain why records must be packed into blocks, and weigh fixed against variable block sizes and large against small blocks.
- Compute the blocking factor, the unused bytes per block and the number of blocks a file of fixed-length records needs.
- Describe fixed blocking, variable-length spanned blocking and variable-length unspanned blocking, with the strengths and costs of each.
- Pack a list of variable-length records under each method and count the blocks used, the bytes wasted and the records split across blocks.

Records versus blocks

A **record** is the logical unit of access: the piece of data a program asks for, such as one customer or one order. A **block** (sometimes called a physical record) is the unit of I/O: the run of bytes, usually of one fixed size, that moves between secondary storage and main memory in a single operation. The two sizes rarely match, so the file system must pack records into blocks before they are written and unpack them after a block is read. This is **record blocking**.

Reading records through a buffer shows why blocks pay off. With 120-byte records and 512-byte blocks, 4 records travel in each block. Reading 12 records in order costs only 3 block reads, because every block brought into the buffer serves the next 3 requests too. In a file of many blocks, reading records in random order rarely finds the needed block already in the buffer, so almost every request costs a full block read that carries 3 records nobody asked for.

Three design questions follow: should all blocks be the same size, how big should they be, and how should records be laid out inside them?

Fixed blocking and the blocking factor

In **fixed blocking** every record has the same length R, and each block of B bytes holds the same whole number of records. That number is the **blocking factor**:

```
bfr = ⌊ B / R ⌋        (rounded down: a record cannot be cut)
unused bytes per block = B − bfr × R
blocks for a file of n records = ⌈ n / bfr ⌉
```

Worked example. B = 512, R = 120: bfr = ⌊4.27⌋ = 4 records per block, using 480 bytes. The remaining 512 – 480 = 32 bytes (6.25% of each block) are too short for another record and are wasted. This waste inside an allocated unit is **internal fragmentation**. A file of 1,000 such records needs ⌈1,000 / 4⌉ = 250 blocks, or 128,000 bytes on the disk for 120,000 bytes of records. Rounding up instead (⌈4.27⌉ = 5) is a classic mistake: 5 records would need 600 bytes.

Because every position is fixed, record k is found by arithmetic: it lies in block ⌊k / bfr⌋, in slot k mod bfr, starting at byte (k mod bfr) × R of that block. With bfr = 4, record 742 is in block 185, slot 2, bytes 240 to 359. Fixed blocking is the usual layout for sequential files of fixed-length records. If a record is longer than a block, bfr = 0 and fixed blocking cannot store the file at all.

Block size: fixed or variable, large or small

Most systems use **one fixed block size**: every transfer moves the same amount, every buffer in main memory is the same size and interchangeable, and blocks are easy to lay out and locate on the disk.

The size itself is a trade-off:

- **Larger blocks** carry more records per I/O, so **sequential processing** (reading every record in stored order) needs fewer block reads.
- For **random access** (single records in unpredictable order) each lookup still moves a whole block, so a larger block moves more records that are not needed.
- Larger blocks need larger I/O buffers in main memory, which are harder to manage.

Block size (10,000 records of 200 B)	bfr	Block reads, full pass	Unwanted records per lookup
512 B	2	5,000	1
4 KB	20	500	19
32 KB	163	62	162

Real systems settle in the middle: ext4 on Linux and NTFS on Windows (which calls its blocks clusters) both default to 4 KB on typical disks.

Variable-length records: spanned and unspanned

When records differ in length (a long address here, a short one there), two methods are available.

Variable-length spanned blocking

Records are packed end to end with no unused space. A record that reaches the end of a block continues at the start of the next block; a **continuation pointer** stored with the first block names the block that holds the rest. Such a record is a **spanned record**.

- Advantages: storage is used efficiently, and there is no limit on record size (a record longer than a block simply continues across as many blocks as it needs).
- Drawbacks: it is hard to implement; a spanned record needs one block read for every block it touches to read on its own, assuming none of them is already cached (two when it crosses one boundary; a record of R bytes starting at offset o in B-byte blocks touches $\lceil (o + R) / B \rceil$ blocks, so a 1,200-byte record at offset 0 in 512-byte blocks touches 3, and at offset 400 it touches $\lceil 1{,}600 / 512 \rceil = 4$; a disk may merge neighbouring blocks into one larger request, but each block is still a separate block access); and growing or shrinking a record shifts every byte after it, so the file is hard to update.
- The fewest blocks any method can use is $\lceil$total bytes / B$\rceil$, and spanned blocking achieves it (real implementations also store a pointer per block and a length per record, which this guide's examples leave out).

Variable-length unspanned blocking

Records are never split. If the next record does not fit in the space left in the current block, that space is abandoned and the record starts a new block.

- Advantages: every record is read with a single I/O, and the layout is simpler.
- Drawbacks: space is wasted at the end of most blocks, and no record may be longer than a block.

Fixed blocking with records that vary

Fixed blocking needs every record to have the same length. To use it for records that vary, the file takes the **longest** record's length L as its record length and **pads** every shorter record with filler bytes up to L. Each block then holds $\lfloor B / L \rfloor$ slots, and the padding is wasted on top of the unused tail ($B - \lfloor B / L \rfloor \times L$) of every block, so one long record can make fixed blocking very wasteful. When all records really are the same length, nothing is padded and fixed blocking is at its best.

Worked example (512-byte blocks; records A–H of 220, 180, 270, 250, 180, 310, 210 and 100 bytes, 1,720 bytes in all):

Method	Blocks	Layout	Wasted
Spanned	4 = ⌈1,720 / 512⌉	[A B C…] […C D E…] […E F G…] […G H]; C, E and G each span two blocks	0 bytes
Unspanned	5	[A B] [C] [D E] [F] [G H]	112 + 242 + 82 + 202 = 638 bytes of gaps
Fixed (padded to 310)	8	one 310-byte slot per block, since ⌊512 / 310⌋ = 1	760 bytes of padding + 8 × 202 = 2,376 bytes

Counting blocks by hand: for fixed blocking compute ⌊B / L⌋ for the slot length L, then ⌈n / bfr⌉; for unspanned, fill each block until the next record does not fit; for spanned, add up all the bytes and divide by B, rounding up. Example: records of 300, 450, 200 and 380 bytes in 512-byte blocks need 4 blocks unspanned (no two neighbours fit together) but only ⌈1,330 / 512⌉ = 3 spanned.

Summary comparison

	Fixed	Variable spanned	Variable unspanned
Record lengths	all equal	vary	vary
Records split?	never	yes, with a pointer	never
Unused space	tail of each block (B – bfr × R)	none	gap at the end of most blocks
Longest record	one block	no limit	one block
Reading one record	one I/O	one I/O per block touched (two if it spans two)	one I/O
Ease	simplest	hardest; hard to update	in between

In real systems. UNIX, Linux and Windows store an ordinary file as a stream of bytes, so a line of text that reaches the end of a block simply carries on in the file's next block: spanning in effect. There the file system's own map of the file's blocks, not a pointer inside each block, says which block comes next. IBM mainframe data sets still name these choices in their record formats: FB (fixed, blocked), VB (variable, blocked, unspanned) and VBS (variable, blocked, spanned).

Key terms

Record
: The logical unit a program reads or writes, such as one customer or one order. Programs ask for records; the hardware never sees them.

Block (physical record)
: The unit of I/O between main memory and secondary storage: a run of bytes, usually of one fixed size, that is always read or written as a whole.

Record blocking
: Packing a file's records into blocks before they are written, and unpacking them again after a block is read.

Blocking factor
The number of fixed-length records that fit in one block: ⌊B / R⌋ for block size B and record size R, rounded down because a record cannot be cut.

Fixed-length record
A record whose length is the same for every record of the file, so the position of record k can be computed directly.

Variable-length record
A record whose length differs from record to record, for example because a text field is long in one record and short in another.

Fixed blocking
Record blocking for fixed-length records: each block holds the same whole number of records, and any bytes left at the end of each block go unused.

Variable-length spanned blocking
Record blocking that packs variable-length records end to end with no gaps; a record that reaches the end of a block continues in the next block.

Variable-length unspanned blocking
Record blocking that never splits a record: when the next record does not fit in the space left, that space is abandoned and the record starts a new block.

Spanned record
A record that is split across two (or more) blocks, so reading it on its own takes one block read per piece.

Continuation pointer
A pointer stored with a block whose last record is incomplete, naming the block that holds the rest of that record.

Internal fragmentation
Space wasted inside an allocated unit because what it holds does not fill it. In fixed blocking it is the tail of each block that is too short for another record.

Padding
Filler bytes added to a shorter record so that it occupies the full fixed record length.

Sequential processing
Reading or writing the records of a file one after another in their stored order, as a payroll run or a backup does.

Random access
Reading single records in an unpredictable order, such as looking up one customer, without needing the records stored around it.

Review questions with answers

1. Fixed blocking uses 4,096-byte blocks and 300-byte records. What is the blocking factor (records per block)?

Answer: 13

⌊4,096 / 300⌋ = ⌊13.65⌋ = 13. Thirteen records use 3,900 bytes; the other 196 bytes of each block are too short for a fourteenth record and stay unused.

2. Fixed blocking uses 512-byte blocks and 120-byte records. How many bytes at the end of each block are unused?

Answer: 32 bytes

⌊512 / 120⌋ = 4 records fit, using 4 × 120 = 480 bytes, so 512 − 480 = 32 bytes (6.25% of the block) are internal fragmentation.

3. A file of 2,000 fixed-length records of 120 bytes is stored with fixed blocking in 512-byte blocks. How many blocks does it occupy?

Answer: 500 blocks

The blocking factor is ⌊512 / 120⌋ = 4, so the file needs ⌈2,000 / 4⌉ = 500 blocks.

4. Variable-length unspanned blocking, 512-byte blocks. The records arrive in this order: 300, 450, 200 and 380 bytes. How many blocks are used?

Answer: 4 blocks

No two neighbours fit together (300 + 450 = 750, 450 + 200 = 650, 200 + 380 = 580, all over 512), so every record starts a new block: 4 blocks. Spanned blocking would need only ⌈1,330 / 512⌉ = 3.

5. Which blocking method can store a record that is longer than one block?

A. Fixed blocking, by giving that record a block of its own
B. Variable-length unspanned blocking, by starting it in a fresh block
C. Variable-length spanned blocking, continuing it in later blocks
D. None of them, because a record must always fit inside one block

Answer: C. Variable-length spanned blocking, continuing it in later blocks

Only spanned blocking lets a record continue past the end of a block, linked by a pointer to the next block, so record size is not limited by block size.

6. In variable-length unspanned blocking, a record that does not fit in the space left in a block is split, and the rest goes into the next block.

Answer: False

Unspanned means records are never split. The leftover space is abandoned and the whole record starts the next block; splitting is what spanned blocking does.

7. Which of these are drawbacks of variable-length spanned blocking? Choose all that apply.

A. It is harder to implement than the other two methods
B. Reading a record that crosses a block boundary takes one block read per block it touches
C. No record may be longer than one block
D. Most blocks end with a stretch of unused space
E. Growing or shrinking a record shifts every byte after it, so the file is hard to update

Answer: A, B, E

Spanned blocking wastes no space and has no record-size limit; its costs are complexity, one block read for every block a spanned record touches (two when it crosses one boundary, more for a record longer than a block), and painful updates. The size limit and the end-of-block gaps belong to unspanned blocking.

8. Sort each statement under the blocking method it describes.

Fixed	Variable spanned	Variable unspanned
Every block holds the same whole number of records Record k is in block ⌊k / bfr⌋, slot k mod bfr	A block may end with a pointer to the block that holds the rest of a record Records are packed end to end with no gaps at all	Records vary in length, and leftover space is abandoned when the next record does not fit Variable-length records are never cut, so each one is read with a single I/O

Fixed blocking: equal records and a constant blocking factor, so positions are pure arithmetic. Spanned: no gaps, records may continue in the next block via a pointer. Unspanned: varying records kept whole, so each takes one I/O but gaps are left behind.

9. A program asks for record k of a file that uses fixed blocking. Put the file system's steps in order.

Correct order:

1. Compute the block number ⌊k / bfr⌋
2. Check whether that block is already in a buffer in main memory
3. If it is not, read the whole block from the disk into a buffer
4. Copy the record at offset (k mod bfr) × R in the buffer to the program

The block number comes first, because that is what the disk understands. Only a block that is not already buffered costs an I/O; the record is then unpacked from its slot in the buffer.

10. Match each term to its meaning.

Record	**The logical unit a program reads or writes**
Block	**The unit of I/O with secondary storage**
Blocking factor	**The number of records that fit in one block**
Internal fragmentation	**Unused bytes at the end of each fixed block**
Continuation pointer	**Names the block that holds the rest of a spanned record**

Programs work in records, the disk moves blocks, and record blocking bridges the two: the blocking factor counts records per block, and each method leaves its own kind of waste or pointer.

11. A payroll program reads every record of a large file in stored order. What does switching the file to larger blocks do for this program?

A. Every read now moves records the program does not need, so the run slows down
B. More records arrive with each I/O, so the run needs fewer block reads
C. Nothing, since the number of block reads depends only on the number of records
D. It removes the internal fragmentation at the end of every block

Answer: B. More records arrive with each I/O, so the run needs fewer block reads
Sequential processing uses every record a block brings, so larger blocks mean fewer I/Os for the same file. Random access is the workload that suffers from large blocks.

12. Why do most systems use a single fixed block size?

A. Fixed blocks always hold more records than variable blocks of the same average size
B. Fixed blocks remove internal fragmentation from every file
C. Fixed blocks make it impossible for any record to span two blocks
D. Fixed blocks keep I/O, buffer allocation and disk layout simple

Answer: D. Fixed blocks keep I/O, buffer allocation and disk layout simple
With one block size every transfer moves the same amount, every buffer in main memory is interchangeable, and blocks are easy to place and find on the disk.

12.7 Secondary Storage Management

You should be able to

- Weigh preallocation against dynamic allocation, and explain how portion size trades speed, table size and wasted space.
- Lay out files with contiguous, chained and indexed allocation, and compare their tables, fragmentation and random-access cost.
- Track free space with a bit table, chained free portions, an index or a free block list, and compute how big a bit table is.
- Explain what a volume is, and how a crash or two concurrent allocators can hand the same block to two files.
- Apply the lock-search-allocate-update-unlock procedure that keeps allocation safe.

Two records the file system must keep

For every block of secondary storage the OS must know (1) which file owns it and where it comes in that file, and (2) whether it is free. Deciding and recording the first is **file allocation**; its record is the **file allocation table**. Tracking the second is **free-space management** (also called free storage management); its record is the **disk allocation table**. Every create, grow and delete changes both. All of this happens inside a **volume**: a logical disk, meaning a set of addressable sectors that an OS or application can use for data. A volume's sectors need not be contiguous and may be spread over several physical disks.

Three allocation decisions

- **When: preallocation** gives a file its declared maximum size at creation; **dynamic allocation** gives space in portions as the file grows. Final sizes are hard to guess, so people overestimate and preallocation wastes space; guessing too low forces a copy to a bigger area or an error.
- **How much:** a **portion** can be one block or the whole file. Contiguity (big portions) speeds reading; many small portions make the allocation tables big; fixed-size portions make reallocation simple; variable or small fixed portions reduce the space lost to overallocation. Example: a 23-block file in 4-block portions needs 6 portions (6 table entries) with 1 unused block.
- **Two main options:** large, variable-size contiguous portions (good performance, little waste, but complex tables and fragmented free space) or small fixed blocks (flexible, simple, but large tables and scattered files).
- **Where:** a variable portion is placed with **first fit** (first free portion big enough), **best fit** (smallest one big enough) or **nearest fit** (the one closest to the file's previous portion, for locality), as with main memory in chapter 7. Example with free portions 2–4, 9–14, 16–17, 24–28, 33–36, a previous portion at 20–23 and a 4-block request: first fit picks 9–14, best fit 33–36, nearest fit 24–28.

The three allocation methods

Contiguous allocation

One run of consecutive blocks per file, chosen at creation (preallocation is necessary). Table entry: **start block and length**. Best for sequential reading (no seeks inside the file) and direct access is easy: block k is at start + k, one read. Weakness: **external fragmentation** of free space; a new file can be refused although enough blocks are free in total, so the disk needs **compaction** now and then, which copies many blocks. A file that grows may have to be moved.

Chained (linked) allocation

Blocks can be anywhere; each block holds a pointer to the next, and the last says end. Table entry: **start block and length**. Any free block can be added, so there is no external fragmentation and growth is easy. Sequential reading is fine, but reaching block k needs k + 1 reads (average (n + 1)/2 for an n-block file), and blocks scatter (poor locality), so files are **consolidated** now and then. **FAT** (MS-DOS and Windows) keeps the chain pointers in one table with an entry per cluster instead of inside the blocks; cached in memory, the chain can be followed without disk reads.

Indexed allocation

Each file has a one-level index, kept in its own **index block**, with one entry per block or portion of the file; the table entry points to the index block. With fixed-size blocks there is no external fragmentation; with variable portions (start, length) the index is smaller and locality is better. It supports sequential and direct access (read the index, then block k: 2 reads, or 1 if the index is cached). It is the most widely used method.

Comparison

	Contiguous	Chained	Indexed (blocks)	Indexed (portions)
Preallocation	Necessary	Possible	Possible	Possible
Fixed or variable portions	Variable	Fixed blocks	Fixed blocks	Variable
Portion size	Large	Small	Small	Medium
Allocation frequency	Once	Low to high	High	Low
Time to allocate	Medium	Long	Short	Medium
File allocation table size	One entry	One entry	Large	Medium

Free-space management

- **Bit table (bit map):** one bit per block, 0 = free, 1 = in use. Small, and runs of free blocks are easy to find. Size = number of blocks ÷ 8 bytes. A large table is slow to scan, so **summary tables** record, for each range of blocks, the number of free blocks and the longest free run, and the allocator scans only ranges that can satisfy the request.
- **Chained free portions:** free portions are linked; each stores its length and a pointer to the next free portion. Almost no space overhead, but free space fragments over time, and allocating a block first requires reading it to recover the pointer it holds.
- **Indexing:** free space is treated like a file with an index table; for efficiency, one entry per free (variable-size) portion.
- **Free block list:** each free block's number is kept in a list on disk, 24 or 32 bits per entry (so the list is 24 or 32 times the size of a bit table). It is used as a stack or a FIFO queue, with only a small part held in memory.

Worked example

A 16-GB disk (2^{34} bytes) with 512-byte blocks (2^{9}) has 2^{25} = 33,554,432 blocks. Its bit table is 2^{25} bits = 2^{22} bytes = **4 MB**, which itself fills 8,192 blocks (1/4096 of the disk). Block numbers need 25 bits, so a free block list would use 32-bit entries: 2^{25} × 4 bytes = **128 MB** when the disk is empty.

Reliability

For speed, the OS keeps copies of the disk allocation table and file allocation data in memory. Two dangers follow. **Crash:** A is given blocks 20–21 in memory, a pointer to block 20 is written into A's file on disk, then the power fails before the on-disk table records 20–21 as used; after the restart the stale table gives 20–21 to B, and both files are corrupted. **Race:** two allocators both search the table, both see the same free blocks before either marks them, and both take them. The cure is to allocate in this order:

1. Lock the disk allocation table on disk.
2. Search it for free space (the memory copy is trustworthy while the lock is held).
3. Allocate the space, update the disk allocation table and write it to disk (for chained allocation, also update the pointers on disk).
4. Update the file allocation table and write it to disk.

5. Unlock the disk allocation table.

The lock stops interleaving. Writing the free-space record before the file's own records means a crash can at worst leave **lost blocks** (marked used, owned by no file), which a disk check can reclaim, but never a block owned by two files.

Key terms

Preallocation
Giving a file its maximum size in disk space at the moment it is created, before any data is written.

Dynamic allocation
Giving a file disk space in portions only as it actually needs them, while it grows.

Portion
A contiguous piece of disk space handed to a file in one allocation. It can be as small as one block or as large as the whole file.

File allocation table
The record the OS keeps of which portions of the disk belong to each file. What one entry holds depends on the allocation method.

Contiguous allocation
An allocation method that gives each file one run of consecutive blocks when it is created; the table needs only the start block and the length.

Chained allocation (linked allocation)
An allocation method in which a file's blocks may sit anywhere; each block stores the number of the next block of the file, and the table holds the first block and the length.

Indexed allocation
An allocation method in which each file has an index (normally kept in its own block) listing the file's blocks or portions in order; the table entry points to that index.

Index block
The disk block that holds a file's index: one entry per block or portion of the file, in file order.

FAT (file allocation table file system)
The file system of MS-DOS and early Windows, still common on memory cards and USB sticks. It is chained allocation with every next-pointer moved out of the data blocks into one table, one entry per cluster.

Nearest fit
A placement rule for variable-size portions: choose the free portion closest to the file's previous portion, so the file's pieces stay near each other.

Consolidation
Rewriting a scattered file (or all files) so each file's blocks sit next to each other again, which shortens the seeks needed to read it.

Bit table (bit map)
A free-space record with one bit per disk block: 0 means the block is free and 1 means it is in use.

Summary table

A small table kept beside a bit table that records, for each range of blocks, how many are free and the longest free run, so most ranges can be skipped without scanning their bits.

Chained free portions

A free-space method that links the free portions together; each free portion stores its own length and a pointer to the next free portion.

Free block list

A free-space method that keeps the number of every free block in a list on disk (24 or 32 bits per entry), used as a stack or a FIFO queue with only a small part held in memory.

Disk allocation table

The OS's record of which blocks of a volume are free, kept with any of the free-space methods. It is updated on every allocation and every deletion.

Volume

A logical disk: a set of addressable sectors on secondary storage that an OS or application can use for data. Its sectors need not be contiguous and may be spread over several physical disks.

Lost block

A block marked as in use that no file refers to. It wastes space but corrupts nothing, and a disk-checking utility can find it and free it.

Review questions with answers

1. Which file allocation method **requires** preallocation, so that a file's full size must be reserved when the file is created?

A. Chained allocation, where each block names the next one
B. Indexed allocation with single fixed-size blocks
C. Contiguous allocation, one run of blocks per file
D. Indexed allocation with variable-size portions

Answer: C. Contiguous allocation, one run of blocks per file

A contiguous file is one run, so the whole run must be chosen at creation; later there may be no free block next to it. The other methods can grow a file one block or portion at a time.

2. A 16-GB disk (2^{34} bytes) uses 512-byte blocks. How many megabytes (1 MB = 2^{20} bytes) does its bit table occupy?

Answer: 4 MB

$2^{34} \div 2^{9} = 2^{25}$ blocks, one bit each: 2^{25} bits = 2^{22} bytes = 4 MB.

3. A disk has 2^{25} blocks. A free block list stores each free block's number in a 32-bit entry. How many megabytes does the list need when every block is free?

Answer: 128 MB

2^{25} entries × 4 bytes = 2^{27} bytes = 128 MB, which is 32 times the size of the 4-MB bit table for the same disk.

4. A file uses chained allocation and none of its blocks are in memory. Blocks are numbered from 0 in file order. How many disk reads does the OS need to get block 6?

Answer: 7 reads

The address of each block is stored only in the block before it, so the OS must read blocks 0 to 6: k + 1 = 7 reads.

5. How does the FAT file system differ from plain chained allocation?

A. It stores each file in one contiguous run of clusters, so no next pointers are needed at all
B. It gives each file its own index block that lists all of that file's clusters in order
C. It replaces the chain with a bit table that marks which clusters belong to which file
D. It moves every next pointer out of the data blocks into one table, one entry per cluster

Answer: D. It moves every next pointer out of the data blocks into one table, one entry per cluster

FAT is still a chain, but the links live in one table with an entry per cluster. The table can be cached in memory, so following the chain needs no disk reads.

6. Which statement about a **volume** is correct?

A. It is always exactly one whole physical drive, never part of one or several
B. It is a logical disk whose sectors may be spread over several physical disks
C. It is the region of the file system that records which of its blocks are free
D. It is the set of blocks that belong to one file, wherever on the disk they lie

Answer: B. It is a logical disk whose sectors may be spread over several physical disks

A volume is a collection of addressable sectors that an OS or application can use for storage. They need not be contiguous and may sit on several physical disks.

7. After a crash and restart, the OS gives file B two blocks that file A is still using. What made this possible?

A. A reference to the blocks reached the disk before the on-disk table recorded them as used
B. The disk allocation table on disk was locked before the search for free blocks began
C. The file allocation table was written to disk only after the disk allocation table
D. The OS searched its cached copy of the free-space table instead of reading the table on disk

Answer: A. A reference to the blocks reached the disk before the on-disk table recorded them as used

The memory copy said "used", but the disk still said "free" when the crash erased memory. After the restart the stale on-disk table handed the blocks out again.

8. Chained allocation suffers from external fragmentation, so the disk must be compacted from time to time to keep it usable.

Answer: False

Any free block can join a chain, so free space never becomes unusable. Files may be consolidated now and then, but only to improve locality (fewer seeks).

9. Put the steps of safe block allocation in order.

Correct order:

1. Lock the disk allocation table on disk
2. Search the table for free space
3. Allocate the space and write the updated table to disk
4. Update the file allocation table and write it to disk
5. Unlock the disk allocation table

Locking stops two allocators from interleaving. Writing the free-space table before the file's own records means a crash can only leave lost blocks, never blocks owned twice.

10. Match each method to the property that best describes it.

Contiguous allocation	**Needs compaction now and then to cure external fragmentation**
Chained allocation	**Must follow pointers block by block to reach block k**
Indexed allocation	**Keeps a separate block listing the file's blocks in order**
FAT	**Keeps every next pointer in one table that can be cached**

Each method's strength and weakness follows from where it records the order of a file's blocks: implied by position, inside each block, in a per-file index, or in one shared table.

11. Which free-space method does each statement describe?

Bit table	Chained free portions	Free block list
One bit per block: 0 free, 1 in use Summary tables per range of blocks speed up searching	Each free portion stores its length and a pointer to the next one Allocating a block first means reading that block from disk	Each entry is a 24- or 32-bit block number Often used as a stack with only its top part in memory

A bit table maps every block to one bit; chained free portions keep their bookkeeping inside the free space itself; a free block list stores full block numbers.

12. Which statements about the size of allocation portions are true?

A. Larger portions mean fewer entries in the file allocation table
B. Fixed-size portions make reallocating space simple
C. Small fixed-size blocks give a file the best contiguity
D. Variable-size portions can leave free space in many odd-sized holes

Answer: A, B, D

Big portions cut table size and seeks; fixed sizes make any free portion fit any need. Small blocks are flexible but scatter a file, and variable portions fragment free space.

12.8 UNIX File Management

You should be able to

- Name the six classic UNIX file types and say what each one is for.
- Describe the main fields of an inode and explain why the file's name is not one of them.
- Explain hard links, symbolic links and the link count, and predict what happens when a name is removed.
- Compute how far the direct, single, double and triple indirect pointers reach, and which blocks must be read to find any byte of a file.
- Trace a path lookup from the root inode and describe the layout of a UNIX volume.

UNIX file management

In UNIX a file is three separate things kept in three places: a **name** in a directory, an **inode** (index node) that describes the file, and **data blocks** that hold its bytes. A directory entry pairs a name with an **inode number**; everything else the system knows about the file lives in the inode. This split is what makes links, quick lookups and very large files possible.

The six classic file types

Type	ls -l letter	What it is
Regular (ordinary)	-	Any bytes a program wrote: text, photos, programs. The system imposes no structure; programs give the bytes meaning. An executable program is a regular file too.
Directory	d	A list of (name, inode number) pairs that builds the tree. Programs may read it; only the file system writes it.
Special	c or b	One per I/O device (character or block). Reading or writing it drives the device; the major and minor device numbers select the driver and the unit.
Named pipe (FIFO)	p	A pipe with a name in the tree, so unrelated processes can use it. Bytes pass first in, first out, through a kernel buffer; only the name and inode are on disk, never the bytes.
Link (hard link)	the file's own letter	Not a new object: one more directory entry naming an existing inode.
Symbolic link	l	Its own small file whose contents are the path name of another file.

Later UNIX systems (from 4.2BSD, 1983) add a seventh type, the socket (s), a named two-way endpoint between processes on one machine. Because devices and pipes are files, the same open, read, write and close calls work on all of them.

The inode

Every file has exactly one inode; several names may lead to the same inode, but an inode belongs to one file. The inodes sit in the **inode table** on disk, and the inode number is simply the position in that table (so the number itself is not stored inside). When a file is opened, the kernel copies its inode into the **in-**

memory inode table and adds bookkeeping such as a count of open references. Typical fields (a modern UNIX such as FreeBSD):

- **File mode**: 4 type bits, then set-user-ID, set-group-ID and sticky bits, then read/write/execute for owner, group and others. Example: `-rw-r--r--` = octal 0100644. On open the kernel compares the process's user and group IDs with the owner and group IDs and applies the matching three bits; a set-user-ID program runs with its owner's rights, and the sticky bit on a shared directory such as /tmp stops users from deleting each other's files.
- **Link count**: how many directory entries name the inode.
- **Owner and group IDs** (numbers only).
- **Size** in bytes (the logical length) and **block count** (space really allocated, indirect blocks included; often counted in 512-byte units).
- **Times**: last access (atime), last modification of the data (mtime), last change of the inode itself (ctime). ctime is *not* creation time; classic UNIX kept none, while UFS2 and ext4 later added a birth time.
- **Block addresses**: 12 direct pointers plus single, double and triple indirect pointers.
- **Flags** (such as immutable or append-only), a **generation number** (changed whenever the slot is reused, so stale network file handles are detected) and the **size of the extended attributes**.

The file's **name is not in the inode**; it lives only in directory entries.

Hard links, symbolic links and the link count

- `ln a b` adds a directory entry b with a's inode number; the link count rises by 1. No data is copied and neither name is the "original".
- `rm` (the unlink call) removes one name and lowers the link count. The inode and its blocks are freed only when the count reaches 0 and no process still has the file open.
- `ln -s a s` creates a new inode of type symbolic link whose data is the text "a". The target's link count does not change, and the target need not exist. Opening s makes the kernel look the stored path up again, so if a is removed, s is a **dangling link**; if a new file named a appears, s finds that new file.
- Hard links cannot cross file systems (inode numbers are local to one file system) and normally cannot name directories; symbolic links can do both.

File allocation: indexed, with the index in the inode

Space is allocated dynamically, one block at a time, and a file's blocks need not be contiguous. The index of the file's blocks is kept in the inode: 12 **direct pointers** name data blocks; the **single indirect pointer** names an index block full of data-block addresses; the **double indirect pointer** names an index block of pointers to index blocks; the **triple indirect pointer** adds a third level. Older System V inodes held 13 addresses (10 direct + 3 indirect) packed into 39 bytes, 3 bytes each.

With block size B and address size p, one index block holds P = B / p addresses.

Level	Data blocks	4 KB blocks, 8-byte addresses (P = 512)	4 KB blocks, 4-byte addresses (P = 1,024)
Direct	12	48 KB	48 KB
Single indirect	P	512 blocks = 2 MB	1,024 blocks = 4 MB
Double indirect	P^2	262,144 blocks = 1 GB	1,048,576 blocks = 4 GB
Triple indirect	P^3	134,217,728 blocks = 512 GB	about 1.07 billion blocks = 4 TB
Largest file	$12 + P + P^2 + P^3$	134,480,396 blocks ≈ 513 GB	≈ 4 TB

(1 KB = 1,024 bytes, 1 MB = 1,024 KB, and so on.) The tree's reach is only an upper bound. A block address of p bytes can number at most 2^{8p} blocks: with 4-byte addresses that is 4,294,967,296 blocks, or 16 TB of 4 KB blocks, so there the 4 TB tree is the tighter limit; with 8 KB or larger blocks the block numbers run out before the tree does. Real file systems may set still lower limits through other fields.

Finding byte N. Logical block $L = \lfloor N / B \rfloor$. If $L < 12$ use direct pointer L. Otherwise subtract 12; if the rest is below P, use entry (L – 12) of the single indirect block. Otherwise subtract P; below P^2 use the double indirect pointer with entries $\lfloor r / P \rfloor$ and r mod P; otherwise subtract P^2 and use the triple indirect pointer, whose three entries are the base-P digits of the rest. Worked examples (4 KB blocks, 8-byte addresses, file open):

- N = 100,000 → L = 24 → single indirect, entry 12 → 1 index read + 1 data read = 2 reads.
- N = 5,242,880 (5 MB) → L = 1,280 → double indirect, r = 756 → entries 1 and 244 → 3 reads.
- N = 2,147,483,648 (2 GB) → L = 524,288 → triple indirect, r = 261,620 → entries 0, 510, 500 → 4 reads.

Why this design works. Most files are small, and every file up to 48 KB is mapped by the inode alone, with no index block to store or read. Large files remain possible, and reaching any byte costs at most 3 extra index reads (fewer once those blocks are cached). The index is cheap in space: a 1 GB file needs 262,144 data blocks and only 513 index blocks (about 0.2%); a 49 KB file needs 13 data blocks and 1 index block.

Directories and path lookup

A directory is a file of (name, inode number) pairs; each one also lists `.` (itself) and `..` (its parent). To open /home/ann/notes.txt the kernel starts at the root directory, whose inode number is fixed in advance (2 on many UNIX file systems), and for each name reads the directory's inode (to find its blocks), reads the directory block, and finds the name to get the next inode number. With one block per directory and nothing cached that is 2 × 3 + 1 = 7 reads: root inode, root block, /home inode, /home block, /home/ann inode, /home/ann block, notes.txt inode. A relative path starts at the working directory's inode instead. The kernel checks search (execute) permission on each directory, and caches recent names and inodes, so repeated lookups usually need no disk reads.

Volume structure

- **Boot block**: space at the start of the volume for code that starts the operating system.
- **Superblock**: describes the file system: its size, block size, number of inodes and where the inode table is, free block and free inode counts, and its state. Real systems keep backup copies.

- **Inode table**: one fixed-size inode for every file the volume can hold; the count is fixed when the file system is created.
- **Data blocks**: file contents, directory contents and indirect blocks.

Example: a 64 GB volume with 4 KB blocks has 16,777,216 blocks. With one 256-byte inode per 16 KB of space it has 4,194,304 inodes, an inode table of 1 GB (about 1.6% of the volume), and 16,515,070 data blocks. Running out of inodes makes the disk "full" even with free blocks left. Modern UNIX file systems split the volume into groups (cylinder groups in FreeBSD, block groups in Linux ext4), each with its own inodes and data blocks, and keep backup copies of the superblock in several of them.

Key terms

Inode (index node)
The small fixed-size record a UNIX file system keeps for every file. It holds the file's type, permissions, owner, size, times, link count and the addresses of its data blocks, but not its name.

Inode number (i-number)
The position of an inode in the inode table. A directory entry ties a file name to this number.

Inode table
The array of inodes kept at known places on the volume, one slot for every file the file system can hold.

In-memory inode table
The kernel's table of inode copies for files that are open or in use, with extra bookkeeping such as how many open references each one has.

Regular file (ordinary file)
A file holding whatever bytes a program wrote into it: text, a photo, a program. The file system imposes no structure on its contents.

Named pipe (FIFO)
A pipe that has a name in the directory tree, so unrelated processes can open it. Bytes written by one process come out, first in first out, for another; only the name and inode are on disk, never the bytes.

Hard link
An extra directory entry for an existing file: another name holding the same inode number. Every name is equal; none of them is the "original".

Symbolic link (soft link)
A small file whose contents are the path name of another file. Opening it makes the kernel follow that path.

Link count
The inode field that counts the directory entries (hard links) naming the file. The inode and its blocks are freed only when the count reaches 0 and no process still has the file open.

Dangling link
A symbolic link whose stored path no longer names an existing file, so following it fails.

File mode
The inode field that encodes the file's type and its permission bits: read, write and execute for the owner, the group and everyone else, plus a few special bits.

Direct pointer
One of the twelve block addresses in an inode that name a data block of the file directly.

Indirect block (index block)
A disk block that holds nothing but block addresses. The indirect pointers in an inode lead to such blocks.

Single indirect pointer
The inode entry that names one indirect block whose entries point straight at data blocks.

Double indirect pointer
The inode entry that names an indirect block full of pointers to more indirect blocks, which point at data blocks: two levels of index.

Triple indirect pointer
The inode entry that adds a third level of indirect blocks, so that very large files can still be mapped.

Indexed allocation
A file allocation method that keeps, for each file, an index of the addresses of its blocks, so any block can be found without reading the blocks before it.

Boot block
The area at the very start of a volume set aside for the code that starts (boots) the operating system.

Superblock
The block that describes a whole file system: its size, block size, how many inodes it has and where they are, how much space is free, and its state.

Path lookup (pathname resolution)
Turning a path such as /home/ann/notes.txt into an inode number by starting at the root (or working) directory and reading one directory at a time.

Review questions with answers

1. In a UNIX file system, what does a directory entry hold?

A. The file's owner and permission bits
B. The file's name and its inode number
C. The file's size and its block addresses
D. The file's name and its access times

Answer: B. The file's name and its inode number
A UNIX directory is a file of (name, inode number) pairs. Everything else about the file lives in its inode, which is why one file can have several names.

2. An inode's ctime field records when the file was created.
Answer: False
The c stands for change: ctime is the last time the inode itself changed (a write, chmod, chown or a new link). Classic UNIX kept no creation time; some newer file systems add a separate birth time.

3. A UNIX file system uses 4 KB blocks and 12 direct pointers per inode. What is the largest file size, in KB, that needs no indirect block at all?
Answer: 48 KB
12 direct pointers × 4 KB per block = 48 KB. Any file up to that size is mapped by the inode alone.

4. Blocks are 4 KB and each block address takes 8 bytes. How many data blocks can the double indirect pointer reach?

Answer: 262144 blocks

One block holds 4,096 / 8 = 512 addresses. The double indirect pointer leads to 512 index blocks, each pointing to 512 data blocks: 512 × 512 = 262,144 blocks (1 GB).

5. Blocks are 4 KB, block addresses are 8 bytes, and the inode has 12 direct pointers. Which pointer leads to the block that holds byte offset 100,000 of a file?

A. The double indirect pointer
B. One of the direct pointers
C. The triple indirect pointer
D. The single indirect pointer

Answer: D. The single indirect pointer

⌊100,000 / 4,096⌋ = 24. Blocks 0 to 11 are direct; blocks 12 to 523 go through the single indirect block, so block 24 is entry 24 − 12 = 12 of that block.

6. Blocks are 4 KB, addresses are 8 bytes, 12 direct pointers. A file is open (its inode is in memory) and nothing else is cached. How many disk reads does it take to read the byte at offset 2,147,483,648 (2 GB)?

Answer: 4 reads

Logical block 524,288 is past 12 + 512 + 262,144 = 262,668, so it is reached through the triple indirect pointer: 3 index blocks plus 1 data block = 4 reads.

7. Ann runs `ln report.txt copy.txt` and then `rm report.txt`. What happens to the data?

A. copy.txt becomes a dangling link to a missing file
B. The data is freed because report.txt was the original
C. copy.txt still reaches it; the link count drops to 1
D. rm refuses, because the file still has two names

Answer: C. copy.txt still reaches it; the link count drops to 1

`ln` made a second entry for the same inode (link count 2). `rm` removed one entry (link count 1), so the inode and its blocks stay, reachable through copy.txt.

8. Sort each property: does it describe a hard link or a symbolic link?

Hard link	Symbolic link
Is another directory entry holding the same inode number	Has its own inode whose contents are a path name
Raises the target's link count	Can name a file on a different file system
Still works after the other name is removed	Breaks (dangles) if the target name is deleted

A hard link is the file itself under another name, so it shares the inode and its link count. A symbolic link is a separate small file holding a path, so it can cross file systems but fails once that path names nothing.

9. Nothing is cached. Put the disk reads for opening `/home/ann/notes.txt` in order.

Correct order:

1. Read the root directory's inode
2. Read the root directory's block and find "home"
3. Read the inode of /home
4. Read the block of /home and find "ann"
5. Read the inode of /home/ann
6. Read the block of /home/ann and find "notes.txt"
7. Read the inode of notes.txt

Path lookup starts at the root inode and alternates: a directory's inode tells where its blocks are; its block gives the next inode number. Three names cost 2 × 3 + 1 = 7 reads.

10. Match each UNIX file type to its description.

Regular file	**Arbitrary bytes, with no structure imposed by the system**
Directory	**A list of (name, inode number) pairs**
Special file	**Gives programs access to an I/O device**
Named pipe	**A first-in, first-out channel between processes**
Symbolic link	**A small file whose contents are a path name**

All of them live in the same directory tree and are used with the same open, read, write and close calls; the type in the inode's mode field tells the kernel how to treat each one.

11. Which of these are kept in a UNIX inode?

A. The file mode (type and permission bits)
B. The link count
C. The file's name
D. The time of the last modification
E. The addresses of the file's blocks
F. The name of the parent directory

Answer: A, B, D, E

The inode holds the mode, link count, owner and group, size, times and block addresses. Names, including the parent's, live only in directory entries.

12. Which part of a classic UNIX volume records how big the file system is, how many inodes it has and how much space is free?

A. The superblock
B. The boot block
C. The root directory
D. The inode table

Answer: A. The superblock

The superblock describes the whole file system: its size, block size, inode count and location, and free counts. Because it is so important, real systems keep backup copies of it.

12.9 Linux Virtual File System

You should be able to

- Explain how the Linux virtual file system lets one set of system calls work on ext4, a USB stick, a network share or /proc.
- Trace a read() from a process through the VFS to a specific file system and its device, and say which parts change with the file system.
- Describe the superblock, inode, dentry and file objects: what each one stands for, what it holds and which operations table it carries.
- Explain how structs with tables of function pointers make the VFS object-oriented in C, and how a new file system plugs in.
- Show how the dentry cache, including negative dentries, saves work on repeated path lookups.

Why Linux has a virtual file system

One Linux machine often uses several file systems at once: **ext4**, **XFS**, **Btrfs** (copy-on-write snapshots), **vfat** (USB sticks, SD cards), **ntfs3** (Windows disks, read-write since Linux 5.15), **NFS** (files on another computer) and **procfs** (/proc, on no device). Programs use them all with the same calls.

The **virtual file system (VFS)** is the kernel layer that makes this possible. The VFS works with one generic picture of files, the **common file model**, does the jobs that are the same for every file system (path lookup, permission checks, file descriptors, the dentry cache) and hands only the format-specific work to the right file system. In the classic picture each file system supplies a **mapping function** between the common model and its own format; in Linux that mapping is the set of operations tables the file system fills in.

Layers, top to bottom: process → system call interface → VFS → one file system → page cache (not for procfs) → block layer and driver (NFS: RPC, TCP/IP, network driver) → device or server.

Tracing one read(3, buf, 4096)

1. The process calls read; the call names a descriptor, never a file system.
2. The VFS finds the process's file object for descriptor 3 (opened for reading, offset 0).
3. The VFS calls `file->f_op->read_iter`.
4. The page cache is searched for the page. On a miss, the file system finds the data and a driver fetches it.
5. The bytes are copied into buf, the offset moves forward by the number of bytes read, and read returns that number.

	read_iter slot holds	finds the data by	below it
ext4	ext4_file_read_iter	the file's extent tree (file block → disk block)	block layer, NVMe driver, SSD
vfat	generic_file_read_iter (a shared helper)	first cluster in the directory entry, then the FAT chain	block layer, USB storage driver, stick
NFS	nfs_file_read (revalidates the cache first)	a READ request to the server	RPC, TCP/IP, network driver, server
procfs	procfs's read function	printing kernel counters on the spot	nothing: no cache, no device

The top three layers never change; only what sits behind f_op differs.

Objects in C: data plus a table of functions

C has no classes, so the VFS builds objects by hand. Each object is a struct of data plus a pointer to an **operations table**: a struct whose members are **function pointers**, one slot per job. Every file system fills in its own copy of each table. To get a job done the VFS calls through the slot, for example `file->f_op->read_iter(...)`, and runs whatever function the file system put there. An empty (NULL) slot means the VFS default applies (no lock slot: generic POSIX locks in local memory; no open slot: nothing extra to do) or the job is not supported (vfat's directories have no link slot, so ln on a FAT stick fails). With tables the VFS never names a file system, so adding one changes no VFS code.

Examples: ext4 leaves lock empty; vfat uses shared generic_ helpers and leaves open empty; NFS fills lock with nfs_lock so that other machines see the lock.

How a new file system plugs in

1. Its module calls `register_filesystem()` with its type name and mount routine.
2. At mount, that routine reads the on-disk superblock (NFS asks the server), fills in a superblock object and sets s_op and the root dentry.
3. As files are looked up, it builds inode objects whose i_op and i_fop point at its own tables.
4. At open, the VFS creates a file object and copies the inode's i_fop into the file's f_op.

The four primary objects

Object	Stands for	Created	On disk?	Table
Superblock	one mounted file system	at mount	filled from the on-disk superblock	super_operations
Inode	one file (a directory is a file too)	when the file is first needed	usually built from the on-disk inode	inode_operations
Dentry	one component of a path	during path lookup	never, memory only	dentry_operations
File	one open() of a file by a process	open(); freed at the last close()	never, memory only	file_operations

Superblock object. Device, type, block size (ext4: 4,096 bytes), magic number, largest file (ext4 with 4 KB blocks: about 16 TiB; FAT: 4 GiB – 1 byte), mount flags such as read-only, the root directory's dentry (s_root), the s_umount semaphore held while (un)mounting, private data and s_op. super_operations include alloc_inode, write_inode, evict_inode, put_super (unmount), sync_fs and statfs (free space for df). Older kernels also had read_inode (removed in 2.6.25, 2008); 2.6-era superblocks also kept a dirty flag and a list of open files, both dropped in the 3.x series.

Inode object. Inode number, mode (type and permission bits), owner, group, size, link count (i_nlink), blocks used in 512-byte units (a 2,140-byte ext4 file on 4 KB blocks: 8), three times, i_sb, i_op and i_fop (copied to f_op at open). A directory's inode_operations hold create, lookup, link, unlink, symlink, mkdir, rmdir, mknod and rename; a regular file's hold mostly attribute jobs such as setattr and getattr. vfat has no on-disk inodes, so it invents the inode number and takes owner and permissions from the mount options.

Dentry object. d_name, d_parent, d_inode (NULL in a negative dentry), d_sb and d_op. Looking up /bin/vi uses three dentries: "/", "bin" and "vi". A dentry is not the on-disk directory entry, but an in-memory object built only for names that were looked up. dentry_operations include d_revalidate (is a cached entry still right? NFS needs this because other machines change directories), d_hash and d_compare (vfat fills them to ignore case, so PHOTO.JPG finds photo.jpg), d_delete, d_release and d_iput. ext4 normally leaves this table empty.

File object. f_path (the dentry and its mount), f_inode, f_pos (the current offset), f_mode (read and/or write), f_flags (as passed to open), a use count (f_ref; f_count before 6.13) and f_op. file_operations include llseek, read and read_iter, write and write_iter, mmap, open, release (after the last close), fsync and lock. read_iter/write_iter (Linux 3.16, 2014) replaced read/write in disk file systems; some drivers still use the old slots.

Traps: write_inode is a *superblock* operation (the file system knows how its inodes are stored), and lookup belongs to the directory's *inode*, not to the dentry it produces.

Who shares what

- **fork()** copies the descriptor table, so parent and child point to the **same file object** and share one offset. Example: P1 reads 100 bytes, forks, the child reads 50 bytes; P1's next read starts at 100 + 50 = 150. dup() shares a file object the same way.
- **Two open() calls** (by two processes or one) create **two file objects** with separate offsets that share one dentry and one inode.
- **A hard link** (ln notes.txt n2) adds a second name: two dentries, one inode, and i_nlink becomes 2.

The dentry cache

Path lookup resolves one component at a time, starting from the root (or working) directory. For each component the VFS first searches the **dentry cache (dcache)** by parent and name. A hit costs nothing more. A miss is a slow lookup: the VFS calls the parent directory inode's i_op->lookup, the file system searches that directory (possibly reading the disk), and the result is cached as a new dentry (with its inode object). If the name does not exist, the answer is cached as a **negative dentry**, so the next attempt fails at once with ENOENT. The root dentry is pinned while the file system is mounted. Under memory pressure the

kernel frees unused dentries. When the walk reaches a mount point, such as /media/usb, it continues in the mounted file system's root dentry.

Worked example. Only the root is cached. Open /home/ana/notes.txt (home, ana, notes.txt: 3 misses), then /home/ana/todo.txt (2 hits, 1 miss), then /home/ana/notes.txt again (3 hits). That is 9 components, 5 hits and 4 slow lookups; without a dentry cache all 9 would be slow.

Key terms

Virtual file system (VFS)
The layer of the Linux kernel that receives every file-related system call and serves it through one common file model, calling the right file system's code through tables of function pointers. It lets dozens of file system types work behind the same calls.

Common file model
The one generic picture of files that the VFS works with: superblock, inode, dentry and file objects, each with a standard set of operations. Every file system must present its data in this shape.

Mapping function
In the classic picture of a VFS, the code each file system supplies to translate between the common file model and its own format. In Linux, that translation is the set of operations tables the file system fills in.

Superblock object
The VFS object that stands for one mounted file system: its device, type, block size and flags, the dentry of its root directory, and a pointer to its super_operations table.

Inode object
The VFS object that stands for one file (a directory is a file too): owner, group, permissions, size, times and link count, plus pointers to its inode_operations table and to the file_operations used when it is opened.

Dentry object
A directory entry object: the VFS object for one component of a path, such as bin in /bin/vi. It links a name to its inode and to its parent's dentry. Dentries exist only in main memory.

VFS file object
The VFS object for one open of a file. open() creates it, fork() and dup() let several descriptors share it, and the last close() frees it. It holds the current file offset, the access mode, a pointer to the dentry and a pointer to the file_operations table.

Operations table
A C struct whose members are function pointers, one slot per job (read_iter, lookup, statfs...). Each file system fills in its own copy, and each VFS object points to the right one.

Function pointer
A variable that holds the address of a function. Calling through it runs whichever function it points to at that moment, which lets C code choose its behaviour at run time.

Dentry cache (dcache)
The kernel's cache of recently used dentry objects, found by parent and name, so a path lookup can skip the file system's slow directory search for every component it has met recently.

Negative dentry

A cached dentry that records that a name does NOT exist in a directory, so a repeated lookup of a missing file fails at once without searching the directory again.

Mount (mount point)
Attaching a file system's directory tree to a directory of the tree already in use, called the mount point. After a USB stick is mounted at /media/usb, paths below that directory lead into the stick.

ext4
The fourth extended file system, the default on many Linux distributions. It keeps a journal and records where files live as extents (runs of neighbouring blocks).

vfat
Linux's driver for the FAT family of file systems with long file names, as found on USB sticks and SD cards. FAT stores no inodes, owners or permission bits, so vfat makes them up in memory.

NFS (Network File System)
A file system whose files live on another computer. The NFS client in the kernel turns each file operation into a request sent over the network to the server.

procfs
A pseudo file system mounted at /proc. Its files are stored on no device: the kernel generates their contents from its own data at the moment they are read.

Review questions with answers

1. What is the main job of the Linux virtual file system (VFS)?

A. To convert every file into one common on-disk format, so that every storage device looks the same
B. To translate a file from one file system's format into another's whenever it is copied between them
C. To present one set of calls and objects for all file systems, handing format-specific work to each
D. To keep recently used disk blocks in main memory so that repeated reads from slow devices finish sooner

Answer: C. To present one set of calls and objects for all file systems, handing format-specific work to each

The VFS receives every file system call, does the shared work once (path lookup, permissions, descriptors, caching of names), and calls the right file system's functions through its operations tables.

2. Match each VFS object to what it stands for.

Superblock object	**One mounted file system**
Inode object	**One file, with its owner, size and permissions**
Dentry object	**One path component, linking a name to an inode**
File object	**One open of a file, with its current offset**

Superblock = a mounted file system; inode = a file; dentry = a name in a path; file = an open instance of a file held by a process.

3. Sort each operation into the table that holds it.

super_operations	inode_operations	dentry_operations	file_operations
statfs put_super	lookup rename	d_revalidate d_hash	llseek release

Whole-file-system jobs go with the superblock; jobs that change a directory or a file's attributes go with the inode; name checks and matching go with the dentry; jobs on an open file (offset, reading, last close) go with the file object.

4. Put the steps of a read() of a file on ext4 in order, for data that is not yet in the page cache.

Correct order:

1. The process calls read(fd, buf, 4096)
2. The VFS finds the file object for that descriptor
3. The VFS calls the read_iter slot of the file object's operations table
4. The page cache is searched for the page and misses
5. ext4 maps the file block to a disk block, and the driver reads it
6. The bytes are copied into buf and the file offset moves forward

System call → VFS → the file system's function through f_op → page cache → (on a miss) the file system finds the block and the driver transfers it → copy to the user and advance f_pos.

5. A process looks up `/bin/vi` on a freshly mounted file system whose dentry cache holds only the root. When the lookup is done, how many dentry objects describe this path, counting the root?

Answer: 3 dentries

One dentry per component: "/", "bin" and "vi". The root existed already; the other two were built by the lookup and are now cached.

6. Process P1 opens a 10,000-byte file and reads 100 bytes. It then calls fork(), and the child reads 50 bytes from the same descriptor. At which byte offset does P1's next read begin?

Answer: 150 bytes

fork() copies the descriptor table, so both descriptors point to the same file object and share one f_pos: 100 + 50 = 150.

7. When two processes each call open() on the same file, they share one file object, and therefore one file offset.

Answer: False

Every open() creates a new file object with its own offset. The two file objects share the dentry and the inode, not the offset. Copying a descriptor, as fork() and dup() do, is what shares a file object.

8. A program asks for a byte-range lock on a file stored on ext4. The lock slot in ext4's file_operations table is NULL. What happens?

A. The request fails, because ext4 does not supply any lock function of its own
B. The VFS falls back to its generic POSIX locks, kept in this machine's memory
C. The VFS forwards the request to the disk driver, which locks the disk blocks
D. The lock is silently ignored and the call reports success without locking anything

Answer: B. The VFS falls back to its generic POSIX locks, kept in this machine's memory

When a slot is NULL the VFS uses its default behaviour. For locks that is the generic POSIX lock code, which is enough on a local disk. NFS fills the slot because its locks must be seen by other machines.

9. Which statements about dentry objects are true? (Pick all that apply.)

A. They exist only in main memory; no dentry object is ever written to the disk
B. Looking up /home/ana/notes.txt involves a dentry for /, home, ana and notes.txt
C. A dentry stores the file's size, owner and permission bits
D. A negative dentry records that a name does not exist in a directory
E. Every process that opens a file gets its own private dentry for it

Answer: A, B, D

Dentries are in-memory objects, one per path component, kept in the dentry cache, and negative ones cache failed lookups. Size, owner and permissions belong to the inode, and processes share dentries.

10. Why does the NFS client supply a d_revalidate function in its dentry operations?

A. To check the user's permission bits again before every single read of the file
B. To make name comparisons ignore the difference between upper and lower case
C. To write the cached dentries back to the server's disk just before unmounting
D. Another machine may have changed the directory since the name was cached

Answer: D. Another machine may have changed the directory since the name was cached

On a local disk only this kernel changes directories, so a cached dentry stays right. With NFS other clients can rename or delete files, so a cached dentry must be checked (revalidated) before it is trusted.

11. The dentry cache holds only the root. A process opens `/home/ana/notes.txt`, then `/home/ana/todo.txt`, then `/home/ana/notes.txt` again. All files exist. How many slow lookups (cache misses that make the file system search a directory) happen in total?

Answer: 4 lookups

First open: home, ana and notes.txt all miss (3). Second: home and ana hit, todo.txt misses (1). Third: all three hit (0). 3 + 1 + 0 = 4.

12. A FAT-formatted USB stick is mounted at /media/usb. Which statement is true?

A. vfat builds an inode object in memory for each file, though FAT stores none
B. Programs must use special vfat system calls in place of open() and read()
C. The VFS will not create dentries for files that live on a mounted stick
D. All files on the stick share one inode object, because FAT has no inodes

Answer: A. vfat builds an inode object in memory for each file, though FAT stores none

The common file model requires an inode object per file, so vfat fills one in from the FAT directory entry: it invents the inode number and takes the owner and permissions from the mount options.

12.10 Windows File System

You should be able to

- List the key features of NTFS and say what each one gives users and programs.
- Relate sectors, clusters and volumes, and compute the number of clusters, the cluster bitmap size and the slack for a given cluster size.
- Describe the regions of an NTFS volume and the job of each system file.
- Explain how an MFT record describes a file as a set of attributes, and when an attribute is resident or nonresident.
- Translate a byte offset in a file into a VCN and an LCN through its data runs, and explain how logging lets NTFS repair its metadata after a crash.

NTFS and its seven key features

NTFS (New Technology File System) is the Windows file system, introduced with Windows NT in 1993. Key features:

- **Recoverability.** After a crash NTFS restores its own structures to a consistent state from a log of metadata transactions, and keeps spare copies of vital data (MFT mirror, backup boot sector).
- **Security.** Every file and folder has a security descriptor (owner plus access control list), checked at every open; since NTFS 3.0 (Windows 2000) descriptors are stored once in $Secure and files refer to them by ID.
- **Large disks and large files.** Cluster numbers are 64 bits on disk; current Windows allows up to $2^{32} - 1$ clusters per volume, so the size limit grows with the cluster size: about 16 TB at 4 KB, 256 TB at 64 KB, 8 PB at 2 MB (clusters above 64 KB need Windows 10 version 1709 or Windows Server 2019 and later). A single file can be almost as big as its volume, unlike FAT32, which stops at 4 GB per file.
- **Multiple data streams.** Besides its unnamed main stream, a file may carry named streams (alternate data streams), written `file:name`. Downloaded files get a Zone.Identifier stream, so Windows warns before opening them.
- **Journaling.** The change journal (USN journal) lists every create, delete, rename and write with an update sequence number, so backup and search tools find changes without scanning the volume.
- **Compression and encryption.** Per file or per folder (new files inherit the folder's setting), transparent to programs. Encryption is the Encrypting File System (EFS); a file cannot be both compressed and encrypted.
- **Hard and symbolic links.** A hard link is an extra name for the same MFT record (files only, same volume only); the file is deleted when its last name goes. A symbolic link (Windows Vista onward) is a small file holding a path; it can point to folders, other volumes or other computers, and dangles if the target is deleted.

Sectors, clusters and volumes

- **Sector:** the disk's smallest unit of transfer: 512 bytes, or 4 KB on newer drives.
- **Cluster:** one or more adjacent sectors, always a power of two; the unit in which NTFS allocates space. **Cluster factor** = sectors per cluster = cluster size ÷ sector size (4 KB ÷ 512 B = 8). A cluster can never

be smaller than a sector.
- **Volume:** a logical disk with one file system: part of a disk, a whole disk, or space on several disks (software RAID). Its clusters are numbered from 0: the **logical cluster number (LCN)**. A cluster's byte offset on the volume is LCN × cluster size. Inside one file, clusters are numbered from 0 again: the **virtual cluster number (VCN)**.

Volume size	Typical default cluster (recent Windows)
up to 16 TB	4 KB
16 – 32 TB	8 KB
32 – 64 TB	16 KB
64 – 128 TB	32 KB
128 – 256 TB	64 KB
256 TB – 8 PB	128 KB to 2 MB (newer versions only)

Defaults change between versions (Windows NT 4.0 used 512 B to 2 KB on small volumes); each is the smallest size from 4 KB up that keeps the volume within $2^{32} - 1$ clusters.

Worked example. A 512 GB volume with 4 KB clusters has $2^{39} \div 2^{12} = 2^{27} = 134{,}217{,}728$ clusters (LCN 0 to 134,217,727). The cluster bitmap needs one bit per cluster: 2^{27} bits = 16 MB. A 10,000-byte file needs $\lceil 10{,}000 \div 4{,}096 \rceil = 3$ clusters = 12,288 bytes, leaving 2,288 bytes of **slack** (internal fragmentation). Over many files of random sizes, a file wastes about half a cluster on average (a file much smaller than a cluster wastes most of it); bigger clusters mean a smaller bitmap but more slack.

Layout of an NTFS volume

1. **Partition boot sector** (up to 16 sectors): bytes per sector, sectors per cluster, volume size, the LCN of the MFT and of the MFT mirror, the MFT record size, and boot startup code. Backup copy in the last sector.
2. **Master file table (MFT).**
3. **System files:** the **MFT mirror** (copy of the first four MFT records), the **log file** (transaction steps for recovery), the **cluster bitmap** (one bit per cluster: used or free) and the **attribute definition table** (the attribute types the volume supports, with their type codes and rules: may they be nonresident, indexed, how big).
4. **File area:** the clusters of everything too big to live in an MFT record.

Every system file is itself a file with an MFT record, so this order describes roles rather than fixed positions. Records 0–15 are reserved: 0 $MFT, 1 $MFTMirr, 2 $LogFile, 3 $Volume, 4 $AttrDef, 5 the root folder, 6 $Bitmap, 7 $Boot, 8 $BadClus, 9 $Secure, 10 $UpCase, 11 $Extend (change journal, quotas, object IDs, reparse points), 12–15 unused.

The master file table

The MFT is the heart of NTFS: a table of records, usually 1 KB each (4 KB on disks with 4 KB native sectors), one per file or folder. A record holds a header (in use?, folder?, sequence number, link count) and

then typed **attributes**:

- **Standard information** (0x10): flags such as read-only and hidden, four time stamps; since NTFS 3.0 also owner ID, security ID, quota charge and latest change-journal number.
- **Attribute list** (0x20): only when the attributes do not fit in one record; says which extra record holds each one.
- **File name** (0x30): the name, the parent folder's record number, copies of size and times. One per name: a long name and a short 8.3 name, or one per hard link.
- **Security descriptor** (0x50): owner and access list; in records only up to NTFS 1.2 (Windows NT 4.0), now kept in $Secure.
- **Volume name** (0x60), **volume information** (0x70): only in $Volume: label, NTFS version (3.1), dirty flag.
- **Data** (0x80): the contents; unnamed main stream plus optional named streams.
- **Index root** (0x90), **index allocation** (0xA0) and **bitmap** (0xB0): a folder's index of names, a balanced tree from the B-tree family (often called a B+ tree, though like a plain B-tree it also keeps real entries in its upper nodes). The root is resident; further 4 KB index blocks are nonresident; the bitmap marks which index blocks are in use. A folder has no data attribute.

A **resident** attribute is stored inside the record; a **nonresident** one lives in clusters, and the record keeps only its **data runs**. Standard information, file names and the index root are always resident. A small file's data is resident too: with a short name, roughly 700 bytes fit after the header (56 bytes), standard information (96) and file name (about 112) in a 1 KB record, so such a file uses no cluster and needs no extra disk read. Once data moves out, NTFS does not move it back.

Data runs, VCN and LCN

A run is a stretch of clusters that are adjacent on disk, stored as (start VCN, start LCN, length). To find byte N of a file: VCN = ⌊N ÷ cluster size⌋; find the run holding that VCN; LCN = run's start LCN + (VCN – run's start VCN). Example with 4 KB clusters and runs VCN 0–29 at LCN 10,240, VCN 30–51 at LCN 52,000, VCN 52–73 at LCN 31,500 (a 300,000-byte file, 74 clusters): byte 150,000 is in VCN 36, byte 2,544 of it; VCN 36 is in run 2, so LCN = 52,000 + 6 = 52,006 and the byte sits at 52,006 × 4,096 + 2,544 = 213,019,120 on the volume. These three runs take about 14 bytes of the record. A badly fragmented file needs a run per piece, and on a hard disk each run costs a seek.

Recoverability

NTFS groups the metadata changes of one operation (for example: allocate an MFT record, fill it in, add the name to the folder, mark clusters used) into a **transaction**. Each log record is numbered by its **log sequence number (LSN)**, and NTFS follows **write-ahead logging**:

1. write the log record (with redo and undo information) into the log, in the cache;
2. make the metadata change in the cache;
3. flush the log records to the log file on disk;
4. only then let the cache manager write the changed metadata to the volume.

After a crash, recovery reads the log on disk. A transaction whose commit record reached the disk is **redone**; one without a commit record is **undone**, newest step first. Redoing or undoing a step is harmless

even if its result is already (or never was) on disk. The result: the structures are always consistent, either as if the transaction completed or as if it never ran. Breaking the rule (metadata on disk before its log records) leaves changes the log cannot explain, so a full chkdsk is needed. Only metadata is logged: file contents waiting in the cache for the lazy writer (11.10) can be lost, and a just-committed change can vanish if its commit record was still in memory. Do not confuse the log file (private, for recovery, circular) with the change journal (for applications, a list of which files changed).

Key terms

NTFS (New Technology File System)
The standard file system of Windows since Windows NT. It keeps one master table that describes every file, logs its own metadata changes so it can recover from crashes, and supports security, very large volumes, multiple data streams, compression, encryption and links.

Cluster
The unit in which NTFS hands out disk space: a fixed number of adjacent sectors, always a power of two, such as 8 sectors = 4 KB. A file stored outside its MFT record always occupies a whole number of clusters.

Cluster factor
The number of sectors in one cluster: the cluster size divided by the sector size. 4 KB clusters on a disk with 512-byte sectors give a cluster factor of 8.

Logical cluster number (LCN)
The position of a cluster on the volume, counting from 0 at the volume's first cluster. Multiplying an LCN by the cluster size gives that cluster's byte offset from the start of the volume.

Virtual cluster number (VCN)
The position of a cluster inside one file, counting from 0 at the file's first cluster. NTFS turns a VCN into an LCN by looking it up in the file's data runs.

Partition boot sector
The first sectors of an NTFS volume (up to 16). They describe the volume's layout, such as the sector and cluster sizes and where the MFT starts, and hold the code that starts loading the operating system if the volume is bootable.

Master file table (MFT)
The heart of NTFS: a table with one record, usually 1 KB, for every file and folder on the volume, including the file system's own system files. Each record describes its file as a list of attributes.

MFT mirror
A second copy of the first four MFT records, stored apart from the MFT, so NTFS can still find the MFT, the log file and the volume information if the start of the MFT becomes unreadable.

NTFS log file ($LogFile)
The system file in which NTFS writes every change to its metadata as a step of a transaction, with what is needed to redo it and to undo it, before the change itself reaches the disk.

Cluster bitmap ($Bitmap)
The system file that holds one bit for each cluster of the volume: 1 if the cluster is in use, 0 if it is free.

Attribute (NTFS)

One typed piece of a file's description in its MFT record, such as its standard information, a file name, its data or a folder's index. NTFS stores everything about a file, contents included, as attributes.

Resident attribute

An attribute whose value is stored inside the MFT record itself. Names and time stamps always are, and a file of a few hundred bytes keeps even its data in the record, using no clusters at all.

Nonresident attribute

An attribute whose value is too big for the MFT record, so it lives in clusters elsewhere on the volume and the record keeps only a list of data runs that says where.

Data run (extent)

A stretch of a file's clusters that sit next to each other on disk, recorded as a starting VCN, a starting LCN and a length in clusters. A contiguous file needs one run; a fragmented file needs one per piece.

Alternate data stream (ADS)

An extra, named stream of data attached to a file besides its main unnamed contents, written as file:name. Windows marks files downloaded from the Internet with a small Zone.Identifier stream.

Change journal (USN journal)

An NTFS system file that lists every change made to the files of a volume (create, delete, rename, write), each entry stamped with an update sequence number, so backup, search and replication tools find what changed without scanning everything.

Write-ahead logging

The rule that the log record describing a change must reach the disk before the changed metadata does, so that after a crash the log always holds enough to redo or undo whatever is found on disk.

Transaction (NTFS)

A group of metadata changes that must take effect completely or not at all, such as every change needed to create one file. After a crash NTFS redoes the transactions that committed and undoes the ones that did not.

Hard link

An extra name for the same file: another directory entry pointing to the same MFT record. The file is removed only when its last name is deleted. NTFS hard links work only for files, and only within one volume.

Symbolic link

A small file that holds the path of another file or folder; opening the link opens the target. It may point to another volume or another computer, and it dangles if the target is deleted. NTFS has them since Windows Vista.

Review questions with answers

1. A 512 GB volume (1 GB = 2^{30} bytes) is formatted with 4 KB clusters. How many megabytes (1 MB = 2^{20} bytes) does its cluster bitmap occupy?

Answer: 16 MB

512 GB ÷ 4 KB = $2^{39} \div 2^{12} = 2^{27}$ = 134,217,728 clusters. One bit each gives 2^{27} bits = 2^{24} bytes = 16 MB.

2. A volume uses 4 KB (4,096-byte) clusters. A file holds 10,000 bytes, too many to stay inside its MFT record. How many bytes of slack are left unused in its last cluster?

Answer: 2288 bytes

The file needs ⌈10,000 ÷ 4,096⌉ = 3 clusters = 12,288 bytes. 12,288 – 10,000 = 2,288 bytes of slack (internal fragmentation).

3. A file on a volume with 4 KB clusters has three data runs: VCN 0–29 at LCN 10,240; VCN 30–51 at LCN 52,000; VCN 52–73 at LCN 31,500. Which LCN holds byte 150,000 of the file?

Answer: 52006

VCN = ⌊150,000 ÷ 4,096⌋ = 36. VCN 36 lies in the second run (VCN 30–51), so LCN = 52,000 + (36 – 30) = 52,006.

4. Where does NTFS keep the contents of a 300-byte text file?

A. In one cluster of the file area, like every other file
B. In the cluster bitmap, next to the bit for its cluster
C. Inside its own MFT record, as a resident data attribute
D. In the log file, until the lazy writer moves it out

Answer: C. Inside its own MFT record, as a resident data attribute

A 1 KB record has room for several hundred bytes of data after the header, standard information and file name, so a small file's data attribute stays resident: no cluster, no extra disk read.

5. NTFS logging guarantees that the data a program wrote just before a power failure survives the crash.

Answer: False

NTFS logs metadata (the MFT, folders, bitmaps) so its structures are always consistent after recovery. File contents still waiting in the cache for the lazy writer are not logged and can be lost.

6. Put the steps of a logged metadata change in the order NTFS performs them.

Correct order:

1. Write the log record for the change into the log, in the cache
2. Make the change to the metadata, in the cache
3. Flush the log record to the log file on disk
4. Write the changed metadata to the volume on disk

This is write-ahead logging: the log record always reaches the disk before the metadata change it describes, so recovery can redo or undo anything it finds on disk.

7. Match each part of an NTFS volume to its job.

Partition boot sector	**Records the cluster size and where the MFT starts**
MFT mirror	**Copy of the first four MFT records**
Log file	**Redo and undo steps of metadata transactions**
Cluster bitmap	**One bit per cluster: in use or free**
Attribute definition table	**The attribute types the volume supports and their rules**

The boot sector describes the layout, the mirror protects the start of the MFT, the log supports recovery, the bitmap tracks free space and $AttrDef defines the attribute types.

8. Is each attribute value stored inside the MFT record (resident) or in clusters outside it (nonresident)?

Resident	Nonresident
The standard information of any file A file name attribute The index root of a folder The data of a 200-byte note	The data of a 2 MB photo The index allocation of a large folder

Standard information, file names and the index root are always resident. Data stays resident only while it fits in the record; 2 MB never does. The index allocation is by definition the part of a folder's index stored outside the record.

9. Which of these are features of NTFS?

A. Several named data streams in one file
B. Compression of single files or folders
C. A maximum file size of 4 GB
D. Hard links and symbolic links
E. Logging of all user data, so no written byte is lost

Answer: A, B, D

Multiple data streams, compression and links are NTFS features. A 4 GB file limit belongs to the older FAT32, and NTFS logs only metadata, not user data.

10. NTFS is creating a file. The log on disk holds all four update records and the commit record of the transaction, but the folder index change had not reached the volume when the power failed. What does recovery do?

A. Undoes all four updates, because the folder change never reached the disk
B. Redoes all four updates, so the file exists and its folder lists it
C. Nothing: it leaves the half-written structures for chkdsk to find
D. Redoes only the folder change and undoes the other three updates

Answer: B. Redoes all four updates, so the file exists and its folder lists it

A commit record on disk means the transaction committed, so recovery redoes every one of its logged updates. Redoing updates that already reached the disk is harmless.

11. Backup software wants the list of files changed since last night, without scanning the whole volume. Which NTFS structure gives it that list?

A. The NTFS log file ($LogFile)
B. The cluster bitmap ($Bitmap)
C. The MFT mirror ($MFTMirr)
D. The change journal ($UsnJrnl)

Answer: D. The change journal ($UsnJrnl)

The change journal records every create, delete, rename and write with an update sequence number, so a program can ask for everything after the number it saw last time.

12. On an NTFS volume, a file has two hard links, A.txt and B.txt. You delete A.txt. What happens?

A. B.txt still opens the file; the link count drops from 2 to 1
B. B.txt now dangles, because its target was deleted
C. The file's clusters are freed and B.txt becomes empty
D. Windows refuses: a file with two names cannot be deleted

Answer: A. B.txt still opens the file; the link count drops from 2 to 1

Both names point to the same MFT record. Deleting one removes that name and lowers the link count; the file itself disappears only when the count reaches zero.

12.11 Android File Management

You should be able to

- Explain why files on an Android phone are Linux files underneath, and name the file systems phones use (ext4 and F2FS).
- Describe the classic partitions (boot, system, recovery, data, cache, misc) and shared storage, and predict what a factory reset or a system update touches.
- Explain how A/B updates and dynamic partitions change the layout on newer phones.
- Decide whether one app may read another app's files, using Linux user IDs and permissions for private files and the scoped-storage rules for shared storage.
- Describe SQLite as an embedded, single-file database and explain what a transaction guarantees.

1. Android files are Linux files

Android runs on the Linux kernel, so it manages files with ordinary Linux machinery. When an app saves something, its Java or Kotlin call is turned by libraries in its own process (Java libraries, SQLite, the C library) into Linux system calls such as `open()`, `write()` and `fsync()`. The kernel's virtual file system (VFS, section 12.9) checks permissions and passes each call to the file system that owns the path. The block layer and the flash driver then move the blocks to the flash chip (UFS or eMMC). There is no special "Android file system".

- **ext4** (fourth extended file system): the standard Linux disk file system, used by Android since version 2.3 (2010).
- **F2FS** (Flash-Friendly File System): a Linux file system Samsung designed for flash; it writes new data in long sequential runs. Many phones use it for the data partition.
- Read-only system partitions use ext4 or, on some newer phones, EROFS (Enhanced Read-Only File System), which can compress its contents.

What Android adds is policy: a fixed set of partitions with clear jobs, a separate Linux user for every app, and SQLite as the standard database.

2. The classic partition layout

A **partition** is a fixed, separately managed region of the storage device with its own file system (or raw contents), so one can be erased or rewritten without touching the others. Phone makers vary the exact set.

Partition	Holds	Normal use	Factory reset	Classic system update
boot	Linux kernel + ramdisk (with init)	raw image, not mounted	unchanged	rewritten
recovery	a second bootable system: own kernel, ramdisk, recovery program	started only when needed	used (it does the wiping)	used (verifies and installs)
misc	tiny flags, e.g. “boot into recovery and do X”; some hardware settings	raw, not mounted	used (carries the wipe message)	used (carries the install message)
system (/system)	framework, system libraries, runtime, preinstalled apps	mounted read-only, checked as read	unchanged	rewritten
cache (/cache)	throw-away data: downloaded update package, recovery logs	read-write	erased	used (holds the package)
data (/data, “userdata”)	installed apps (/data/app), app-private directories (/data/data/<package>), accounts and settings (/data/system), emulated shared storage (/data/media)	read-write, encrypted	erased	unchanged

The **bootloader** runs first at power-on and loads the kernel from boot, or from recovery when misc asks it to. The **ramdisk** is a small file-system image copied into memory at boot; init runs from it and mounts the real partitions. Since Android 8.0 the maker’s drivers and hardware libraries (camera, radio, sensors) sit in a separate **vendor** partition.

Shared storage (external storage) is what apps see as `/sdcard` or `/storage/emulated/0`: DCIM, Pictures, Music, Download. On most phones it is emulated: a folder under /data/media (/data/media/0 for the first user) inside the data partition, not a separate card. A removable microSD card, if present, is a separate device (usually exFAT or FAT32) that a reset normally leaves alone.

Factory reset

Android writes a wipe command into misc and reboots; the bootloader starts recovery; recovery formats data and cache; the phone restarts into the setup screen. Boot, system and recovery are untouched, so the phone keeps the Android version of its last update. Photos in emulated shared storage are erased with data.

Classic system update (over-the-air, OTA)

1. Android downloads the update package into cache (some phones use /data) and checks it.
2. It writes “install this package” into misc and reboots.
3. The bootloader reads misc and starts recovery.
4. Recovery verifies the package’s signature and rewrites system and boot.
5. The phone restarts into the new version; your apps and files in data are unchanged.

Normal Android cannot safely rewrite the read-only system it is running from, which is why the work is handed to recovery.

3. Newer phones: A/B updates and dynamic partitions

- **A/B (seamless) updates**, Android 7.0: two copies, slot A and slot B, of boot and the system partitions. The update is written into the inactive slot while the phone keeps running; misc (on many phones) records "try slot B next"; one reboot switches; if the new slot fails to boot, the bootloader falls back to the old one. Such phones need no cache partition, and most have no recovery partition either: the recovery program lives inside a boot image.
- **Dynamic partitions**, Android 10: system, vendor, product and similar partitions are logical partitions inside one physical partition named **super**, managed by the kernel's device mapper (which maps each logical partition onto ranges of real blocks), so an update can resize them.
- **Virtual A/B**, Android 11: saves space by keeping only the changes, as a snapshot (in super, or in free space on userdata if super is full), until the new version has booted; then they are merged in.

A factory reset on these phones still erases only userdata; both OS slots survive. An update leaves the user's apps and files in userdata as they were.

4. The app sandbox: one Linux user per app

At install time every app gets its own Linux user ID, and all its processes run as that user. Its **app-private directory** (internal storage), e.g. `/data/data/com.example.notes` (also reachable as /data/user/0/...), is owned by that UID with mode `rwx------` (for apps targeting Android 7.0 or later). Each app also gets its own Linux group, so its files are typically `rw-rw----` with that private group: the app may read and write, all others get nothing. When another app opens such a file, the kernel's ordinary owner/group/others check fails and `open()` returns "Permission denied". No Android permission and no Android version changes this; SELinux, a second kernel security layer with its own rule for every app, adds an independent wall.

Naming: `u0_a145` = device user 0, app 145. UID = 100000 × user + 10000 + app. Worked example: u0_a145 → 0 + 10000 + 145 = 10145; the same app for user 10 is u10_a145 = 1,000,000 + 10,000 + 145 = 1010145. App UIDs start at 10000.

Shared storage rules

- **Before scoped storage** (Android 4.4 to 9): Android did not track who created a shared file. One permission, READ_EXTERNAL_STORAGE, opened all of shared storage, other apps' folders included. (Before 4.4, reading shared storage needed no permission at all.)
- **Scoped storage**: introduced in Android 10 (apps targeting Android 10 could still opt out), required for apps targeting Android 11 or later.
  - An app may always use its own folder `Android/data/<package>` on shared storage, with no permission (true since Android 4.4).
  - It may add photos, videos and audio through MediaStore (Android's index of shared media) and read back what it added, with no permission.
  - Other apps' media need a user-granted permission: READ_EXTERNAL_STORAGE on Android 10 to 12; since Android 13 separate ones for images, video and audio (READ_MEDIA_IMAGES,

READ_MEDIA_VIDEO, READ_MEDIA_AUDIO); since Android 14 the user may grant only selected photos and videos.
    - Other apps' documents (e.g. a downloaded PDF) can only be opened when the user picks them in the system file picker (the Storage Access Framework), which grants access to exactly those files. (Only special apps such as file managers may request "all files access", Android 11 and later.)
    - Since Android 11 no ordinary app can open another app's Android/data folder.

5. SQLite

SQLite is Android's standard on-device relational database: a small but complete SQL engine that runs as a library inside the app's own process. There is no database server and no network connection. A whole database (table definitions and every row) is one ordinary file, usually `/data/data/<package>/databases/name.db`, so the app sandbox protects it like any other file. Apps use it through Android's database classes or the Room library; system components such as the contacts and messaging providers keep their data in SQLite databases too.

Statements: `INSERT` adds rows, `UPDATE` changes them, `DELETE` removes them, `SELECT` reads them (optionally with `WHERE`).

A **transaction** groups changes so that all of them become permanent (COMMIT) or none do (ROLLBACK), even if the power fails part-way. Outside `BEGIN … COMMIT`, each statement is its own transaction (autocommit). With the classic rollback journal, the first change inside a transaction creates `name.db-journal` holding original copies of the pages being changed. COMMIT writes the new pages into the database and then deletes the journal; that deletion is the commit point. If the power fails before it, the next open finds the leftover ("hot") journal and copies the original pages back over any changes that had already reached the file, so the database looks exactly as it did before BEGIN: never half a transaction. Android often runs SQLite in write-ahead-log mode (a `name.db-wal` file) instead; the all-or-nothing guarantee is the same.

Key terms

Partition
: A fixed, separately managed region of a storage device. The OS treats each partition as its own volume, usually with its own file system, so one partition can be erased or rewritten without touching the others.

ext4 (fourth extended file system)
: The standard Linux disk file system since the late 2000s, a journaling successor of ext2 and ext3. Android has used it for its main partitions since version 2.3 (2010).

F2FS (Flash-Friendly File System)
: A Linux file system that Samsung designed for flash storage. It writes new data in long sequential runs, which suits how flash chips work; many phones use it for their data partition.

Bootloader
: The small program a device runs first when it is switched on. It checks and loads a kernel, normally from the boot partition or, when asked to, from recovery, and hands control to it.

Boot partition

The partition that holds the Linux kernel and a small ramdisk, the two things the bootloader loads to start Android. It is read as one raw image, not mounted as a folder.

Ramdisk (initial ramdisk)

A small file-system image that is copied into main memory at boot and used as the first root file system. On Android it holds init and the files init needs to mount the real partitions.

System partition

The partition mounted at /system that holds the Android operating system: the framework, system libraries and preinstalled apps. It is read-only during normal use and changes only when the system is updated.

Recovery partition

A second bootable partition with its own small kernel and a recovery program. The phone starts it instead of normal Android to install a system update or to carry out a factory reset.

Data partition (userdata)

The partition mounted at /data that holds everything that belongs to the user: installed apps, each app's private files and databases, accounts and settings. A factory reset erases it.

Cache partition

The partition mounted at /cache for data that may be thrown away, classically a downloaded system-update package and the recovery program's logs. Many newer phones have no cache partition.

Misc partition

A tiny partition for small flags and settings. Its best-known job is carrying a message such as "start recovery on the next boot" from Android to the bootloader.

Factory reset

Erasing the data partition (and the cache) so the phone starts again as if new: every installed app, account, file and setting is gone, while the operating-system partitions stay as they are.

Over-the-air update (OTA)

A system update that the phone downloads and installs by itself, without being connected to a computer.

A/B updates (seamless updates)

An update scheme, introduced in Android 7.0, that keeps two copies (slots A and B) of the system partitions. The update is written to the slot not in use while the phone keeps working; a reboot switches slots, and a failed boot falls back to the old slot.

Dynamic partitions

Since Android 10, system, vendor and the other read-only system partitions can be logical partitions inside one physical partition named super, so an update can change their sizes.

Shared storage (external storage)

The storage area for user-visible files that apps share, such as photos, music and downloads, seen as /sdcard or /storage/emulated/0. On most phones it is "emulated": a folder inside the data partition, not a separate card.

App-private directory (internal storage)

The directory Android creates for each app inside /data, such as /data/data/com.example.notes. It belongs to the app's own Linux user ID, so other apps cannot open it.

Scoped storage
: The shared-storage rules introduced in Android 10 (apps targeting Android 10 could still opt out) and required for apps that target Android 11 or later: an app sees its own files and the media it created, and anything else only if the user grants access.

SQLite
: A small but complete SQL database engine that runs as a library inside the app that uses it, with no separate server process, and keeps a whole database in one ordinary file. It is Android's standard on-device database.

Transaction
: A group of database changes applied as one unit: either all of them become permanent (commit) or none of them do (rollback), even if the power fails part-way through.

Review questions with answers

1. Which file system is the data partition of a typical Android phone most likely to use?

A. NTFS, the Windows file system
B. ext4 or F2FS
C. FAT32, as on a USB stick
D. An Android-only file system

Answer: B. ext4 or F2FS

Android runs on the Linux kernel, so it stores files with Linux file systems: ext4 on many phones and F2FS, designed for flash, on many others.

2. A factory reset erases the system partition, so the phone goes back to the Android version it originally shipped with.

Answer: False

A factory reset formats only data (and cache). The system partition is untouched, so the phone keeps whatever Android version its last update installed.

3. On most phones, the shared storage that apps see as /sdcard is physically a folder inside the data partition.

Answer: True

Shared storage is usually "emulated": it lives in /data/media and is shown to apps through a special layer. That is why a factory reset also erases photos and downloads.

4. Match each classic Android partition to what it holds.

boot	**The Linux kernel and its ramdisk**
system	**The framework and preinstalled apps, read-only**
recovery	**A separate system that installs updates and wipes data**
data	**Installed apps and their private files**
cache	**A downloaded update package and recovery logs**
misc	**A message telling the bootloader to start recovery**

Each partition has one job. The OS lives in boot, system and recovery; the user's things live in data; cache and misc hold throw-away files and small flags.

5. A phone with the classic layout is factory-reset. Which of these are erased, and which are kept?

Erased	Kept
A game the user installed Camera photos in the built-in (emulated) shared storage, never backed up A chat app's database of conversations	The preinstalled dialer app The Linux kernel The most recent system update the user installed

The reset formats data, and the emulated shared storage lives inside data. Boot and system are not touched, so the kernel, the preinstalled apps and the updated OS all survive.

6. An app runs as the Android Linux user `u0_a57`. What is its numeric UID? (UID = 100000 × user number + 10000 + app number)

Answer: 10057

User 0, app 57: 100000 × 0 + 10000 + 57 = 10057. App UIDs start at 10000, and each further person on the device adds 100000.

7. The Game app runs as UID 10145. It tries to open `/data/data/com.example.notes/databases/notes.db`, owned by user 10123 and group 10123 (Notes' own group) with mode `rw-rw----`. What happens?

A. It succeeds once the user grants Game the storage permission.
B. It succeeds on Android 9 and earlier, but fails since Android 11.
C. It fails: Game's UID is not the owner and others get no access.
D. It succeeds, because both apps belong to the same person, user 0.

Answer: C. It fails: Game's UID is not the owner and others get no access.

The kernel checks the process UID against the file's owner. 10145 is not 10123 and not in its group, so the "others" bits (no rights) apply and open() fails with "Permission denied".

8. Under scoped storage (Android 11 and later), which of these may an ordinary app do **without** the user granting any permission?

A. Read and write its own private directory in /data
B. Add a photo to the shared Pictures collection and read it back
C. Read photos that other apps saved
D. Open another app's folder under Android/data
E. Use its own folder under Android/data on shared storage

Answer: A, B, E

An app always owns its private directory, its own Android/data folder and the media it added. Other apps' photos need a user-granted media permission, and other apps' Android/data folders are closed to everyone.

9. Put the steps of a classic (non-A/B) system update in order.

Correct order:

1. Android downloads the update package into the cache partition
2. Android writes an "install the update" message into misc and reboots
3. The bootloader reads misc and starts recovery
4. Recovery verifies the package and rewrites system and boot
5. The phone restarts into the new version with its user data untouched

Normal Android cannot rewrite the system it is running from, so it parks the package in cache, leaves a note in misc and hands over to recovery, which does the rewriting.

10. Which statement about SQLite on Android is correct?

A. Each app connects over the network to a shared SQLite server.
B. SQLite keeps every table in its own file under /system.
C. An SQLite database exists only in memory and vanishes on exit.
D. A database is one file inside the app's private directory.

Answer: D. A database is one file inside the app's private directory.

SQLite is an embedded engine: the library inside the app reads and writes one database file, usually in /data/data/<package>/databases/, which the app sandbox protects.

11. An app opens a transaction and inserts three rows. If the battery dies before COMMIT, some of the three rows may still be in the database after restart.

Answer: False

A transaction is all-or-nothing. On restart SQLite finds the leftover journal and restores the old pages, so none of the three rows appear.

12. What is the main benefit of A/B (seamless) updates?

A. The update goes to the unused slot as you keep working; a failed boot falls back.
B. Every app keeps two copies of its private data, so a reset can be undone.
C. The data partition is split in two, so photos never crowd out installed apps.
D. The bootloader installs it from the cache partition while the screen is off.

Answer: A. The update goes to the unused slot as you keep working; a failed boot falls back.

With two slots, the inactive one is updated in the background, a single reboot switches over, and if the new slot does not boot, the bootloader returns to the old one.

Chapter 13 · Embedded Operating Systems

Most computers are hidden inside other things. Their operating systems must be small, fast to react and built for one job.

Far more processors live inside cars, appliances, sensors and phones than inside desktop computers. This chapter explains what an embedded system is, what its operating system must do differently, how Linux is slimmed down for embedded use, and how TinyOS runs on tiny wireless sensor nodes.

Learning objectives

- Explain what an embedded system is and how microcontrollers differ from microprocessors.
- List the characteristics of an embedded operating system.
- Describe how embedded Linux differs from desktop Linux.
- Explain the component and task model of TinyOS.

How Chapter 13 fits together

Most computers are not desktops or phones but small processors built into products, each doing one job. The overview's farm hides seven of them, running four kinds of software: no OS at all (a greenhouse thermostat and a gate remote, each a super loop), a small RTOS (a pump controller and a tractor's engine controller, both with hard deadlines), embedded Linux (a barn gateway and a cab display, which need big ready-made software) and a TinyOS-style sensor OS (soil motes that must last years on two AA cells). The rule behind every choice: use the smallest system software that meets every requirement, because each extra feature costs memory, energy and testing.

Embedded systems (13.1)

An embedded system combines hardware, software and often mechanical parts to perform one dedicated function, usually inside a larger product, and such systems far outnumber general-purpose computers. Requirements range widely in cost, safety, reliability, timing, lifetime and environment. Sensors and actuators, A/D and D/A converters, a human interface, a diagnostic port and sometimes an FPGA or ASIC surround the processor and close a feedback loop with the physical world. A microcontroller puts processor, flash, RAM, clock and I/O on one cheap, frugal chip.

Embedded operating systems (13.2)

An embedded OS is set apart by real-time and reactive operation, configurability, I/O device flexibility, streamlined protection and direct use of interrupts. One road adapts a general-purpose OS such as Linux (rich and familiar, but larger and less predictable); the other is a purpose-built RTOS with fast switching, bounded interrupt latency, small size, fixed memory pools and timeouts.

Embedded Linux (13.3)

An embedded distribution keeps only one device's drivers and programs, is cross-compiled with tools such as Buildroot or the Yocto Project, often saves space with a small C library (musl, uClibc-ng) and BusyBox (some keep glibc), and stores its files on flash with squashfs, JFFS2 or UBIFS. Android is the largest

embedded Linux; without an MMU (µClinux, mainline since 2.6) programs start others with vfork() instead of fork().

TinyOS (13.4)

Motes have kilobytes of RAM and run for months or years on batteries. TinyOS compiles one image from nesC components wired through interfaces of commands and events. Tasks wait in a FIFO queue and run to completion, interrupts may preempt them, long operations are split-phase, and the processor sleeps whenever the queue is empty.

Why it matters

Embedded systems gather the whole guide under tight limits: chapter 10's deadlines, chapter 11's interrupts and devices and chapter 12's files on flash all return, cut down to fit one job.

13.1 Embedded Systems

You should be able to

- Define an embedded system and recognise one inside everyday products, from washing machines to cars and pacemakers.
- Describe the ways embedded systems differ in size, cost, quality requirements, lifetime, environment and behaviour.
- Name the elements of an embedded system and trace its feedback loop from sensor through A/D conversion, computation and D/A conversion to actuator.
- Explain how a microcontroller differs from a microprocessor, and an application processor from a dedicated processor.
- Explain what makes a system deeply embedded and why battery-powered sensor nodes must sleep most of the time.

Embedded systems: the computers you never see

An **embedded system** is a combination of computer hardware and software, often with mechanical or other parts, designed to perform **one dedicated function**, usually as part of a **larger product**. Example: a car's anti-lock brakes (ABS). A wheel-speed sensor reports each wheel's speed, a controller (microcontroller plus program) spots a wheel about to lock, and a hydraulic unit (valves, pump) releases and reapplies its brake pressure many times a second. Embedded processors far outnumber the rest: roughly 30 billion microcontrollers ship a year against about 0.26 billion PCs (30 ÷ 0.26 ≈ 115 per PC) and 1.2 billion phones. A **general-purpose computer** (laptop, server) runs whatever its user installs, so it is *not* embedded, though it contains embedded controllers.

Application areas: consumer electronics (TVs, cameras, game consoles); household appliances (washers, microwave ovens, thermostats); automotive (engine control, ABS, airbags, tire-pressure sensors; a typical new car has dozens of ECUs, some high-end models over 100); industrial control (robot arms, process control); medical devices (pacemakers, infusion pumps); office automation (printers, copiers); aerospace and defense (flight control, satellites).

Seven ways embedded systems differ

1. **Size**: one tiny chip to a large networked system.
2. **Cost**: cents matter for mass products, much less for one-off systems.
3. **Quality requirements**, relaxed to very strict and usually combined: **safety**, **reliability**, **real-time**, **flexibility** (updates), **legislation** (laws, certification).
4. **Lifetime**: two or three years to decades.
5. **Environment**: mild indoors to heat, cold, vibration, humidity, radiation.
6. **Workload** (application characteristics): static or changing load, slow or fast, compute- or interface-intensive.
7. **Model of computation**: **discrete-event** (separate events) to **hybrid** (continuous change mixed with events).

A fitness band is tiny, cost-driven, short-lived and event-driven; a pacemaker is safety-critical, regulated and long-lived; an engine controller is cost-driven yet real-time, harsh-environment and hybrid; a Mars rover is large, extremely reliable, radiation-exposed and updatable from Earth. Often one requirement **dominates**: safety (infusion pump, level-crossing gate), cost (greeting card, $10 toy), power (buried soil sensor, wildlife collar) or real-time timing (spark timing: at 6,000 rpm the crankshaft turns 36° per millisecond; a drone stabiliser).

Elements of an embedded system (car engine controller in brackets)

- **Sensors** measure the world, usually as a voltage (crankshaft position, air flow, coolant temperature).
- **A/D converter** turns a voltage into a number. 10 bits = 1,024 steps; over 0–3.3 V each step is 3,300 ÷ 1,024 ≈ 3.22 mV.
- **Processor, memory, software**: usually a microcontroller; flash/ROM holds the program and tables, RAM the changing data; the firmware has one job (fuel and spark timing).
- **D/A converter** turns a number into a voltage for an actuator; many actuators are instead switched by timed digital outputs (injector pulses).
- **Actuators** act on the world: motors, valves, heaters, relays (injectors, ignition coils, throttle).
- **Human interface**: lights, buttons, small display (check-engine light).
- **Diagnostic port**: a technician's tool reads faults of the *controlled system*, not just the computer (OBD-II socket, required on US cars since the 1996 model year).
- **Special-purpose hardware**: FPGA (reconfigurable logic), ASIC (custom chip) or nondigital circuits, for speed or reliability (a pulse-timing unit).
- **Electromechanical backup and safety** keeps things safe if the computer fails (springs pull the throttle to a "limp-home" position).
- **Auxiliary systems**: power supply and cooling (12 V regulated to steady low voltages).

The **feedback loop**: environment → sensors → A/D → computation → D/A → actuators → environment → sensors again.

The thermostat lab (a simple model)

Outside 5 °C; each minute the room loses 1 % of its indoor-outdoor difference (2.5 % with a window open); full heater power adds 0.25 °C. Sensor: 0.5 V + 10 mV per °C, read by a 10-bit A/D over 0–3.3 V. Example: 21 °C → 0.710 V → 0.710 ÷ 3.3 × 1,024 ≈ 220.3 → code 220 → 20.90 °C (one step ≈ 0.32 °C). **On/off**: a digital pin drives a relay, on below setpoint − 0.5 °C, off above setpoint + 0.5 °C, unchanged in between, so

the room swings in a band. **Smooth**: an 8-bit D/A (0–255) sets power = 60 % per °C × error + a running correction, limited to 0–100 %. With the window open, 100 % power holds only 5 + 0.25 ÷ 0.025 = 15 °C: no program beats a too-weak actuator.

Microprocessor versus microcontroller

A **microprocessor** is a processor on one chip; memory, clock and I/O are separate chips. A **microcontroller** puts processor, nonvolatile program memory (flash/ROM), RAM, clock and I/O (timers, serial ports, A/D) on one chip: slow (MHz to a few hundred MHz), small (KB to a few MB; the biggest car chips hold over 10 MB of flash), cheap (cents to dollars), frugal (milliwatts, microwatts asleep). Arduino Uno's chip: 16 MHz, 32 KB flash, 2 KB RAM; a desktop CPU: GHz, gigabytes of separate RAM, tens of watts or more. A phone's system on a chip sits in between.

Application, dedicated and deeply embedded

Application processors run a full OS such as Linux or Android and many apps (phones, tablets, infotainment). **Dedicated processors** do one or a few fixed tasks (washer, microwave, printer engine, engine control); one phone holds both kinds. A **deeply embedded system** is microcontroller-based, dedicated, not programmable by its user, with no OS or a tiny one, often a battery-powered node of a **wireless sensor network**.

Battery life. 20 mA awake, 5 µA asleep, 2,500 mAh. Awake 50 ms per minute: fraction 0.05 ÷ 60 ≈ 0.00083; average ≈ 20 × 0.00083 + 0.005 ≈ 0.0217 mA; life ≈ 2,500 ÷ 0.0217 ≈ 115,000 h ≈ 13.2 years on paper (self-discharge caps it near ten). Never sleeping: 2,500 ÷ 20 = 125 h, about 5 days. Awake 100 ms every 10 s (1 %): 0.2 + 0.00495 ≈ 0.205 mA = 205 µA.

Key terms

Embedded system
: A combination of computer hardware and software, and often mechanical or other parts, designed to perform one dedicated function. It is usually part of a larger product, such as the anti-lock braking system of a car.

General-purpose computer
: A computer built to run whatever programs its user chooses, such as a laptop, desktop or server. The opposite of a computer dedicated to one job.

Electronic control unit (ECU)
: The car industry's name for an embedded computer that controls one part of a vehicle, such as the engine, the brakes or the airbags.

Microprocessor
: A processor on a single chip. On its own it cannot run a program: main memory, the clock and most I/O sit in separate chips around it.

Microcontroller (MCU)
: A whole small computer on one chip: a processor, nonvolatile memory for the program, RAM for data, a clock and I/O such as timers, serial ports and A/D converters.

Application processor

A powerful processor, usually the heart of a system on a chip, that runs a full operating system such as Linux or Android and many apps. Phones, tablets and car infotainment units use them.

Dedicated processor

A processor devoted to one task or a few fixed tasks, such as running a washing machine's wash cycles. Its program rarely or never changes.

Deeply embedded system

A dedicated system built around a microcontroller that its user cannot program, usually with no operating system or only a tiny one. Sensor nodes are typical examples.

Wireless sensor network (WSN)

Many small battery-powered nodes that each measure something (temperature, moisture, vibration) and send their readings by radio to each other or to a gateway.

Firmware

The program stored in a device's nonvolatile memory (flash or ROM) that runs the device. It starts by itself when the power comes on.

Flash memory

Nonvolatile memory that can be erased and rewritten electrically, a block at a time. Most microcontrollers keep their program in on-chip flash.

A/D converter (analog-to-digital converter)

A circuit that measures an analog voltage, such as a sensor's output, and turns it into a number the processor can use. A 10-bit converter gives one of 1,024 possible numbers.

D/A converter (digital-to-analog converter)

A circuit that turns a number from the processor into an analog voltage, for example to set the power of a heater or the speed of a motor.

Field-programmable gate array (FPGA)

A chip full of logic blocks whose wiring can be configured after manufacture, so engineers can build custom hardware circuits without making a new chip.

Application-specific integrated circuit (ASIC)

A chip designed and manufactured for one particular job. Fast, small and cheap in large volumes, but its circuit can never be changed.

Diagnostic port

A connector through which a technician's tool reads the state and fault records of the system being controlled (the engine, for example), not only of its computer.

Electromechanical backup

A mechanical or electrical mechanism that keeps the system safe, or keeps it working in a limited way, when the computer fails.

Feedback loop

A cycle in which sensors measure the physical world, the computer decides, actuators act, the world changes, and the sensors measure again.

Setpoint

The target value a control system tries to hold, such as 21 °C on a thermostat.

Hybrid system

A system whose behaviour mixes continuous change (a temperature, a speed) with separate discrete events (a switch flips, a gear changes). A pure discrete-event system reacts only to separate events.

Review questions with answers

1. Which of these is **not** an embedded system?

A. The controller that runs a washer's wash cycles
B. The computer that decides when airbags fire
C. A laptop that runs whatever programs you install
D. The computer in a printer that times the paper motors

Answer: C. A laptop that runs whatever programs you install

An embedded system is dedicated to one function inside a larger product. A laptop is a general-purpose computer: its user decides what it runs.

2. Most of the processors made each year end up in general-purpose computers such as PCs and servers.

Answer: False

Roughly 30 billion microcontrollers ship a year against about a quarter of a billion PCs: around a hundred microcontrollers for every PC, before even counting the other embedded chips.

3. Put one pass of a thermostat's feedback loop in order, starting with the room.

Correct order:

1. The room's temperature changes
2. The sensor's output voltage follows the temperature
3. The A/D converter turns the voltage into a number
4. The processor compares the number with the setpoint and decides
5. The output stage (a D/A converter or a relay) drives the heater
6. The heater warms the room, and the loop begins again

A feedback loop runs environment → sensor → A/D → computation → D/A (or switch) → actuator → environment, over and over.

4. Match each element of an embedded system to its job.

A/D converter	**Turns a sensor's voltage into a number**
D/A converter	**Turns a number into a voltage for an actuator**
Diagnostic port	**Lets a technician read faults of the controlled system**
FPGA or ASIC	**Special hardware for extra speed or reliability**
Electromechanical backup	**Keeps things safe if the computer fails**
Auxiliary systems	**Supply power and cooling**

Converters bridge the analog world and the digital computer; the diagnostic port serves technicians; special hardware, backup and auxiliary systems support the main loop.

5. Which of these are normally found on a single microcontroller chip?

A. A processor core
B. Flash memory that holds the program
C. Gigabytes of RAM
D. Timers and serial ports
E. A hard disk
F. An A/D converter

Answer: A, B, D, F

A microcontroller holds a processor, nonvolatile program memory, a little RAM (kilobytes), a clock and I/O such as timers, serial ports and A/D converters. Gigabytes of RAM and disks do not fit on such a chip.

6. A 10-bit A/D converter measures voltages from 0 to 3.3 V. How wide is each step, in millivolts? (two decimals)

Answer: 3.22 mV

A 10-bit converter has 2^{10} = 1,024 steps. 3,300 mV ÷ 1,024 ≈ 3.22 mV per step.

7. A wireless sensor node draws 20 mA while awake and 5 µA while asleep. It wakes for 100 ms every 10 seconds. What is its average current, in µA? (nearest whole number)

Answer: 205 µA

It is awake 0.1 s ÷ 10 s = 1 % of the time. Average = 20 mA × 0.01 + 0.005 mA × 0.99 = 0.2 + 0.00495 ≈ 0.205 mA = 205 µA.

8. Why does an embedded system have a **diagnostic port**?

A. It lets the user install new apps on the device
B. It supplies backup power if the main supply fails
C. It turns every sensor voltage into a number for the processor
D. It reports on the controlled system too, such as engine faults

Answer: D. It reports on the controlled system too, such as engine faults

A diagnostic port lets a technician's tool read stored faults and readings about the system being controlled (a misfiring cylinder, for example), not only about the computer.

9. Sort each processor: application processor or dedicated processor?

Application processor	Dedicated processor
Main chip of a smartphone running Android	Microwave oven controller
Car infotainment unit running Linux	Engine control unit
Tablet that runs many apps	Printer engine controller
	Washing-machine controller

Application processors run full operating systems such as Linux or Android and many apps. Dedicated processors do one or a few fixed tasks, usually on a microcontroller.

10. A microcontroller differs from a microprocessor mainly because it…

A. runs at a far higher clock speed than a desktop or laptop CPU
B. puts memory, clock and I/O on the same chip as the processor
C. can be programmed only in assembly language, never in C
D. always needs a full operating system such as Linux to run

Answer: B. puts memory, clock and I/O on the same chip as the processor

A microcontroller is a whole small computer on one chip: processor, program memory, RAM, clock and I/O. A microprocessor is just the processor, with everything else in separate chips.

11. Which description best fits a **deeply embedded** system?

A. A phone chip that runs Android and the apps its owner installs
B. A desktop PC used for one program only, such as a cash register
C. A server that runs many virtual machines for different customers
D. A battery-powered sensor node that its user cannot program

Answer: D. A battery-powered sensor node that its user cannot program

Deeply embedded systems are microcontroller-based, dedicated, not programmable by their users, and run no OS or a tiny one, often as networked sensors on batteries.

12. An engine controller must fire each spark at an exact crankshaft angle, hundreds of times a second. Which requirement does this describe most directly?

A. Real-time behaviour
B. Low unit cost
C. Flexibility for updates
D. A long lifetime

Answer: A. Real-time behaviour

A result that is right but late is wrong here: at 6,000 rpm the crankshaft turns 36° every millisecond, so the spark must fire on time. That is a real-time requirement.

13.2 Characteristics of Embedded Operating Systems

You should be able to

- Explain when a device needs no operating system, a small real-time OS, or an adapted general-purpose OS.
- Name and explain the six characteristics that set embedded operating systems apart from general-purpose ones.
- Show how build-time configuration keeps an OS image small, and how short interrupts-off intervals keep interrupt latency low.
- Compare adapting an existing OS such as Linux with using a purpose-built embedded OS, and choose between them for a device.
- List the typical features of a purpose-built real-time OS and name some real examples.

Characteristics of embedded operating systems

An **embedded operating system** runs inside a device built for one dedicated job (a router, a pump, a car controller). It gives up much of what a desktop OS offers in exchange for a small **memory footprint** (flash for code, RAM for data), predictable timing and easy trimming.

How much OS does a device need?

- **None:** a **bare-metal program (super loop)** reads inputs, computes and drives outputs forever, helped by a few interrupt handlers. Enough for one simple job on a tiny **microcontroller** (one chip with processor, flash, RAM, clock and I/O hardware).
- **A small RTOS** (an OS whose services take a known, bounded time): several jobs with deadlines on one microcontroller, such as a drone flight controller. A kernel of a few KB gives tasks, priorities, timers and queues.
- **An adapted general OS** (usually Linux) for rich devices such as routers and smart TVs, with megabytes to gigabytes of memory.

The six characteristics

Characteristic	General-purpose OS	Embedded OS
Real-time operation	Good average speed and fairness.	A result is correct only if delivered on time; deadlines come from the physical system.
Reactive operation	Runs programs a user started.	Responds to outside events at unpredictable moments, so the worst-case response must be bounded.
Configurability	One big build for everyone.	Only the needed functions: modules chosen at build or link time, conditional compilation, or object-oriented components. Each configuration must be tested.
I/O device flexibility	Fixed device classes; drivers are kernel subsystems.	Device handling is done by special tasks added per device.
Streamlined protection	Privileged I/O; processes walled off by the MMU.	Trusted, tested code: tasks may do I/O directly; memory protection minimal or absent.
Direct use of interrupts	Only the kernel handles interrupts.	Tasks may attach their own handlers, skipping kernel layers.

These are tendencies, not laws: QNX keeps full memory protection, and connected or safety-critical devices often add protection back.

Configurability: a worked example (illustrative sizes)

Modules (flash/RAM in KB): kernel core 6/1 (always), multitasking 4/3, timers 2/1, queues 3/1, low-power radio 14/4, TCP/IP + Wi-Fi 110/48, flash file system 28/6, touch-screen GUI 140/64, MPU support 6/2, debug shell 16/4; all together 329/134.

- **Door sensor** (budget 32/8) needs timers and radio: 6 + 2 + 14 = 22 KB flash and 1 + 1 + 4 = 6 KB RAM. Adding multitasking raises RAM to 9 KB: no longer fits.

- **Smart thermostat** (512/160) needs multitasking, timers, queues, networking, file system, GUI: 293/124. **Drone controller** (128/32) needs multitasking, timers, queues, radio, MPU: 35/12.

Conditional compilation: code inside `#if CONFIG_NET ... #endif` is never compiled when CONFIG_NET is off, so it takes no flash. An unneeded module that fits is still dead weight: more to test, more to go wrong.

Interrupt latency

Interrupt latency runs from a device raising an interrupt to the first instruction of its handler. An interrupt arriving while interrupts are disabled must wait, so **worst-case latency = longest interrupts-off stretch + entry time**.

Example (illustrative): a general kernel with a 240 µs longest stretch and 4 µs entry has a worst-case latency of 244 µs; its driver (10 µs) and the wake-up and switch to the process (30 µs) push the start of the application's code to 284 µs. Averaged over all arrival moments, that code starts after only about 60 µs, but reactive operation means the worst case is what counts. An RTOS with a 6 µs longest stretch and 1 µs entry has a worst case of 7 µs, and with direct use of interrupts the application's own handler starts right then. A typical RTOS target is under 10 µs.

Protection and devices

- Desktop OS: reading a device register from user mode needs a system call (two mode switches plus the driver). Embedded OS: one ordinary instruction.
- A stray write into another task's data: an MMU faults and stops only the buggy process; no protection means silent corruption; a **memory protection unit (MPU)** (a few guarded regions, no address translation) faults and the kernel stops the faulty task.
- A new device: a desktop OS needs a privileged kernel driver; an embedded OS adds one task that owns the device's registers.

Two approaches

Adapt an existing OS (usually Linux): add real-time capability, streamline it, add embedded functions. Linux's real-time changes, PREEMPT_RT, were kept outside the main kernel for many years and merged in version 6.12 (2024). Pros: familiar interface, huge stock of drivers and software. Cons: not designed for real-time or tiny devices, so the worst case is harder to bound; larger, needing more memory, processor power and energy.

Purpose-built embedded OS: designed from the start for embedded, real-time use. Pros: small, fast, predictable. Cons: fewer ready-made drivers and applications, its own interface, extras as add-ons. Rich devices with soft timing adapt Linux; small, battery-powered or hard-deadline devices use an RTOS; some products run both on two processors.

Typical features of a purpose-built RTOS

1. Fast, lightweight task switch (one shared address space: only registers change).
2. Real-time scheduling, with the dispatcher built into the scheduler, so picking and switching to the next task is one short, bounded step.
3. Small size (a few KB to tens of KB).
4. Quick response to external interrupts.

5. Short, bounded interrupts-off intervals.
6. Memory partitions set up in advance, plus **memory locking** (never paged out).
    - **Fixed-size block pools** (memory pools) cut memory into equal-size blocks kept on a free list. Taking or returning a block is a fixed, small number of steps, and the pool has no external fragmentation, because any free block fits any request the pool serves. The cost is internal fragmentation: a 20-byte request in a 64-byte pool wastes the other 44 bytes of its block.
    - **Variable-size partitions** give each request about the size it asks for, so less is wasted inside blocks. A suitable allocator can put an upper bound on how long an allocation takes, but variable sizes do not remove external fragmentation: free 32 bytes, used 32, free 32 holds 64 free bytes, yet a 48-byte request fails because no single gap is big enough.
    - A fixed number of steps means a bounded, predictable cost, not an identical elapsed time on every call: cache misses and interrupts can still stretch a single call.
7. Special sequential files that accumulate data fast (append-only, space reserved).
8. Timing services: real-time clock, bounded-time primitives, alarms and **timeouts** (no wait lasts forever), delay and suspend/resume, deadline-ordered queues, urgent messages at the front of a queue.

Examples: FreeRTOS, Zephyr (configured with Kconfig), VxWorks (ran Mars Pathfinder), QNX (microkernel with full memory protection), ThreadX (now Eclipse ThreadX), eCos (build-time configuration).

Common mistake: "real-time means fast". It means predictable: a bounded worst case matters more than a good average.

Key terms

Embedded operating system
: An operating system built to run inside a device that has one dedicated job, such as a router, a pump or a car controller. It gives up general-purpose features in exchange for small size, predictable timing and easy trimming.

Bare-metal program (super loop)
: Software for a simple device that runs with no operating system at all: one endless loop that reads inputs, computes and drives outputs, helped by a few interrupt handlers.

Real-time operating system (RTOS)
: An operating system whose services take a known, bounded amount of time, so that the designer can guarantee that tasks meet their deadlines.

Microcontroller
: A single chip holding a processor, flash memory for the program, RAM for data, a clock and I/O hardware such as timers and converters. Cheap and frugal with power, with kilobytes rather than gigabytes of memory.

Memory footprint
: How much memory a piece of software occupies on the device: flash (or ROM) for its code and constants, RAM for its changing data.

Real-time operation
: An embedded-OS trait: a result counts as correct only if it is delivered on time, because the deadlines are set by the physical system being controlled.

Reactive operation
An embedded-OS trait: the software exists to respond to events from the outside world that can arrive at any moment, so its worst-case response time must be bounded and planned for.

Configurability
An embedded-OS trait: the OS can easily be built to contain only the functions one device needs, by choosing modules at build time, by conditional compilation, or with object-oriented components.

Conditional compilation
Telling the compiler to include or skip blocks of source code depending on configuration settings (for example with #if in C), so an unneeded feature never becomes part of the program at all.

I/O device flexibility
An embedded-OS trait: because devices vary so widely, the OS does not build fixed device support into its kernel; device handling is done by special tasks that can be added or swapped for each device.

Streamlined protection
An embedded-OS trait: because the software is fixed, tested and trusted, protection is kept light. Tasks may run I/O instructions themselves, and memory protection may be minimal or absent.

Direct use of interrupts
An embedded-OS trait: application tasks may attach their own handlers to interrupts or be woken by them directly, instead of every interrupt being handled only deep inside the kernel.

Interrupt latency
The time from the moment a device raises an interrupt to the moment the first instruction of its handler runs. Long stretches with interrupts disabled make the worst case long.

Memory protection unit (MPU)
Simple protection hardware found on many microcontrollers. It marks a handful of memory regions as readable, writable or executable for the running task, but does not translate addresses the way an MMU does.

Purpose-built embedded OS
An operating system designed from the start for embedded and real-time use, rather than cut down from a general-purpose OS.

PREEMPT_RT
A set of Linux kernel changes that make almost all kernel code preemptible and run most interrupt handling in schedulable threads, so latency stays bounded. Kept outside the main kernel for many years, it was merged into mainline Linux in version 6.12 (2024).

Memory pool (fixed-size partition)
Memory set aside in advance and cut into equal-size blocks. Taking or returning a block takes the same short time every time, and the pool has no external fragmentation (a request smaller than a block still wastes the rest of that block).

Memory locking
Pinning a task's code and data in main memory so they are never paged or swapped out, which removes page-fault delays from time-critical code.

Timeout

A limit on how long a task will wait for something, such as a semaphore, a message or a device. When the time is up the wait ends with an error code instead of lasting forever.

Review questions with answers

1. Which statement best describes **real-time operation** in an embedded operating system?

A. The OS runs every task as fast as the hardware allows it to.
B. A result counts as correct only if it arrives on time.
C. The OS shares the processor evenly among all of its tasks.
D. Every task must finish in less than ten microseconds.

Answer: B. A result counts as correct only if it arrives on time.
In real-time operation, when a result is delivered is part of whether it is correct; the deadlines come from the physical system being controlled.

2. A crash sensor can fire at any instant, even while the controller is busy with other work, and the designers must plan for the worst possible moment. Which embedded-OS characteristic does this illustrate most directly?

A. Configurability
B. Streamlined protection
C. Reactive operation
D. I/O device flexibility

Answer: C. Reactive operation
Reactive operation: the software responds to outside events that arrive unpredictably, so its worst-case response time must be bounded.

3. Why can an embedded OS often get away with **streamlined protection**?

A. Embedded processors cannot raise memory protection faults.
B. Embedded tasks never contain bugs, so the checks would never fire anyway.
C. Users of the device install their own apps, so trust is shared.
D. Its software is fixed and tested, not supplied by unknown users.

Answer: D. Its software is fixed and tested, not supplied by unknown users.
A desktop OS must defend against unknown, possibly hostile programs. An embedded device runs one fixed set of tested code, so light protection is an acceptable trade for speed and simpler hardware.

4. In a typical general-purpose OS, a user process may install its own interrupt handler and have it run directly when the device interrupts.

Answer: False
Only the kernel handles interrupts in a general-purpose OS; it may then wake the process. Letting tasks use interrupts directly is a characteristic of embedded systems.

5. A purpose-built RTOS usually makes the dispatcher part of the scheduler, so choosing the next task and switching to it happen in one short, bounded step.

Answer: True
Merging the two removes a separate stage between “this task must run now” and “it runs”, keeping that path short and predictable.

6. Which of these are standard ways to configure an embedded OS so that it contains only the functions a device needs? Select all that apply.

A. Selecting OS modules at build or link time
B. Conditional compilation
C. Object-oriented components, with subclasses for each need
D. Paging unused kernel code out to disk at run time
E. Disabling interrupts during start-up

Answer: A, B, C

Module selection, conditional compilation and object-oriented components all keep unneeded code out of the image. Paging is a run-time memory technique, and disabling interrupts has nothing to do with configuration.

7. An RTOS never keeps interrupts disabled for more than 6 µs at a time, and the processor needs 1 µs to save its state and jump to a handler. What is the worst-case interrupt latency, in µs?

Answer: 7 µs

In the worst case the interrupt arrives just as the longest interrupts-off stretch begins: 6 µs of waiting plus 1 µs of entry = 7 µs.

8. An OS image contains a 6 KB kernel core, a 4 KB scheduler, a 2 KB timer module and a 14 KB radio stack. The microcontroller has 32 KB of flash. How many KB are left for the application?

Answer: 6 KB

6 + 4 + 2 + 14 = 26 KB for the OS image, and 32 − 26 = 6 KB remain.

9. Building an embedded OS by adapting an existing general-purpose OS such as Linux: advantage or disadvantage?

Advantage	Disadvantage
Familiar programming interface Large stock of existing drivers and software Developers already know how to use it	Needs more memory and processor power Not designed for real-time, so the worst case is harder to bound Larger code and higher energy use

Adapting brings a familiar interface and a huge software base, but the OS was not optimized for real-time or tiny devices, so it is larger and hungrier than a purpose-built one.

10. Match each feature of a purpose-built RTOS to the problem it addresses.

Memory pools of fixed-size blocks	**Allocation time that never varies, and no external fragmentation**
Locking code and data in memory	**No page-fault delays in time-critical code**
Short interrupts-off intervals	**A low worst-case interrupt latency**
Timeouts on waits	**A task can never be stuck waiting forever**
Fast sequential files	**Logging data at a high rate without pausing**

Each feature removes one source of unpredictable delay, which is what a real-time system cares about most.

11. On a general-purpose OS, a device interrupts while the kernel has interrupts disabled. Put these events in order, from the interrupt to the user process that needs the data running.

Correct order:

1. The device raises an interrupt
2. The processor waits until interrupts are enabled again
3. The processor saves its state and enters the kernel's handler
4. The kernel driver services the device and marks the process ready
5. The scheduler switches to the user process

Waiting out an interrupts-off stretch, entering the handler, the driver, and the wake-up and switch all add up. An embedded OS shortens the first and may skip the last two by letting the task's own handler run.

12. A delivery robot needs motor control every 100 µs, plus camera navigation, Wi-Fi and a touch screen. Which design is most common?

A. An RTOS on a microcontroller for motors, Linux on another processor
B. Stock desktop Linux alone, since it has the most drivers and libraries
C. No operating system: one super loop that handles everything
D. A purpose-built RTOS alone, with a GUI written from scratch

Answer: A. An RTOS on a microcontroller for motors, Linux on another processor

The two halves want different things, so many devices split them: a purpose-built RTOS for the hard deadlines and an adapted general OS for the rich features.

13.3 Embedded Linux

You should be able to

- Explain how an embedded Linux distribution differs from desktop Linux: kernel configuration, memory, cross-compiled toolchain, C library and BusyBox.
- Estimate how kernel features, the C library, the command-line tools and compression change the size of a firmware image.
- Explain erase blocks and wear leveling, and choose squashfs, JFFS2, UBIFS or ext4/F2FS for a given flash device.
- List the advantages of embedded Linux and describe Android as a Linux kernel with its own, non-GNU user space.
- Explain why Linux without an MMU (µClinux) cannot offer fork(), and how vfork() plus exec replaces it.

What an embedded Linux distribution is

An **embedded Linux distribution** is Linux tailored to one kind of device: the kernel keeps only the drivers and features that hardware needs, every program is compiled for its processor, and the system is sized to fit its memory and flash.

Compared with desktop Linux: the kernel holds only this board's drivers (often 1–3 MB compressed), not thousands of modules; RAM is tens of MB (a few on the smallest systems), not gigabytes; software is cross-compiled on a host PC; and to save flash, constrained systems often swap glibc for musl or uClibc-

ng and the GNU commands for BusyBox, although others keep glibc and fuller user-space tools when they need full compatibility and have the room. Those library and command choices are configuration decisions; what makes the system embedded is that it is built around one device's needs.

Building it: host, target and cross-compilation

Software is built on a **host** (usually an x86-64 PC) for a **target** (the device's processor: ARM, MIPS, RISC-V). A **cross-compiler** runs on the host but produces target machine code; with an assembler, linker, the target's C library and kernel headers it forms a **cross toolchain**, named after its target, e.g. `arm-linux-gnueabihf-gcc` (ARM, Linux, GNU C library with ARM calling rules and hardware floating point).

Machine code belongs to one instruction set: an x86-64 program on an ARM board is refused ("Exec format error"), and an ARM program is refused on the PC unless an emulator such as QEMU runs it. **Embedded build systems** automate everything: **Buildroot** (menu-configured; produces kernel and root file system images) and the **Yocto Project** (layers of recipes run by BitBake, for complete custom distributions).

Making the image small

- **Kernel configuration**: switched-off features are not compiled in and cost nothing.
- **C library**: **glibc** is complete but the largest. **musl** is small, written from scratch, easy to link statically (used by OpenWrt and Alpine). **uClibc-ng** is small, configurable and supports MMU-less processors (it continues uClibc, inactive since about 2012). newlib mostly serves bare-metal and RTOS firmware. Android uses **Bionic**.
- **BusyBox**: one program with hundreds of slimmed-down commands; `/bin/ls`, `/bin/sh` ... are links to it, and it reads `argv[0]` (the name it was started under) to pick the command.
- **Compression**: a **squashfs** root is packed and compressed at build time (about half here); the kernel is already compressed. Settings need a separate writable partition.

Worked example (lab's illustrative sizes): kernel 1.6 + 0.9 networking = 2.5 MB; root = 0.6 musl + 0.9 BusyBox + 0.6 network programs + 0.2 config = 2.3 MB, squashfs at 50% = 1.15 MB; total 3.65 MB, fitting 8 MB of flash. With glibc, GNU tools and no compression: 2.5 + 2.4 + 12 + 0.6 + 0.2 = 17.7 MB.

Flash memory: why it is not a disk

- Erased flash reads as all 1s. **Programming** can only change 1s into 0s (a byte or word at a time on NOR, a page at a time on NAND).
- Changing a 0 back into a 1 needs an **erase**, and erases work only on a whole **erase block** (typically tens of KB to a few MB). Changing one byte in place in a 128 KB block means erasing all 131,072 bytes and writing the others back.
- Each block survives a limited number of erases (roughly 1,000 to 100,000, depending on the chip type).

A disk file system rewrites in place, so on raw flash it erases the same block again and again (toy chip, 8 blocks × 4 pages, 100 erases each: dead after 100 updates). **Wear leveling** spreads erases. **Dynamic**: each new version goes to a fresh page; full blocks of stale copies are erased, least-worn first (1,200 updates, but static-data blocks never share the wear). **Static**: rarely changed data is also moved off

barely-worn blocks (2,820 updates; ideal 8 × 100 × 4 = 3,200, less the erases spent on moves and life left unused in other blocks).

Flash file systems

Raw flash is driven through the kernel's **MTD** layer, so the file system must level wear, avoid bad blocks and never overwrite in place. **Managed flash** (eMMC, SD, USB sticks, SSDs) has a controller whose **flash translation layer (FTL)** does this and shows ordinary sectors.

File system	Writes?	Runs on	Key points
cramfs	no	any	Older, compressed, files up to 16 MB; largely replaced by squashfs
squashfs	no	any	Compressed, built in one go; usual router/camera root
JFFS2	yes	raw NOR/NAND	Levels wear, compresses; scans whole chip at mount: small partitions
YAFFS2	yes	raw NAND	NAND-specific; early Android; outside mainline
UBIFS	yes	raw NAND via UBI	UBI levels wear chip-wide, handles bad blocks; index on flash, fast mount
ext4	yes	managed flash	Standard journaling FS; the controller levels wear
F2FS	yes	managed flash	Log-structured, sequential writes; phone data partitions

Guide: never written → squashfs; behind a controller → ext4 or F2FS; large raw NAND → UBIFS; small raw partition → JFFS2. Never ext4 on raw flash.

Why device makers choose Linux

Vendor independence (no lock-in to one supplier), **varied hardware support** (many architectures, thousands of drivers), **low cost** (no per-device license fee; the real costs are engineering and security updates) and **open source** (code can be read and changed; the GPL requires sharing kernel changes you ship). The price: Linux needs megabytes of RAM and usually an MMU (but see µClinux), so tiny microcontrollers use an RTOS; and it is not hard real-time out of the box (PREEMPT_RT, merged in Linux 6.12 in 2024, shortens worst-case delays).

Android as an embedded Linux

Android runs the Linux kernel (plus Android additions; the Binder driver is now in mainline) but its own user space, not the GNU one: Bionic, Toybox and mksh, its own init, Binder IPC, the Android Runtime (ART, which replaced Dalvik in Android 5.0) and its app framework. It runs on phones, tablets, TVs, cars, watches.

µClinux: Linux without an MMU

µClinux began in 1998 as a port of Linux to processors with no MMU; its changes entered the main kernel in the 2.5 series, so since Linux 2.6 (2003) mainline can be built without MMU support. Every address is then physical: no memory protection; no virtual memory, paging or swapping; programs are relocated to

where they load (flat or FDPIC binaries) with stack sizes fixed at build time; large requests need contiguous memory, so external fragmentation can make them fail; shared libraries are limited; and **there is no fork()**.

With an MMU, fork() copies the parent's page table: same virtual addresses, same frames shared copy-on-write, and pointers stay valid because each process is translated separately. Without an MMU a copy must sit at other physical addresses, yet its pointers still hold the parent's addresses, so the child would overwrite the parent's memory. (Copy on write only makes fork() cheap; fork() truly needs address translation.)

vfork() + exec instead: (1) the parent calls vfork(); (2) the child runs in the parent's borrowed memory while the parent is suspended, and may only call exec or _exit; (3) the child calls exec; (4) the kernel loads and relocates the program in free, contiguous memory; (5) the parent resumes, its memory untouched.

Key terms

Embedded Linux distribution
: A version of Linux cut down and configured for one kind of device: only the kernel features, drivers, libraries and programs that device needs, compiled for its processor and sized to fit its memory and storage.

Cross-compiler (cross toolchain)
: A compiler, together with its assembler, linker and libraries, that runs on one kind of computer (the host, often an x86-64 PC) but produces programs for a different processor (the target, such as an ARM board).

Embedded build system
: A tool such as Buildroot or the Yocto Project that fetches source code, cross-compiles the toolchain, kernel, libraries and programs, and packs them into images ready to write to a device.

C library (libc)
: The library of standard C functions (printf, malloc, fopen…) and system-call wrappers that almost every Linux program uses. Desktop Linux uses glibc; embedded systems often use a smaller one.

glibc (GNU C Library)
: The full-featured C library of desktop and server Linux distributions: very complete and widely compatible, but larger than the embedded alternatives.

musl
: A small C library for Linux written from scratch with a focus on correctness and easy static linking. OpenWrt and Alpine Linux use it.

uClibc-ng
: A small, highly configurable C library for embedded Linux, including processors without an MMU. It continues the older uClibc project, which stopped releasing around 2012.

BusyBox
: One small executable that contains slimmed-down versions of hundreds of standard commands (ls, cp, sh, mount…). Each command name is a link to the same file, and BusyBox looks at the name it was started under to decide which command to act as.

Root file system (rootfs)

The file system mounted at / when Linux starts. It holds the programs, libraries and configuration files; on a device it is usually built as one image and written to flash.

Flash memory
Nonvolatile memory with no moving parts. It is programmed in small units (a page on NAND, a byte or word on NOR), but bits can be turned back into 1s only by erasing a whole erase block, and each block survives a limited number of erases.

Erase block
The smallest region of flash that can be erased at once, typically tens of KB to a few MB. Erasing sets every bit in it to 1; programming can then only change 1s into 0s.

Wear leveling
Spreading erases evenly over all the blocks of a flash chip, so that no block wears out early while others are barely used.

Raw flash (MTD)
A flash chip that Linux drives directly through its memory technology device (MTD) layer, with no controller in between. The file system on top must handle erase blocks, wear leveling and bad blocks (blocks that have failed) itself.

Flash translation layer (FTL)
Firmware inside the controller of managed flash (eMMC, SD cards, USB sticks, SSDs) that hides erase blocks and levels wear, so the device looks like an ordinary disk with rewritable sectors.

squashfs
A compressed, read-only Linux file system. The whole tree is packed into one image when the firmware is built; widely used for the root file systems of routers and other devices.

JFFS2
Journalling Flash File System version 2: a read-write file system for raw flash that writes every change to a new place, levels wear and compresses data. It scans the whole chip when mounted, so it suits small partitions.

UBIFS
A read-write file system for raw NAND flash that runs on UBI, a layer that levels wear and handles bad blocks across the whole chip. It keeps its index on flash, so it mounts quickly even on large chips.

YAFFS2
Yet Another Flash File System 2: a log-structured file system designed for NAND flash and used in many devices, including early Android phones. It is maintained outside the mainline Linux kernel.

µClinux (uClinux)
Microcontroller Linux: Linux for processors without a memory management unit. Its changes entered the mainline kernel during the 2.5 development series, so kernels from 2.6 on can be built for MMU-less chips.

vfork()
A variant of fork() in which the child borrows the parent's memory instead of getting a copy, and the parent is suspended until the child calls exec or _exit. It is how programs start other programs on Linux without an MMU.

Review questions with answers

1. Which description best fits an **embedded Linux distribution**?

A. A complete desktop distribution copied unchanged onto a device's flash chip
B. Linux cut down for one device: only its drivers and programs, built for its processor
C. A separate OS that copies Linux's commands but runs its own, non-Linux kernel
D. Any ordinary Linux system that happens to run without a keyboard or a screen

Answer: B. Linux cut down for one device: only its drivers and programs, built for its processor

An embedded Linux distribution keeps only the kernel features, drivers, libraries and programs one kind of device needs, compiles them for its processor and sizes them to its memory and storage.

2. A program compiled with the host's own `gcc` on an x86-64 PC will run on an ARM router, as long as both run Linux.

Answer: False

Machine code belongs to one instruction set. The router's kernel sees x86-64 code in the file and refuses it (Exec format error). The program must be built with a cross-compiler for ARM, such as arm-linux-gnueabihf-gcc.

3. How can a single BusyBox file act as `ls`, `cp` and `sh`?

A. The kernel unpacks a different hidden program from the file for each command
B. Each command is a small shell script that calls BusyBox with a long option list
C. Each command name links to it, and it reads argv[0] to choose the command
D. It copies itself under every command's name each time the system boots up

Answer: C. Each command name links to it, and it reads argv[0] to choose the command

/bin/ls, /bin/cp and /bin/sh are links to /bin/busybox. When started, BusyBox looks at argv[0], the name it was started under, and runs the matching built-in command.

4. Which of these are **C libraries** used instead of glibc to save space on embedded or mobile Linux?

A. musl
B. uClibc-ng
C. BusyBox
D. Bionic
E. squashfs

Answer: A, B, D

musl, uClibc-ng and Android's Bionic are smaller C libraries. BusyBox provides commands, not the C library, and squashfs is a read-only file system.

5. A flash chip has 128 KB erase blocks (1 KB = 1,024 bytes). A program changes one byte, and one of its bits must go from 0 to 1. How many bytes are erased to make that change?

Answer: 131072 bytes

A 0 → 1 change needs an erase, and an erase always covers the whole block: 128 × 1,024 = 131,072 bytes. The other 131,071 bytes must be saved first and written back.

6. A flash chip has 8 erase blocks of 4 pages each, and every block survives 100 erases. If wear were spread perfectly and every erase made room for 4 new single-page updates, how many updates could the chip absorb before its blocks wear out?

Answer: 3200 updates

8 blocks × 100 erases × 4 pages per erase = 3,200 updates. Real wear leveling only gets close to this: moving static data costs extra erases, and when the first block wears out the others usually have a little life left.

7. Match each file system to the job it suits best.

squashfs	**Read-only, compressed root image built with the firmware**
JFFS2	**Small read-write partition on a raw NOR chip**
UBIFS	**Large, busy raw NAND chip that must mount quickly**
ext4 or F2FS	**eMMC or SD card whose own controller levels wear**

Read-only images use squashfs. On raw flash the file system must handle wear: JFFS2 for small partitions, UBIFS (on UBI) for large NAND. Managed flash hides erase blocks, so ordinary file systems such as ext4 or F2FS work.

8. Who levels the wear in each setup?

Linux (file system or UBI)	**The storage device's controller (FTL)**
UBIFS on a raw NAND chip JFFS2 on a raw NOR chip YAFFS2 on raw NAND	ext4 on a smart TV's eMMC F2FS on a microSD card ext4 on an SSD

On raw flash, Linux sees the chip directly, so the flash file system (or UBI) must level wear. eMMC, SD cards and SSDs contain a controller whose flash translation layer does it, and Linux sees ordinary sectors.

9. Why does Linux built for a processor **without an MMU** not offer `fork()`?

A. Small chips are far too slow to copy a whole process in a reasonable time
B. Without an MMU the kernel cannot run more than one process at a time
C. Without page tables a new process cannot get its own process ID
D. A copy must sit at other addresses, yet its pointers still lead into the parent

Answer: D. A copy must sit at other addresses, yet its pointers still lead into the parent

With no address translation, the child's copy must live at different physical addresses, but every pointer copied from the parent still holds the parent's addresses. The child would read and corrupt the parent's memory, so vfork() + exec is used instead.

10. Put the steps of starting a new program with `vfork()` on Linux without an MMU in order.

Correct order:

1. The parent calls vfork()
2. The child runs in the parent's borrowed memory while the parent is suspended
3. The child calls exec for the new program
4. The kernel loads the program into free, contiguous memory and relocates it
5. The parent resumes with its memory untouched

vfork() lends the parent's memory to the child and suspends the parent. The child immediately calls exec; the kernel loads and relocates the new program elsewhere, and the parent continues.

11. Which part of a desktop GNU/Linux system does **Android** share?

A. The Linux kernel
B. glibc, the GNU C library
C. GNU coreutils and the bash shell
D. The X11 or Wayland display system

Answer: A. The Linux kernel

Android keeps the Linux kernel (with some Android additions) but replaces the user space: Bionic, Toybox, its own init, Binder IPC, the ART runtime and its app framework.

12. Which are genuine advantages of embedded Linux?

A. Independence from any single OS vendor
B. Support for many processor types and drivers
C. No license fee to pay per device
D. Hard real-time guarantees without any changes
E. Runs in a few kilobytes of RAM

Answer: A, B, C

Linux gives vendor independence, broad hardware support, low cost and open source. It needs megabytes of memory, and it is not hard real-time out of the box (PREEMPT_RT, mainline since 6.12, improves its worst case).

13.4 TinyOS

You should be able to

- Describe a wireless sensor network, what a mote contains, and the goals an operating system for motes must meet.
- Explain why TinyOS has no separate kernel, no processes and no virtual memory, and how one program image is built from components.
- Read a nesC wiring: tell modules from configurations, and the commands of an interface from its events.
- Trace how tasks, interrupt handlers and split-phase operations share one processor and one stack, and why the mote sleeps whenever the task queue is empty.
- Tell dedicated, virtualized and shared resources apart, predict an arbiter's grants, and estimate battery life from a duty cycle.

Wireless sensor networks and motes

A **wireless sensor network (WSN)** is a large number of small, battery-powered computers spread over an area. Each node, called a **mote**, measures something physical and radios its readings toward a **base station** (the sink), which passes them to an ordinary computer. Readings are relayed hop by hop; motes near the base station relay for many others and use more energy.

A typical mote of the mid-2000s (TelosB-class) has a 16-bit microcontroller at 8 MHz, 10 KB of RAM, 48 KB of program flash (plus a separate 1 MB flash chip for data), a low-power radio of 250 kbit/s with a range of tens of metres, a few sensors and two AA cells. Older MICA-class motes had only 4 KB of RAM.

An operating system for motes must: (1) use almost no power, sleeping nearly all the time; (2) fit in a tiny memory; (3) handle concurrency, since sensing, computing, sending and relaying overlap; (4) be robust, running unattended for months; (5) support many hardware designs and applications, building only what each needs.

TinyOS is not a conventional OS

TinyOS is an open-source OS for motes, begun at UC Berkeley around 2000 (last release 2.1.2 in 2012; newer systems such as Contiki-NG, RIOT and Zephyr now run many sensor devices). It is a library of small software components. The application is written as components too, and the compiler joins them with exactly the TinyOS components they use into **one program image**, which is loaded into the mote's flash. Unused components are left out; blinking an LED from a timer needs a few kilobytes of code and well under a kilobyte of RAM.

Conventional OS	TinyOS	Why
Kernel and user modes, system calls	No boundary: one privilege level, ordinary function calls	One trusted program; often no protection hardware
Many processes	One application, which is the whole system	A mote does one job
Threads, each with its own stack	One stack shared by all tasks and handlers	RAM: a few thread stacks would fill it
Virtual memory	None: physical addresses	No MMU, no disk
Run-time allocation (malloc)	All memory reserved at compile time	No failures months later; no fragmentation
Programs loaded at run time	Whole program compiled at once	Smaller code; the compiler can inline and check for data races

Illustrative stack budget: six 512-byte thread stacks plus 1,200 bytes of data need 4,272 bytes (42 % of 10 KB); TinyOS needs 1,200 + 256 = 1,456 bytes (14 %). TinyOS 2.1 added an optional thread library for those who accept that cost.

Components, interfaces, commands and events (nesC)

TinyOS is written in **nesC**, a dialect of C. A **component** *provides* some interfaces and *uses* others, and reaches the rest of the system only through them. An **interface** is two-way:

- **Commands** go down: the *user* of the interface calls them and the *provider* implements them (start a timer, send a packet, read a sensor).
- **Events** go up: the *provider* signals them and the *user* must implement a handler for every one, even an empty one (timer fired, packet sent, reading done).

Examples: Boot has only an event (booted) and Leds only commands; Timer's main commands are startPeriodic, startOneShot and stop, and its event is fired; Read has read/readDone; AMSend has send/sendDone; SplitControl has start/stop and startDone/stopDone.

There are two kinds of component. A **module** is written as nesC code: variables, tasks, command implementations and event handlers. A **configuration** contains no ordinary code: it lists components and **wires** them, writing `User.Interface -> Provider.Interface` (the arrow points from user to provider). Configurations can contain other configurations, all the way down to modules that touch the hardware. Generic components are instantiated with `new`, e.g. `new TimerMilliC()` gives a timer of one's own. In the example, module SenseC uses six interfaces and configuration SenseAppC wires them to MainC, ActiveMessageC, TimerMilliC, a sensor, AMSenderC and LedsC. Wiring is fixed at compile time, so the compiler knows where every call goes.

Tasks, interrupt handlers and split-phase operations

- **Tasks** are deferred work. `post t();` puts task t at the tail of a first-in, first-out queue. The scheduler runs the task at the head **to completion**. Tasks never preempt one another and, with the default scheduler, have no priorities; a long task delays every other task, so long work should be split into short tasks that post the next piece. In TinyOS 2.x each task has one queue slot, so posting a task that is already waiting fails.
- **Interrupt handlers** run the moment hardware interrupts, even in the middle of a task, on the same stack; when they return, the task resumes where it stopped. Good handlers are short: save the data, post a task, return. A task posted by a handler waits its turn in the queue. In TinyOS 2.x, completion events such as fired, readDone and sendDone are signalled from a task that the low-level handler posts, so these events never cut into another task and a long task delays them too; only code marked `async` may run inside an interrupt.
- **Split-phase operations**: a command starts a slow job (sensor reading, packet send, radio power-up) and returns at once; an event reports completion later. Nobody waits, so nobody needs a stack of their own: one stack serves the whole mote. A common mistake is expecting `read()` to return the value; the value arrives in readDone.
- When the task queue is empty, the scheduler puts the microcontroller into **low-power sleep** until the next interrupt.

One sense-and-send cycle in TinyOS 2.x: the timer's compare interrupt posts a task that signals Timer.fired, whose handler posts senseTask → senseTask calls read() and finishes → the processor sleeps while the sensor converts → the conversion-done interrupt posts a task that signals readDone, whose handler stores the value and posts sendTask → sendTask calls send() and finishes → the processor sleeps while the radio transmits → the radio interrupt posts a task that signals sendDone, whose handler toggles an LED → the queue is empty, so the processor sleeps until the next timer interrupt. In the illustrative simulation the CPU is awake 12 ticks of each 30, yet the radio uses about 82 % of the charge although it is on for only 6 ticks.

Resources in TinyOS 2.x

- **Dedicated**: exactly one component uses it, so no sharing rules are needed (a hardware timer's compare register, an interrupt line).
- **Virtualized**: software gives every client its own logical copy. Every `new TimerMilliC()` is a virtual timer; the timer system keeps each client's next deadline and sets the one hardware alarm to the *earliest*. With periods 250, 400 and 1000 ms the alarm fires at 250 (A), 400 (B), 500 (A), 750 (A), 800 (B), 1000 (A and C), 1200 (B)...
- **Shared**: clients take turns, one at a time (an SPI bus, the serial link the microcontroller uses to reach the radio chip and the data flash chip). An **arbiter** decides: a client calls `request`, is told `granted` by an event, uses the resource and calls `release`. A first-come-first-served arbiter grants the longest-waiting request; a round-robin arbiter checks clients in a fixed circular order starting just after the one that released. Example: clients 1, 2, 3; client 3 holds the bus, 2 requests, then 1; when 3 releases, FCFS grants 2 and round robin grants 1. When nobody wants the resource, its default owner (a power manager) can switch it off.

An arbiter is not the task scheduler: the scheduler picks the next task, the arbiter picks which client may use one shared resource.

Duty cycle and battery life

The **duty cycle** d is the fraction of time the mote is awake. Average current and lifetime:

$I_{avg} = d \times I_{awake} + (1 - d) \times I_{sleep}$ and **lifetime = capacity ÷ I_{avg}**.

Worked example (illustrative TelosB-class numbers: 20 mA awake, 5 µA = 0.005 mA asleep, two AA cells of 2,500 mAh). At d = 1 %: $I_{avg} = 0.01 \times 20 + 0.99 \times 0.005 = 0.2 + 0.00495 = 0.20495$ mA, so the lifetime is 2,500 ÷ 0.20495 ≈ 12,198 hours ≈ 508 days ≈ 1.4 years. Always awake (d = 100 %): 2,500 ÷ 20 = 125 hours, about 5 days. If the processor idles awake at 2 mA instead of sleeping, d = 1 % gives 0.01 × 20 + 0.99 × 2 = 2.18 mA and only about 48 days. At very low duty cycles (below about 0.025 %) the sleep current becomes the larger part of the average, and beyond about ten years the battery's own self-discharge sets the limit. A radio that only listens draws about as much as one that transmits, which is why sensor networks also switch the radio off between messages.

Key terms

Wireless sensor network (WSN)
A network of many small, battery-powered computers spread over an area. Each one measures something in the physical world and radios its readings, often relayed hop by hop, to a collection point.

Mote (sensor node)
One small computer in a wireless sensor network: a microcontroller with a few kilobytes of RAM and some flash memory, plus sensors, a low-power radio and a battery.

Base station (sink)
The collection point of a sensor network. It receives the readings the motes send and passes them on to an ordinary computer or the internet.

TinyOS

An open-source operating system for wireless sensor motes, begun at UC Berkeley around 2000. It is a library of small components that is compiled together with one application into a single program image.

nesC

The dialect of C that TinyOS and its applications are written in. It adds components, interfaces and wiring to C, and its compiler analyses the whole program at once.

Component (nesC component)

A building block of a TinyOS program. It provides some interfaces and uses others, and it can reach the rest of the system only through those interfaces.

Interface (nesC interface)

A named, two-way contract between two components. It lists the commands the user may call and the events the provider will signal back.

Command (nesC command)

A function in an interface that the user of the interface calls and the provider implements. Commands travel down toward the hardware and ask for something, such as “start a timer” or “send this packet”.

Event (nesC event)

A function in an interface that the provider signals and the user must implement. Events travel up toward the application and report that something happened, such as “the timer fired” or “the packet was sent”.

Module (nesC module)

A component whose behaviour is written as nesC code: its variables, the commands it implements, its event handlers and its tasks.

Configuration (nesC configuration)

A component that contains no ordinary code. It names other components and wires their interfaces together; it can itself be used as a component.

Wiring

Connecting an interface that one component uses to a component that provides it. Wiring is written in a configuration and fixed when the program is compiled.

Task (TinyOS task)

A function that is posted to run later. The scheduler keeps posted tasks in a first-in, first-out queue and runs each one to completion; tasks never preempt one another.

Run to completion

Once a task starts, it keeps the processor until it returns. Interrupt handlers may cut in briefly, but no other task runs in the middle.

Split-phase operation

A long operation divided into two halves: a command starts it and returns at once, and an event later reports that it has finished. Nobody blocks while waiting.

Duty cycle

The fraction of time a device is awake and drawing full power. A mote whose processor and radio are on 1 % of the time has a 1 % duty cycle.

Dedicated resource

A hardware resource that exactly one component ever uses, so no sharing rules are needed.

Virtualized resource
: A resource that software splits into logical copies so that each client seems to have its own, as when many software timers run on one hardware timer.

Shared resource
: A resource that several clients must take turns using, one at a time, under the control of an arbiter.

Arbiter
: The component that decides which client may use a shared resource next. Clients call request and release; the arbiter signals granted to the client it picks, following a policy such as first-come-first-served or round robin.

Review questions with answers

1. On a TinyOS mote, a task is running when a hardware interrupt arrives. What happens?

A. The interrupt is held back until the task returns, and then its handler runs.
B. The task is moved to the tail of the task queue so that the handler can run.
C. The handler runs at once, and then the task continues from where it stopped.
D. The scheduler starts the next queued task while the handler is running.

Answer: C. The handler runs at once, and then the task continues from where it stopped.
Interrupt handlers preempt tasks. The handler runs briefly on the same stack (in TinyOS 2.x it is short async code that typically just posts a task); when it returns, the task carries on exactly where it was.

2. Task A is running and task B is already waiting in the TinyOS task queue. While running, A posts task C. Which task runs as soon as A returns?

A. C, because it was posted most recently.
B. A again, because it was the one that posted C.
C. Whichever of B and C needs less time.
D. B, because it was posted before C.

Answer: D. B, because it was posted before C.
TinyOS keeps posted tasks in a first-in, first-out queue. B was posted first, so B runs next and C waits its turn.

3. In TinyOS, a running task can be preempted by another task that the scheduler considers more important.

Answer: False
Tasks run to completion and never preempt one another; the default scheduler is plain first-in, first-out. Only interrupt handlers can cut into a running task.

4. Which of these does a TinyOS mote **not** have? Select all that apply.

A. A separate kernel running in a privileged mode
B. Virtual memory with page tables
C. A queue of posted tasks
D. Run-time memory allocation with malloc
E. Components joined by wiring

Answer: A, B, D

TinyOS compiles the application and the OS components it needs into one image with no kernel boundary, no virtual memory and no run-time allocation. It does have a task queue, and it is built entirely from wired components.

5. Match each nesC idea to its description.

Command	**Called by an interface's user, implemented by its provider**
Event	**Signalled by an interface's provider, handled by its user**
Module	**A component whose behaviour is written as code**
Configuration	**A component that only lists components and wires them**
Task	**A function posted to run later, to completion**

Commands go down and events come up through an interface. Modules hold code; configurations hold only wiring; tasks are deferred work run by the scheduler.

6. Put one sense-and-send cycle of a TinyOS mote in order.

Correct order:

1. The timer interrupt posts a task, which signals Timer.fired; its handler posts a sensing task
2. The sensing task calls read(), which starts the sensor and returns at once
3. The conversion-done interrupt posts a task, which signals readDone with the value; its handler posts a sending task
4. The sending task calls send(), which starts the radio and returns at once
5. The radio interrupt posts a task, which signals sendDone: the packet has gone
6. The task queue is empty, so the processor sleeps until the next interrupt

Each slow step is split-phase: a command starts it inside a task; later a hardware interrupt posts a task that signals the completion event, and that event's handler posts the next task. In between, the processor sleeps.

7. Sort each example by the kind of resource it is in TinyOS 2.x.

Dedicated	Virtualized	Shared
A hardware timer's compare register, owned only by the timer system An interrupt line wired to exactly one component	Each new TimerMilliC() an application creates	A serial (SPI) bus that the radio chip and the flash chip take turns using A flash chip that a data logger and a settings store both need, one at a time

A dedicated resource has one owner; a virtualized one gives every client its own logical copy (virtual timers on one hardware timer); a shared one is used by one client at a time under an arbiter.

8. A mote draws 20 mA when awake and 0.005 mA asleep, and it is awake 1 % of the time. What is its average current, in mA? (Give three decimal places.)

Answer: 0.205 mA

$I_{avg} = 0.01 \times 20 + 0.99 \times 0.005 = 0.2 + 0.00495 = 0.20495$ mA, about 0.205 mA.

9. A mote's average current is 0.25 mA. How many hours will a 2,500 mAh battery last, ignoring self-discharge?

Answer: 10000 h

Lifetime = capacity ÷ average current = 2,500 mAh ÷ 0.25 mA = 10,000 hours, a little over 416 days.

10. Why does TinyOS use split-phase operations instead of calls that wait until a device finishes?

A. Split-phase calls make the sensor and radio chips themselves work faster.
B. Waiting calls need a stack for every waiting activity, and RAM is tiny.
C. C has no way to write a call that waits until a device finishes.
D. Split-phase calls remove the need for components to handle events.

Answer: B. Waiting calls need a stack for every waiting activity, and RAM is tiny.

A call that waits needs a thread whose stack holds its state during the wait. Split-phase code keeps no waiting stack, so one shared stack serves the whole mote.

11. A round-robin arbiter controls a bus shared by clients 1, 2 and 3, checked in that circular order. Client 3 holds the bus; client 2 requests it, then client 1 does. Client 3 releases. Who is granted the bus?

A. Client 1, the first waiting client after 3 in circular order.
B. Client 2, because its request arrived before client 1's.
C. Client 3 again, because it was the most recent owner.
D. Nobody, because a released bus is always powered down first.

Answer: A. Client 1, the first waiting client after 3 in circular order.

Round robin starts looking just after the client that released: after 3 comes 1, which is waiting, so client 1 gets granted.

12. In a nesC configuration, the line `App.Timer -> TimerMilliC.Timer` means that App uses the Timer interface and TimerMilliC provides it.

Answer: True

The wiring arrow always points from the user to the provider: App calls the timer's commands, and TimerMilliC implements them and signals fired back to App.